```
================================================================================
  THE AMD INFERENCE STACK -- FROM PYTHON TO SILICON
  A Deep Beginner's Textbook on ONNX Runtime + MIGraphX + ROCm

  Target Hardware: Radeon 8060S (gfx1151, RDNA 3.5)
  With comparisons to MI210 (CDNA2) and MI300X (CDNA3)
================================================================================
```

TABLE OF CONTENTS


```
-----------------
  Chapter 0: Foundations for Beginners -- Everything You Need Before Page 1
    0.1-0.23: Core Concepts (CPU/GPU, Memory, Bandwidth, Kernels, ISA, ...)
    0.24: How to Read C++ Source Code (for Python Developers)
    0.25: Environment Variables Reference (HSA_XNACK, MIGraphX, ORT, System)

  Preface: The 8-Layer Stack Overview

  PART I -- THE APPLICATION LAYERS
    Chapter 1: ONNX Runtime Core (Layer 2) -- Protobuf, Graphs, Partitioning, and Subgraphs
    Chapter 2: MIGraphX Execution Provider (Layer 3) -- The AMD Translator
    Chapter 3: MIGraphX Library, libmigraphx.so (Layer 4) -- The Brain

  PART II -- THE RUNTIME LAYERS
    Chapter 4: HIP Runtime, libamdhip64.so (Layer 5) -- The GPU Remote Control
    Chapter 5: HSA Runtime, libhsa-runtime64.so (Layer 6) -- The Hardware Interface

  PART III -- THE SYSTEM LAYERS
    Chapter 6: amdgpu Kernel Driver (Layer 7) -- The Gatekeeper
    Chapter 7: AMD GPU Hardware (Layer 8) -- The Silicon

  PART IV -- REFERENCES AND APPENDICES
    Appendix A: Libraries Used by ORT + MIGraphX -- The Complete Map
    Appendix B: The Complete API Call Chain -- All 8 Layers, Unified
    Appendix C: Summary Tables and Key Insights
    Appendix D: Annotated Source Code Walkthroughs (D.1-D.12)
      D.1-D.7:  Line-by-line walkthroughs of 7 key functions
      D.8:      target::get_passes() -- Complete pass list
      D.9:      Numbered 50-pass table
      D.10:     Source file reference table (Layers 2-5)
      D.11:     Profiling and observability tools
      D.12:     Precision formats (BF16, FP8, INT4/INT8)
    Appendix E: Deployment Engineering
      E.1:      Ten Engineering Lessons
      E.2:      Infrastructure Engineering (swap, cache, disk)
      E.3:      Per-Model Workaround Patterns
      E.4:      Complete Deployment Checklist
```


```
================================================================================
```

## CHAPTER 0: FOUNDATIONS FOR BEGINNERS

```
  Everything You Need Before Page 1
  (If you only know Python basics, start here)
================================================================================
```

This chapter exists because the rest of this textbook uses dozens of terms from computer science, hardware engineering, and systems programming. If you only know Python, many of these words would send you to Google for hours. This chapter explains every foundational concept you will encounter, using Python analogies wherever possible. Read this FIRST, then move to the Preface.

### 0.1 What is a CPU vs a GPU?

You already know the CPU - it runs your Python code. But what IS it?

CPU (Central Processing Unit):
- The "brain" of your computer
- Has a few very powerful cores (typically 4-16 on a laptop)
- Each core is incredibly smart - it can handle ANY task
- But it does them mostly one at a time (sequential)
- Think: a brilliant professor who solves one problem at a time, very fast

GPU (Graphics Processing Unit):
- Originally designed to draw pixels on your screen
- Has THOUSANDS of tiny, simple cores (hundreds to tens of thousands)
- Each core is very dumb compared to a CPU core
- But they all work TOGETHER on the same problem simultaneously (parallel)
- Think: an army of 10,000 students who each solve one simple math problem

Why GPUs for AI?
  AI models are essentially giant piles of matrix math (multiply millions of

```
  numbers together). This is exactly what GPUs are built for:

  CPU approach:
    "Multiply these 1,000,000 numbers one by one"
    Time: 1,000,000 steps

  GPU approach:
    "All 10,000 of you, each multiply 100 numbers"
    Time: 100 steps (10,000x faster conceptually!)

  In Python terms:
    # CPU-like (sequential)
    result = []
    for i in range(1000000):
        result.append(a[i] * b[i])

    # GPU-like (parallel) - all happen AT ONCE
    result = a * b  # NumPy does this in parallel internally

  +-- DIAGRAM: CPU vs GPU Architecture -------------------------------------+
  |                                                                          |
  |   CPU (few powerful cores)          GPU (thousands of simple cores)     |
  |  +-------------------------+         +--------------------------------+   |
  |  | +-------+  +-------+     |         | +---++---++---++---++---++---++---++---+   |   |
  |  | |Core0|  |Core1|     |         | |.||.||.||.||.||.||.||.|   |   |
  |  | |SMART|  |SMART|     |         | `----+`----+`----+`----+`----+`----+`----+`----+   |   |
  |  | `--------+  `--------+     |         | +---++---++---++---++---++---++---++---+   |   |
  |  | +-------+  +-------+     |         | |.||.||.||.||.||.||.||.|   |   |
  |  | |Core2|  |Core3|     |         | `----+`----+`----+`----+`----+`----+`----+`----+   |   |
  |  | |SMART|  |SMART|     |         | +---++---++---++---++---++---++---++---+   |   |
  |  | `--------+  `--------+     |         | |.||.||.||.||.||.||.||.|   |   |
  |  |   4-16 cores          |         | `----+`----+`----+`----+`----+`----+`----+`----+   |   |
  |  |   Each handles ANY    |         |  ... hundreds to thousands    |   |
  |  |   task (complex)      |         |  Each does ONE simple op      |   |
  |  `-------------------------+         `--------------------------------+   |
  |                                                                          |
  |  Task: Multiply 1,000,000 numbers                                        |
  |                                                                          |
  |  CPU: --[1]--[2]--[3]--[4]-- ... --[1M]--  (sequential, 1M steps)    |
  |                                                                          |
  |  GPU: --[1..100]--     (10,000 cores each do 100, done in 100 steps)  |
  |        --[101..200]--                                                    |
  |        --[201..300]--  ... all 10,000 cores working simultaneously     |
  |        --[...     ]--                                                    |
  `--------------------------------------------------------------------------+


0.2 What is Memory (RAM, VRAM, HBM)?
-------------------------------------

Memory is where your computer stores data that programs are actively using.
It is NOT your hard drive/SSD (that is "storage"). Memory is much faster
but much smaller.

RAM (Random Access Memory):
  - Your computer's main memory (8 GB, 16 GB, 32 GB, etc.)
  - Where Python stores your variables, lists, and objects
  - When you do "x = [1, 2, 3]" in Python, that list lives in RAM
  - Speed: ~40-80 GB/s (how fast you can read/write data)
  - Analogy: your desk - everything you're actively working on is here

VRAM (Video RAM):
  - The GPU's own private memory (separate chip on discrete GPUs)
  - When the GPU needs to do math, the data must be in VRAM first
  - Your CPU cannot directly touch VRAM, and the GPU cannot directly
    touch RAM (they must copy data back and forth)
  - Speed: varies widely (~256 GB/s on your APU, 1600+ GB/s on data center GPUs)
  - Analogy: a separate desk in another room that only the GPU can use

HBM (High Bandwidth Memory):
  - A special, super-fast type of VRAM used on expensive data center GPUs
  - Stacked vertically on the chip (3D packaging) for extreme speed
  - MI210 has 64 GB of HBM2e at 1,638 GB/s
  - MI300X has 192 GB of HBM3 at 5,300 GB/s
  - Your laptop GPU does NOT have HBM - it shares LPDDR5x system RAM

LPDDR5x:
  - The type of RAM in your Strix Halo system (Low Power DDR5, extended speed)
  - 256-bit bus at 8000 MT/s for high bandwidth in a mobile package
  - Speed: ~256 GB/s in your system (shared between CPU and GPU)

APU (Accelerated Processing Unit):
  - YOUR system has an APU - a chip where CPU and GPU are ON THE SAME chip
```

```
  - They share the SAME LPDDR5x memory (no separate VRAM chip)
  - 512 MB of that LPDDR5x is "carved out" exclusively for the GPU (your VRAM)
  - The rest (~28 GB) the GPU can also access (GTT pool)
  - Advantage: no slow copying between CPU and GPU memory
  - Disadvantage: GPU must share bandwidth with CPU (still slower than HBM)

  +-- DIAGRAM: Memory Hierarchy (fastest/smallest at top) -------------------+
  |                                                                          |
  |          +-------------------+                                          |
  |          |  GPU Registers   |  <- ~20 TB/s, 256 KB per CU              |
  |          `-----------+----------+                                        |
  |             +-------+--------+                                           |
  |             |  L1 Cache   |  <- ~10 TB/s, 32 KB per CU                 |
  |             `---------+-------+                                          |
  |           +----------+----------+                                        |
  |           |    L2 Cache     |  <- ~3 TB/s, 2 MB shared                 |
  |           `-----------+----------+                                        |
  |        +-------------+-------------+                                     |
  |        |  Infinity Cache (MALL) |  <- ~1 TB/s, 32 MB                   |
  |        `--------------+-------------+                                    |
  |     +----------------+----------------+                                  |
  |     |  LPDDR5x (System Memory)     |  <- ~256 GB/s, 30 GB             |
  |     `-----------------+----------------+                                 |
  |  +-------------------+-------------------+                               |
  |  |  SSD / NVMe (Storage)              |  <- ~7 GB/s, 512 GB+          |
  |  `-------------------------------------+                                |
  `---------------------------------------------------------------------------+

  +-- DIAGRAM: APU Shared Memory Layout (Your Strix Halo) -------------------+
  |                                                                          |
  |           +-----------------------------------------------+              |
  |           |            Single APU Die                     |              |
  |           |  +------------+        +--------------------+   |              |
  |           |  |   CPU     |        |      GPU          |   |              |
  |           |  | 12 cores  |        |    40 CUs         |   |              |
  |           |  `--------+------+        `-----------+-----------+   |              |
  |           |        |      shared bus       |              |              |
  |           |        `-------------+--------------+              |              |
  |           `---------------------+-----------------------------+              |
  |                              |                                           |
  |                              v                                           |
  |  +-------------------------------------------------------------+        |
  |  |              LPDDR5x  ~30 GB  (~256 GB/s)                 |        |
  |  |  +------------+  +-----------------------+  +----------------+  |        |
  |  |  |   VRAM    |  |        GTT          |  |  CPU / OS     |  |        |
  |  |  |  512 MB   |  |      ~28 GB         |  |   ~2 GB       |  |        |
  |  |  | (GPU only)|  | (GPU can access)    |  | (kernel, etc) |  |        |
  |  |  `-------------+  `-----------------------+  `-----------------+  |        |
  |  `-------------------------------------------------------------+        |
  `---------------------------------------------------------------------------+


0.3 What is Bandwidth vs Latency?
-----------------------------------

These are the two most important performance concepts in this textbook.

Bandwidth = how much data you can move per second
  Analogy: a highway. Bandwidth is how many cars fit per hour.
  Measured in GB/s (gigabytes per second)
  Your LPDDR5x: ~256 GB/s
  MI210 HBM:    ~1,638 GB/s (~6x more "highway lanes")

Latency = how long it takes for ONE piece of data to arrive
  Analogy: how long it takes one car to drive from A to B.
  Measured in nanoseconds (ns) or microseconds (mus)
  1 mus (microsecond) = 1,000 ns (nanoseconds)
  1 ms (millisecond) = 1,000 mus = 1,000,000 ns

Time units you will see in this book:
  1 second      = 1,000 ms  = 1,000,000 mus = 1,000,000,000 ns
  1 ms (milli)  = 0.001 seconds   (you can feel this delay)
  1 mus (micro)  = 0.000001 seconds (faster than you can blink)
  1 ns (nano)   = 0.000000001 seconds (light travels 30 cm)

Why this matters for AI:
  AI models need to read billions of numbers (weights) from memory.
  If your memory bandwidth is low, the GPU sits idle waiting for data.
  This is why bandwidth is usually the bottleneck - the GPU can compute
  faster than memory can feed it data.

  +-- DIAGRAM: Bandwidth vs Latency ----------------------------------------+
  |                                                                          |
```

```
  |  BANDWIDTH = how wide the pipe is (data per second)                    |
  |  LATENCY   = how long the pipe is (time for first byte to arrive)     |
  |                                                                         |
  |  Your LPDDR5x (~256 GB/s bandwidth, ~100 ns latency):                 |
  |  ========================================>  wide pipe, medium length   |
  |  ========================================>                             |
  |  ========================================>  (256 GB/s = 256 lanes)     |
  |  ========================================>                             |
  |                                                                         |
  |  MI210 HBM2e (~1,638 GB/s bandwidth, ~100 ns latency):                |
  |  ========================================>                             |
  |  ========================================>  MUCH wider pipe            |
  |  ========================================>  (6x more lanes)            |
  |  ========================================>                             |
  |  ========================================>                             |
  |  ========================================>                             |
  |                                                                         |
  |  AI inference:  GPU compute (18 TFLOPS) >> memory feed (256 GB/s)     |
  |                 GPU is STARVING for data -> bandwidth-bound             |
  |                                                                         |
  |  Timeline of a bandwidth-bound kernel:                                 |
  |   GPU: [WAIT for data][compute][WAIT][compute][WAIT][compute]         |
  |   MEM: [----read----][----read----][----read----][----read---]        |
  |         ^^^^^^^^^^^^ GPU idle, waiting for memory                      |
  `-------------------------------------------------------------------------+


0.4 What is a GPU "Kernel"? (Not the Linux Kernel!)
-----------------------------------------------------

WARNING: The word "kernel" means TWO completely different things in this book.
This is the #1 source of confusion for beginners.

GPU Kernel:
  A function (piece of code) that runs on the GPU.
  It runs on thousands of GPU threads simultaneously.
  Think of it as a Python function, but instead of running once,
  it runs 10,000 copies of itself at the same time, each working
  on a different piece of data.

  Python analogy:
    # This is like a GPU kernel (conceptual - not real Python)
    def relu_kernel(data, thread_id):
        if data[thread_id] < 0:
            data[thread_id] = 0
    # GPU launches 10,000 copies of this function simultaneously

Linux Kernel:
  The core of the Linux operating system.
  It manages hardware, memory, processes, and security.
  When we say "kernel driver" we mean a piece of software
  inside the Linux kernel (the OS), NOT a GPU function.

How to tell which one we mean:
  "launch a kernel"         -> GPU kernel (a function running on GPU)
  "kernel driver"           -> Linux kernel (the operating system)
  "kernel space"            -> Linux kernel (OS internals)
  "GPU kernel launch"       -> GPU kernel
  "kernel module"           -> Linux kernel module (like amdgpu.ko)

  +-- DIAGRAM: GPU Kernel Execution Hierarchy -------------------------------+
  |                                                                          |
  |  hipLaunchKernel(relu_kernel, grid, block, args)                        |
  |                                                                          |
  |  Grid (entire work)                                                     |
  |  +-------------------------------------------------------+               |
  |  |  Block(0,0)      Block(1,0)      Block(2,0)         |               |
  |  |  +-------------+  +-------------+  +-------------+       |               |
  |  |  | T0 T1 T2  |  | T0 T1 T2  |  | T0 T1 T2  |       |               |
  |  |  | T3 T4 T5  |  | T3 T4 T5  |  | T3 T4 T5  |       |               |
  |  |  | T6 T7 T8  |  | T6 T7 T8  |  | T6 T7 T8  |       |               |
  |  |  `-------------+  `-------------+  `-------------+       |               |
  |  |  Block(0,1)      Block(1,1)      Block(2,1)         |               |
  |  |  +-------------+  +-------------+  +-------------+       |               |
  |  |  | T0 T1 T2  |  | T0 T1 T2  |  | T0 T1 T2  |       |               |
  |  |  | T3 T4 T5  |  | T3 T4 T5  |  | T3 T4 T5  |       |               |
  |  |  | T6 T7 T8  |  | T6 T7 T8  |  | T6 T7 T8  |       |               |
  |  |  `-------------+  `-------------+  `-------------+       |               |
  |  `-------------------------------------------------------+               |
  |                                                                          |
  |  Grid    = all blocks (3x2 = 6 blocks here)                             |
  |  Block   = group of threads that share fast local memory (LDS)          |
  |  Thread  = one execution of the kernel function                         |
```

```
  |  Wave32  = 32 threads running in lockstep on one SIMD unit             |
  |                                                                         |
  |  Total threads = 6 blocks x 9 threads = 54                             |
  |  Real models: grid = 10,000+ blocks, block = 256 threads each         |
  `-------------------------------------------------------------------------+
```

## 0.5 What is a Compiler? What is Compilation?

In Python, you write code and run it directly:

```
  python my_script.py  ->  Python interpreter reads and runs it line by line
```

This is called "interpreted" - the computer reads your human-readable code at runtime and figures out what to do.

In C/C++ (which most of this stack is written in), you must COMPILE first:

```
  gcc my_program.c -o my_program   ->  creates a binary executable
  ./my_program                     ->  runs the compiled binary
```

Compilation = translating human-readable source code into machine code (binary instructions that the CPU or GPU hardware directly understands)

```
  Source code (human-readable):    int x = a + b;
                     |
                  compiler
                     |
                     v
  Machine code (binary):           add r1, r2, r3  (CPU instruction)
```

Why this matters:

GPU code MUST be compiled into GPU machine code before it can run. The GPU cannot read C++ or Python - it only understands its own binary instruction set (called ISA - see below).

This is why "compilation" takes time when you first load a model - the system is translating the math operations into GPU instructions.

JIT Compilation (Just-In-Time):

Compiling code at runtime, right when you need it, instead of ahead of time. MIGraphX uses JIT to compile custom fused operations on the fly. Think: Python's @jit decorator in Numba - same concept.

LLVM:

A popular open-source compiler framework. AMD uses LLVM to compile GPU code. It is not a single program but a collection of tools that transform code step by step from human-readable to machine code. Think: LLVM is the factory that builds GPU executables.

COMGR (Code Object Manager):

AMD's wrapper around LLVM specifically for GPU compilation. It takes HIP C++ code and produces AMD GPU machine code.

```
  +-- DIAGRAM: Compilation Pipeline ----------------------------------------+
  |                                                                          |
  |  +----------------+    +----------------+    +----------------+             |
  |  |  Source Code  |    |  LLVM IR      |    |  Machine Code|             |
  |  |  (.cpp/.hip)  |--->| (intermediate)|--->|  (.co/.hsaco)|             |
  |  |  Human-       |    |  Portable,    |    |  Binary GPU  |             |
  |  |  readable     |    |  optimizable  |    |  instructions|             |
  |  `-----------------+    `-----------------+    `-----------------+             |
  |                                                                          |
  |  Detailed pipeline for GPU code:                                        |
  |                                                                          |
  |  HIP C++ source   (v_add_kernel.hip)                                    |
  |       |                                                                  |
  |       v  [Clang front-end]                                              |
  |  LLVM IR           (platform-independent intermediate)                  |
  |       |                                                                  |
  |       v  [LLVM optimizer passes]                                        |
  |  Optimized IR      (dead code removed, loops unrolled)                  |
  |       |                                                                  |
  |       v  [AMDGPU back-end, target=gfx1151]                             |
  |  AMDGPU assembly   (v_add_f32 v0, v1, v2)                               |
  |       |                                                                  |
  |       v  [Assembler]                                                    |
  |  ELF code object   (.co binary, ready for GPU hardware)                 |
  |       |                                                                  |
  |       v  [hipModuleLoad]                                                |
  |  Loaded into GPU memory, ready to execute                               |
  `-------------------------------------------------------------------------+
```

## 0.6 What is an ISA (Instruction Set Architecture)?

ISA = the set of instructions a processor understands.

Every CPU and GPU has its own "language" - the binary instructions
it can execute. This is the ISA.

```
  Intel/AMD CPUs speak: x86-64 ISA
    Example: add eax, ebx    (add two numbers)

  AMD GPUs speak: AMDGPU ISA
    Example: v_add_f32 v0, v1, v2   (add two floating-point numbers on GPU)

  Your GPU (gfx1151) speaks: RDNA 3.5 variant of AMDGPU ISA
  MI210 (gfx90a) speaks: CDNA 2 variant of AMDGPU ISA

  Code compiled for one ISA will NOT run on another!
  A binary built for gfx90a will crash on your gfx1151.
  This is why the compiler must know your exact GPU model.
```

ELF Binary:
  The file format for compiled executables on Linux.
  Both CPU programs (/usr/bin/python) and GPU code objects (.co files)
  use the ELF format. It is a standard container for machine code.
  Think: a .zip file but for compiled programs.

```
  +-- DIAGRAM: ISA Compatibility --------------------------------------------+
  |                                                                           |
  |  Source code          Compiler              Binary                      |
  |  (portable)           (targets ISA)         (NOT portable)             |
  |                                                                           |
  |  same.hip ---+---> [compile for gfx90a]  --> kernel_mi210.co            |
  |             |       MI210 only                [Y] runs on MI210          |
  |             |                                 [N] CRASHES on gfx1151     |
  |             |                                                            |
  |             |----> [compile for gfx1151] --> kernel_8060s.co            |
  |             |       8060S only                [Y] runs on 8060S          |
  |             |                                 [N] CRASHES on gfx90a      |
  |             |                                                            |
  |             `----> [compile for gfx942]  --> kernel_mi300x.co           |
  |                     MI300X only               [Y] runs on MI300X         |
  |                                               [N] CRASHES on others      |
  |                                                                           |
  |  The --offload-arch= flag in the compiler determines which ISA         |
  |  the binary targets. Wrong arch = guaranteed crash.                    |
  `---------------------------------------------------------------------------+
```

## 0.7 What is a Library? What is a .so File?

In Python, you import libraries:
```
  import numpy
  import torch
```

In C/C++, libraries work similarly but come as compiled binary files.

Shared Library (.so file on Linux, .dll on Windows):
  A file containing pre-compiled functions that programs can use.
  ".so" stands for "shared object"

```
  Example:
    libmigraphx.so   -> MIGraphX's compiled library (the brain of AMD inference)
    libamdhip64.so    -> HIP runtime library (GPU memory and kernel launches)

  When MIGraphX needs to do a matrix multiply, it calls a function
  inside rocBLAS (which is the file librocblas.so on your system).
  It does NOT contain the source code - just compiled machine code.

  Python equivalent:
    When you do "import numpy", Python loads the numpy .so files
    (you can find them: numpy/__init__.py loads compiled C extensions)
```

Static Library (.a file):
  Similar to .so but the code gets baked directly into the program
  at compile time, rather than loaded at runtime.

dlopen/dlsym:
  C functions used to load .so files at runtime.
  Like Python's importlib.import_module() but for compiled C/C++ code.

```
  +-- DIAGRAM: Dynamic Library Loading ---------------------------------------+
```

```
  |                                                                        |
  |  Your application (e.g., ORT)                                         |
  |  +-----------------------------------------------------+                 |
  |  |  main() {                                          |                 |
  |  |    handle = dlopen("libmigraphx.so", RTLD_LAZY);  |  <- load at     |
  |  |    func = dlsym(handle, "migraphx_compile");      |    runtime      |
  |  |    func(program);                                  |  <- call it     |
  |  |  }                                                 |                 |
  |  `-------------------------+-----------------------------+                 |
  |                          | dlopen()                                      |
  |                          v                                               |
  |  +-------------------------------------------+                           |
  |  |  /opt/rocm-7.2.0/lib/libmigraphx.so      |                           |
  |  |  |---- migraphx_compile()                 |                           |
  |  |  |---- migraphx_parse_onnx()              | pre-compiled              |
  |  |  |---- migraphx_quantize_fp16()           | binary code               |
  |  |  `---- ... hundreds of functions          |                           |
  |  `-------------------------+----------------------+                          |
  |                          | internally calls                              |
  |                          v                                               |
  |  +-------------------------------------------+                           |
  |  |  /opt/rocm-7.2.0/lib/libamdhip64.so      |                           |
  |  |  |---- hipMalloc()                        |                           |
  |  |  |---- hipLaunchKernel()                  |                           |
  |  |  `---- hipDeviceSynchronize()             |                           |
  |  `-------------------------------------------+                           |
  |                                                                        |
  |  Chain: app -> libmigraphx -> libamdhip64 -> libhsa-runtime64 -> driver  |
  `-------------------------------------------------------------------------+


0.8 What is Serialization and Deserialization?
----------------------------------------------

Serialization = converting an in-memory object into a format that can be
  saved to a file or sent over a network.

Deserialization = the reverse - reading that file back into an in-memory object.

Python analogy:
  import json

  # Serialization: Python dict -> JSON string (saveable to file)
  data = {"name": "Alice", "age": 30}
  json_string = json.dumps(data)  # '{"name": "Alice", "age": 30}'

  # Deserialization: JSON string -> Python dict (usable in code)
  data_back = json.loads(json_string)  # {"name": "Alice", "age": 30}

In the AI stack:
  - Your model weights are serialized into a .onnx file (using Protobuf format)
  - ORT deserializes (loads) them back into C++ objects in memory
  - This is exactly like json.load() but for neural network models

Protocol Buffers (Protobuf):
  Google's binary serialization format. Like JSON but:
  - Binary (not human-readable) -> much smaller and faster to parse
  - Schema-based (has a .proto file defining the structure)
  - The .onnx model file is a protobuf-encoded file

  Python analogy: think pickle (Python's binary serialization)
    pickle.dump(model, file)   # serialize to file
    model = pickle.load(file)  # deserialize from file
  Protobuf is like a faster, cross-language version of pickle.

  +-- DIAGRAM: Serialization / Deserialization ------------------------------+
  |                                                                          |
  |  IN MEMORY (C++ objects)              ON DISK (binary file)              |
  |  +-----------------------+              +-----------------------+          |
  |  |  Graph {             |   serialize  |  model.onnx          |          |
  |  |    nodes: [Conv,     | ------------>|  (protobuf binary)   |          |
  |  |            ReLU,     |              |  compact, portable   |          |
  |  |            MatMul]   | <------------|  ~14 GB for 7B model|          |
  |  |    weights: [...]    |  deserialize |                      |          |
  |  |    shapes: [...]     |              `-----------------------+          |
  |  |  }                   |                                                |
  |  `-----------------------+                                                |
  |                                                                          |
  |  Python analogy:                                                         |
  |  pickle.dump(model, f)  ==  serialize   (object -> bytes)                |
  |  pickle.load(f)         ==  deserialize (bytes -> object)                |
  |  protobuf               ==  like pickle but faster + cross-language      |
  `-------------------------------------------------------------------------+
```

## 0.9 What is a Graph in Computer Science?

Not a bar chart or line chart! In computer science, a "graph" is a data structure made of NODES connected by EDGES.

```
  Node (also called "vertex") = a thing
  Edge = a connection between two things

  Python analogy:
    # A simple graph as a Python dictionary
    graph = {
        "Conv":     ["BatchNorm"],     # Conv connects to BatchNorm
        "BatchNorm": ["ReLU"],          # BatchNorm connects to ReLU
        "ReLU":     ["MatMul"],         # ReLU connects to MatMul
        "MatMul":   ["Output"],         # MatMul connects to Output
    }

  In a neural network:
    - Each NODE is an operation (Conv, ReLU, MatMul, etc.)
    - Each EDGE is data flowing from one operation to the next
    - The graph describes the entire computation flow of your model
```

DAG (Directed Acyclic Graph):
A graph where edges have direction (A -> B, not A <-> B) and there are no cycles (you can't follow edges and end up back where you started). Neural networks are DAGs - data flows forward through operations.

```
  "Directed" = arrows go one way (input -> conv -> relu -> output)
  "Acyclic" = no loops (data doesn't flow in circles)
```

In-Memory Graph:
When ORT "builds an in-memory graph," it creates a data structure in your computer's RAM that represents all the operations and their connections. It is like building that Python dictionary above, but using C++ objects with much more detail (shapes, types, weights).

```
  +-- DIAGRAM: Neural Network as a DAG -------------------------------------+
  |                                                                          |
  |  Input Image (1, 3, 224, 224)                                            |
  |       |                                                                  |
  |       v                                                                  |
  |  +-----------+     +-----------+                                         |
  |  |  Conv    |---->| BN+ReLU |-- branch A                                |
  |  `------------+     `-------+------+                                     |
  |                        |                                                 |
  |           +-------------+-------------+                                  |
  |           v            v             v                                   |
  |     +-----------+  +-----------+  +-----------+                          |
  |     | Conv 3x3|  | Conv 1x1|  | MaxPool  |    <- 3 parallel branches  |
  |     `-------+------+  `-------+------+  `-------+------+                |
  |          |             |            |                                    |
  |          `---------------+-------------+                                 |
  |                        v                                                 |
  |                  +-------------+                                         |
  |                  |  Concat   |  <- merge point                           |
  |                  `--------+-------+                                      |
  |                        v                                                 |
  |                  +-------------+                                         |
  |                  |  MatMul   |                                           |
  |                  `--------+-------+                                      |
  |                        v                                                 |
  |                     Output                                               |
  |                                                                          |
  |  Directed: arrows go one way (input -> output)                           |
  |  Acyclic:  no loops (data never flows backwards)                         |
  |  = DAG (Directed Acyclic Graph)                                          |
  `--------------------------------------------------------------------------+
```

## 0.10 What is an API?

API = Application Programming Interface

An API is a set of functions that a library provides for you to call. You already use APIs every day in Python:

```
  # The "os" module's API includes:
  os.listdir(".")       # function provided by the os API
  os.path.exists("f")   # another function in the API
```

```
  # NumPy's API includes:
  np.array([1,2,3])     # creating arrays
  np.dot(a, b)          # matrix multiply

In this textbook:
  - HIP API: functions like hipMalloc(), hipLaunchKernel()
    (These are the buttons you push to control the GPU)
  - HSA API: lower-level functions like hsa_queue_create()
    (These are the internal mechanisms inside those buttons)
  - Python ORT API: ort.InferenceSession(), session.run()
    (The simple Python functions you actually call)

  Each "layer" in our stack has its own API. The layer above calls
  the API of the layer below it. Like a chain of function calls.

  +-- DIAGRAM: API Layers -- Who Calls Whom ----------------------------------+
  |                                                                           |
  |  YOUR CODE   session.run(input)                                          |
  |       |       v calls Python ORT API                                     |
  |  +-----v-------------------------------------------------------------+  |
  |  |  ORT API:  InferenceSession(), run(), get_providers()            |  |
  |  `-------+-------------------------------------------------------------+  |
  |       | calls EP API                                                      |
  |  +-----v-------------------------------------------------------------+  |
  |  |  MIGraphX API: parse_onnx_buffer(), compile(), run()             |  |
  |  `-------+-------------------------------------------------------------+  |
  |       | calls HIP API                                                     |
  |  +-----v-------------------------------------------------------------+  |
  |  |  HIP API: hipMalloc(), hipLaunchKernel(), hipMemcpy()            |  |
  |  `-------+-------------------------------------------------------------+  |
  |       | calls HSA API                                                     |
  |  +-----v-------------------------------------------------------------+  |
  |  |  HSA API: hsa_queue_create(), hsa_signal_store() (doorbell)      |  |
  |  `-------+-------------------------------------------------------------+  |
  |       | talks to hardware                                                 |
  |  +-----v-------------------------------------------------------------+  |
  |  |  GPU HARDWARE: reads AQL packets, executes kernels                |  |
  |  `-----------------------------------------------------------------+  |
  |                                                                           |
  |  Each layer only knows the API below it -- never skips layers.           |
  `---------------------------------------------------------------------------+


0.11 What is a Thread? What is Parallelism?
-------------------------------------------

A thread is a sequence of instructions that a processor executes.

In Python:
  # Single-threaded (one thing at a time)
  result1 = compute_a()  # wait for this to finish
  result2 = compute_b()  # then do this

  # Multi-threaded (two things at once)
  import threading
  t1 = threading.Thread(target=compute_a)
  t2 = threading.Thread(target=compute_b)
  t1.start()  # starts running
  t2.start()  # starts running AT THE SAME TIME
  t1.join()   # wait for both
  t2.join()

On a GPU, you don't have 2 threads - you have THOUSANDS:
  - A GPU "kernel launch" might create 262,144 threads
  - Each thread does a tiny piece of work (like one addition)
  - They all run simultaneously across the GPU's cores

Wavefront (AMD) / Warp (NVIDIA):
  GPUs don't execute one thread at a time - they execute groups:
  - AMD RDNA (your GPU): groups of 32 threads (Wave32)
  - AMD CDNA (MI210):    groups of 64 threads (Wave64)
  - NVIDIA:              groups of 32 threads (Warp)

  All threads in a wavefront execute the SAME instruction at the
  same time, but on different data. This is called SIMD:

SIMD (Single Instruction, Multiple Data):
  One instruction, applied to 32 (or 64) pieces of data simultaneously.

  Python analogy:
    # Non-SIMD (one at a time)
    for i in range(32):
```

```
        result[i] = a[i] + b[i]

    # SIMD (all 32 at once, in one clock cycle)
    result[0:32] = a[0:32] + b[0:32]  # happens in ONE step on hardware

Asynchronous Execution:
  "Fire and forget" - you tell the GPU to do work, and your CPU
  immediately moves on to the next task without waiting.

  Conceptually similar to (but NOT the same as) Python async/await:
    async def do_work():
        task = asyncio.create_task(gpu_compute())  # starts GPU work
        prepare_next_batch()  # CPU does this WHILE GPU is computing
        result = await task   # NOW we wait for the GPU result

  IMPORTANT DIFFERENCE: Python async/await uses cooperative coroutines
  on ONE thread. GPU async is TRUE hardware parallelism -- the CPU and
  GPU are separate processors running simultaneously. The GPU has its
  own hardware command queue that it processes independently.

  +-- DIAGRAM: Sequential vs Parallel Execution -----------------------------+
  |                                                                          |
  |  Task: Process 4 images (each takes 10ms)                               |
  |                                                                          |
  |  1 CPU Thread (sequential):                              Total: 40ms   |
  |  +---img1---++---img2---++---img3---++---img4---+                            |
  |  0        10        20        30        40 ms                           |
  |                                                                          |
  |  4 CPU Threads (parallel):                               Total: 10ms   |
  |  Thread 0: +---img1---+                                                  |
  |  Thread 1: +---img2---+                                                  |
  |  Thread 2: +---img3---+                                                  |
  |  Thread 3: +---img4---+                                                  |
  |  0         10 ms                                                         |
  |                                                                          |
  |  GPU (10,000 threads, one kernel):                       Total: ~0.1ms |
  |  +--- all pixels of all 4 images processed simultaneously ---+           |
  |  0                                                        0.1 ms       |
  |                                                                          |
  |  CPU+GPU async (overlap):                                                |
  |  CPU: [prepare batch 1][prepare batch 2][prepare batch 3] ...           |
  |  GPU:     [--compute batch 1--][--compute batch 2--] ...                |
  |       ^ CPU and GPU work simultaneously on different batches            |
  `--------------------------------------------------------------------------+


0.12 What is a Cache?
----------------------

A cache is a small, very fast memory that stores recently-used data.

Real-world analogy:
  You're writing a research paper.
  - Your desk (cache): holds the 5 books you're actively using. FAST access.
  - Your bookshelf (RAM): holds 100 books. Takes 10 seconds to walk over.
  - The library (disk): holds 1 million books. Takes 30 minutes to go there.

  When you need a book:
    1. Check your desk (cache) -> found? Great, instant access!  (cache HIT)
    2. Not on desk? Check bookshelf (RAM) -> found? Walk over.   (cache MISS)
    3. Not there? Go to library (disk) -> very slow.

On a GPU, there are multiple cache levels:
  L0 Cache (smallest, fastest) -> ~32 KB per Compute Unit
  L1 Cache (bigger, still fast)
  L2 Cache (on-chip, shared)   -> 2 MB shared across all CUs on your GPU
  Infinity Cache / MALL (L3)   -> 32 MB memory-side last-level cache, ~1 TB/s
  Main Memory (off-chip, slow) -> LPDDR5x ~256 GB/s on your GPU

  Cache Hit = data was in the cache (fast!)
  Cache Miss = data was NOT in the cache (slow - must fetch from main memory)

  For AI: if your model weights fit in the 32 MB Infinity Cache, inference
  is FAST. If they don't fit, every access goes to slower LPDDR5x memory.

  +-- DIAGRAM: Cache Hit / Miss Flow ----------------------------------------+
  |                                                                          |
  |  GPU Compute Unit requests data at address 0xABCD                      |
  |       |                                                                  |
  |       v                                                                  |
  |  +------------+  HIT (data found)                                       |
  |  | L1 Cache  |------------------> Return data (~1 cycle, ~0.5 ns)     |
  |  |  32 KB    |                                                          |
```

```
  |  `-------+-------+  MISS                                                    |
  |       |                                                                  |
  |       v                                                                  |
  |  +------------+  HIT                                                     |
  |  | L2 Cache  |------------------> Return data (~50 cycles, ~25 ns)    |
  |  |   2 MB    |                                                          |
  |  `-------+-------+  MISS                                                    |
  |       |                                                                  |
  |       v                                                                  |
  |  +------------+  HIT                                                     |
  |  | Infinity  |------------------> Return data (~100 cycles, ~50 ns)   |
  |  |Cache 32MB |                                                          |
  |  `-------+-------+  MISS                                                    |
  |       |                                                                  |
  |       v                                                                  |
  |  +------------+                                                          |
  |  | LPDDR5x  |------------------> Return data (~200+ cycles, ~100 ns) |
  |  |  30 GB   |  SLOW! GPU stalls waiting for data                      |
  |  `-------------+                                                          |
  |                                                                          |
  |  Each level: ~2-4x slower than the one above it                         |
  `--------------------------------------------------------------------------+
```

## 0.13 What is a Pointer?

A pointer is a variable that stores a memory address (where some data lives in memory), rather than the data itself.

Python hides pointers from you, but they exist underneath:

```
  x = [1, 2, 3]
  # "x" is actually a pointer (reference) to a list object in memory
  # The actual list data lives at some memory address like 0x7f1234000000
```

In C/C++ (which this stack is written in), pointers are explicit:

```
  int* ptr = malloc(sizeof(int));  // allocate memory, get a pointer
  *ptr = 42;                       // write to that memory location
  free(ptr);                       // release the memory
```

Why this matters:

When we say "hipMalloc returns a device pointer," it means:

- hipMalloc gives you an ADDRESS on the GPU's memory
- That address is just a number (like 0x7f1234000000)
- The GPU can use that address to read/write data
- The CPU CANNOT use that address (it is on the GPU's side)

This is why you need hipMemcpy() - to copy data between CPU memory addresses and GPU memory addresses.

```
  +-- DIAGRAM: CPU Pointer vs GPU Pointer -----------------------------------+
  |                                                                          |
  |  CPU Side (host)                    GPU Side (device)                    |
  |  +--------------------+              +--------------------+                |
  |  | host_ptr          |              | dev_ptr           |                |
  |  | = 0x7f00_0000     |              | = 0x7f80_0000     |                |
  |  |     |              |              |     |              |                |
  |  |     v              |              |     v              |                |
  |  | +--------------+    |              | +--------------+    |                |
  |  | | [1.0, 2.0, |    |              | | [1.0, 2.0, |    |                |
  |  | |  3.0, 4.0] |    |              | |  3.0, 4.0] |    |                |
  |  | `---------------+    |              | `---------------+    |                |
  |  |   LPDDR5x RAM     |              |  VRAM (or GTT)    |                |
  |  `---------------------+              `---------------------+                |
  |                                                                          |
  |           [N] CPU cannot dereference dev_ptr (CRASH!)                    |
  |           [N] GPU cannot dereference host_ptr (CRASH!)                   |
  |           [Y] hipMemcpy(dev, host, N, HostToDevice) bridges them         |
  |                                                                          |
  |  Exception: hipMallocManaged() returns a SINGLE pointer that            |
  |  both CPU and GPU can use (unified memory, via page faults/XNACK).    |
  `--------------------------------------------------------------------------+
```

## 0.14 What is a Driver?

A driver is software that tells the operating system how to talk to a specific piece of hardware.

Without a driver, your OS has no idea what your GPU is or how to use it.

Python analogy:

```
    # Without a database driver, Python can't talk to PostgreSQL
    import psycopg2  # this is the "driver" for PostgreSQL
    conn = psycopg2.connect(...)  # now Python can talk to the database

  Similarly:
    # Without amdgpu.ko (the GPU driver), Linux can't talk to your GPU
    # The driver is loaded automatically at boot time

  The AMD GPU driver (amdgpu.ko) runs inside the Linux kernel (the OS).
  It has special privileges - it can directly access hardware registers,
  allocate physical memory, and handle hardware interrupts.
  Normal programs (including MIGraphX) CANNOT do these things directly.

  +-- DIAGRAM: Driver as Gatekeeper ----------------------------------------+
  |                                                                          |
  |  +------ User Space ----------------------------------------------+     |
  |  |                                                                |     |
  |  |  Python App -> ORT -> MIGraphX -> HIP (libamdhip64.so)          |     |
  |  |                                                                |     |
  |  |  These programs CANNOT:                                        |     |
  |  |    [N] Touch GPU hardware registers directly                    |     |
  |  |    [N] Allocate physical memory pages                           |     |
  |  |    [N] Handle hardware interrupts                               |     |
  |  `--------------------------+-----------------------------------------+     |
  |                             | ioctl() system calls                         |
  |  ========================-+-========================== (privilege wall)   |
  |                             |                                              |
  |  +------ Kernel Space ----v-------------------------------------+       |
  |  |                                                              |       |
  |  |  amdgpu.ko (kernel driver)                                   |       |
  |  |                                                              |       |
  |  |  This code CAN:                                              |       |
  |  |    [Y] Write to GPU MMIO registers                            |       |
  |  |    [Y] Allocate physical memory (VRAM/GTT pages)              |       |
  |  |    [Y] Set up page tables (GPUVM)                             |       |
  |  |    [Y] Handle GPU interrupts (completion signals)             |       |
  |  `-------------------------+-----------------------------------------+       |
  |                            | PCIe / Fabric / MMIO                         |
  |  +------ GPU Hardware ---v-------------------------------------+       |
  |  |  Command Processor, CUs, Memory Controllers, etc.            |       |
  |  `-----------------------------------------------------------------+       |
  `-------------------------------------------------------------------------------+


0.15 What is Virtual Memory and Page Tables?
---------------------------------------------

When you allocate memory in Python (x = [1,2,3]), you get an address.
But that address is NOT a real physical location in your RAM chips.
It is a "virtual address" - a fake address that the OS translates
to a real physical address using a data structure called a page table.

Why?
  - Multiple programs can each think they have all the memory
  - Programs can't accidentally (or maliciously) access each other's data
  - The OS can move data around in physical memory without programs knowing

  Virtual address:  0x7f1234000000  (what your program sees)
         |
    Page Table (lookup table maintained by OS/hardware)
         |
         v
  Physical address: 0x000380000000  (actual location on RAM chip)

GPUs have their OWN virtual memory system (called GPUVM):
  - GPU programs see virtual addresses
  - The GPU hardware translates them to physical device memory locations
    (HBM on discrete GPUs, LPDDR5x carve-out on APUs like 8060S)
  - This is managed by the amdgpu kernel driver

Page:
  Memory is divided into fixed-size chunks called "pages" (typically 4 KB).
  The page table maps virtual pages to physical pages.

Page Fault:
  When a program accesses a virtual address that is NOT currently mapped
  to physical memory. The OS must handle this - either by loading the data
  from disk or by signaling an error.

  +-- DIAGRAM: Virtual Memory (CPU and GPU) ---------------------------------+
  |                                                                          |
  |  CPU Process Virtual Address Space     GPU Virtual Address Space         |
  |  +--------------------------+           +--------------------------+     |
```

```
|  | 0x0000 +----------------+ |           | 0x0000 +----------------+ |     |
|  |        |  Code (.text) | |           |        | Kernel code   | |     |
|  |        |------------------| |           |        |------------------| |     |
|  |        |  Heap (malloc)| |           |        | Weight buffers| |     |
|  |        |------------------| |           |        |------------------| |     |
|  |        |  Stack        | |           |        | Workspace     | |     |
|  |        |------------------| |           |        |------------------| |     |
|  |        |  Shared libs  | |           |        | Output buffers| |     |
|  | 0xFFFF `-----------------+ |           | 0xFFFF `-----------------+ |     |
|  `---------------+--------------+           `---------------+--------------+     |
|               |                                  |                  |
|          CPU Page Table                        GPU Page Table          |
|          (managed by OS)                       (managed by amdgpu)    |
|               |                                  |                  |
|               v                                  v                  |
|  +-------------------------------------------------------------+        |
|  |              Physical LPDDR5x Memory (30 GB)               |        |
|  |  [CPU page][GPU page][CPU page][GPU page][GPU page] ...   |        |
|  `-------------------------------------------------------------+        |
|                                                                        |
|  Page Fault Flow:                                                      |
|  GPU accesses VA 0x1234 -> TLB miss -> page table walk ->               |
|    page not mapped -> PAGE FAULT interrupt -> driver allocates          |
|    physical page -> updates page table -> GPU retries -> success         |
`-------------------------------------------------------------------------+
```

## 0.16 What is Floating Point? (FP32, FP16, INT8)

Computers store numbers in binary (0s and 1s). "Floating point" is the standard way computers represent decimal numbers (like 3.14159).

```
  In Python:
    x = 3.14  # this is stored as a 64-bit floating point number by default

  The number of bits determines precision and range:

  FP32 (32-bit float):
    - Uses 32 bits (4 bytes) per number
    - Precision: ~7 decimal digits
    - Range: +/-3.4 x 10^38
    - This is Python's "float" by default (actually Python uses FP64)
    - Standard training precision for AI models

  FP16 (16-bit float, "half precision"):
    - Uses 16 bits (2 bytes) per number -- HALF the memory of FP32
    - Precision: ~3 decimal digits
    - Range: +/-65,504
    - Good enough for most AI inference
    - GPUs can process FP16 2-8x faster than FP32

  INT8 (8-bit integer):
    - Uses 8 bits (1 byte) per number -- QUARTER the memory of FP32
    - Range: -128 to 127 (no decimal places)
    - Used in "quantized" models for maximum speed

  Why smaller is better for AI inference:
    A model with 7 billion parameters:
      FP32: 7B x 4 bytes = 28 GB  (might not fit in memory!)
      FP16: 7B x 2 bytes = 14 GB  (fits, and GPU processes 2x faster)
      INT4: 7B x 0.5 bytes = 3.5 GB  (easily fits, very fast)

  TFLOPS (Tera Floating-Point Operations Per Second):
    How many trillion math operations a GPU can do per second.
    Your GPU: ~18 TFLOPS (FP16, estimated with 40 CUs + WMMA)
    MI210: ~181 TFLOPS (FP16) -- roughly 10x faster

  +-- DIAGRAM: Floating Point Bit Layouts -----------------------------------+
  |                                                                          |
  |  FP32 (32 bits = 4 bytes):                                              |
  |  +---+------------+-----------------------------+                        |
  |  |S| Exponent |        Mantissa            |                            |
  |  |1|  8 bits  |       23 bits              |                            |
  |  `----+------------+-----------------------------+                        |
  |  Precision: ~7 decimal digits    Range: +/-3.4 x 10^38                   |
  |                                                                          |
  |  FP16 (16 bits = 2 bytes):                                              |
  |  +---+---------+----------------+                                        |
  |  |S| Exp   |   Mantissa   |                                             |
  |  |1| 5 bits|   10 bits    |                                             |
  |  `----+---------+----------------+                                        |
  |  Precision: ~3 decimal digits    Range: +/-65,504                        |
```

```
  |  HALF the memory, 2-8x faster on GPU                                  |
  |                                                                        |
  |  INT8 (8 bits = 1 byte):                                              |
  |  +------------------+                                                  |
  |  |   8 bits total  |  Range: -128 to 127 (no decimals)               |
  |  `------------------+                                                  |
  |                                                                        |
  |  Model Size Impact (7 billion parameters):                            |
  |  FP32: ########################################  28 GB               |
  |  FP16: ####################                      14 GB               |
  |  INT8: ##########                                 7 GB               |
  |  INT4: #####                                    3.5 GB               |
  `------------------------------------------------------------------------+
```

## 0.17 What is PCIe?

PCIe = Peripheral Component Interconnect Express

It is the physical connection (slot/cable on the motherboard) between your CPU and add-on cards like GPUs, SSDs, and network cards.

Think: a highway between two cities (CPU-city and GPU-city)

PCIe Gen4 x16 (common today):
Speed: ~64 GB/s total (32 GB/s each direction)

For AI: when data moves between CPU RAM and GPU VRAM, it travels over the PCIe bus. This is MUCH slower than GPU's internal memory:
PCIe: ~64 GB/s
MI210 HBM: ~1,638 GB/s (25x faster!)

This is why copying data to/from the GPU is expensive.

YOUR APU is special: since CPU and GPU are on the same chip, there is NO PCIe between them. They share the same memory bus. This eliminates the copy bottleneck (but the shared bus is slower than dedicated HBM).

```
  +-- DIAGRAM: Discrete GPU vs APU Topology ---------------------------------+
  |                                                                          |
  |  DISCRETE GPU (e.g., MI210, RX 7900):                                   |
  |  +-------------+    PCIe Gen4 x16     +----------------+                |
  |  |    CPU     |<==================>|     GPU       |                  |
  |  |            |   ~32 GB/s each dir  |              |                  |
  |  `--------+-------+                       `---------+---------+                  |
  |        |                                     |                          |
  |   +------+------+                            +------+-------+                    |
  |   |  DDR5   |                          |   HBM    |                    |
  |   |  RAM    | separate memory          |  VRAM    |                    |
  |   | ~64GB/s | spaces, must copy        | ~1638GB/s|                    |
  |   `------------+                          `-------------+                     |
  |                                                                          |
  |  YOUR APU (Strix Halo):                                                 |
  |  +-------------------------------------+                                 |
  |  |         Single Die (APU)           |                                 |
  |  |  +--------+ <- on-die bus -> +--------+|                                |
  |  |  | CPU  |  (no PCIe!)     | GPU  ||                                 |
  |  |  `-----+-----+                 `-----+-----+|                                |
  |  |     `------------+----------------+    |                                |
  |  `-----------------+--------------------+                                 |
  |                    |  shared memory controller                           |
  |            +--------+--------+                                             |
  |            |   LPDDR5x   |  ~256 GB/s, shared between CPU and GPU     |
  |            |   30 GB     |  NO copy needed! Same physical memory.      |
  |            `----------------+                                             |
  `------------------------------------------------------------------------+
```

## 0.18 What is DMA?

DMA = Direct Memory Access

A hardware feature that lets devices copy data between memory regions WITHOUT using the CPU. The CPU just says "copy this block," and a dedicated DMA engine does the work while the CPU does something else.

Without DMA:
CPU copies byte-by-byte from RAM to GPU memory (CPU is busy the whole time)

With DMA:

```
    CPU says "copy these 14 GB" -> DMA engine copies -> CPU goes to do other work

  On AMD GPUs, the DMA engine is called SDMA (System DMA).
  MIGraphX uses SDMA to copy model weights to the GPU while the
  CPU prepares other things - this overlap is called "pipelining."

  +-- DIAGRAM: DMA (Direct Memory Access) -----------------------------------+
  |                                                                          |
  |  WITHOUT DMA (CPU copies):                                              |
  |  CPU: [copy byte 1][copy byte 2][copy byte 3]...[copy byte N] BUSY!    |
  |  GPU: [idle....................................][can start work]      |
  |       CPU is stuck copying, can't do anything else                      |
  |                                                                          |
  |  WITH DMA (SDMA engine copies):                                         |
  |  CPU: [tell SDMA: copy 14GB][prepare next batch][optimize][ready!]     |
  |  SDMA:[--------- copying 14 GB in background ------------][done]       |
  |  GPU: [idle, waiting for data.......................][start compute]    |
  |       CPU is FREE to do other work while SDMA copies                    |
  |                                                                          |
  |  +--------+  "copy 14GB"   +---------+  DMA transfer   +------------+      |
  |  | CPU  |--------------->| SDMA  |================>| GPU VRAM |      |
  |  |      | (one command)  |engine | (hardware does   | / GTT    |      |
  |  |      | then CPU is    |       |  the heavy lift) |          |      |
  |  |      | free           `----------+                  `-------------+      |
  |  `---------+                                                            |
  `-----------------------------------------------------------------------------+


0.19 What is an Interrupt?
---------------------------

An interrupt is a hardware signal that tells the CPU "stop what you're
doing, something needs attention."

  Real-world analogy:
    You're reading a book (CPU doing work).
    Your phone rings (interrupt!).
    You stop reading, answer the phone (handle the interrupt).
    Hang up, go back to reading (resume previous work).

  In our GPU stack:
    The GPU finishes a computation and fires an interrupt.
    The CPU stops what it was doing, runs the interrupt handler
    (a special function in the amdgpu driver), which updates a
    signal to say "GPU is done." Then the CPU goes back to whatever
    it was doing.

  MSI-X (Message Signaled Interrupts):
    A modern way for hardware to send interrupts to the CPU.
    Instead of a physical wire, the device writes a small message
    to a special memory location, which triggers the CPU interrupt.

  +-- DIAGRAM: GPU Interrupt Flow (Kernel Completion) -----------------------+
  |                                                                          |
  |  TIME ------------------------------------------------------------>  |
  |                                                                          |
  |  CPU:  [running Python code...] [ISR handler] [resume Python]           |
  |                     ^                  |                                 |
  |                     | interrupt!        | update hsa_signal              |
  |                     |                  v                                 |
  |  GPU:  [=== executing kernel ===]  DONE!                                |
  |                                   |                                      |
  |                                   |---- writes to MSI-X address          |
  |                                   |   (special memory location)          |
  |                                   `---- CPU interrupt triggered          |
  |                                                                          |
  |  Detail:                                                                 |
  |  +-----------+  "I'm done"  +------------+  IRQ   +-------------+          |
  |  | GPU CU  |------------->| Interrupt |------>|  amdgpu   |          |
  |  | finishes|  MSI-X msg   | Controller|       |  driver    |          |
  |  | kernel  |              | (IOMMU)   |       |  ISR()     |          |
  |  `------------+              `-------------+       `--------+--------+          |
  |                                                  |                |
  |                           +---------------------------v-----------+    |
  |                           | hsa_signal value: 1 -> 0 ("done!")   |    |
  |                           | waiting thread wakes up              |    |
  |                           `----------------------------------------+    |
  `-----------------------------------------------------------------------------+


0.20 What is MMIO?
-------------------
```

```
MMIO = Memory-Mapped I/O

A technique where hardware registers (tiny control switches on the GPU)
are mapped to memory addresses. The CPU can "talk to" the GPU by simply
reading from or writing to specific memory addresses.

  Normal memory write:
    Write to address 0x1234 -> stores data in RAM

  MMIO write:
    Write to address 0xFFFF8800 -> the GPU hardware REACTS to this write
    (for example: the GPU's Command Processor wakes up and starts work)

  This is how kernel launches are so fast - the CPU just writes to
  a "doorbell" address (an MMIO register), and the GPU hardware
  immediately notices and starts processing.

  No system call, no OS involvement, no context switch - just a
  memory write. That is why it takes only ~5 microseconds.

  +-- DIAGRAM: MMIO -- Doorbell Write (Kernel Launch) -----------------------+
  |                                                                          |
  |  REGULAR RAM Write:                                                      |
  |  CPU writes 0x42 to addr 0x0010_0000  ->  stored in LPDDR5x RAM         |
  |                                              (nothing special happens)   |
  |                                                                          |
  |  MMIO Write (doorbell):                                                  |
  |  CPU writes to addr 0xFFFF_8800      ->  GPU hardware REACTS!            |
  |                                                                          |
  |  +-------------+     write to doorbell addr      +-------------------+    |
  |  |  CPU core | ------------------------------>| GPU Command     |    |
  |  |  (no      |     (looks like a memory write   | Processor (CP)  |    |
  |  |  syscall!)|      but hardware intercepts it) | wakes up!       |    |
  |  `--------------+                                  `-----------+----------+    |
  |                                                          |             |
  |                                          +------------------v-----------+ |
  |                                          | CP reads AQL packet from   | |
  |                                          | queue -> dispatches to CUs  | |
  |                                          `-------------------------------+ |
  |                                                                          |
  |  Cost comparison:                                                        |
  |    ioctl (syscall):   ~1,000-5,000 ns  (crosses kernel boundary)        |
  |    MMIO doorbell:          ~40 ns      (direct memory write)             |
  |    -> 25-125x faster! That's why launches are cheap.                     |
  `--------------------------------------------------------------------------+


0.21 What is ioctl?
--------------------

ioctl = Input/Output Control

A system call (a way for programs to ask the OS to do something) used to
communicate with device drivers. It is the primary way userspace programs
(like HIP runtime) ask the kernel driver (amdgpu.ko) to do things that
require special privileges:

  "Allocate 14 GB of GPU memory" -> ioctl(fd, AMDGPU_IOC_ALLOC_MEMORY, ...)
  "Set up a GPU command queue"   -> ioctl(fd, AMDGPU_IOC_CREATE_QUEUE, ...)

  Python analogy:
    # When you open a file, Python uses a system call internally
    f = open("data.txt")  # internally calls: open() system call

    # ioctl is similar - a system call for device-specific operations
    # It is more specialized than open/read/write

  ioctl is SLOW (~1-5 mus) because it crosses the user/kernel boundary
  (a "context switch"). This is why it is only used for rare operations
  (memory allocation, setup) and NOT for kernel launches.

  +-- DIAGRAM: ioctl Path (Userspace to Hardware) ---------------------------+
  |                                                                          |
  |  USERSPACE (your application, MIGraphX, HIP)                             |
  |  +-------------------------------------------------------+               |
  |  |  hip_result = hipMalloc(&ptr, 14 * GB);              |               |
  |  |    `---> internally calls ioctl(gpu_fd, ALLOC, ...)    |               |
  |  `---------------------------+------------------------------+               |
  |                              |  ioctl() system call                      |
  |  =========================-+-========================== (user/kernel    |
  |                              |                            boundary)      |
  |  KERNEL SPACE (Linux kernel + amdgpu.ko)                                 |
  |  +------------------------+-----------------------------+               |
```

```
  |  |                        v                              |              |
  |  |  amdgpu_gem_create()                                  |              |
  |  |    |---> TTM: find free pages in GTT pool               |              |
  |  |    |---> Update GPU page tables                         |              |
  |  |    `---> Return virtual address to userspace            |              |
  |  `---------------------------+------------------------------+              |
  |                              |  MMIO register writes                      |
  |                              v                                            |
  |  +-------------------------------------------------------+              |
  |  |  GPU HARDWARE (page table registers updated)          |              |
  |  `-------------------------------------------------------+              |
  |                                                                          |
  |  SLOW PATH (ioctl): ~1-5 mus -- used for setup (alloc, queue create)   |
  |  FAST PATH (MMIO doorbell): ~40 ns -- used for kernel launch           |
  `--------------------------------------------------------------------------+
```

## 0.22 What are C++ Objects? (Coming from Python)

Most of this inference stack is written in C++. If you only know Python, here is a quick mental model:

```
  Python class -> C++ class (very similar concept)
  Python object -> C++ object (instance of a class)

  Python:
    class Node:
        def __init__(self, op_type, inputs, outputs):
            self.op_type = op_type
            self.inputs = inputs
            self.outputs = outputs

  C++ (roughly):
    class Node {
        string op_type;
        vector<string> inputs;
        vector<string> outputs;
    };

  When we say "ORT builds a C++ object for each node," it means ORT
  creates a Node instance in memory for each operation in your model,
  just like you would in Python.

  The key difference: C++ objects live at specific memory addresses,
  and you access them via pointers. Python hides all of this from you.

  +-- DIAGRAM: Python Object vs C++ Object (Memory Layout) -----------------+
  |                                                                          |
  |  PYTHON (hidden complexity):              C++ (explicit layout):       |
  |                                                                          |
  |  node = Node("Conv", ["x"], ["y"])        Node* node = new Node();    |
  |                                            node->op_type = "Conv";     |
  |  What's really happening:                                              |
  |  +---- Python object header ----+           +--- C++ object (no header)---+|
  |  | refcount: 1                |           | op_type: "Conv"     8B   ||
  |  | type_ptr: -> Node class     |           | inputs:  ["x"]     24B  ||
  |  | __dict__: -> {              |           | outputs: ["y"]     24B  ||
  |  |   "op_type": "Conv"        |           `----------------------------+|
  |  |   "inputs": ["x"]          |           Total: ~56 bytes, contiguous|
  |  |   "outputs": ["y"]         |                                       |
  |  | }                          |           No GC overhead, no refcount |
  |  `-----------------------------+           Destructor runs at scope end|
  |  Total: ~200+ bytes, scattered                                          |
  |  GC must track every object!              RAII: new -> use -> delete    |
  |                                                                          |
  |  For 10,000 nodes (typical model):                                      |
  |    Python: ~2 MB overhead    |    C++: ~0.5 MB (4x smaller)           |
  `--------------------------------------------------------------------------+
```

## 0.23 Quick Reference: Units and Abbreviations

```
Size Units:
  1 KB (kilobyte)   = 1,024 bytes
  1 MB (megabyte)   = 1,024 KB = ~1 million bytes
  1 GB (gigabyte)   = 1,024 MB = ~1 billion bytes
  1 TB (terabyte)   = 1,024 GB = ~1 trillion bytes

Speed Units:
  1 GB/s            = 1 gigabyte per second (data transfer speed)
  1 TFLOPS          = 1 trillion floating-point operations per second
```

```
  1 GHz            = 1 billion clock cycles per second (processor speed)

Time Units:
  1 s   (second)    = 1,000 ms
  1 ms  (milli)     = 1,000 mus      (visible delay)
  1 mus  (micro)     = 1,000 ns      (imperceptible)
  1 ns  (nano)      = light travels 30 cm

Common Abbreviations in This Book:
  ORT     = ONNX Runtime (the framework)
  EP      = Execution Provider (backend that runs operations)
  HBM     = High Bandwidth Memory (fast GPU memory)
  LPDDR5x = Type of system RAM (Low-Power DDR5, extended speed)
  CU      = Compute Unit (processing block on AMD GPU)
  WGP     = Work Group Processor (contains 2 CUs)
  LDS     = Local Data Share (fast shared memory per CU)
  ISA     = Instruction Set Architecture (GPU's "language")
  IR      = Intermediate Representation (partially compiled code)
  AQL     = Architected Queuing Language (GPU command packet format)
  TTM     = Translation Table Manager (GPU memory manager)
  BO      = Buffer Object (a chunk of allocated GPU memory)
  SDMA    = System DMA (hardware data copy engine)
  GPUVM   = GPU Virtual Memory (GPU's page table system)
  MMIO    = Memory-Mapped I/O (controlling hardware via memory writes)
  PCIe    = PCI Express (bus connecting CPU to GPU)
  APU     = Accelerated Processing Unit (CPU + GPU on one chip)

  +-- DIAGRAM: Units -- Scale Ladder ----------------------------------------+
  |                                                                          |
  |  SIZE:     1 B -> 1 KB -> 1 MB -> 1 GB -> 1 TB                            |
  |            |      |       |       |       |                              |
  |           char   L1$    weight   model   dataset                        |
  |                         tensor   file    (ImageNet)                     |
  |                                                                          |
  |  BANDWIDTH:  1 MB/s -> 1 GB/s -> 100 GB/s -> 1 TB/s -> 5 TB/s            |
  |              |         |         |          |         |                  |
  |             USB2     PCIe1    LPDDR5x     HBM2e    HBM3                 |
  |                               8060S      MI210    MI300X                 |
  |                                                                          |
  |  TIME:  1 s -> 1 ms -> 1 mus -> 1 ns -> 1 ps                             |
  |          |      |       |       |      |                                |
  |        human  model   ioctl   MMIO   light                              |
  |        blink  infer   call   write   1 mm                               |
  |                                                                          |
  |  COMPUTE:  1 GFLOPS -> 1 TFLOPS -> 18 TFLOPS -> 181 TFLOPS               |
  |            |           |           |            |                       |
  |           old CPU    modern CPU   8060S FP16   MI210 FP16               |
  `--------------------------------------------------------------------------+
```

## 0.24 How to Read C++ Source Code (for Python Developers)

Appendix D contains line-by-line walkthroughs of real ROCm source code. This section teaches you just enough C++ syntax to read those walkthroughs. You do NOT need to write C++ -- only read it.

NOTE: This is a condensed primer covering the patterns that appear in ROCm source. For a deeper beginner tutorial (= vs ==, stack vs heap, new/delete, loops, for/while, constructors/destructors, inheritance, #include, casting, typedef/using, enum, and codebase navigation strategies), see the companion document Layer1-5_CallFlow_SourceGuide.md, Sections 0.0-0.33.

```
COMPANION DOCUMENTS (in the same repository):
  Layer1-5_CallFlow_SourceGuide.md  -- Deep beginner C/C++ tutorial + Layers 1-5 walkthrough
  API_Call_Flow.md                  -- Slide-format API call flow (L1 through L5)
  presentation_diagrams.md          -- 14 Mermaid diagram slides (8-layer stack overview)
  PPT_Mermaid_Slides.md             -- Layers 1-5 Mermaid slides for presentations
  all_slides_LR.md                  -- Complete PPT slide deck (all layers, all topics)
  ML-onnx-student-handbook/         -- 25-module student handbook (exercises + cheatsheets)
  docs/R1_R2_OFFICIAL_REPORT.txt    -- 29-section deployment report (23 models, 4 ORT fixes)
  docs/SETUP_GUIDE.txt              -- Step-by-step setup from fresh Ubuntu 24.04
  profiling/README.txt              -- rocprofv3 profiling methodology and data guide

VARIABLES
  Python:   x = 42              # type inferred
  C++:      int x = 42;         // type is explicit
            size_t size = 1024; // size_t = unsigned integer for sizes
            float val = 3.14f;  // f suffix = single precision
            auto y = x + 1;    // auto = "compiler, figure out the type"

SEMICOLONS AND BRACES
  Every C++ statement ends with ;
  Code blocks use { } (like Python's indentation):
```

```
    Python:   if x > 0:         C++:   if (x > 0) {
                  print(x)                 printf("%d", x);
                                       }

FUNCTIONS
  Python:   def allocate(size):     C++: hipError_t allocate(size_t size) {
                return ptr                   return hipSuccess;
                                         }
  Return type is written BEFORE the function name.
  hipError_t = the return type (a number: 0 = success, other = error)

POINTERS (*) AND REFERENCES (&)
  A POINTER holds a memory address (like a GPS coordinate to data):
    int* ptr;          // ptr is a "pointer to an integer"
    *ptr = 42;         // DEREFERENCE: write 42 to the address ptr points to

  WHY void** IN hipMalloc?
    hipMalloc(void** ptr, size_t size)
    - void*  = "pointer to anything" (the GPU memory address)
    - void** = "pointer to a pointer" -- so hipMalloc can WRITE BACK the address
    - You pass &your_variable (address of your pointer), hipMalloc fills it in

  A REFERENCE (&) is an alias (like two Python names pointing to one object):
    void quantize_fp16(program& prog)
    - program& = "reference to a program" -- same object, not a copy
    - Modifying prog inside the function modifies the ORIGINAL

ARROW (->) AND DOT (.)
  Both access members of an object, like Python's dot:
    Python:   obj.method()           Always dot
    C++:      obj.method()           When obj is the OBJECT itself
              ptr->method()          When ptr is a POINTER to the object
  ptr->method() is shorthand for (*ptr).method()

const
  "This value will NOT be modified":
    const void* data       // "I promise not to modify the data"
    const target& t        // "read-only reference to the target"
  Helps catch bugs -- if you accidentally try to modify a const, compiler errors.

NAMESPACES (::)
  C++ organizes code into namespaces (like Python packages):
    migraphx::parse_onnx_buffer()    // parse_onnx_buffer in the migraphx namespace
    hip::getCurrentDevice()          // getCurrentDevice in the hip namespace
    amd::SvmBuffer::malloc()         // malloc in the SvmBuffer class, amd namespace
  "::" means "belongs to" -- read right to left:
    program::compile = "compile that belongs to program"

TEMPLATES (<...>)
  Like Python generics / type hints:
    Python:   list[str]              C++: std::vector<std::string>
    std::vector<pass>                // a list of pass objects
    allocate<migraphx_program_t>()   // allocate, specialized for migraphx_program_t
  The angle brackets < > specify the type parameter.

CLASSES AND STRUCTS
  Both are containers for data + methods:
    struct migraphx_program { program object; };
    class program { public: void compile(const target& t); };
  struct = all members public by default
  class  = all members private by default
  In practice, ROCm uses both interchangeably.

extern "C"
  "Make this function callable from C code (and therefore from any language)":
    extern "C" migraphx_status migraphx_parse_onnx_buffer(...)
  Without extern "C", C++ "name mangles" function names (adds type info to
  the symbol name), making them impossible to call from C, Python, etc.
  The MIGraphX C API uses extern "C" on every public function.

PREPROCESSOR: #ifdef, #if, #define
  Lines starting with # are preprocessor directives (processed before compilation):
    #define MIGRAPHX_USE_MIOPEN 1     // define a constant
    #if MIGRAPHX_USE_MIOPEN           // conditional compilation
        compile_miopen{&gctx},        // only included if MIOpen is available
    #endif
  In target.cpp, this controls which library integrations are compiled in.

MACROS (ALL_CAPS)
  HIP_INIT_API, CHECK_STREAM_CAPTURE_SUPPORTED, MIGRAPHX_THROW, etc.
  ALL_CAPS names are macros -- they expand into other code at compile time.
  In the walkthroughs, macros are marked [SKIP] because they are boilerplate
  (logging, error checking) -- not the core logic.
```

```
LAMBDA [&] { ... }
  A lambda is an inline function (like Python's lambda but more powerful):
    auto api_error_result = migraphx::try_([&] {
        // ... code here ...
    });
  [&] = "capture all local variables by reference"
  The code inside { } runs as a function. migraphx::try_() catches any
  exceptions thrown inside and converts them to error codes.

COMMON PATTERNS IN ROCm SOURCE
  1. Public wrapper -> internal function:
       hipMalloc -> ihipMalloc  (the "i" prefix = internal)
  2. Handle pattern (opaque pointers):
       migraphx_program_t handle;       // caller holds this
       handle->object;                  // internal C++ object inside
  3. Guard clause early return:
       if (ptr == nullptr) return hipErrorInvalidValue;   // fail fast
       if (size == 0) { *ptr = nullptr; return hipSuccess; }
  4. Safety wrapper:
       migraphx::try_([&] { /* risky code */ });  // catches C++ exceptions

READING STRATEGY FOR THE WALKTHROUGHS
  1. Skip ALL_CAPS macros on first read (they are [SKIP] markers)
  2. Focus on function calls -- follow the chain deeper
  3. Read the [KEY] lines -- those are the actual logic
  4. Ignore error checking on first pass -- come back to it later
  5. Read bottom-up: the FULL CALL CHAIN summary at the end of each
     walkthrough gives the big picture before you read the line-by-line detail

  +-- DIAGRAM: Python -> C++ Translation Cheat Sheet ------------------------+
  |                                                                           |
  |  PYTHON                                C++ (ROCm style)                  |
  |  -----------------                     ------------------------          |
  |  x = 42                                int x = 42;                       |
  |  data = [1,2,3]                        std::vector<int> data = {1,2,3}; |
  |  name = "Conv"                         std::string name = "Conv";        |
  |  ptr = None                            int* ptr = nullptr;               |
  |                                                                           |
  |  def add(a, b):                        int add(int a, int b) {           |
  |      return a + b                          return a + b;                 |
  |                                        }                                  |
  |                                                                           |
  |  class Foo:                            class Foo {                        |
  |      def bar(self):                    public:                           |
  |          ...                               void bar();                    |
  |                                        };                                 |
  |                                                                           |
  |  if x > 0:                             if (x > 0) {                      |
  |      print(x)                              printf("%d", x);              |
  |                                        }                                  |
  |                                                                           |
  |  for i in range(10):                   for (int i = 0; i < 10; i++) {    |
  |      total += i                            total += i;                   |
  |                                        }                                  |
  |                                                                           |
  |  KEY PATTERNS IN ROCm CODE:                                              |
  |  +-------------------------------------------------------------+      |
  |  | hipMalloc -> ihipMalloc     "i" prefix = internal version    |      |
  |  | program::compile           "::" = belongs to (method call)  |      |
  |  | extern "C" func()          callable from Python/C (no mangle)|      |
  |  | [&] { ... }                lambda capturing local variables  |      |
  |  | MIGRAPHX_THROW(...)        ALL_CAPS = macro (skip on 1st read)|     |
  |  `-------------------------------------------------------------+      |
  `---------------------------------------------------------------------------+


0.25 Environment Variables Reference
-------------------------------------

The AMD inference stack is configured through environment variables at
multiple layers. Setting the wrong variable (or forgetting one) can cause
silent CPU fallback, out-of-memory crashes, or 10x slower inference.

This section is your one-stop reference. Set these BEFORE launching Python.

+---------------------------------------------------------------------------+
|  CRITICAL -- Required for APU (Radeon 8060S / Strix Halo)               |
|---------------------------------------------------------------------------|
|                                                                           |
|  export HSA_XNACK=1                                                      |
|    Layer:   HSA Runtime (Layer 6)                                        |
|    Purpose: Enable unified virtual addressing / page fault handling    |
```

```
|    Why:     Without this, hipMallocManaged() falls back to device-     |
|             only allocation (512 MB VRAM limit). With XNACK=1, GPU    |
|             page faults are handled by migrating pages from GTT/swap   |
|             to VRAM on demand. MANDATORY for any model > 500 MB.      |
|    Effect:  Compiler targets gfx1151 with +xnack (ISA includes page   |
|             fault handling instructions). Code compiled without xnack  |
|             will crash on page faults instead of handling them.        |
|    Values:  0 = disabled (default on discrete GPUs)                    |
|             1 = enabled (required on APU for unified memory)           |
|                                                                         |
|  export no_system_mem_limit=Y                                          |
|    Layer:   MIGraphX Library (Layer 4)                                 |
|    Purpose: Remove the artificial GPU system memory cap                |
|    Why:     MIGraphX limits GPU allocations by default. On an APU      |
|             with 28 GB GTT, this cap prevents using available memory.  |
|    Values:  Y = unlimited (use all available GTT + swap)               |
|                                                                         |
`-------------------------------------------------------------------------+

+-------------------------------------------------------------------------+
|  MIGraphX Compiler Configuration                                        |
|-------------------------------------------------------------------------|
|                                                                         |
|  export MIGRAPHX_GPU_HIP_FLAGS="-ftemplate-depth=2048                  |
|                                 -Wno-error=stack-exhausted"            |
|    Layer:   MIGraphX -> hipRTC (Layer 4 -> Layer 5)                     |
|    Purpose: Raise C++ template recursion limit for kernel codegen      |
|    Why:     Large transformer models (LLaMA, OpenVLA, CogACT) generate |
|             deeply nested template instantiations during MIGraphX      |
|             fusion. Default depth (256-1024) causes compile crashes.   |
|                                                                         |
|  export MIGRAPHX_GPU_COMPILE_PARALLEL=1                                |
|    Layer:   MIGraphX Library (Layer 4)                                 |
|    Purpose: Serialize kernel compilation (1 kernel at a time)          |
|    Why:     Parallel compilation of large graphs can exhaust RAM +     |
|             swap. Setting to 1 trades compile speed for memory safety. |
|             Essential on memory-constrained Xen VMs (29 GB RAM).      |
|                                                                         |
|  export MIGRAPHX_ENABLE_CK=1                                           |
|    Layer:   MIGraphX Library (Layer 4)                                 |
|    Purpose: Enable Composable Kernel (CK) backend for fused ops        |
|    Why:     CK provides hand-optimized fused attention, GEMM, and      |
|             normalization kernels. Can improve throughput for          |
|             transformer models vs. default MIOpen/hipBLAS path.        |
|                                                                         |
`-------------------------------------------------------------------------+

+-------------------------------------------------------------------------+
|  MIGraphX Debug / Tracing                                               |
|-------------------------------------------------------------------------|
|                                                                         |
|  export MIGRAPHX_TRACE_COMPILE=1                                       |
|    Prints each optimization pass name and duration during              |
|    program::compile(). Invaluable for finding which pass is slow.      |
|                                                                         |
|  export MIGRAPHX_TRACE_EVAL=1                                          |
|    Prints each kernel launch during program::eval(). Shows the         |
|    execution plan step by step.                                        |
|                                                                         |
|  export MIGRAPHX_TRACE_QUANTIZATION=1                                  |
|    Prints which ops are converted to FP16/BF16 during quantization.    |
|                                                                         |
|  export MIGRAPHX_TRACE_MATCHES=1                                       |
|    Traces pattern matching during fusion passes. Very verbose.         |
|                                                                         |
`-------------------------------------------------------------------------+

+-------------------------------------------------------------------------+
|  ORT (ONNX Runtime) Configuration                                       |
|-------------------------------------------------------------------------|
|                                                                         |
|  sess_options.graph_optimization_level = ORT_DISABLE_ALL               |
|    (Set in Python, not an env var)                                     |
|    Purpose: Disable ORT's built-in graph optimizers                    |
|    Why:     ORT's fusions can conflict with MIGraphX's own fusion      |
|             passes -- the EP re-parses the subgraph into MIGraphX IR   |
|             anyway. Some models (YOLOv12, CenterPoint) produce         |
|             incorrect results or crashes without disabling this.       |
|                                                                         |
|  Verify EP is active (not silently on CPU):                            |
|    providers = session.get_providers()                                 |
|    assert providers[0] == 'MIGraphXExecutionProvider'                  |
|    ORT does NOT raise an error if the GPU EP fails to load -- it       |
```

```
|    silently falls back to CPUExecutionProvider. Always check.         |
|                                                                        |
`------------------------------------------------------------------------+

+------------------------------------------------------------------------+
|  System-Level Configuration                                            |
|------------------------------------------------------------------------|
|                                                                        |
|  ulimit -s unlimited                                                   |
|    Purpose: Remove stack size limit for the current shell              |
|    Why:     MIGraphX compilation of large graphs recurses deeply      |
|             through C++ template instantiation. Default 8 MB stack    |
|             causes segfaults on models like LLaMA-11B, OpenVLA-7B.   |
|                                                                        |
|  MIGraphX compilation cache: /tmp/migraphx/                            |
|    First compile of a model writes cached kernels here (~1-11 GB).    |
|    Subsequent runs reuse cached binaries (much faster).               |
|    Clear with: rm -rf /tmp/migraphx/  (forces recompilation)         |
|                                                                        |
`------------------------------------------------------------------------+

+------------------------------------------------------------------------+
|  WSL2 / Docker Overrides (Windows development)                         |
|------------------------------------------------------------------------|
|                                                                        |
|  export HSA_ENABLE_DXG_DETECTION=1                                     |
|    Required inside WSL2 Docker containers to detect AMD GPU via        |
|    DirectX Graphics (DXG) interop layer.                               |
|                                                                        |
|  export HSA_OVERRIDE_GFX_VERSION=11.0.0                                |
|    Override reported GPU architecture. Used when the WSL2 driver       |
|    reports a generic version instead of gfx1151.                       |
|                                                                        |
`------------------------------------------------------------------------+
```

Quick copy-paste block for your .bashrc or run script:

```
  # === AMD APU Inference Environment ===
  export HSA_XNACK=1
  export no_system_mem_limit=Y
  export MIGRAPHX_GPU_HIP_FLAGS="-ftemplate-depth=2048 -Wno-error=stack-exhausted -Wno-stack-exhausted"
  export MIGRAPHX_GPU_COMPILE_PARALLEL=1
  ulimit -s unlimited

  +-- DIAGRAM: Where Each Env Var Acts in the 8-Layer Stack -----------------+
  |                                                                          |
  |  L1  Python API                                                         |
  |  |    graph_optimization_level = ORT_DISABLE_ALL  <-- ORT config       |
  |  |    session.get_providers() check                                     |
  |  v                                                                       |
  |  L2  ORT Core ---------------------------------------- (no env vars)    |
  |  v                                                                       |
  |  L3  MIGraphX EP ------------------------------------- (no env vars)    |
  |  v                                                                       |
  |  L4  libmigraphx.so                                                     |
  |  |    no_system_mem_limit=Y              <-- remove memory cap         |
  |  |    MIGRAPHX_GPU_COMPILE_PARALLEL=1    <-- serialize compilation     |
  |  |    MIGRAPHX_GPU_HIP_FLAGS="..."       <-- compiler flags            |
  |  |    MIGRAPHX_TRACE_COMPILE=1           <-- debug: pass timing        |
  |  |    MIGRAPHX_TRACE_EVAL=1              <-- debug: kernel launches    |
  |  |    MIGRAPHX_ENABLE_CK=1              <-- enable CK backend         |
  |  |    MIGRAPHX_DISABLE_MLIR=1           <-- use rocBLAS not MLIR      |
  |  |    MIGRAPHX_SET_GEMM_PROVIDER=rocblas <-- GEMM library choice      |
  |  v                                                                       |
  |  L5  HIP Runtime ------------------------------------- (no env vars)    |
  |  v                                                                       |
  |  L6  HSA Runtime                                                        |
  |  |    HSA_XNACK=1                        <-- enable page faults        |
  |  |    HSA_ENABLE_DXG_DETECTION=1         <-- WSL2 GPU detection        |
  |  |    HSA_OVERRIDE_GFX_VERSION=11.0.0    <-- WSL2 arch override        |
  |  v                                                                       |
  |  L7  amdgpu.ko --------------------------------------- (no env vars)    |
  |  v                                                                       |
  |  L8  GPU Hardware ----------------------------------- (no env vars)    |
  `--------------------------------------------------------------------------+
```

Now you are ready for the Preface. Every term used in the rest of this textbook has been introduced above. When you encounter something unfamiliar, come back to this chapter.

==========

```
                    PREFACE: THE 8-LAYER STACK OVERVIEW
================================================================================

+-- EXECUTIVE SUMMARY (for senior engineers) ---------------------------+
|                                                                        |
|  Two Python lines -> 8 software/hardware layers -> GPU result.         |
|                                                                        |
|  L1  Python API         ort.InferenceSession / session.run()         |
|  L2  ORT Core (C++)     Protobuf load -> graph -> partition -> EPs    |
|  L3  MIGraphX EP        Thin adapter: parse -> quantize -> compile    |
|  L4  libmigraphx.so     50+ passes, fusion, scheduling, kernels      |
|  L5  HIP Runtime        hipMalloc, hipLaunchKernel, hipSync          |
|  L6  HSA Runtime        AQL packets, doorbell MMIO, signals          |
|  L7  amdgpu.ko driver   TTM memory mgr, VRAM 512 MB + GTT 28 GB      |
|  L8  Radeon 8060S       40 CUs, Wave32, 2 MB L2 + 32 MB MALL         |
|                                                                        |
|  Session creation: one-time (seconds to minutes for large models).    |
|  Inference hot path: L1->L2->L4->L5->L6->L8, ~40 us software overhead. |
|  Bottleneck on APU: memory bandwidth (~256 GB/s shared LPDDR5x).      |
`------------------------------------------------------------------------+

+-----------------------------------------------------------------------+
|  LAYER 1: Python / ONNX Runtime API                                   |
|    session = ort.InferenceSession("model.onnx", providers=[...])      |
|    output = session.run(None, {"input": data})                        |
|-----------------------------------------------------------------------|
|  LAYER 2: ONNX Runtime Core (C++)                                     |
|    Loads ONNX protobuf -> builds in-memory graph                      |
|    Graph partitioning -> assigns nodes to Execution Providers         |
|    Creates subgraphs per EP -> allocates memory via EP allocators     |
|-----------------------------------------------------------------------|
|  LAYER 3: MIGraphX Execution Provider (C++ plugin inside ORT)         |
|    Thin adapter -- receives subgraph from ORT, delegates to Layer 4   |
|    Calls migraphx::parse_onnx_buffer() -> Layer 4 does the parsing    |
|    Calls migraphx::quantize_fp16()     -> Layer 4 does conversion     |
|    Calls prog.compile(target, options) -> Layer 4 does all passes     |
|    (COMGR/LLVM compilation happens in Layer 5, invoked by Layer 4)    |
|-----------------------------------------------------------------------|
|  LAYER 4: MIGraphX Library (libmigraphx.so)                           |
|    Graph-level optimizations (fusion, dead code elimination)          |
|    Instruction scheduling across HIP streams                          |
|    Emits HIP kernel launch calls                                      |
|-----------------------------------------------------------------------|
|  LAYER 5: HIP Runtime (libamdhip64.so)                                |
|    hipMalloc() / hipMallocManaged() for GPU memory                    |
|    hipLaunchKernel() to dispatch work to GPU                          |
|    hipDeviceSynchronize() to wait for results                         |
|-----------------------------------------------------------------------|
|  LAYER 6: HSA Runtime (libhsa-runtime64.so)                           |
|    Hardware-level abstraction (agent/pool discovery)                  |
|    Manages GPU command queues (AQL packets)                           |
|    Page table management (XNACK for unified memory)                   |
|-----------------------------------------------------------------------|
|  LAYER 7: amdgpu Kernel Driver (amdgpu.ko, kernel 6.18.0+)            |
|    Direct hardware register access (MMIO)                             |
|    GPU memory manager (TTM subsystem)                                 |
|    Manages VRAM (512 MB) + GTT (28 GB) pools                          |
|-----------------------------------------------------------------------|
|  LAYER 8: AMD GPU Hardware (Radeon 8060S, gfx1151, RDNA 3.5)          |
|    Compute Units execute shader/compute kernels (Wave32)              |
|    L2 Cache (2 MB) + Infinity Cache / MALL (32 MB, ~1 TB/s)           |
|    Shared LPDDR5x (~256 GB/s to system memory)                        |
`-----------------------------------------------------------------------+

+-- DIAGRAM: Data Flow Between Layers ----------------------------------+
|                                                                        |
|  L1  Python          session = ort.InferenceSession("model.onnx")     |
|       |                                                                |
|       |  passes: file path + provider config (Python objects)         |
|       v                                                                |
|  L2  ORT Core        Load protobuf -> build graph -> partition        |
|       |                                                                |
|       |  passes: ONNX subgraph (serialized protobuf bytes)            |
|       v                                                                |
|  L3  MIGraphX EP     parse_onnx_buffer -> quantize_fp16 -> compile    |
|       |                                                                |
|       |  passes: MIGraphX program object (C++ pointer)                |
|       v                                                                |
|  L4  libmigraphx     50+ optimization passes -> schedule -> emit kernels |
|       |                                                                |
|       |  passes: HIP API calls (hipMalloc, hipLaunchKernel)           |
|       v                                                                |
|  L5  HIP Runtime     Translate HIP calls -> HSA queue operations      |
```

```
|       |                                                                  |
|       |  passes: HSA API calls (hsa_signal, AQL packet writes)          |
|       v                                                                  |
|  L6  HSA Runtime     Write AQL packet to queue -> ring doorbell          |
|       |                                                                  |
|       |  passes: ioctl() for setup, MMIO doorbell write for dispatch    |
|       v                                                                  |
|  L7  amdgpu.ko       Program page tables, manage BO placement           |
|       |                                                                  |
|       |  passes: register writes, DMA commands, interrupt signals       |
|       v                                                                  |
|  L8  GPU Silicon     Command Processor -> Shader Engines -> CUs -> result |
|                                                                          |
|  Setup:     L1->L2->L3->L4->L5->L6->L7->L8  (seconds to minutes, once)      |
|  Inference: L1->L2->L4->L5->L6->L8         (milliseconds, every run)       |
|             L3 skipped (already compiled)                                |
|             L7 skipped (pages already mapped)                            |
`--------------------------------------------------------------------------+
```

```
==============================================================================
                      PART I -- THE APPLICATION LAYERS
==============================================================================
```

```
==============================================================================
  CHAPTER 1: ONNX Runtime Core (Layer 2)
  Protobuf, Graphs, Partitioning, and Subgraphs
==============================================================================

+-- EXECUTIVE SUMMARY -- Layer 2: ORT Core ------------------------------+
|                                                                         |
|  What:  C++ engine that owns the model lifecycle.                      |
|  Input: .onnx file (protobuf) from Python API.                        |
|  Output: Compiled subgraphs handed to Execution Providers.             |
|                                                                         |
|  Key operations (all happen inside InferenceSession::Initialize):      |
|    1. Model::Load()       -- deserialize protobuf -> in-memory graph |
|    2. ApplyTransformers() -- ORT-level graph optimizations             |
|    3. Partition()         -- assign nodes to EPs (MIGraphX vs CPU)     |
|    4. EP::Compile()       -- hand subgraphs to MIGraphX (-> Layer 3) |
|                                                                         |
|  At inference (session.run):                                           |
|    SequentialExecutor walks the execution plan, calling each EP's      |
|    compute_func in order. MIGraphX nodes -> Layer 4 -> GPU.            |
|                                                                         |
|  Key files:                                                            |
|    inference_session.cc, graph_partitioner.cc, sequential_executor.cc|
|                                                                         |
|  APU note: ORT's allocator was patched (Fix B) to use                 |
|  hipMallocManaged instead of hipMalloc for the 512 MB VRAM limit.      |
`-------------------------------------------------------------------------+

  +-- SOURCE REFERENCES -- Layer 2 (ORT Core) -------------------------------+
  |                                                                          |
  |  Repository: https://github.com/microsoft/onnxruntime                   |
  |  Branch:     rel-1.25.0 (custom build with Fixes A-D)                  |
  |                                                                          |
  |  File                                              Key Functions         |
  |  ------------------------------------------------  ------------------- |
  |  onnxruntime/core/session/inference_session.cc     Initialize(), Run() |
  |  onnxruntime/core/framework/model.cc               Model::Load()         |
  |  onnxruntime/core/optimizer/graph_transformer_utils.cc                   |
  |                                        GenerateTransformers()            |
  |  onnxruntime/core/framework/graph_partitioner.cc   Partition()           |
  |  onnxruntime/core/framework/sequential_executor.cc Execute()             |
  |                                                                          |
  |  GitHub links:                                                           |
  |  https://github.com/microsoft/onnxruntime/tree/rel-1.25.0/             |
  |    onnxruntime/core/session/inference_session.cc                         |
  |    onnxruntime/core/framework/graph_partitioner.cc                       |
  |    onnxruntime/core/framework/sequential_executor.cc                     |
  `--------------------------------------------------------------------------+

+-------------------------------------------------------------------------+
|  BEGINNER'S CONTEXT: What Just Happened Before This Chapter?            |
|                                                                         |
|  In Python, you wrote two lines:                                       |
|    import onnxruntime as ort                                            |
|    session = ort.InferenceSession("model.onnx")                        |
|                                                                         |
|  That second line is where EVERYTHING in this chapter happens.          |
|  Behind that simple Python call, thousands of lines of C++ code run.   |
```

```
|  This chapter explains what those C++ internals actually do.           |
|  You never see this - it is hidden behind the Python API.              |
`-------------------------------------------------------------------------+
```

Let me walk through each of these four stages in the order they actually happen when you call ort.InferenceSession("model.onnx").

## 1.1 ONNX Protobuf - "The Blueprint File"

What is a Protobuf?
Protocol Buffers (protobuf) is a binary file format invented by Google. Think of it like a ZIP file but for structured data - compact,
Analogy: Imagine you're building a house. The .onnx file is the architect's blueprint rolled up in a tube. It's not the house itself -

What's Inside the .onnx File?
The ONNX spec defines a protobuf schema. When you export a PyTorch model to ONNX, it writes this structure:

```
ModelProto                          <- the entire file
 |---- graph: GraphProto              <- the computation graph
 |    |---- node[0]: NodeProto        <- e.g., "Conv" operation
 |    |    |---- op_type: "Conv"
 |    |    |---- inputs: ["X", "W"]
 |    |    |---- outputs: ["Y"]
 |    |    `---- attributes: {kernel_shape: [3,3], strides: [1,1]}
 |    |---- node[1]: NodeProto        <- e.g., "Relu"
 |    |---- node[2]: NodeProto        <- e.g., "MatMul"
 |    |    ...
 |    |---- initializer[0]: TensorProto  <- weight tensor (actual numbers)
 |    |---- initializer[1]: TensorProto  <- bias tensor
 |    |---- input: [ValueInfoProto]      <- model input shapes/types
 |    `---- output: [ValueInfoProto]     <- model output shapes/types
 `---- opset_import: {version: 17}    <- which ONNX op version
```

On AMD - Why Does This Matter?
When you run on an AMD GPU (via MIGraphX or ROCm EP), the ONNX protobuf is the universal exchange format. Your model was probably trai
PyTorch (NVIDIA training) -> export -> model.onnx -> ORT + MIGraphX EP -> AMD GPU
The protobuf is the bridge between ecosystems.

```
  +-- DIAGRAM: ONNX File -> Deserialization -> In-Memory Graph ---------------+
  |                                                                           |
  |  +------------------+     +--------------------+     +------------------+   |
  |  |  model.onnx     |     |  Protobuf Parser  |     | C++ Graph      |   |
  |  |  (binary file   |---->|  (deserializer)   |---->| (in RAM)       |   |
  |  |   on disk)      |     |                    |     |                |   |
  |  |  ~14 GB for     |     |  Reads binary,     |     | Nodes: Conv,  |   |
  |  |  OpenVLA-7B     |     |  creates C++        |     | MatMul, ReLU  |   |
  |  |                 |     |  objects            |     | Edges: data   |   |
  |  `------------------+     `---------------------+     | flow links     |   |
  |                                                    | Weights: in    |   |
  |  Step 1: fread() loads     Step 2: protobuf       | TensorProto    |   |
  |  raw bytes from disk       schema maps bytes      | objects (RAM)  |   |
  |                             to typed structures    `------------------+   |
  |                                                                           |
  |  Timing: small model (<100 MB) = milliseconds                            |
  |          large model (14+ GB)  = several seconds (disk I/O bound)     |
  `---------------------------------------------------------------------------+
```

## 1.2 Builds In-Memory Graph - "Unrolling the Blueprint"

What Happens Here?
When ORT loads the .onnx file, it deserializes (unpacks) the protobuf and builds a live graph data structure in RAM.
Analogy: The architect unrolls the blueprint on a table and builds a 3D model of the house with wooden blocks. Now you can walk around

The In-Memory Graph Structure

```
         [Input: image, shape=(1,3,224,224)]
                     |
                     v
              +---------------+
              |  Conv (node) |  <- weights stored in RAM
              |  kernel=3x3  |
              `---------+--------+
                     |
                     v
              +---------------+
              |  BatchNorm   |
              `---------+--------+
                     |
                     v
              +---------------+
              |    ReLU      |
              `---------+--------+
                     |
```

```
                    v
              +---------------+
              |   MatMul     |  <- weights stored in RAM
              `---------+--------+
                        |
                        v
           [Output: class_scores]
```

Each node in this graph is a C++ object containing:
* What operation it performs (Conv, Relu, MatMul, etc.)
* Pointers to its input/output edges
* Metadata (shapes, data types)

Each edge connects one node's output to another node's input.

```
+-------------------------------------------------------------------------+
|  BEGINNER'S NOTE: What is a "C++ Object" here?                          |
|                                                                          |
|  Remember from Chapter 0: a C++ object is just like a Python object.   |
|  Think of each node as a Python class instance:                         |
|                                                                          |
|    class Node:                                                           |
|        def __init__(self):                                               |
|            self.op_type = "Conv"       # what operation                  |
|            self.inputs = ["X", "W"]    # input connections (edges)       |
|            self.outputs = ["Y"]        # output connections (edges)      |
|            self.shape = (1,64,112,112) # output tensor shape            |
|                                                                          |
|  ORT creates one such object for every operation in your model.         |
|  These objects are connected to each other via "edges" (pointers),      |
|  forming the graph structure you see in the diagram above.              |
`-------------------------------------------------------------------------+
```

Graph Optimizations (Before Partitioning)

```
+-------------------------------------------------------------------------+
|  BEGINNER'S NOTE: What is "Graph Optimization"?                         |
|                                                                          |
|  Think of it like optimizing a cooking recipe. The recipe says:         |
|    1. Boil water                                                         |
|    2. Add pasta                                                          |
|    3. Boil more water (wait, you already have boiling water!)           |
|    4. Make sauce while pasta cooks                                       |
|                                                                          |
|  An optimizer would rewrite this as:                                    |
|    1. Boil water                                                         |
|    2. Add pasta + start making sauce in parallel                        |
|    (Step 3 was redundant -- removed!)                                    |
|                                                                          |
|  ORT does the same thing with your model's computation graph:           |
|  it removes redundant operations, combines multiple steps into one,     |
|  and reorders things for efficiency. The math result is identical --    |
|  it's just computed smarter.                                            |
`-------------------------------------------------------------------------+
```

ORT runs graph transformations at this stage - rewriting the graph to be faster:
Optimization
What It Does
Example
Constant Folding
Pre-computes anything that doesn't depend on input
Reshape(constant_tensor) -> just store the result
Operator Fusion
Merges consecutive ops into one
Conv -> BatchNorm -> Relu becomes ConvBnRelu (single kernel)
Dead Node Elimination
Removes nodes whose outputs nobody uses
Leftover debug outputs
Shape Inference
Figures out tensor shapes at every edge
Needed so the GPU knows how much memory to allocate

On AMD - Why This Matters

```
+-------------------------------------------------------------------------+
|  BEGINNER'S NOTE: What is "Operator Fusion"?                            |
|                                                                          |
|  In Python terms, imagine you have:                                     |
|    step1 = conv(image)     # save result to RAM                         |
|    step2 = batchnorm(step1) # load from RAM, save to RAM              |
|    step3 = relu(step2)      # load from RAM, save to RAM              |
|                                                                          |
|  Each step saves to memory and the next step loads from memory.         |
|  That loading/saving is SLOW on a GPU.                                  |
|                                                                          |
|  Fusion combines them into ONE step:                                    |
|    result = fused_conv_bn_relu(image)  # do all 3 at once!            |
```

```
|                                                                      |
|  The intermediate results never touch slow memory -- they stay in       |
|  fast on-chip storage (registers/cache). Like doing three recipe       |
|  steps without putting the pan down between each one.                  |
`------------------------------------------------------------------------+
```

Operator fusion is critical for AMD GPUs. An AMD MI210/MI300 GPU has enormous compute power but memory bandwidth is the bottleneck. Eve

```
WITHOUT fusion (3 separate GPU kernel launches):
  GPU Memory -> [Conv kernel] -> GPU Memory -> [BN kernel] -> GPU Memory -> [ReLU kernel] -> GPU Memory
  ^^^^^^^^^^^^                 ^^^^^^^^^^^^                ^^^^^^^^^^^^
  3 round-trips to slow global memory = SLOW
WITH fusion (1 fused kernel):
  GPU Memory -> [ConvBnRelu kernel] -> GPU Memory
  ^^^^^^^^^^^^
  1 round-trip = FAST
```

MIGraphX EP is especially aggressive about fusion - it can fuse dozens of ops into a single kernel.

## 1.3 Graph Partitioning - "Assigning Workers to Rooms"

What is an Execution Provider (EP)?

An EP is a backend engine that knows how to run operations on specific hardware:

```
EP
Hardware
Library Underneath
CPUExecutionProvider
Any CPU
Built-in ORT kernels
MIGraphXExecutionProvider
AMD GPU
AMD MIGraphX library
ROCmExecutionProvider
AMD GPU
ROCm/HIP (lower-level)
CUDAExecutionProvider
NVIDIA GPU
cuDNN, cuBLAS
```

How Partitioning Works

Analogy: You're renovating a house. You have an electrician (MIGraphX EP) and a general handyman (CPU EP). The electrician is amazing a

The algorithm:

```
For each node in the graph:
    Ask MIGraphX EP: "Can you run this operation?"
        If YES -> mark node as "MIGraphX"
        If NO  -> mark node as "CPU" (fallback)
```

Concretely, MIGraphX EP supports ~90% of common ops: Conv, MatMul, Relu, Softmax, LayerNorm, Attention, etc. But some rare or custom op

Real Example - A Transformer Model on AMD

```
Node 0:  Embedding         -> MIGraphX [Y]
Node 1:  LayerNorm         -> MIGraphX [Y]
Node 2:  MatMul (Q,K,V)    -> MIGraphX [Y]
Node 3:  Attention          -> MIGraphX [Y]
Node 4:  Add (residual)    -> MIGraphX [Y]
Node 5:  CustomOp_xyz      -> CPU [N] (MIGraphX doesn't know this op)
Node 6:  MatMul (FFN)      -> MIGraphX [Y]
Node 7:  GELU              -> MIGraphX [Y]
Node 8:  Final LayerNorm   -> MIGraphX [Y]
```

Why CPU Fallback is Expensive

When a node falls back to CPU, data must travel:

```
AMD GPU (device memory)
    |
    |  <- PCIe transfer on discrete GPUs (~32 GB/s on PCIe Gen4)
    |     or memory controller remap on APU (faster, but still a stall)
    v
CPU (LPDDR5x memory)
    |
    |  <- CPU computes the op
    v
CPU (LPDDR5x memory)
    |
    |  <- Transfer back to GPU (same overhead)
    v
AMD GPU (device memory)
```

Compare this to staying on GPU the whole time:

```
GPU device memory bandwidth: ~1,638 GB/s (MI210 HBM2e) or ~256 GB/s (8060S LPDDR5x)
PCIe Gen4 bandwidth:          ~32 GB/s
```

That's a 50x-160x slowdown for every CPU fallback!

This is why you want maximum GPU coverage - every CPU fallback is extremely painful.

```
  +-- DIAGRAM: Graph Partitioning -------------------------------------------+
  |                                                                          |
  |  Original Graph (all nodes):                                            |
```

```
|  +--------+ +--------+ +--------+ +------------+ +--------+ +--------+ +--------+|
|  |Embed |->|LNorm |->|MatMul|->|CustomOp  |->|MatMul|->| GELU |->|LNorm ||
|  `---------+ `---------+ `---------+ `-------------+ `---------+ `---------+ `---------+|
|   GPU [Y]    GPU [Y]    GPU [Y]    CPU only [N]    GPU [Y]    GPU [Y]    GPU [Y]  |
|                                                                                  |
|  After partitioning (split into subgraphs):                                     |
|                                                                                  |
|  +-- MIGraphX Subgraph 1 (GPU) ---------------+                                  |
|  |  Embed -> LayerNorm -> MatMul               |                                   |
|  `-----------------------------------+--------------+                                |
|                                      | copy to CPU                                |
|                    +----------------+----------------+                              |
|                    |  CPU Subgraph (fallback)     |                                |
|                    |  CustomOp                    |                                |
|                    `-----------------+----------------+                              |
|                                      | copy to GPU                                |
|  +-- MIGraphX Subgraph 2 (GPU) ---+-----------------+                              |
|  |  MatMul -> GELU -> LayerNorm                     |                               |
|  `---------------------------------------------------+                              |
|                                                                                  |
|  Ideal: one big GPU subgraph (no copies, no fallback)                           |
|  Reality: occasional CPU fallback for unsupported ops                            |
`-----------------------------------------------------------------------------+
```

1.4 Creates Subgraphs Per EP - "Giving Each Worker Their Blueprint"
---

What Are Subgraphs?

```
+-----------------------------------------------------------------------------+
|  BEGINNER'S NOTE: Think of "subgraph" in Python terms                      |
|                                                                             |
|  If the full model is one big Python function:                              |
|    def full_model(x):                                                       |
|        x = conv(x)          # -> GPU can do this                            |
|        x = relu(x)          # -> GPU can do this                            |
|        x = custom_op(x)     # -> GPU can't do this (CPU only)               |
|        x = matmul(x, W)     # -> GPU can do this                            |
|        return x                                                             |
|                                                                             |
|  ORT splits this into three "subgraphs" (sub-functions):                    |
|    def subgraph1_gpu(x):       # runs on GPU                                |
|        return relu(conv(x))                                                 |
|    def subgraph2_cpu(x):       # runs on CPU                                |
|        return custom_op(x)                                                  |
|    def subgraph3_gpu(x):       # runs on GPU                                |
|        return matmul(x, W)                                                  |
|                                                                             |
|  Each subgraph is handed to the appropriate "Execution Provider"            |
|  (GPU or CPU), which compiles and runs it independently.                    |
`-----------------------------------------------------------------------------+
```

After partitioning, ORT groups consecutive nodes assigned to the same EP into subgraphs. Each subgraph is a self-contained mini-graph t
Analogy: After assigning rooms to each worker, you don't hand the electrician the full house blueprint. You cut out just the pages for

Visual Example
Starting from the partitioned graph above:
FULL GRAPH (after partitioning):
---

```
+---------------------------------------+
|          SUBGRAPH 1 (MIGraphX)      |
|                                     |
|  Embedding -> LayerNorm -> MatMul ->   |
|  Attention -> Add                    |
`-----------------+-----------------------+
               | (data transfer: GPU -> CPU)
+---------------v-----------------------+
|          SUBGRAPH 2 (CPU)           |
|                                     |
|  CustomOp_xyz                       |
`-----------------+-----------------------+
               | (data transfer: CPU -> GPU)
+---------------v-----------------------+
|          SUBGRAPH 3 (MIGraphX)      |
|                                     |
|  MatMul(FFN) -> GELU -> LayerNorm    |
`---------------------------------------+
```

What Each EP Does With Its Subgraph
MIGraphX EP (Layer 3) receives Subgraph 1 and 3, then delegates to
libmigraphx (Layer 4) via three C++ API calls (wrappers around the C API):
Step 1: migraphx::parse_onnx_buffer() -- convert ORT subgraph -> MIGraphX IR
Step 2: migraphx::quantize_fp16() -- convert FP32 ops to FP16 (if enabled)
Step 3: prog.compile(target, options) -- Layer 4 runs its OWN optimizer:

```
        - Fuse ops further (50+ MIGraphX-specific passes)
        - Choose optimal data layout (NCHW vs NHWC)
        - Select best GPU kernel for each op on YOUR specific AMD GPU
        - Allocate device memory (HBM on discrete GPUs, LPDDR5x on APU)
        - Compile everything into GPU machine code via hipRTC (Layer 5)
Step 4: Cache the compiled result (so next session loads instantly)
CPU EP receives Subgraph 2, then:
Step 1: Use ORT's built-in CPU kernels
Step 2: Allocate regular DDR RAM
Step 3: Ready to run (no compilation needed)

Memory Allocation - EP-Specific Allocators
Each EP manages its own memory:
+--------------------------------+     +--------------------------------+
|     MIGraphX EP Allocator    |     |       CPU EP Allocator       |
|                              |     |                              |
|  hipMalloc() on AMD GPU      |     |  malloc() / aligned_alloc()  |
|  Memory: HBM (up to 192GB   |     |  Memory: LPDDR5x system RAM  |
|          on MI300X)          |     |                              |
|                              |     |                              |
|  Pools memory to avoid       |     |  Uses ORT's arena allocator  |
|  repeated alloc/free         |     |  for efficiency              |
|  on the GPU                  |     |                              |
`--------------------------------+     `--------------------------------+
Why pooling matters on AMD GPUs: hipMalloc() (AMD's equivalent of cudaMalloc()) is expensive - it can take milliseconds. If you alloca


1.5 The Complete Flow - Start to Finish
----------------------------------------

model.onnx (protobuf file on disk)
    |
    |  (1) ORT deserializes protobuf
    v
In-Memory Graph (C++ objects in RAM)
    |
    |  (2) Graph optimizations (fusion, constant folding, etc.)
    v
Optimized Graph
    |
    |  (3) Ask each EP: "which nodes can you run?"
    v
Partitioned Graph (each node tagged with an EP)
    |
    |  (4) Group consecutive same-EP nodes into subgraphs
    v
+---------------+  +---------------+  +---------------+
| Subgraph 1  |  | Subgraph 2  |  | Subgraph 3  |
|  (MIGraphX) |  |   (CPU)     |  |  (MIGraphX) |
`---------+--------+  `---------+--------+  `---------+--------+
       |                |                |
       v                v                v
  MIGraphX            ORT CPU         MIGraphX
  compiles to         kernels          compiles to
  AMD GPU code        ready            AMD GPU code
       |                |                |
       v                v                v
    ------------ INFERENCE TIME ------------
    Run subgraph1 -> transfer -> Run subgraph2 -> transfer -> Run subgraph3
     (on GPU)      (GPU->CPU)    (on CPU)       (CPU->GPU)   (on GPU)

Key Takeaway for AMD
The fewer subgraphs you have, the better. The ideal case is one single MIGraphX subgraph covering the entire model - zero CPU fallbacks


1.6 Source Code Walkthrough -- ORT Core Internals
--------------------------------------------------

Model Loading (Model::Load)
  File: onnxruntime/core/framework/model.cc

  Status Model::Load(const PathString& file_path,
                     std::shared_ptr<Model>& model) {
      // 1. Read raw bytes from .onnx file
      // 2. Deserialize using protobuf schema:
      //    onnx::ModelProto model_proto;
      //    model_proto.ParseFromArray(data, size);
      // 3. Build in-memory Graph:
      //    Nodes: Conv, MatMul, Add, ReLU...
      //    Edges: data flow connections
      //    Initializers: weight tensor values (as OrtValue on CPU)
      //    Metadata: input/output shapes, types
      // 4. Return Graph ready for optimization
```

```
  }

ORT Graph Transformers (Level 1 and 2)
  File: onnxruntime/core/optimizer/graph_transformer_utils.cc

  Level 1 transformers (always applied):
    ConstantFolding           -- pre-compute constant subgraphs
    MatMulAddFusion           -- MatMul + Add -> single Gemm
    ReluClip6Fusion           -- Relu + Clip(0,6) -> Clip
    ReshapeFusion             -- consecutive reshapes -> one reshape
    FreeDimensionOverrideTransformer

  Level 2 transformers (extended):
    GemmActivationFusion      -- Gemm + Relu/Sigmoid -> fused
    ConvActivationFusion      -- Conv + Relu -> fused conv
    AttentionFusion           -- Q/K/V matmuls + softmax -> Attention
    BiasGeluFusion            -- Add + Gelu -> BiasGelu
    LayerNormFusion           -- Mean + Sub + Pow + Mean + Div -> LayerNorm
    SkipLayerNormFusion       -- Add + LayerNorm -> SkipLayerNorm

  Each transformer walks the graph looking for matching patterns and
  rewrites them in-place. These are EP-independent (run before partitioning).

  NOTE: For some models, ORT's fusions conflict with MIGraphX's own fusion
  passes. Set sess_options.graph_optimization_level = ORT_DISABLE_ALL to
  let MIGraphX handle all optimizations itself (see Section 0.25).

  +-- DIAGRAM: ORT Core Internal Flow -------------------------------------+
  |                                                                         |
  |  InferenceSession("model.onnx")                                        |
  |       |                                                                 |
  |       v  (1) LOAD                                                      |
  |  +----------------+                                                      |
  |  | Model::Load  |  read bytes -> protobuf -> in-memory Graph            |
  |  `---------+---------+                                                      |
  |         v  (2) OPTIMIZE                                                |
  |  +----------------------------+                                          |
  |  | Graph Transformers (L1)  |  ConstantFolding, MatMulAddFusion       |
  |  | Graph Transformers (L2)  |  AttentionFusion, LayerNormFusion       |
  |  `---------+--------------------+                                          |
  |         v  (3) PARTITION                                               |
  |  +-----------------------------------------------+                      |
  |  | GraphPartitioner::Partition                   |                      |
  |  |  MIGraphX EP claims: Conv, MatMul, Softmax...|                      |
  |  |  Unclaimed nodes -> CPU fallback              |                      |
  |  `---------+----------------------------------------+                      |
  |         v  (4) COMPILE  (calls into Layer 3 = MIGraphX EP)            |
  |  +-----------------------------------------------+                      |
  |  | MIGraphXEP::Compile()                        |                      |
  |  |  parse_onnx -> quantize_fp16 -> compile        |                      |
  |  |  -> stores compute_func lambda                 |                      |
  |  `---------+----------------------------------------+                      |
  |         v                                                              |
  |  session.run(input)  <- inference hot path                             |
  |         |                                                              |
  |         v  (5) EXECUTE                                                 |
  |  +-----------------------------------------------+                      |
  |  | SequentialExecutor::Execute                   |                      |
  |  |  for each node: compute_func(ctx)            |                      |
  |  |    -> prog.run_async(args, stream)            |                      |
  |  |    -> GPU kernel launches (pre-compiled)       |                      |
  |  `-----------------------------------------------+                      |
  |                                                                         |
  |  Steps 1-4 happen ONCE (session creation, seconds to minutes)          |
  |  Step 5 happens EVERY inference (milliseconds, no recompilation)       |
  `-------------------------------------------------------------------------+

Graph Partitioning (GraphPartitioner::Partition)
  File: onnxruntime/core/framework/graph_partitioner.cc

  Status GraphPartitioner::Partition(
      Graph& graph,
      const ExecutionProviders& providers) {
      for (auto& provider : providers) {
          auto capabilities = provider->GetCapability(graph);
          // MIGraphX EP claims: Conv, MatMul, Add, ReLU, Softmax,
          //   BatchNorm, Gemm, Reshape, Transpose, Sigmoid, Tanh,
          //   MaxPool, AveragePool, GlobalAveragePool, Concat,
          //   Gather, ReduceMean, LayerNorm, Attention...
          // Claimed nodes -> GPU subgraph
          // Unclaimed nodes -> CPU fallback
      }
      // Result: hybrid plan (GPU + CPU regions)
```

```
  }

Sequential Executor (the inference hot path)
  File: onnxruntime/core/framework/sequential_executor.cc

  Status SequentialExecutor::Execute(
      const SessionState& session_state,
      const std::vector<int>& feed_mlvalue_idxs,
      const std::vector<OrtValue>& feeds,
      const std::vector<int>& fetch_mlvalue_idxs,
      std::vector<OrtValue>& fetches) {
      for (auto node_idx : execution_plan) {
          if (node is CPU op) {
              // Run ORT's built-in CPU kernel
          }
          else if (node is MIGraphX fused subgraph) {
              // compute_func(context)
              //   -> prog.run_async(args, stream)
              //   -> launches pre-compiled GPU kernels
          }
      }
  }


1.7 Where Every Byte Lives -- Layer 1 (Python) and Layer 2 (ORT Core)
----------------------------------------------------------------------

Layer 1 (Python side):
  +-------------------------------------------------------------------+
  |  NumPy array (input)                                              |
  |    Location: Python heap (CPU RAM, malloc'd by Python/NumPy)      |
  |    Layout:   contiguous C-order (row-major), dtype=float32        |
  |    Size:     shape x sizeof(dtype) bytes                          |
  |    Lifecycle: reference-counted by Python GC                      |
  |    Example:  np.float32 [1,3,224,224] = 602,112 bytes             |
  |    Address:  array.ctypes.data -> raw pointer (passed to C++)     |
  `-------------------------------------------------------------------+

Layer 2 (ORT Core C++ side):
  +-------------------------------------------------------------------+
  |  .onnx file bytes                                                 |
  |    Where: std::string buffer in C++ heap                          |
  |    Size:  exact file size (protobuf binary)                       |
  |                                                                    |
  |  onnx::ModelProto (after deserialization)                         |
  |    Where: C++ heap (protobuf-generated objects)                   |
  |    Contains: repeated fields for nodes, initializers, metadata    |
  |                                                                    |
  |  Graph (ORT internal representation)                              |
  |    Where: C++ heap (std::vector<Node>, std::vector<NodeArg>)      |
  |    Nodes:  pointers to Op implementations                         |
  |    Edges:  data flow connections (producer -> consumer indices)   |
  |                                                                    |
  |  Initializer weights (model parameters)                           |
  |    Where: OrtValue tensors, CPU allocator (C++ heap)              |
  |    Layout: contiguous buffer, shape + strides metadata            |
  |    Size:   sum of all weight tensors (can be GBs)                 |
  |    Note:   these stay on CPU until EP copies them to GPU          |
  |                                                                    |
  |  Execution plan (after partitioning)                              |
  |    Where: SessionState in C++ heap                                |
  |    Contains: ordered list of {node_index, EP_tag} pairs           |
  `-------------------------------------------------------------------+


================================================================================
  CHAPTER 2: MIGraphX Execution Provider (Layer 3)
  The AMD Translator -- parse, quantize, compile
================================================================================

+-- EXECUTIVE SUMMARY -- Layer 3: MIGraphX EP ---------------------------+
|                                                                         |
|  What:  Thin C++ adapter plugin inside ORT. Bridges ORT -> AMD GPU.  |
|  Input: ONNX subgraph (serialized buffer) from ORT's partitioner.  |
|  Output: Compiled GPU program (migraphx::program) + compute_func.  |
|                                                                         |
|  Three lifecycle methods:                                              |
|    GetCapability() -- "I can handle these ONNX ops on GPU"            |
|    Compile()       -- re-parse ONNX -> quantize FP16 -> compile       |
|    compute_func    -- lambda that calls program.eval() at inference  |
|                                                                         |
|  Inside Compile(), three C API calls into libmigraphx.so (Layer 4): |
|    migraphx_parse_onnx_buffer()  -> parse ONNX into MIGraphX IR    |
```

```
|    migraphx_quantize_fp16()      -> convert FP32 weights to FP16   |
|    migraphx_program_compile()    -> 50+ optimization passes + JIT   |
|                                                                      |
|  Custom ORT fixes in this layer:                                     |
|    Fix A: Skip double-allocation of constant initializers (PR#27551)|
|    Fix B: hipMallocManaged allocator (migraphx_allocator.cc)       |
|    Fix C: C++20 [=,this] lambda capture                            |
|    Fix D: hipMemAdviseSetCoarseGrain for Infinity Cache utilization |
|                                                                      |
|  Key file: migraphx_execution_provider.cc, migraphx_allocator.cc   |
`----------------------------------------------------------------------+

  +-- SOURCE REFERENCES -- Layer 3 (MIGraphX EP) ----------------------------+
  |                                                                           |
  |  Repository: https://github.com/microsoft/onnxruntime (contrib EP)     |
  |  Branch:     rel-1.25.0 (custom build)                                  |
  |                                                                           |
  |  File                                            Key Functions          |
  |  ----------------------------------------------  ------------------     |
  |  onnxruntime/core/providers/migraphx/            GetCapability()        |
  |    migraphx_execution_provider.cc                Compile()              |
  |                                                   compute_func          |
  |  onnxruntime/core/providers/migraphx/            OrtMIGraphXAlloc()     |
  |    migraphx_allocator.cc                         (Fix B: Managed)       |
  |                                                                           |
  |  GitHub links:                                                            |
  |  https://github.com/microsoft/onnxruntime/tree/rel-1.25.0/              |
  |    onnxruntime/core/providers/migraphx/migraphx_execution_provider.cc   |
  |    onnxruntime/core/providers/migraphx/migraphx_allocator.cc            |
  |                                                                           |
  |  Custom Fix PRs:                                                          |
  |    Fix A: https://github.com/microsoft/onnxruntime/pull/27551            |
  |    Fix B: Custom (migraphx_allocator.cc, hipMallocManaged)              |
  |    Fix D: Custom (hipMemAdviseSetCoarseGrain after every allocation)    |
  `---------------------------------------------------------------------------+

This is where the AMD-specific magic happens. The MIGraphX EP is a C++ plugin that lives inside ORT and acts as the translator between

+-------------------------------------------------------------------------+
|  BEGINNER'S CONTEXT: What is a "C++ Plugin"?                            |
|                                                                          |
|  In Python, you might have a base framework and install plugins:        |
|    pip install flask          # the framework                           |
|    pip install flask-cors     # a plugin that adds functionality        |
|                                                                          |
|  Similarly, ORT is the framework, and MIGraphX EP is a plugin.          |
|  It is a .so file (shared library) that ORT loads at runtime.           |
|  When you specify providers=["MIGraphXExecutionProvider"], ORT          |
|  loads this plugin and hands it the parts of your model that            |
|  can run on the AMD GPU. Without this plugin, ORT would only            |
|  use the CPU -- which would be much slower for AI workloads.            |
|                                                                          |
|  "EP" stands for "Execution Provider" -- the backend that actually      |
|  executes the math. Think of it as a "driver" for a specific            |
|  hardware vendor's implementation.                                      |
`-------------------------------------------------------------------------+


2.1 The Big Picture First
-------------------------

The EP has TWO lifecycle methods plus a callback. Two at setup, one at inference:

ORT partitions graph --> GetCapability() --> "I can handle these nodes"
ORT compiles subgraph --> Compile()      --> "Here's a compiled GPU program"
ORT runs inference    --> compute_func   --> "Here are the output tensors"
                          (lambda returned by Compile, stored in NodeComputeInfo)

Inside Compile(), the EP uses C++ API wrappers into Layer 4 (libmigraphx.so):

ORT hands over a subgraph
        |
        v
+-----------------------------------------------------------------+
|         MIGraphX Execution Provider (C++ plugin in ORT)         |
|                                                                  |
|  CALL 1: migraphx::parse_onnx_buffer()  <- re-parse into IR     |
|          Serialize ORT subgraph to ONNX bytes, then parse       |
|          into MIGraphX's internal "program" format              |
|                                                                  |
|  CALL 2: migraphx::quantize_fp16()      <- optional FP16        |
|          Convert safe ops to FP16, keep sensitive ops in FP32   |
|                                                                  |
```

```
|  CALL 3: prog.compile(target, options)  <- full compilation    |
|          50+ optimization passes, kernel selection, memory     |
|          planning, scheduling, JIT compilation via hipRTC      |
|          (ALL the heavy work happens inside Layer 4)           |
|                                                                 |
|  NOTE: The EP does NOT compile GPU machine code itself.        |
|  It delegates everything to libmigraphx.so (Layer 4), which   |
|  in turn invokes HIP/hipRTC (Layer 5) for GPU code generation. |
`-----------------------------------------------------------------+
        |
        v
  Compiled MIGraphX program (ready-to-run on AMD MI210/MI300)

During inference, the compute_func callback calls prog.run_async() -- the
pre-compiled kernels are launched asynchronously on the GPU stream (very fast).

Note: The EP uses the C++ API (migraphx:: namespace wrappers), which internally
calls the C API functions (migraphx_program_compile, etc.) in libmigraphx.so.
Let's look at what each call does at a high level.


2.2 CALL 1: migraphx::parse_onnx_buffer() - "Re-reading the Blueprint in AMD's Language"
-----------------------------------------------------------------------------------------
(EP calls this C API -> Layer 4 does the actual parsing)

Why Re-Parse? ORT Already Parsed It!
This is a common question. Here's what's happening:
Analogy: ORT is a general contractor who reads blueprints in a universal format. MIGraphX is an AMD-specialized engineer who has their
ORT's Graph (generic)          MIGraphX's IR (after parsing)
+--------------------+          +----------------------------+
| NodeProto: Conv  |          | migraphx::op::convolution|
|  generic shape   |   -->   |  output shape known      |
|  generic attrs   |          |  data type: float32      |
|                  |          |  input refs: @0, @1      |
`--------------------+          `----------------------------+
(After compile(), instructions also get: hip_stream, memory address,
 specific kernel handle, etc. But at parse time, only shapes/types.)

What Actually Happens in Code
ORT serializes its subgraph back into an ONNX protobuf buffer (bytes in memory)
         |
         v
migraphx::parse_onnx_buffer(buffer, buffer_size)
         |
         v
MIGraphX builds its OWN internal graph: migraphx::program
The migraphx::program is MIGraphX's core data structure - a directed acyclic graph (DAG) of instructions, not "nodes." This distinction
ORT world:                    MIGraphX world:
  "node" = an operation         "instruction" = an operation +
                                 its output shape +
                                 its data type +
                                 references to input instructions
At this stage (after parsing, BEFORE compilation), instructions know their
shapes and types, but NOT yet their memory locations, scheduling info, or
hardware targets. Those are determined later during compile() in Layer 4.

The MIGraphX Internal IR (Intermediate Representation)
+-------------------------------------------------------------------------+
|  BEGINNER'S NOTE: What is an "IR" (Intermediate Representation)?       |
|                                                                         |
|  Think of translating a book from English to Japanese.                 |
|  You might first translate to a universal "middle language":            |
|    English -> Middle Language -> Japanese                               |
|                                                                         |
|  In compilers, the same idea applies:                                  |
|    Human-readable code -> IR (middle form) -> Machine code              |
|                                                                         |
|  The IR is not the original code, and not the final machine code.      |
|  It is an intermediate form that is easier to optimize and transform.  |
|                                                                         |
|  MIGraphX's IR is a list of numbered instructions like @0, @1, @2...  |
|  Each instruction knows its exact output shape and data type.          |
|  After compile(), instructions also gain memory locations, stream      |
|  assignments, and hardware targets. But at parse time, only shapes    |
|  and types are known -- the rest is determined during compilation.      |
`-------------------------------------------------------------------------+
After parsing, MIGraphX holds something like this:
migraphx::program {
  module "main" {
    @0 = @param("input")              : float_type, {1, 3, 224, 224}
    @1 = @param("conv1.weight")       : float_type, {64, 3, 7, 7}
    @2 = convolution(@0, @1)          : float_type, {1, 64, 112, 112}
    @3 = @param("bn1.weight")         : float_type, {64}
```

```
    @4 = @param("bn1.bias")           : float_type, {64}
    @5 = batch_norm(@2, @3, @4, ...)  : float_type, {1, 64, 112, 112}
    @6 = relu(@5)                     : float_type, {1, 64, 112, 112}
    @7 = @param("fc.weight")          : float_type, {1000, 2048}
    @8 = dot(@6_reshaped, @7)         : float_type, {1, 1000}
    @9 = @return(@8)
  }
}
Each @N is an instruction with a known output shape and type. This is a flat, linear representation - much simpler than ORT's nested gr

IMPORTANT NOTE: Optimization passes and operator fusion are NOT part of parsing.
They happen later, when the EP calls prog.compile() (CALL 3).
The compile() function inside libmigraphx.so (Layer 4) runs 50+ optimization
passes including fusion, dead code elimination, constant folding, scheduling,
and memory planning. See Chapter 3 for the full details of every optimization
pass, fusion type, kernel selection strategy, and memory planning approach.


2.3 CALL 2: migraphx::quantize_fp16() - "Shrinking the Numbers"
---------------------------------------------------------------
(EP calls this C API -> Layer 4 does the actual work)

What is FP16?
Every number (weight, activation) in your model is stored as a floating-point number. FP32 uses 32 bits per number. FP16 uses 16 bits.
FP32 (32 bits):  1 sign | 8 exponent | 23 mantissa
                 |---------+------------+-----------------------|
                 Range: +/-3.4 x 10^3^8    Precision: ~7 decimal digits
FP16 (16 bits):  1 sign | 5 exponent | 10 mantissa
                 |---------+------------+-------------|
                 Range: +/-65,504        Precision: ~3 decimal digits

Why Convert to FP16?
Two reasons - both matter enormously on AMD:
Reason 1: Half the Memory
A 7B parameter model:
  FP32: 7 billion x 4 bytes = 28 GB  <- might not fit in one GPU
  FP16: 7 billion x 2 bytes = 14 GB  <- fits in MI210 (64 GB)
Reason 2: Double the Compute Throughput
AMD GPUs have specialized hardware for FP16 math:
AMD MI210 (CDNA2):
  FP32 throughput:  45.3 TFLOPS
  FP16 throughput:  181.0 TFLOPS  <- 4x faster!
AMD MI300X (CDNA3):
  FP32 throughput:  163.4 TFLOPS
  FP16 throughput:  1,307.4 TFLOPS <- 8x faster!
FP16 is not just "a little faster" - it's a fundamental throughput multiplier on AMD Matrix Cores.

What quantize_fp16() Actually Does
migraphx::program (FP32)               migraphx::program (FP16)
+-------------------------+             +-------------------------+
| @0 = param: float32    |             | @0 = param: float16    |
| @1 = param: float32    |   ------>   | @1 = param: float16    |
| @2 = conv: float32     |  quantize   | @2 = conv: float16     |
| @3 = relu: float32     |   _fp16()   | @3 = relu: float16     |
| @4 = matmul: float32   |             | @4 = matmul: float16   |
`-------------------------+             `-------------------------+
But it's not blind - MIGraphX is smart about which ops to convert:
SAFE to convert to FP16:          RISKY / kept in FP32:
  [Y] Convolution                     [N] Loss computation (if present)
  [Y] MatMul / Dot                    [N] Softmax (needs precision for
  [Y] ReLU, GELU, SiLU                   exp() to avoid overflow)
  [Y] Add, Mul                        [N] LayerNorm (reduction ops
  [Y] Pooling                            need FP32 accumulation)
  [Y] Concatenation                   [N] Certain reduction ops
For ops kept in FP32, MIGraphX inserts cast instructions at boundaries:
  @3 = relu(@2)              : float16   <- fast FP16 compute
  @4 = convert(@3, float32)  : float32   <- cast to FP32
  @5 = softmax(@4)           : float32   <- precise FP32 compute
  @6 = convert(@5, float16)  : float16   <- cast back to FP16
  @7 = matmul(@6, @w)        : float16   <- fast FP16 again
This is called mixed precision - most of the graph runs in FP16 for speed, but precision-sensitive ops use FP32 for accuracy.

+-------------------------------------------------------------------------+
|  BEGINNER'S NOTE: Why Not Just Use FP16 for Everything?                |
|                                                                         |
|  Some operations involve very large or very small numbers that need    |
|  more precision. For example, Softmax uses exp() which can produce     |
|  astronomically large numbers that FP16 cannot represent (max 65,504). |
|                                                                         |
|  Python analogy:                                                        |
|    import math                                                          |
|    math.exp(100)  # = 2.688 x 10^43 -- WAY bigger than FP16's 65,504! |
|    # If this overflows to infinity, your entire model's output breaks  |
```

```
|                                                                          |
|  Mixed precision = use fast FP16 where safe, keep precise FP32 where  |
|  needed. The "convert" (cast) instructions at boundaries are like      |
|  currency exchange -- converting between the two number formats.        |
`--------------------------------------------------------------------------+
```

## 2.4 CALL 3: prog.compile(target, options) - "The Heavy Lifting"

(EP calls this C API -> ALL the work below happens inside Layer 4)

```
+--------------------------------------------------------------------------+
|  CRITICAL ARCHITECTURAL NOTE                                              |
|                                                                           |
|  Everything described below happens INSIDE libmigraphx.so (Layer 4).  |
|  The EP (Layer 3) simply calls prog.compile(target, options) and waits. |
|  The EP does NOT orchestrate these sub-steps -- Layer 4 does.           |
|                                                                           |
|  This section provides a high-level PREVIEW so you understand what     |
|  "compile" means. See Chapter 3 for the full deep-dive into every     |
|  optimization pass, fusion type, kernel selection, and memory plan.    |
`--------------------------------------------------------------------------+
```

What "Compile" Means Here
When the EP calls prog.compile(target, options), Layer 4 transforms the parsed IR into a concrete execution plan for the GPU. This is the most complex step. Analogy: You've designed the house, chosen your materials (FP16), now the head chef (Layer 4) creates the exact construction schedule.

The 5 Pass Groups Inside compile() (all in Layer 4):

```
  PASS GROUP 1: Graph Optimizations (simplify, fold constants, eliminate dead code)
  PASS GROUP 2: Fusion (fuse_pointwise, fuse_ops, fuse_ck, fuse_mlir)
  PASS GROUP 3: Lowering (map ops to MIOpen, rocBLAS, hipBLASLt, CK, or JIT)
  PASS GROUP 4: Memory Planning (memory_coloring -- reuse buffers, hipMalloc)
  PASS GROUP 5: Scheduling (assign HIP streams, insert sync events)
```

During Pass Group 3, JIT kernels are compiled via hipRTC (which calls COMGR/LLVM inside Layer 5 -- the HIP Runtime) to produce GPU machine code (ISA).

See Chapter 3, Sections 3.2-3.4 for every optimization pass, memory planning, scheduling, and kernel emission explained in detail.

## 2.5 Where Does GPU Machine Code Get Generated? (COMGR/LLVM)

```
+--------------------------------------------------------------------------+
|  CRITICAL CORRECTION: COMGR/LLVM is NOT a Layer 3 (EP) step!          |
|                                                                           |
|  In the original version of this document, COMGR/LLVM was listed as   |
|  "Step 4" of the EP. This was INCORRECT.                                |
|                                                                           |
|  The correct call chain for GPU machine code generation:               |
|                                                                           |
|  Layer 3 (EP):  calls prog.compile(target, options)                     |
|       v                                                                   |
|  Layer 4 (libmigraphx.so): during compile(), generates HIP C++        |
|       source code for JIT kernels, then calls hiprtcCompileProgram()   |
|       v                                                                   |
|  Layer 5 (HIP Runtime / hipRTC): invokes COMGR/LLVM internally        |
|       to compile HIP C++ -> LLVM IR -> AMDGPU ISA -> Code Object (ELF)  |
|                                                                           |
|  So: COMGR/LLVM lives INSIDE Layer 5 (the HIP Runtime), and is        |
|  invoked BY Layer 4 during compilation. The EP never touches it.       |
|                                                                           |
|  See Chapter 4 (Layer 5 - HIP Runtime) for the full COMGR/LLVM        |
|  compilation pipeline and ISA details.                                  |
`--------------------------------------------------------------------------+
```

Brief overview of the compilation chain (detailed in Chapter 4):

```
Layer 4 generates HIP C++ kernel source -> calls hiprtcCompileProgram()
                                                  |
                                            Layer 5 (HIP Runtime)
                                                  |
                                            COMGR/LLVM pipeline:
                                            Clang -> LLVM IR -> AMDGPU ISA
                                                  |
                                            Code Object (ELF binary)
                                            loadable by hipModuleLoadData()
```

Target Architecture:

```
  GPU              Target     Matrix Cores    Key Feature
  ----------       ------     ------------    -----------
  MI100 (CDNA1)    gfx908     MFMA v1        First AMD ML GPU
  MI210 (CDNA2)    gfx90a     MFMA v2        Unified memory, 2x perf
  MI300X (CDNA3)   gfx942     MFMA v3        Chiplet design, 192GB HBM3


2.6 The Complete Layer 3 Flow - Start to Finish
-----------------------------------------------

The EP (Layer 3) is a THIN ADAPTER. It translates between ORT and MIGraphX
via C++ API wrappers (which internally call the C API). It does NOT do
optimization, fusion, compilation, or memory management itself.

=== SETUP TIME (InferenceSession creation) ===

ORT calls GetCapability(graph)
|  EP examines graph nodes -> reports which ops MIGraphX can handle
|  Returns: list of subgraph capsules
v
ORT calls Compile(subgraph) for each MIGraphX subgraph
|
|  STEP 1: Serialize ORT subgraph to ONNX protobuf bytes
|          (ORT's internal format -> standard ONNX bytes in memory)
|
|  STEP 2 (CALL 1): migraphx_parse_onnx_buffer(bytes, len)  ---> Layer 4
|          Layer 4 deserializes ONNX and builds MIGraphX IR
|          Returns: migraphx::program (graph of instructions)
|
|  STEP 3 (CALL 2): migraphx_quantize_fp16(program)         ---> Layer 4
|          Layer 4 converts safe ops to FP16, inserts casts
|          Returns: modified program (mixed precision)
|
|  STEP 4 (CALL 3): migraphx_program_compile(program, "gpu") ---> Layer 4
|          Layer 4 runs 50+ optimization passes:
|            * Graph optimizations (fold constants, eliminate dead code)
|            * Operator fusion (pointwise, MIOpen, CK, MLIR)
|            * Kernel selection (MIOpen, rocBLAS, hipBLASLt, CK, JIT)
|            * JIT compilation via hipRTC                      ---> Layer 5
|              (COMGR/LLVM: HIP C++ -> LLVM IR -> GPU ISA)
|            * Memory planning (memory_coloring, hipMalloc)    ---> Layer 5
|            * Instruction scheduling (streams, events)
|          Returns: fully compiled program (kernel handles + memory + plan)
|
|  STEP 5: Store compiled program for later inference
v

=== INFERENCE TIME (every session.run()) ===

ORT invokes compute_func(inputs)
|  EP maps ORT tensors to MIGraphX arguments
|  CALL: prog.run_async(arguments, stream)                  ---> Layer 4
|        Layer 4 walks its execution plan:
|          hipMemcpy(input to GPU)                           ---> Layer 5
|          hipLaunchKernel(conv, relu, matmul, ...)          ---> Layer 5
|          hipStreamSynchronize()                            ---> Layer 5
|          hipMemcpy(output to CPU)                          ---> Layer 5
|  Returns: output tensors
v
Results returned to Python

Key Architectural Insight
The EP is the WAITER, not the CHEF:
  Layer 3 (EP)     = Waiter -- takes the order, passes it to the kitchen
  Layer 4 (libmigraphx.so) = Head Chef -- plans, optimizes, orchestrates
  Layer 5 (HIP/hipRTC)     = Kitchen equipment -- actually runs GPU code
  COMGR/LLVM       = lives INSIDE Layer 5, compiles GPU machine code

The EP never directly touches optimization passes, kernel selection,
memory planning, or GPU machine code generation.

  +-- DIAGRAM: Setup Time vs Inference Time ---------------------------------+
  |                                                                          |
  |  ============== SETUP (once, at InferenceSession creation) ==========  |
  |                                                                          |
  |  Time: seconds to MINUTES for large models                              |
  |  +----------+  +------------+  +------------+  +---------------------------+|
  |  | Load   |->| Parse to |->|Quantize  |->|  Compile (50+ passes)    ||
  |  | .onnx  |  | MIGraphX |  |  FP16    |  |  optimize + fuse + JIT   ||
  |  | (disk) |  |   IR     |  |          |  |  hipRTC -> GPU ISA        ||
  |  | ~3s    |  |  ~2s     |  |  ~1s     |  |  ~30s to 15min           ||
  |  `-----------+  `-------------+  `-------------+  `----------------------------+|
  |                                                                          |
```

```
  |  =========== INFERENCE (every session.run() call) ===================  |
  |                                                                         |
  |  Time: milliseconds (pre-compiled kernels, just dispatch)              |
  |  +------------+  +----------------------------+  +------------+           |
  |  | Copy     |->| Launch pre-compiled GPU    |->| Copy     |           |
  |  | input    |  | kernels (hipLaunchKernel)  |  | output   |           |
  |  | to GPU   |  | ~5-500 ms (model size)    |  | to CPU   |           |
  |  | ~0.1 ms  |  | NO recompilation!         |  | ~0.1 ms  |           |
  |  `-------------+  `----------------------------+  `-------------+           |
  |                                                                         |
  |  First run: setup + inference (heavy)                                  |
  |  Every subsequent run: inference only (lightweight, fast)              |
  `-------------------------------------------------------------------------+
```

## 2.7 Custom ORT 1.25.0 Build -- Four Critical Fixes for APU

Standard ORT 1.24.2 (pip install) could not run models > 500 MB on the MIGraphX EP due to memory allocation design. A custom ORT 1.25.0 was built from source with four fixes:

```
FIX A: Double Allocation Prevention (Upstream PR #27551)
+-------------------------------------------------------------------------+
|  File: migraphx_execution_provider.cc                                   |
|  PR:   https://github.com/microsoft/onnxruntime/pull/27551              |
|                                                                          |
|  PROBLEM:                                                                |
|  When ORT partitions a graph for MIGraphX EP, it includes constant    |
|  initializers (model weights) as subgraph inputs. ORT allocates GPU   |
|  memory for those weights -- but MIGraphX ALSO loads them internally   |
|  via parse_onnx_buffer(). Every weight is allocated TWICE.            |
|                                                                          |
|  IMPACT BEFORE: A 28 GB FP32 model consumed 56 GB GPU memory = OOM.  |
|                                                                          |
|  FIX: Skip constant initializers when building MIGraphX subgraph      |
|  inputs. ~10 lines of code.                                            |
|                                                                          |
|  IMPACT AFTER: GPU memory usage halved.                                |
|                                                                          |
|  WHERE IN THE STACK:                                                    |
|    Layer 3 (EP) -> Compile() -> builds subgraph input list              |
|    The fix filters out ORT NodeArg entries that are initializers       |
|    before serializing the subgraph for MIGraphX parse_onnx_buffer()   |
`-------------------------------------------------------------------------+

FIX B: hipMallocManaged Allocator (Custom, Not Upstream)
+-------------------------------------------------------------------------+
|  File: migraphx_allocator.cc                                            |
|                                                                          |
|  PROBLEM:                                                                |
|  ORT's MIGraphX EP allocator used hipMalloc() for all GPU buffers.    |
|  hipMalloc on APU only allocates from 512 MB VRAM -- catastrophically  |
|  insufficient for any model > 500 MB.                                  |
|                                                                          |
|  FIX: Replace hipMalloc with hipMallocManaged (fallback to hipMalloc):|
|                                                                          |
|    hipError_t err = hipMallocManaged(                                   |
|        (void**)&p, size, hipMemAttachGlobal);                          |
|    if (err != hipSuccess) {                                             |
|        HIP_CALL_THROW(hipMalloc((void**)&p, size));                    |
|    }                                                                     |
|                                                                          |
|  WHY: With HSA_XNACK=1, hipMallocManaged returns unified virtual      |
|  addresses. GPU page table maps to wherever physical memory is        |
|  available -- VRAM, GTT, or swap-backed system RAM.                    |
|                                                                          |
|  WHERE IN THE STACK:                                                    |
|    Layer 3 (EP) -> ORT allocator interface -> hipMallocManaged          |
|      -> Layer 5 (HIP) -> Layer 6 (HSA) -> hsa_amd_memory_pool_allocate  |
|      -> Layer 7 (Driver) -> TTM places pages in VRAM or GTT            |
`-------------------------------------------------------------------------+

FIX C: C++20 Build Compatibility
+-------------------------------------------------------------------------+
|  File: migraphx_execution_provider.cc line ~1397                        |
|  Fix:  [=] -> [=, this]  (implicit this capture deprecated in C++20)   |
|  One-line fix. Prevents compiler warning-as-error in ROCm 7.2 builds. |
`-------------------------------------------------------------------------+

FIX D: Coarse-Grained Coherence (hipMemAdviseSetCoarseGrain)
+-------------------------------------------------------------------------+
|  File: migraphx_allocator.cc (same file as Fix B)                       |
```

```
|                                                                          |
|  PROBLEM:                                                                |
|  hipMallocManaged() defaults to fine-grained coherence. Fine-grained  |
|  memory is visible to both CPU and GPU at all times (cache-coherent   |
|  via interconnect snooping), but this BYPASSES the GPU L2 cache       |
|  entirely. Every weight read goes through LPDDR5x bandwidth (~256     |
|  GB/s) instead of hitting Infinity Cache (~1 TB/s on RDNA 3.5).      |
|                                                                          |
|  For inference: weights are written once (load time) and read many    |
|  times (every forward pass). No concurrent CPU writer exists.         |
|  Fine-grained coherence wastes L2 capacity on memory with no writer   |
|  contention.                                                           |
|                                                                          |
|  FIX: After every hipMallocManaged(), call hipMemAdvise():            |
|                                                                          |
|    hipError_t err = hipMallocManaged(                                    |
|        (void**)&p, size, hipMemAttachGlobal);                           |
|    if (err != hipSuccess) {                                             |
|        HIP_CALL_THROW(hipMalloc((void**)&p, size));                     |
|    } else {                                                             |
|        hipMemAdvise(p, size, hipMemAdviseSetCoarseGrain, 0);            |
|    }                                                                     |
|                                                                          |
|  WHY SAFE: ORT MIGraphX EP calls hipDeviceSynchronize() (or stream    |
|  sync) before returning output tensors to CPU. This flush point       |
|  ensures all coarse-grained dirty cache lines are written back        |
|  before CPU reads results. No concurrent CPU-GPU writes exist during  |
|  inference.                                                            |
|                                                                          |
|  IMPACT: L2 cache can now hold frequently-accessed weight tiles,      |
|  effectively giving small models access to ~1 TB/s bandwidth         |
|  instead of ~256 GB/s.                                                 |
|                                                                          |
|  WHERE IN THE STACK:                                                   |
|    Layer 3 (EP allocator) -> hipMallocManaged -> hipMemAdvise          |
|    Layer 5 (HIP) interprets the advise flag                            |
|    Layer 6 (HSA) updates page table coherence attributes               |
|    Layer 8 (Hardware) GPU L2 cache now caches these pages             |
`--------------------------------------------------------------------------+

Rejected Approach: LD_PRELOAD Shim
  An LD_PRELOAD shim (hip_alloc_redirect.cpp/.so) was implemented to
  intercept hipMalloc and redirect to hipMallocManaged at the dynamic
  linker level. Explicitly rejected as a production risk -- fragile,
  non-portable, intercepts ALL hipMalloc calls (not just ORT's).
  Superseded by Fix B (source-level allocator change).

Build Configuration (Custom ORT 1.25.0 for gfx1151)
  python3 tools/ci_build/build.py \
    --config Release --use_migraphx --migraphx_home /opt/rocm \
    --rocm_home /opt/rocm --build_dir ./build --build_wheel --skip_tests \
    --parallel $(nproc) \
    --cmake_extra_defines CMAKE_HIP_ARCHITECTURES=gfx1151 \
      CMAKE_CXX_FLAGS="-Wno-error=free-nonheap-object -Wno-error=deprecated"


2.8 Source Code Walkthrough -- MIGraphX EP Internals
-----------------------------------------------------
  File: onnxruntime/core/providers/migraphx/migraphx_execution_provider.cc

GetCapability -- "Which nodes can MIGraphX handle?"
  std::vector<std::unique_ptr<ComputeCapability>>
  MIGraphXExecutionProvider::GetCapability(
      const GraphViewer& graph_viewer,
      const IKernelLookup& /*kernel_lookup*/) const {
      // Walk every node in the ONNX graph
      // Check: can MIGraphX handle this op?
      // Supported ops: Conv, MatMul, Gemm, Add, Mul, Relu,
      //   Sigmoid, Tanh, Softmax, BatchNorm, MaxPool,
      //   AveragePool, GlobalAveragePool, Reshape, Transpose,
      //   Concat, Gather, ReduceMean, LayerNorm, Attention...
      // Returns: list of ComputeCapability
      //   (groups of connected claimed nodes = subgraph capsules)
  }

Compile -- "Three C++ API calls into Layer 4"
  Status MIGraphXExecutionProvider::Compile(
      const std::vector<FusedNodeAndGraph>& fused_nodes,
      std::vector<NodeComputeInfo>& node_compute_funcs) {

      // CALL 1: Parse ONNX subgraph into MIGraphX IR
      auto prog = migraphx::parse_onnx_buffer(
          onnx_data, onnx_size, onnx_options);
```

```
      // CALL 2: Optional FP16 quantization
      if (fp16_enabled) {
          migraphx::quantize_fp16(prog);
      }

      // CALL 3: Full GPU compilation (50+ optimization passes)
      prog.compile(target, compile_options);
      // After this: prog contains compiled GPU kernels (ELF code objects)

      // Store compute_func lambda in NodeComputeInfo

  +-- DIAGRAM: MIGraphX EP Compile -- Three Calls into Layer 4 --------------+
  |                                                                          |
  |  ORT calls MIGraphXEP::Compile(fused_nodes)                             |
  |       |                                                                  |
  |       v  CALL 1: Parse                                                  |
  |  +-------------------------------------------------------+              |
  |  | migraphx::parse_onnx_buffer(onnx_data, size, opts)   |              |
  |  |  onnx bytes -> protobuf -> MIGraphX IR (program)      |              |
  |  |  each ONNX node -> migraphx::instruction              |              |
  |  `---------+-----------------------------------------------+              |
  |         v  CALL 2: Quantize (optional)                                  |
  |  +-------------------------------------------------------+              |
  |  | migraphx::quantize_fp16(prog)                         |              |
  |  |  FP32 weights/ops -> FP16 (halves memory, 2x throughput)|             |
  |  |  inserts convert ops at FP32<->FP16 boundaries          |              |
  |  `---------+-----------------------------------------------+              |
  |         v  CALL 3: Compile (heavy -- 50+ passes + JIT)                  |
  |  +-------------------------------------------------------+              |
  |  | prog.compile(gpu_target, options)                     |              |
  |  |  -> 50+ optimization passes (Ch. 3)                    |              |
  |  |  -> JIT: HIP C++ -> COMGR/LLVM -> ELF .co binaries     |              |
  |  |  -> GPU memory plan: which buffer at which address     |              |
  |  |  -> stream schedule: which kernel on which queue       |              |
  |  `---------+-----------------------------------------------+              |
  |         v                                                                |
  |  +-------------------------------------------------------+              |
  |  | Store lambda: compute_func(ctx) { prog.run(args); }  |              |
  |  |  Called per-inference by SequentialExecutor            |              |
  |  `-------------------------------------------------------+              |
  `-----------------------------------------------------------------------+

      // This lambda captures the compiled program and will be called
      // by the executor during inference
  }

compute_func -- "The inference hot path" (called every session.run())
  // This is the lambda returned by Compile(), stored in NodeComputeInfo
  // Called by SequentialExecutor for each fused GPU subgraph

  // 1. Map ORT tensors -> MIGraphX arguments
  migraphx_program_parameter_shapes_t shapes;
  migraphx_program_get_parameter_shapes(&shapes, program);

  // 2. Copy input data to GPU if needed (hipMemcpy)

  // 3. Execute compiled program asynchronously
  auto outputs = prog.run_async(arguments, stream);
  //  -> program::eval() walks the execution plan
  //    -> launches pre-compiled GPU kernels via hipLaunchKernel
  //    -> all on the same HIP stream for ordering

  // 4. Copy results back to ORT tensors

  Source files in this directory:
    migraphx_execution_provider.cc  -- GetCapability, Compile, compute_func
    migraphx_execution_provider.h   -- class declaration
    migraphx_allocator.cc           -- hipMallocManaged allocator (Fix B/D)
    migraphx_provider_factory.cc    -- EP factory registration
    migraphx_stream_handle.cc       -- HIP stream management
    gpu_data_transfer.cc            -- hipMemcpy wrappers


2.9 Where Every Byte Lives -- Layer 3 (Execution Provider)
----------------------------------------------------------

  +-----------------------------------------------------------------+
  |  MIGraphXExecutionProvider class instance                       |
  |    Where: C++ heap (constructed by ORT's EP factory)            |
  |    Contains: device_id, fp16_enable flag, provider_options      |
  |                                                                  |
  |  Compiled migraphx::program (after Compile())                   |
```

```
  |    Where: C++ heap (owned by EP, one per fused subgraph)        |
  |    Contains: MIGraphX IR (instruction list), compiled contexts  |
  |    Size:    metadata ~10-100 KB per subgraph                    |
  |    GPU code objects are in GPU memory (loaded by HIP)           |
  |                                                                  |
  |  migraphx::argument structs (during compute_func)               |
  |    Where: C++ stack/heap (temporary, per inference)             |
  |    Contains: {data_ptr -> GPU device pointer, shape metadata}    |
  |    shape:   migraphx::shape on CPU heap (type, lens, strides)  |
  |    data:    the actual tensor bytes are on GPU (device pointers)|
  |                                                                  |
  |  compute_func lambda                                            |
  |    Where: std::function in NodeComputeInfo (C++ heap)           |
  |    Captures: reference to compiled migraphx::program            |
  |    Lifetime: entire session (not freed between inferences)      |
  |                                                                  |
  |  HIP stream                                                     |
  |    Where: hipStream_t handle (C++ heap + GPU hardware queue)    |
  |    Purpose: orders all GPU operations for this EP               |
  `-----------------------------------------------------------------+
```

## CHAPTER 3: MIGraphX Library, libmigraphx.so (Layer 4)

The Brain -- Optimizations, Scheduling, and Kernel Emission

```
+-- EXECUTIVE SUMMARY -- Layer 4: libmigraphx.so ------------------------+
|                                                                         |
|  What:  AMD's graph compiler. ALL heavy lifting happens here.          |
|  Input: ONNX protobuf buffer (via C API from Layer 3).                |
|  Output: Scheduled, fused, JIT-compiled GPU program ready to eval.    |
|                                                                         |
|  Three phases:                                                         |
|    1. Parse    -- ONNX protobuf -> migraphx::program (internal IR)    |
|    2. Quantize -- FP32 -> FP16 (4-pass: convert shapes, transforms,   |
|                   adjust types, dead-code eliminate)                   |
|    3. Compile  -- 50+ ordered passes via target::get_passes():         |
|       - Normalize, simplify, dead-code eliminate                      |
|       - fuse_attention (MHA -> 1 kernel), fuse_ck, fuse_mlir          |
|       - lowering (abstract ops -> gpu::gemm, gpu::conv)               |
|       - compile_miopen (conv), compile_hipblaslt (GEMM+epilogue)      |
|       - compile_ops (custom HIP kernels via hipRTC -> Layer 5)        |
|       - schedule (multi-stream), memory_coloring (50-80% savings)     |
|                                                                         |
|  Inference: program::eval() walks the compiled instruction list,      |
|  dispatching GPU kernels via HIP (Layer 5) in scheduled order.        |
|                                                                         |
|  Key files: api.cpp, program.cpp, target.cpp, compile_hip.cpp         |
|  Env vars: MIGRAPHX_TRACE_COMPILE=1, MIGRAPHX_TRACE_EVAL=1            |
`-------------------------------------------------------------------------+

  +-- SOURCE REFERENCES -- Layer 4 (libmigraphx.so) -------------------------+
  |                                                                            |
  |  Repository: https://github.com/ROCm/AMDMIGraphX                         |
  |  Branch:     rocm-7.2.0                                                   |
  |                                                                            |
  |  File                                    Line   Key Function              |
  |  ------------------------------------   -----  --------------------      |
  |  src/api/api.cpp                         2109   parse_onnx_buffer()       |
  |  src/api/api.cpp                         2270   quantize_fp16()           |
  |  src/api/api.cpp                         1714   program_compile()         |
  |  src/api/api.cpp                         1774   program_run()             |
  |  src/onnx/onnx.cpp                       107    parse_onnx_buffer()       |
  |  src/quantization.cpp                    68     quantize_fp16() pipeline|
  |  src/program.cpp                         313    program::compile()        |
  |  src/program.cpp                         588    program::eval()           |
  |  src/targets/gpu/target.cpp              94     get_passes() (50+ list) |
  |  src/targets/gpu/compile_hip.cpp         202    hipRTC JIT compilation    |
  |  src/targets/gpu/compile_miopen.cpp      varies MIOpen conv integration |
  |  src/targets/gpu/compile_hipblaslt.cpp   varies hipBLASLt GEMM            |
  |                                                                            |
  |  GitHub links:                                                             |
  |  https://github.com/ROCm/AMDMIGraphX/blob/rocm-7.2.0/src/api/api.cpp |
  |  https://github.com/ROCm/AMDMIGraphX/blob/rocm-7.2.0/src/program.cpp |
  |  https://github.com/ROCm/AMDMIGraphX/blob/rocm-7.2.0/src/targets/    |
  |    gpu/target.cpp#L94                                                      |
  `----------------------------------------------------------------------------+
```

This is the brain of the AMD inference stack. Layer 3 (the MIGraphX EP) is a thin adapter that calls THREE C++ API wrappers into this library. ALL the real work -- parsing, optimization, fusion, kernel selection, memory planning, scheduling, and JIT compilation -- happens here inside libmigraphx.so.

```
+-----------------------------------------------------------------------------+
|  BEGINNER'S CONTEXT: Chapter 2 vs Chapter 3 -- What's the Difference?  |
|                                                                         |
|  Chapter 2 described the MIGraphX "Execution Provider" -- a PLUGIN      |
|  inside ORT that acts as the translator (receives subgraphs from ORT). |
|                                                                         |
|  Chapter 3 describes libmigraphx.so -- the ACTUAL LIBRARY that the     |
|  plugin calls to do the heavy work. It's like:                          |
|    Chapter 2 = the waiter who takes your order                          |
|    Chapter 3 = the head chef who plans and executes the cooking        |
|                                                                         |
|  In Python terms: Chapter 2 is like the wrapper/adapter class,        |
|  Chapter 3 is the actual implementation library underneath.            |
|                                                                         |
|  A ".so file" (shared object) is a compiled C++ library on Linux.     |
|  See Chapter 0 section 0.7 for the full explanation.                   |
`-----------------------------------------------------------------------------+
```

### 3.1 What is libmigraphx.so?

It's a shared library (a .so file on Linux - like a .dll on Windows). This is MIGraphX's core engine compiled into machine code.
Analogy: If the entire inference stack is a restaurant:
* ORT is the front desk (takes customer orders)
* MIGraphX EP is the waiter (translates orders for the kitchen)
* libmigraphx.so is the head chef (plans the cooking sequence, assigns tasks to sous-chefs, optimizes everything)
* The AMD math libraries (MIOpen, rocBLAS) are the sous-chefs (actually cook the food)

```
libmigraphx.so contains:
|---- ONNX parser          (parse_onnx / parse_onnx_buffer)
|---- Graph IR             (migraphx::program, migraphx::module)
|---- Optimization passes  (50+ graph transformation passes)
|---- Compiler             (compile() - maps ops to GPU kernels)
|---- Scheduler            (instruction ordering + stream assignment)
|---- Memory planner       (GPU HBM allocation strategy)
|---- JIT engine           (generates + compiles custom HIP kernels)
|---- Execution engine     (actually runs the compiled program)
`---- Python bindings      (via pybind11)

  +-- DIAGRAM: libmigraphx.so Internal Architecture ------------------------+
  |                                                                         |
  |  libmigraphx.so                                                        |
  |  +-----------------------------------------------------------------+  |
  |  |                                                                 |  |
  |  |  +---------------+    +-----------------+    +--------------------+  |  |
  |  |  | ONNX Parser |--->|  Graph IR     |--->| 50+ Opt Passes   |  |  |
  |  |  | protobuf    |    | program/module|    | (dead code elim, |  |  |
  |  |  | -> instrs   |    | instructions  |    |  fusion, lower)  |  |  |
  |  |  `----------------+    `------------------+    `-----------+-----------+  |  |
  |  |                                                     |            |  |
  |  |  +---------------+    +-----------------+    +---------v----------+  |  |
  |  |  |  Execution  |<---|  Scheduler    |<---|  JIT Compiler    |  |  |
  |  |  |  Engine     |    |  stream assign|    |  HIP -> COMGR    |  |  |
  |  |  |  run(args)  |    |  mem planner  |    |  -> LLVM -> ELF  |  |  |
  |  |  `----------------+    `------------------+    `--------------------+  |  |
  |  |                                                                 |  |
  |  |  External library calls: rocBLAS, MIOpen, hipBLASlt            |  |
  |  `-----------------------------------------------------------------+  |
  |                                                                         |
  |  Inputs: ONNX bytes + target (gfx1151)                                 |
  |  Output: ready-to-run compiled GPU program                             |
  `-------------------------------------------------------------------------+
```

### 3.2 Part 1: Graph-Level Optimizations

What "Graph-Level" Means
There are two levels of optimization in any compiler:
GRAPH-LEVEL (MIGraphX does this):
  "Which operations can I merge, remove, or reorder?"
  "Can I compute this at compile time instead of runtime?"
  Think: rearranging the recipe steps
KERNEL-LEVEL (MIOpen/rocBLAS handle this, compiled via COMGR/LLVM in Layer 5):
  "What's the fastest way to execute THIS specific matrix multiply?"
  "Which GPU registers and wavefronts to use?"
  Think: perfecting the technique for one cooking step
MIGraphX handles graph-level. Let's go through every major optimization pass.

```
  +-- DIAGRAM: Graph-Level vs Kernel-Level Optimization ---------------------+
  |                                                                         |
  |  BEFORE (unoptimized graph):          AFTER (optimized graph):         |
```

```
  |                                                                        |
  |  +-------+  +-------+  +--------+         +--------------------------+       |
  |  | Conv |->| BN  |->| ReLU |         | fused_conv_bn_relu     |       |
  |  `--------+  `--------+  `---------+         | (1 kernel, 1 mem r/w)  |       |
  |  3 kernels, 3 mem round-trips        `---------------------------+       |
  |                                                                        |
  |  +--------+  +--------+  +--------+       +--------------------------+       |
  |  | Add  |->| Mul  |->| ReLU |       | pointwise_fused         |       |
  |  `---------+  `---------+  `---------+       | JIT-compiled via hipRTC |       |
  |  3 kernels, 3 mem round-trips        `---------------------------+       |
  |                                                                        |
  |  +---------+  +------------+             +--------------------------+       |
  |  |MatMul |->| Softmax  |             | GEMM_softmax (rocBLAS) |       |
  |  `----------+  `-------------+             | or MLIR fused kernel   |       |
  |  2 kernels                           `---------------------------+       |
  |                                                                        |
  |  Graph-level (MIGraphX, this chapter):                                 |
  |    WHAT to merge  |  passes: DCE, CSE, const-fold, fuse_pointwise     |
  |                                                                        |
  |  Kernel-level (MIOpen/rocBLAS, Ch.5):                                 |
  |    HOW to execute |  tiling, wavefront scheduling, register alloc      |
  `------------------------------------------------------------------------+

3.2a. Operator Fusion - "Combining Steps"
This is the single most impactful optimization. MIGraphX has multiple fusion passes.
What fusion does:
BEFORE FUSION (3 separate GPU kernel launches):
  @0 = load(input)           --- kernel launch (1)
       |                          GPU writes result to HBM
       v                          (slow: ~200ns latency)
  @1 = convolution(@0, W)    --- kernel launch (2)
       |                          GPU writes result to HBM
       v                          (slow again)
  @2 = relu(@1)              --- kernel launch (3)
       |                          GPU writes result to HBM
       v
  @3 = add(@2, residual)     --- kernel launch (4)
  Total: 4 kernel launches, 4 HBM writes, 4 HBM reads
  Kernel launch overhead: ~5-10 mus EACH on AMD GPUs
AFTER FUSION (1 fused kernel):
  @0 = load(input)
       |
       v
  @1 = fused_conv_relu_add(@0, W, residual)  --- kernel launch (1) (ONLY ONE)
       |
       v
  done
  Total: 1 kernel launch, 1 HBM write, 1 HBM read
Why this matters on AMD GPUs - the numbers:
MI210 GPU:
  HBM bandwidth:      1,638 GB/s
  Kernel launch cost:  ~5-10 mus each

For a ResNet-50 model:
  WITHOUT fusion: ~400 kernel launches -> ~2-4 ms just in launch overhead
  WITH fusion:    ~60 kernel launches  -> ~0.3-0.6 ms launch overhead

  That's 3-6x less overhead, BEFORE counting memory savings

Types of Fusion MIGraphX Performs
MIGraphX has specific fusion passes, each targeting different patterns:

Pointwise Fusion (fuse_pointwise):
Merges chains of element-wise operations into a single JIT-compiled kernel.
BEFORE:
  @1 = add(x, bias)       <- element-wise
  @2 = mul(@1, scale)     <- element-wise
  @3 = relu(@2)           <- element-wise
  @4 = add(@3, residual)  <- element-wise
  4 kernels, 4 HBM round-trips
AFTER:
  @1 = pointwise_fused(x, bias, scale, residual)
       // internally: relu((x + bias) * scale) + residual

  1 kernel, 1 HBM round-trip

  MIGraphX JIT-compiles this fused kernel via hipRTC:

  __global__ void fused_kernel(half* x, half* bias,
                               half* scale, half* residual, half* out) {
      int i = threadIdx.x + blockIdx.x * blockDim.x;
      half temp = (x[i] + bias[i]) * scale[i];
      temp = (temp > 0) ? temp : 0;  // relu
```

```
      out[i] = temp + residual[i];
  }

Convolution Fusion (fuse_ops / MIOpen):
MIOpen supports fused conv patterns natively:
Patterns MIOpen can fuse:
  Conv + Bias                    -> single MIOpen call
  Conv + Bias + ReLU             -> single MIOpen call
  Conv + Bias + ReLU + Add       -> single MIOpen call (residual connection)
  Conv + BatchNorm               -> rewritten first, then fused
MIGraphX detects these patterns and maps them to MIOpen's fused API.

GEMM Fusion (fuse_ck - Composable Kernel):
For transformer models, CK can fuse entire attention blocks:
BEFORE (standard attention):
  Q = MatMul(input, Wq)          <- rocBLAS GEMM
  K = MatMul(input, Wk)          <- rocBLAS GEMM
  V = MatMul(input, Wv)          <- rocBLAS GEMM
  scores = MatMul(Q, K^T)        <- rocBLAS GEMM
  weights = Softmax(scores)      <- separate kernel
  output = MatMul(weights, V)    <- rocBLAS GEMM
  5 GEMM launches + 1 Softmax = 6 kernel launches
  Each writes multi-MB tensors to HBM between steps
AFTER (CK fused attention):
  output = ck_gemm_softmax_gemm(Q_proj, K_proj, V_proj)
  Internally: the GEMM results stay in LDS (shared memory)
  Softmax is computed on-chip without touching HBM

  Massive memory bandwidth savings on MI210/MI300X

MLIR Fusion (fuse_mlir - rocMLIR):
For fusion patterns that don't match any library's pre-built kernels, MIGraphX falls back to rocMLIR to generate a custom fused kernel
Example: a weird fusion pattern unique to your model

  @1 = transpose(x)
  @2 = matmul(@1, W)
  @3 = add(@2, bias)
  @4 = gelu(@3)
  No library has this exact fused pattern.

  MIGraphX generates MLIR code -> rocMLIR compiles it ->
  Custom GPU kernel that fuses all 4 ops.

3.2b. Dead Code Elimination - "Throwing Away Unused Work"
What it does: Removes any instruction whose output is never consumed by another instruction.
BEFORE:
  @0 = param("input")
  @1 = conv(@0, W1)           <- used by @3
  @2 = conv(@0, W2)           <- output NEVER used by anything! (DEAD)
  @3 = relu(@1)               <- used by @4
  @4 = @return(@3)
AFTER dead code elimination:
  @0 = param("input")
  @1 = conv(@0, W1)
  @2 = relu(@1)
  @3 = @return(@2)
  Saved: one entire convolution! (could be millions of FLOPs)

  +-- DIAGRAM: Dead Code Elimination (Before -> After) ----------------------+
  |                                                                          |
  |  BEFORE:                              AFTER:                            |
  |  +-------------+                        +-------------+                  |
  |  |  input @0  |                        |  input @0  |                  |
  |  `---------+----+----+                        `---------+------+                    |
  |        |  |                                  |                         |
  |        |  `----> +------------+                  v                        |
  |        |       | conv @2  | <- DEAD!    +------------+                  |
  |        |       | (unused) |  REMOVED   | conv @1   |                  |
  |        v       `-------------+            `-------+-------+                    |
  |  +------------+                                v                        |
  |  | conv @1   |                        +------------+                  |
  |  `-------+-------+                        | relu @2   |                    |
  |       v                                `-------+-------+                  |
  |  +------------+                                v                        |
  |  | relu @3   |                        +------------+                  |
  |  `-------+-------+                        | return @3 |                    |
  |       v                                `-------------+                  |
  |  +------------+                                                         |
  |  | return @4 |                   Saved: entire conv removed            |
  |  `-------------+                      (millions of FLOPs eliminated!)       |
  `--------------------------------------------------------------------------+

How does dead code get into models? Common reasons:
```

```
* Training branches (loss computation, gradient paths) left in the export
* Debug outputs that nobody reads during inference
* Auxiliary heads removed after export but ops left behind
* ONNX export producing redundant reshape/cast chains

3.2c. Constant Folding - "Pre-Compute at Compile Time"
Any computation that depends only on weights/constants (not on runtime input) can be computed once during compile() and stored as a co
BEFORE:
  @0 = param("weight")                    <- constant (known at compile time)
  @1 = transpose(@0)                      <- computed every inference!
  @2 = param("input")                     <- runtime input
  @3 = matmul(@2, @1)
AFTER constant folding:
  @0 = literal(pre_transposed_weight)     <- already transposed! stored as constant
  @1 = param("input")                     <- runtime input
  @2 = matmul(@1, @0)
  Saved: transpose operation removed from every inference run

  +-- DIAGRAM: Constant Folding (Compile-Time Pre-Computation) -------------+
  |                                                                          |
  |  BEFORE (runs transpose EVERY inference):                                |
  |  +--------------+     +---------------+     +------------+                |
  |  | weight @0   |---->| transpose @1|---->|          |                |
  |  | (constant)  |     | (runs every |     | matmul @3|--> output      |
  |  `---------------+     |  time!)     |     |          |                |
  |                        `----------------+  +-->|          |                |
  |  +--------------+                        |  `-------------+                |
  |  | input @2    |------------------------+                                |
  |  | (runtime)   |                                                          |
  |  `---------------+                                                          |
  |                                                                          |
  |  AFTER (transpose computed ONCE at compile, stored as literal):          |
  |  +-------------------+     +------------+                                |
  |  | pre_transposed   |---->|          |                                |
  |  | weight (literal) |     | matmul @2|--> output                      |
  |  | (computed once!) |  +-->|          |                                |
  |  `--------------------+  |  `-------------+                                |
  |  +--------------+       |                                                  |
  |  | input @1    |-------+   Saved: transpose gone from hot path!          |
  |  `---------------+                                                          |
  `--------------------------------------------------------------------------+

More examples:
FOLDABLE (computed once at compile):     NOT FOLDABLE (depends on input):
  reshape(constant_tensor)                reshape(input_tensor)
  transpose(weight_matrix)                matmul(input, weight)
  concat(weight_A, weight_B)              add(input, bias)
  slice(constant, 0:10)                   softmax(scores)
  cast(weight, to=fp16)                   layernorm(hidden_state)

3.2d. Other Optimization Passes
Pass
What It Does
Example
eliminate_common_subexpression
If two instructions compute the exact same thing, keep one
Two branches both computing relu(x) -> compute once, share result
simplify_reshapes
Remove reshape chains that cancel out
reshape(1,64,1) -> reshape(64) -> reshape(1,64,1) -> removed
simplify_algebra
Apply mathematical identities
x * 1.0 -> x ; x + 0 -> x ; x * 0 -> 0
rewrite_batchnorm
Convert BN into multiply + add
BN has 5 inputs; rewrite into x * scale + bias (2 ops, fusible)
rewrite_rnn
Unroll RNN/LSTM/GRU loops
Recurrent ops -> explicit unrolled computation graph
eliminate_contiguous
Remove unnecessary memory copies
If data is already contiguous in memory, skip the copy
propagate_constant
Push constants through the graph
If a reshape has constant inputs, fold it
simplify_qdq
Simplify quantize-dequantize chains
Remove redundant Q->DQ pairs for INT8 inference
eliminate_data_type
Remove unnecessary type casts
fp16->fp32->fp16 -> removed (no-op roundtrip)

  +-- DIAGRAM: Other Optimization Passes -- Quick Reference ------------------+
```

```
  |                                                                        |
  |  PASS                                WHAT IT DOES (ONE LINE)          |
  |  -------------------------           ---------------------------------  |
  |  eliminate_common_subexpr     ->    Two identical ops -> keep one        |
  |  simplify_reshapes            ->    reshape(reshape(x)) -> reshape(x)   |
  |  simplify_algebra             ->    x*1->x, x+0->x, x*0->0              |
  |  rewrite_batchnorm            ->    BN(5 inputs) -> mul+add (fusible)   |
  |  rewrite_rnn                  ->    Unroll LSTM/GRU -> explicit graph   |
  |  eliminate_contiguous         ->    Skip copy if already contiguous     |
  |  propagate_constant           ->    Push constants through the graph    |
  |  simplify_qdq                 ->    Remove redundant Q->DQ pairs        |
  |  eliminate_data_type          ->    fp16->fp32->fp16 -> removed (no-op)   |
  |                                                                        |
  |  Example: eliminate_common_subexpression                               |
  |  BEFORE:                           AFTER:                              |
  |  +--------+   +--------+            +--------+                            |
  |  |relu(x)|   |relu(x)|            |relu(x)|---+---> user A             |
  |  `------+-----+   `------+-----+            `---------+  `----> user B             |
  |      v           v                  (computed once, shared!)           |
  |   user A      user B                                                   |
  `-------------------------------------------------------------------------+
```

Optimization Pass Order Matters
MIGraphX runs these passes in a specific sequence, and some passes are run multiple times because earlier passes create new opportuniti
Pass execution order (simplified):

```
  (1) simplify_reshapes          <- clean up shape mess
  (2) simplify_algebra           <- mathematical simplifications
  (3) eliminate_common_subexpr   <- deduplicate
  (4) dead_code_elimination      <- remove unused ops
  (5) constant_folding           <- pre-compute constants
  (6) rewrite_batchnorm          <- convert BN to mul+add

     (repeat) REPEAT (1)-(6) until no more changes (fixed-point iteration)

  (7) fuse_pointwise             <- fuse element-wise chains
  (8) fuse_ops (MIOpen)          <- fuse conv patterns
  (9) fuse_ck                    <- fuse GEMM/attention (Composable Kernel)
  (10) fuse_mlir                  <- fuse remaining patterns (rocMLIR)
  (11) dead_code_elimination      <- clean up again after fusion
  (12) instruction_scheduling     <- order for parallel execution
  (13) memory_coloring            <- plan memory reuse (after scheduling)
```

## 3.3 Part 2: Instruction Scheduling

After optimization, MIGraphX has a flat list of instructions. Now it needs to decide the execution order and which instructions can ove

HIP Streams - Parallel Lanes on the GPU
AMD GPUs can execute multiple independent operations simultaneously using HIP streams. Think of streams as lanes on a highway - cars in
WITHOUT scheduling (sequential - one stream):

```
  Stream 0: --[Conv_branch1]--[Conv_branch2]--[Add]--[ReLU]--

  Total time: T(conv1) + T(conv2) + T(add) + T(relu)
```

WITH scheduling (parallel - multiple streams):

```
  Stream 0: --[Conv_branch1]-------------+
                                         |----[Add]--[ReLU]--
  Stream 1: --[Conv_branch2]-------------+

  Total time: max(T(conv1), T(conv2)) + T(add) + T(relu)

  If both convs take 1ms: sequential = 2ms, parallel = 1ms
```

How MIGraphX Schedules
MIGraphX analyzes the dependency graph to find independent instructions:
Dependency analysis:

```
  @0 = param("input")
  @1 = conv(@0, W1)         <- depends on @0
  @2 = conv(@0, W2)         <- depends on @0 (but NOT on @1!)
  @3 = pool(@0)             <- depends on @0 (but NOT on @1 or @2!)
  @4 = add(@1, @2)          <- depends on @1 AND @2 (sync point)
  @5 = concat(@4, @3)       <- depends on @4 AND @3 (sync point)
  MIGraphX discovers:
    @1, @2, @3 are INDEPENDENT -> can run on different streams
    @4 must WAIT for @1 and @2 -> insert synchronization event
    @5 must WAIT for @4 and @3 -> insert synchronization event
Schedule:
  Stream 0: --[@1: conv]----------------------[@4: add]--[@5: concat]--
  Stream 1: --[@2: conv]----------[event->]-----+         |
  Stream 2: --[@3: pool]-[event->]------------------------+

  [event->] = HIP event synchronization (one stream waits for another)
```

```
HIP Events - The Synchronization Mechanism
// MIGraphX generates code equivalent to (CUDA-style <<<>>> pseudocode):
hipStream_t stream0, stream1, stream2;
hipEvent_t event1, event2;
// Launch independent ops on different streams
conv_kernel<<<grid, block, 0, stream0>>>(input, W1, out1);  // @1
conv_kernel<<<grid, block, 0, stream1>>>(input, W2, out2);  // @2
pool_kernel<<<grid, block, 0, stream2>>>(input, out3);      // @3
// Record events when done
hipEventRecord(event1, stream1);  // stream1 signals completion
hipEventRecord(event2, stream2);  // stream2 signals completion
// Stream0 waits for stream1 before running @4
hipStreamWaitEvent(stream0, event1, 0);
add_kernel<<<grid, block, 0, stream0>>>(out1, out2, out4);  // @4
// Stream0 waits for stream2 before running @5
hipStreamWaitEvent(stream0, event2, 0);
concat_kernel<<<grid, block, 0, stream0>>>(out4, out3, final); // @5

Scheduling Constraints on AMD
Not everything can be parallelized. MIGraphX respects hardware limits:
Constraint                          What It Means
----------                          --------------
Limited GPU compute units           MI210 has 104 CUs, MI300X has 304 CUs.
                                    Two large convolutions might saturate
                                    all CUs, so parallel != faster.
MIOpen workspace memory             MIOpen may need temporary workspace.
                                    Two concurrent MIOpen calls might
                                    exceed available HBM.
rocBLAS internal streams            rocBLAS may use its own internal
                                    parallelism. Over-parallelizing can
                                    cause contention.
Memory bandwidth saturation         If all streams are memory-bound,
                                    parallelism hits HBM bandwidth wall.
MIGraphX uses a cost model to decide whether parallelism actually helps or hurts.

  +-- DIAGRAM: Multi-Stream Scheduling Timeline -----------------------------+
  |                                                                           |
  |  Time ----------------------------------------------------->             |
  |                                                                           |
  |  Stream 0: +---Conv_branch1---+         +---Add---++--Concat--++--Output--+ |
  |            `-------------------+         `----------+`-----------+`-----------+  |
  |                                         ^ wait    ^ wait              |
  |  Stream 1: +---Conv_branch2---+--[evt]---+  |       |                   |
  |            `-------------------+             |       |                    |
  |                                   |       |                    |
  |  Stream 2: +---MaxPool---+------[evt]---------------+                   |
  |            `--------------+                                              |
  |                                                                           |
  |  [evt] = hipEventRecord() -> hipStreamWaitEvent()                        |
  |                                                                           |
  |  Without multi-stream: Conv1 + Conv2 + Pool + Add + Concat = 5 serial |
  |  With multi-stream:    max(Conv1,Conv2,Pool) + Add + Concat = 3 steps |
  `---------------------------------------------------------------------------+


3.4 Part 3: Emits HIP Kernel Launch Calls
-----------------------------------------

What "Emit" Means
+---------------------------------------------------------------------------+
|  BEGINNER'S NOTE: What Does "Emit HIP Kernel Launch Calls" Mean?      |
|                                                                           |
|  "Emit" in compiler terminology means "to produce" or "to output."    |
|  When we say MIGraphX "emits HIP calls," we mean it generates and     |
|  executes function calls to the HIP library (the GPU remote control). |
|                                                                           |
|  Python analogy: imagine a build system that generates and runs        |
|  commands:                                                                |
|    commands = ["python train.py", "python test.py", "python deploy.py"]|
|    for cmd in commands:                                                   |
|        os.system(cmd)  # "emitting" and executing each command         |
|                                                                           |
|  MIGraphX does the same -- it walks through its execution plan and     |
|  for each step, calls the appropriate HIP function:                    |
|    - hipMemcpy() for data transfers                                     |
|    - hipLaunchKernel() for GPU computations                             |
|    - hipStreamWaitEvent() for synchronization                           |
|  These are the actual C++ function calls that make the GPU do work.   |
`---------------------------------------------------------------------------+
After compilation and scheduling, MIGraphX has a finalized execution plan - an ordered list of GPU operations to execute. When you call
Compiled MIGraphX Program (execution plan):
+-----------------------------------------------------------------+
```

```
|  Instruction 0: allocate 3.2 MB on GPU (output buffer)      |
|  Instruction 1: hipMemcpy input -> GPU                       |
|  Instruction 2: launch fused_conv_bn_relu on stream 0       |
|  Instruction 3: launch conv_branch2 on stream 1             |
|  Instruction 4: hipEventRecord(event1, stream1)             |
|  Instruction 5: hipStreamWaitEvent(stream0, event1)         |
|  Instruction 6: launch add_residual on stream 0             |
|  Instruction 7: launch fused_matmul_gelu on stream 0        |
|  Instruction 8: launch layernorm on stream 0                |
|  Instruction 9: hipMemcpy GPU -> output                      |
`-------------------------------------------------------------+
```

The Three Types of Kernel Launches
MIGraphX emits different types of GPU work depending on the instruction:

```
Type 1: Library kernel calls (MIOpen, rocBLAS, hipBLASLt)
// For a convolution instruction, MIGraphX calls MIOpen's API:
miopenConvolutionForward(
    handle,              // MIOpen handle (bound to a HIP stream)
    &alpha,              // scaling factor
    inputDesc,           // input tensor descriptor
    input_gpu_ptr,       // pointer to input in GPU HBM
    filterDesc,          // weight tensor descriptor
    weight_gpu_ptr,      // pointer to weights in GPU HBM
    convDesc,            // convolution parameters (padding, stride)
    algo,                // algorithm chosen during auto-tuning
    workspace_gpu_ptr,   // temporary workspace in GPU HBM
    workspace_size,      // workspace size in bytes
    &beta,               // scaling factor
    outputDesc,          // output tensor descriptor
    output_gpu_ptr       // pointer to output in GPU device memory
);
// MIOpen internally launches the GPU kernel:
//   hipLaunchKernelGGL(ImplicitGemmV4R4, grid, block, shared, stream, ...)

Type 2: JIT-compiled custom kernels (via hipRTC, which uses COMGR internally)
// For fused pointwise ops, MIGraphX already compiled a custom kernel.
// Now it just launches it:
hipModuleLaunchKernel(
    fused_pointwise_function,    // pre-compiled GPU function handle
    grid.x, grid.y, grid.z,     // grid dimensions
    block.x, block.y, block.z,  // block dimensions (e.g., 256 threads)
    shared_mem_bytes,            // shared memory per block
    stream,                      // which HIP stream
    kernel_args,                 // pointers to GPU memory buffers
    nullptr
);

Type 3: rocMLIR-compiled kernels
// For MLIR-fused subgraphs, the compiled code object is already loaded.
// MIGraphX launches it the same way as JIT kernels - via hipModuleLaunchKernel

What a Full Inference Execution Looks Like
session.run(input_data)
Time ---------------------------------------------------------->
CPU thread:
  | hipMemcpyAsync(gpu_input, cpu_input, size, H2D, stream0)
  | launch_conv1(stream0)           <- MIOpen call
  | launch_conv_branch(stream1)     <- MIOpen call (parallel branch)
  | hipEventRecord(ev1, stream1)
  | hipStreamWaitEvent(stream0, ev1)
  | launch_add_relu(stream0)        <- JIT kernel
  | launch_matmul(stream0)          <- rocBLAS call
  | launch_layernorm(stream0)       <- JIT kernel
  | launch_softmax(stream0)         <- JIT kernel
  | launch_matmul(stream0)          <- hipBLASLt call
  | hipMemcpyAsync(cpu_output, gpu_output, size, D2H, stream0)
  | hipStreamSynchronize(stream0)   <- wait for GPU to finish
  |
  v return output_data
GPU (stream 0):     [Conv1]-------------[wait]-[add+relu]-[matmul]-[LN]-[softmax]-[matmul]-
GPU (stream 1):     [Conv_branch]-[ev1->]
GPU (DMA engine):   [H2D copy]                                                [D2H copy]
Note: CPU issues ALL commands almost instantly (~microseconds)
      GPU executes them asynchronously (takes milliseconds)
      hipStreamSynchronize is where CPU actually waits for results
```

## 3.5 The Big Picture - Where Layer 4 Sits

```
+-------------------------------------------------------------+
|  YOUR CODE: session.run(input)                               |
```

```
|-----------------------------------------------------------------|
|  ORT: manages session, feeds subgraph to MIGraphX EP          |
|-----------------------------------------------------------------|
|  MIGraphX EP: calls parse -> quantize -> compile               |
|-----------------------------------------------------------------|
|                                                                 |
|  libmigraphx.so  <--- YOU ARE HERE (Layer 4)                |
|                                                                 |
|  +-------------------------------------------------------+ |
|  |  COMPILE TIME (once)                                  | |
|  |  (1) Parse ONNX -> build instruction graph              | |
|  |  (2) Run 50+ optimization passes (fusion, DCE, etc.)      | |
|  |  (3) Select/compile kernels (MIOpen, hipBLASLt, JIT/CK)  | |
|  |  (4) Schedule instructions across HIP streams             | |
|  |  (5) Plan memory layout (memory_coloring for reuse)       | |
|  `-------------------------------------------------------+ |
|                                                                 |
|  +-------------------------------------------------------+ |
|  |  RUN TIME (every inference)                           | |
|  |  (1) Copy input to GPU (hipMemcpy H2D)                    | |
|  |  (2) Walk the execution plan, emit HIP kernel launches    | |
|  |  (3) Synchronize streams at dependency points             | |
|  |  (4) Copy output from GPU (hipMemcpy D2H)                 | |
|  `-------------------------------------------------------+ |
|                                                                 |
|-----------------------------------------------------------------|
|  AMD GPU Hardware: actually executes the kernels              |
`-----------------------------------------------------------------+
```

Key Takeaway
libmigraphx.so is the orchestrator. It doesn't do the raw math itself - it figures out the smartest possible plan for how, when, and wh

## 3.6 Source Code Walkthrough -- MIGraphX Library Internals

C API Entry Points (api.cpp)
All three EP calls land in AMDMIGraphX/src/api/api.cpp. Each is an
extern "C" function wrapped in migraphx::try_() for exception safety.

```
// api.cpp:2109-2121 -- Parse ONNX buffer
extern "C" migraphx_status
migraphx_parse_onnx_buffer(migraphx_program_t* out,
                           const void* data, size_t size,
                           migraphx_onnx_options_t options) {
    auto api_error_result = migraphx::try_([&] {
        if(options == nullptr)
            MIGRAPHX_THROW(migraphx_status_bad_param,
                "Bad parameter options: Null pointer");
        *out = allocate<migraphx_program_t>(
            migraphx::parse_onnx_buffer(data, size, options->object));
    });
    return api_error_result;
}

// api.cpp:1714-1728 -- Full compilation
extern "C" migraphx_status
migraphx_program_compile(migraphx_program_t program,
                         migraphx_target_t target,
                         migraphx_compile_options_t options) {
    auto api_error_result = migraphx::try_([&] {
        // Null-checks all 3 params, then:
        (program->object).compile(target->object, options->object);
    });
    return api_error_result;
}

// api.cpp:1774-1786 -- Inference hot path
extern "C" migraphx_status
migraphx_program_run(migraphx_arguments_t* out,
                     migraphx_program_t program,
                     migraphx_program_parameters_t params) {
    auto api_error_result = migraphx::try_([&] {
        // Null-checks, then:
        *out = allocate<migraphx_arguments_t>(
            migraphx::run(program->object, params->object));
    });
    return api_error_result;
}
```

ONNX Parser (onnx.cpp)
File: AMDMIGraphX/src/onnx/onnx.cpp

```
  // parse_onnx_from -- template that does the actual work
  template <class... Ts>
  static program parse_onnx_from(const onnx_options& options, Ts&&... xs) {
      onnx::onnx_parser parser;
      parser.external_data_path   = options.external_data_path;
      parser.map_input_dims       = options.map_input_dims;
      parser.dim_params           = options.dim_params;
      parser.map_dyn_input_dims   = options.map_dyn_input_dims;
      parser.skip_unknown_operators = options.skip_unknown_operators;
      parser.max_loop_iterations  = options.max_loop_iterations;
      // Validates: can't set both static + dynamic dims
      parser.parse_from(std::forward<Ts>(xs)...);
      return std::move(parser.prog);  // complete MIGraphX IR
  }

FP16 Quantization Pipeline (quantization.cpp)
  File: AMDMIGraphX/src/quantization.cpp

  void quantize_fp16(program& prog,
                     const std::vector<std::string>& ins_names) {
      run_passes(prog,
                 {normalize_ops{},                                // Pass 1: standardize ops
                  optimize_module{{"quantizelinear",
                                   "dequantizelinear"}},          // Pass 2: constant fold in FP32
                  truncate_float_pass{ins_names, shape::half_type}, // Pass 3: FP32 -> FP16
                  optimize_module{{"quantizelinear",
                                   "dequantizelinear"}}},         // Pass 4: cleanup
                 quant_tracer());  // enabled via MIGRAPHX_TRACE_QUANTIZATION
  }

  BF16 uses identical pattern with shape::bf16_type (gfx942+ only).

program::compile -- The Compilation Driver (program.cpp)
  File: AMDMIGraphX/src/program.cpp:313-346

  void program::compile(const target& t, compile_options options) {
      assert(not this->is_compiled());
      this->impl->targets  = {t};
      this->impl->contexts = {t.get_context()};

      if(enabled(MIGRAPHX_TRACE_COMPILE{}))
          options.trace = tracer{std::cout};    // env var tracing

      auto&& passes = t.get_passes(this->impl->contexts.front(), options);
      run_passes(*this, passes, options.trace); // * ALL 50+ PASSES RUN HERE

      for(const auto& mod : reverse(this->get_modules())) {
          auto invalid = mod->validate();       // check every instruction
          if(invalid != mod->end())
              MIGRAPHX_THROW("Invalid module ...");
          mod->finalize(this->impl->contexts);  // lock in compilation
      }
  }

program::eval -- The Inference Hot Path (program.cpp)
  File: AMDMIGraphX/src/program.cpp:588-668

  std::vector<argument> program::eval(
      const parameter_map& params, execution_environment exec_env) const {
      auto trace_level = value_of(MIGRAPHX_TRACE_EVAL{});
      if(exec_env.async) {
          contexts.front().wait_for(exec_env.queue);  // wait on HIP stream
      }
      if(trace_level > 0) {
          // Per-instruction timing: prints instruction name, ms elapsed
          ret = generic_eval(*this, contexts, params, [&](auto ins, auto f) {
              ctx.finish();
              timer t{};
              auto result = f();               // run one instruction
              double t1 = t.record<milliseconds>();
              // Level 2: also print output statistics (min, max, mean)
              return result;
          });
      } else {
          // Production path: no tracing overhead
          ret = generic_eval(*this, contexts, params,
              [&](auto&&, auto f) { return f(); });
      }
      if(exec_env.async)
          contexts.front().finish_on(exec_env.queue);
      return ret;
  }
```

```
target::get_passes -- The 50+ Pass Pipeline (target.cpp)
  File: AMDMIGraphX/src/targets/gpu/target.cpp:94-277

  +-- DIAGRAM: MIGraphX 50+ Pass Compilation Pipeline ----------------------+
  |                                                                          |
  |  ONNX protobuf -> parse_onnx_buffer() -> MIGraphX IR (unoptimized)     |
  |       |                                                                  |
  |       v  PHASE 1: CLEANUP                                               |
  |  +-----------------------------------------------------+                 |
  |  | split_single_dyn_dim -> dead_code_elimination ->      |                 |
  |  | simplify_dyn_ops -> normalize_ops -> eliminate_identity|                 |
  |  `---------------------------+----------------------------+                 |
  |       |                                                                  |
  |       v  PHASE 2: TYPE HANDLING                                         |
  |  +-----------------------------------------------------+                 |
  |  | detect unsupported types -> convert_to_supported_types|                 |
  |  | (ensure FP32/FP16/INT8/INT32/BOOL)                  |                 |
  |  `---------------------------+----------------------------+                 |
  |       |                                                                  |
  |       v  PHASE 3: FUSION (biggest perf impact)                          |
  |  +-----------------------------------------------------+                 |
  |  | fuse_attention (MHA -> 1 kernel) ->                   |                 |
  |  | fuse_pointwise_reduce -> fuse_ck -> fuse_mlir ->      |                 |
  |  | fuse_concat                                          |                 |
  |  `---------------------------+----------------------------+                 |
  |       |                                                                  |
  |       v  PHASE 4: LOWERING & GPU COMPILATION                            |
  |  +-----------------------------------------------------+                 |
  |  | lowering (abstract -> gpu ops) ->                     |                 |
  |  | compile_miopen (conv) -> fuse_ops ->                  |                 |
  |  | compile_hipblaslt (GEMM) -> replace_allocate ->       |                 |
  |  | compile_ops (hipRTC -> COMGR -> GPU ISA)              |                 |
  |  `---------------------------+----------------------------+                 |
  |       |                                                                  |
  |       v  PHASE 5: SCHEDULING & MEMORY                                   |
  |  +-----------------------------------------------------+                 |
  |  | promote_literals -> write_literals ->                 |                 |
  |  | schedule (multi-stream) -> memory_coloring ->         |                 |
  |  | sync_device -> preallocate_param -> check_context     |                 |
  |  `---------------------------+----------------------------+                 |
  |       |                                                                  |
  |       v                                                                  |
  |  Fully compiled program (kernel handles + memory plan + schedule)      |
  `--------------------------------------------------------------------------+

  Phase 1 -- Cleanup & Normalization:
    split_single_dyn_dim, dead_code_elimination, simplify_dyn_ops,
    normalize_ops, eliminate_identity

  Phase 2 -- Type Handling:
    unsupported_types set built from GPU capabilities:
      FP32, FP16, BOOL, INT8, INT32 always supported
      BF16 if gfx_has_bf16_intrinsics() (gfx942+)
      FP8 if gfx_has_fp8fnuz/fp8ocp_intrinsics()
    convert_to_supported_types pass applied

  Phase 3 -- Fusion:
    fuse_attention       -- Q.K^T + scale + mask + softmax + V -> one kernel
    fuse_pointwise_reduce -- element-wise + reduction ops
    fuse_ck              -- Composable Kernel (gated by MIGRAPHX_ENABLE_CK)
    fuse_mlir            -- MLIR-based GEMM fusion
    fuse_concat          -- concatenation optimization

  Phase 4 -- Lowering & Library Compilation:
    lowering{&ctx}       -- abstract ops -> GPU ops (gpu::gemm, gpu::conv)
    compile_miopen{&gctx} -- Conv via MIOpen (AMD's cuDNN equivalent)
    fuse_ops{&ctx}       -- fuse remaining pointwise chains
    compile_hipblaslt{&gctx} -- GEMM with fused epilogues
    replace_allocate     -- insert hip::allocate for every buffer
    compile_ops{&ctx}    -- custom HIP kernels via hipRTC -> Layer 5

  Phase 5 -- Scheduling & Memory:
    promote_literals     -- hoist constants for GPU transfer
    write_literals{&ctx} -- copy constants to GPU
    schedule{gpu::schedule_model{nstreams}} -- multi-stream scheduling
    memory_coloring{"hip::allocate"} -- buffer reuse (50-80% savings)
    sync_device          -- insert stream synchronization barriers
    preallocate_param{"scratch"} -- pre-allocate scratch buffers
    eliminate_allocation{"hip::allocate"} -- remove dead allocations
    check_context        -- final sanity check

hipRTC Kernel Compilation (compile_hip.cpp)
  File: AMDMIGraphX/src/targets/gpu/compile_hip.cpp
```

```
std::vector<std::vector<char>>
compile_hip_src_with_hiprtc(std::vector<hiprtc_src_file> srcs,
                            const std::vector<std::string>& params,
                            const std::string& arch, bool quiet) {
    hiprtc_program prog(std::move(srcs));
    auto options = params;
    options.push_back("-DMIGRAPHX_USE_HIPRTC=1");
    options.push_back("-std=c++17");
    options.push_back("-fno-gpu-rdc");
    options.push_back("-O" + string_value_of(MIGRAPHX_GPU_OPTIMIZE{}, "3"));
    options.push_back("--offload-arch=" + arch);  // e.g., gfx1151
    // MIGRAPHX_GPU_HIP_FLAGS env var appended here
    std::vector<std::string> extra_flags =
        split_string(string_value_of(MIGRAPHX_GPU_HIP_FLAGS{}, ""), ' ');
    options.insert(options.end(), extra_flags.begin(), extra_flags.end());
    prog.compile(options, quiet);  // -> hiprtcCompileProgram() -> Layer 5
    return {prog.get_code_obj()};  // ELF binary
}

// The prog.compile() method (same file):
void compile(const std::vector<std::string>& options, bool quiet) const {
    auto result = hiprtcCompileProgram(
        prog.get(), c_options.size(), c_options.data());
    auto prog_log = log();
    if(not prog_log.empty() and not quiet)
        std::cerr << prog_log << std::endl;
    if(result != HIPRTC_SUCCESS)
        MIGRAPHX_HIPRTC_THROW(result, "Compilation failed.");
}
```

## 3.7 Where Every Byte Lives -- Layer 4 (MIGraphX Library)

```
+-----------------------------------------------------------------+
|  COMPILE TIME                                                    |
|                                                                  |
|  migraphx::program IR (instruction list)                        |
|    Where: C++ heap                                               |
|    Contains: ordered instruction_ref list, each with:            |
|      operation (e.g., gpu::gemm, gpu::code_object),              |
|      input argument references, output shape                     |
|                                                                  |
|  Literal constants (model weights, biases)                      |
|    Where: initially C++ heap, copied to GPU by write_literals   |
|    GPU location: hip::allocate buffer in VRAM/GTT               |
|    Lifetime: persists for entire session                         |
|                                                                  |
|  memory_coloring allocation map                                 |
|    Where: C++ heap (metadata only)                               |
|    Contains: offset assignments for buffer reuse                 |
|    Result:  50-80% fewer physical GPU allocations                |
|                                                                  |
|  Compiled kernels (ELF code objects)                            |
|    Where: returned from hipRTC as std::vector<char>              |
|    Then:  loaded to GPU via hipModuleLoadData -> GPU instruction  |
|           cache (SRAM) + code object heap (VRAM)                |
|    Cached: /tmp/migraphx/ on disk (1-11 GB per model)           |
|                                                                  |
|  Scratch buffer                                                  |
|    Where: single pre-allocated GPU buffer (hip::allocate)        |
|    Size:  determined by preallocate_param pass                   |
|    Reused: by all instructions that need temporary workspace     |
|                                                                  |
|  RUN TIME (every inference)                                     |
|                                                                  |
|  generic_eval walks instruction list:                            |
|    Each instruction either:                                      |
|      a) Launches GPU kernel via hipLaunchKernel (-> Layer 5)     |
|      b) Calls library (MIOpen/hipBLASLt) -> library launches GPU |
|      c) Performs CPU-side metadata operation (no GPU launch)     |
|    Results accumulate in GPU buffers (device pointers)           |
|    Final output: migraphx::argument with GPU device pointer     |
`-----------------------------------------------------------------+
```

# PART II -- THE RUNTIME LAYERS

```
  CHAPTER 4: HIP Runtime, libamdhip64.so (Layer 5)
  The GPU Remote Control -- Memory, Kernels, and Synchronization
==================================================================================

+-- EXECUTIVE SUMMARY -- Layer 5: HIP Runtime --------------------------+
|                                                                        |
|  What:  AMD's GPU programming API (analogous to CUDA runtime).       |
|  Input: Memory allocation requests, kernel launch commands from L4. |
|  Output: GPU memory pointers, enqueued kernel dispatches to L6.      |
|                                                                        |
|  Three core operations:                                               |
|    hipMalloc / hipMallocManaged -- allocate device / unified memory  |
|    hipLaunchKernel             -- dispatch GPU kernel to a stream    |
|    hipDeviceSynchronize        -- wait for all GPU work to complete  |
|                                                                        |
|  Also handles:                                                        |
|    hipMemcpy (H2D/D2H/D2D transfers), hipStreamCreate (async ops), |
|    hiprtcCompileProgram (runtime JIT: HIP C++ -> GPU ISA via COMGR) |
|                                                                        |
|  APU-specific behavior:                                               |
|    hipMalloc()        -> 512 MB VRAM carve-out only                  |
|    hipMallocManaged() -> unified VA across VRAM + GTT + swap         |
|    hipMemcpy(H2D)     -> page table remap (no PCIe copy on APU)    |
|                                                                        |
|  Translates to HSA Runtime (Layer 6) internally -- all hip* calls   |
|  become hsa_* calls, AQL packets, and doorbell writes.             |
|                                                                        |
|  Key files: hip_memory.cpp, hip_module.cpp, hiprtc.cpp (clr repo)  |
`------------------------------------------------------------------------+

  +-- SOURCE REFERENCES -- Layer 5 (HIP Runtime) ----------------------------+
  |                                                                           |
  |  Repository: https://github.com/ROCm/clr                                |
  |  Branch:     rocm-7.2.0                                                  |
  |                                                                           |
  |  File                                     Line  Key Function            |
  |  ----------------------------------------- ----- ---------------------  |
  |  hipamd/src/hip_memory.cpp                778   hipMalloc()             |
  |  hipamd/src/hip_memory.cpp                338   ihipMalloc() (real)     |
  |  hipamd/src/hip_memory.cpp                817   hipMemcpy()             |
  |  hipamd/src/hip_memory.cpp                1034  hipMallocManaged()      |
  |  hipamd/src/hip_module.cpp                534   hipModuleLaunchKernel()|
  |  hipamd/src/hip_module.cpp                352   ihipLaunchKernelCmd()  |
  |  hipamd/src/hip_module.cpp                493   command->enqueue()      |
  |  hipamd/src/hiprtc/hiprtc.cpp             68    hiprtcCreateProgram()  |
  |  hipamd/src/hiprtc/hiprtc.cpp             115   hiprtcCompileProgram() |
  |                                                                           |
  |  GitHub links:                                                            |
  |  https://github.com/ROCm/clr/blob/rocm-7.2.0/                          |
  |    hipamd/src/hip_memory.cpp#L778       (hipMalloc)                      |
  |    hipamd/src/hip_memory.cpp#L338       (ihipMalloc)                     |
  |    hipamd/src/hip_module.cpp#L534       (hipModuleLaunchKernel)          |
  |    hipamd/src/hiprtc/hiprtc.cpp#L115    (hiprtcCompileProgram)           |
  `---------------------------------------------------------------------------+

This is the last software layer before we hit actual hardware. HIP is AMD's GPU programming API - the hands that physically reach into

+-----------------------------------------------------------------------------+
|  BEGINNER'S CONTEXT: Why Do We Need HIP?                                  |
|                                                                             |
|  In Python, when you do numpy.dot(a, b), NumPy handles everything:    |
|  allocating memory, computing the result, returning it. Easy!          |
|                                                                             |
|  On a GPU, these are SEPARATE steps that someone must coordinate:      |
|    1. Allocate memory on the GPU         -> hipMalloc()                 |
|    2. Copy your data from CPU to GPU     -> hipMemcpy()                 |
|    3. Tell GPU to run a computation      -> hipLaunchKernel()           |
|    4. Wait for GPU to finish             -> hipDeviceSynchronize()      |
|    5. Copy result back from GPU to CPU   -> hipMemcpy()                 |
|    6. Free GPU memory                    -> hipFree()                   |
|                                                                             |
|  HIP is the library that provides all these functions.                 |
|  You (as a Python user) never call HIP directly -- MIGraphX does it    |
|  for you. But understanding HIP helps you understand what happens      |
|  "behind the curtain" when you call session.run().                     |
|                                                                             |
|  HIP is AMD's answer to NVIDIA's CUDA. They have nearly identical     |
|  APIs. If you have ever seen CUDA code, HIP code looks the same.     |
`-----------------------------------------------------------------------------+


4.1 What is HIP?
----------------
```

```
HIP = Heterogeneous-compute Interface for Portability
HIP provides GPU programming functions (allocate memory, launch kernels,
synchronize) with an API that mirrors CUDA nearly 1:1. On AMD GPUs, HIP
runs natively through the ROCm stack. For NVIDIA GPUs, AMD provides a
source-level porting tool called "hipify" that converts HIP source code
into CUDA -- but the actual runtime binary is different on each vendor.
HIP is NOT a universal binary that runs on both; it is a common API
surface that makes porting straightforward.
NVIDIA world:           AMD world:
  cudaMalloc()     ->      hipMalloc()
  cudaMemcpy()     ->      hipMemcpy()
  cudaLaunchKernel ->      hipLaunchKernel()
  cudaFree()       ->      hipFree()
  cudaStream_t     ->      hipStream_t
Nearly 1:1 mapping. That's by design - HIP was built to make
porting from CUDA painless.

Where HIP Sits in the Stack
+-----------------------------------------------+
|  MIGraphX / MIOpen / rocBLAS                |  <- call HIP functions
|-----------------------------------------------|
|                                               |
|  libamdhip64.so  <-- YOU ARE HERE           |
|  (HIP Runtime)                                |
|                                               |
|  Responsibilities:                            |
|    * GPU memory management                    |
|    * Kernel dispatch (launch)                 |
|    * Stream & event synchronization           |
|    * Device management (which GPU?)           |
|    * Error handling                           |
|                                               |
|-----------------------------------------------|
|  HSA Runtime (libhsa-runtime64.so)          |  <- HIP calls into HSA
|-----------------------------------------------|
|  AMD GPU Kernel Driver (amdgpu.ko)          |  <- kernel-space
|-----------------------------------------------|
|  AMD GPU Hardware (MI210/MI300X)            |  <- silicon
`-----------------------------------------------+

The Three Core Responsibilities
HIP does many things, but for ML inference, three functions dominate:
(1) hipMalloc / hipMallocManaged   -> "Reserve a parking spot on the GPU"
(2) hipLaunchKernel               -> "Tell the GPU to do work"
(3) hipDeviceSynchronize           -> "Wait until the GPU is done"
Let's go deep on each.


4.2 Part 1: GPU Memory - hipMalloc() / hipMallocManaged()
---------------------------------------------------------

The Two Separate Memory Worlds
Discrete GPUs (MI210, MI300X) have separate memory from the CPU:
+-----------------------+          +---------------------------+
|     CPU (Host)       |          |   AMD GPU (Device)       |
|                      |          |                           |
|  DDR5 System RAM     |  PCIe   |  Device Memory           |
|  64-512 GB           |<=======>|  HBM on MI210 (64 GB)    |
|  ~90 GB/s bandwidth  | ~64GB/s |  HBM on MI300X (192 GB)  |
|                      |         |  ~1,600-5,300 GB/s       |
|  your Python data    |         |                           |
|  lives here          |         |  GPU code operates here  |
|                      |         |                           |
|  CPU can't directly  |         |  GPU can't directly       |
|  access GPU memory   |         |  access CPU memory*       |
`-----------------------+          `-----------------------------+
* MI300X and APUs like 8060S share memory -- explained below

On your Radeon 8060S (APU): CPU and GPU share the SAME LPDDR5x RAM.
There is no separate HBM. hipMalloc allocates from a 512 MB carve-out
of the shared LPDDR5x pool. The "two separate worlds" diagram above applies
to discrete GPUs; your APU has unified memory (similar to MI300X).

hipMalloc() - Allocate on GPU
hipMalloc(void** ptr, size_t size)
What it does:
  Reserves a block of device memory (HBM on discrete GPUs like MI210,
  VRAM carve-out from LPDDR5x on APUs like Radeon 8060S).
  Returns a DEVICE pointer - only the GPU can use it.
Analogy:
  Like reserving a table at a restaurant.
  The table (memory) is physically inside the restaurant (GPU).
```

```
  You (CPU) can't sit at it - only the chefs (GPU cores) can use it.
What happens inside hipMalloc:
Your code:  hipMalloc(&d_ptr, 1024 * 1024)  // 1 MB on GPU
Internally:
  +-----------------------------------------------------+
  | HIP Runtime (libamdhip64.so)                        |
  |   1. Check: is 1 MB available in the pool?          |
  |      |---- YES -> return pointer from pool (fast!)   |
  |      `---- NO  -> ask HSA runtime for new memory     |
  |                                                      |
  | HSA Runtime (libhsa-runtime64.so)                    |
  |   2. Calls hsa_amd_memory_pool_allocate()           |
  |   3. This talks to the kernel driver                |
  |                                                      |
  | Kernel Driver (amdgpu.ko)                           |
  |   4. Programs the GPU's memory controller           |
  |   5. Reserves physical pages in device memory        |
  |      (HBM on discrete GPUs, LPDDR5x on APUs)        |
  |   6. Sets up page table entries so GPU can           |
  |      access this address                             |
  |                                                      |
  | Returns: device pointer 0x7f1234000000              |
  `-----------------------------------------------------+

  Time cost:
    From pool:    ~1-5 mus (microseconds)
    Fresh alloc:  ~50-500 mus (much slower!)

  This is why MIGraphX pre-allocates a memory pool!

hipMallocManaged() - Unified Memory
hipMallocManaged(void** ptr, size_t size)
What it does:
  Allocates memory that BOTH CPU and GPU can access using
  the SAME pointer. The system auto-migrates data as needed.
Analogy:
  Like a shared Google Doc - both people can read/edit it.
  The system handles syncing behind the scenes.
How managed memory works:
hipMallocManaged(&ptr, size);
// CPU writes data
ptr[0] = 42;                // data lives in CPU RAM initially
// GPU reads it
gpu_kernel<<<grid, block>>>(ptr);  // CUDA-style pseudocode (HIP uses hipLaunchKernelGGL or similar)
// The system detects GPU needs this page
// AUTOMATICALLY migrates it: CPU RAM -> device memory (via PCIe on discrete GPUs)
// On APUs like 8060S: page table remap, no physical copy needed
// CPU reads result
hipDeviceSynchronize();
printf("%d\n", ptr[0]);
// On discrete GPUs: system migrates back via PCIe
// On APUs: just a page table update
Performance implications:
hipMalloc + explicit hipMemcpy:      hipMallocManaged:
  [Y] YOU control when data moves        [N] System decides (may be suboptimal)
  [Y] Can overlap copy + compute         [N] Page faults cause stalls
  [Y] Predictable performance            [N] First access can be very slow
  [N] More code to write                 [Y] Simpler code
For ML inference: explicit hipMalloc + hipMemcpy is almost always used.
Managed memory is mainly for prototyping.

  +-- DIAGRAM: hipMalloc vs hipMallocManaged --------------------------------+
  |                                                                           |
  |  hipMalloc (explicit copy):     hipMallocManaged (unified address):    |
  |                                                                           |
  |  CPU RAM        GPU VRAM        CPU RAM <----- same -----> GPU VRAM     |
  |  +--------+       +--------+        +---------------------------------+     |
  |  | h_ptr|       | d_ptr|        |        managed_ptr              |     |
  |  |[data]|       |[????]|        |  [data wherever it is needed]  |     |
  |  `-----+-----+       `-----+-----+        `---------------------------------+     |
  |     |              |                      |                                 |
  |     | hipMemcpy()  |             CPU writes: data in CPU RAM           |
  |     `-----------------+             GPU reads:  page fault -> migrate      |
  |    YOU control when              SYSTEM decides when to move            |
  |    data moves                    (auto page migration on access)       |
  |                                                                           |
  |  On APU (your 8060S):                                                   |
  |  +-----------------------------------------+                            |
  |  |  LPDDR5x (shared physical memory)      |                            |
  |  |                                         |                            |
  |  |  hipMalloc:        512 MB VRAM only     |  <- NOT ENOUGH for LLMs   |
  |  |  hipMallocManaged: VRAM + GTT + swap   |  <- CAN fit 7B+ models    |
  |  |                    (~30 GB+ available)  |                            |
```

```
  |  `-----------------------------------------+                            |
  `-----------------------------------------------------------------------------+

MI300X Special Case - Unified HBM
MI300X has a unique architecture where CPU and GPU share the same physical HBM:
TRADITIONAL (MI210):                MI300X (APU-like):
+----------+    PCIe    +----------+   +----------------------------+
|  CPU   |<==========>|  GPU   |   |    CPU + GPU Chiplets     |
|  DDR5  |  ~64 GB/s  |  HBM2e |   |         |                 |
`-----------+            `-----------+   |    +-----v-----+            |
                                    |    |  Shared  |            |
  Must copy data back and forth     |    |  HBM3    |            |
  over slow PCIe bus                |    |  192 GB  |            |
                                    |    | 5.3 TB/s |            |
                                    |    `-------------+            |
                                    `-------------------------------+

                                    CPU and GPU access SAME memory!
                                    No PCIe copy needed!
                                    hipMallocManaged is nearly free!

Memory Types Summary
Function              Where Memory Lives    Who Can Access    Speed to GPU
--------              ------------------    --------------    ------------
hipMalloc()           GPU device memory      GPU only          Fastest (local)
hipHostMalloc()       CPU RAM (pinned)      CPU + GPU(slow)   Slow (PCIe)
hipMallocManaged()    Auto-migrated         CPU + GPU         Depends on location
hipHostRegister()     Existing CPU RAM      CPU + GPU(slow)   Slow (PCIe)

+-----------------------------------------------------------------------------+
|  APU NOTE (Radeon 8060S): On your system, CPU and GPU share the same  |
|  LPDDR5x RAM. There is no separate HBM or GDDR6. This means:          |
|                                                                         |
|  - hipMalloc() allocates from the SHARED LPDDR5x pool (512 MB carve-out)|
|  - hipMemcpy(H2D) does NOT cross PCIe -- it is a page table remap or  |
|    a DMA copy within the SAME physical memory, much faster than PCIe  |
|  - hipMallocManaged() is nearly free -- already unified memory          |
|  - Your bandwidth bottleneck is LPDDR5x (~256 GB/s), not PCIe         |
|                                                                         |
|  Everywhere this chapter says "PCIe" or "HBM", mentally substitute:   |
|    "HBM" -> "VRAM carve-out from LPDDR5x"                              |
|    "PCIe copy" -> "memory controller remap (much faster)"              |
|    "64 GB/s PCIe" -> "256 GB/s LPDDR5x (shared with CPU)"             |
`-----------------------------------------------------------------------------+

Memory Lifecycle in MIGraphX Inference
COMPILE TIME:
  MIGraphX calls hipMalloc() to reserve GPU memory for:
    * Model weights:     hipMalloc(&weights, 14 GB)     // 7B model in FP16
    * Workspace buffers: hipMalloc(&workspace, 512 MB)  // MIOpen scratch space
    * Output buffers:    hipMalloc(&outputs, 64 MB)     // intermediate results
  Then copies weights from CPU to GPU:
    hipMemcpy(weights_gpu, weights_cpu, 14 GB, hipMemcpyHostToDevice)
    (This takes ~220 ms over PCIe Gen4 at 64 GB/s)
RUN TIME (every inference):
  hipMemcpyAsync(input_gpu, input_cpu, 600 KB, H2D, stream)  // input image
  ... kernel launches (weights already on GPU, no copy needed!) ...
  hipMemcpyAsync(output_cpu, output_gpu, 4 KB, D2H, stream)  // class scores

  Note: weights stay on GPU permanently - no re-copying!


4.3 Part 2: Kernel Launch - hipLaunchKernel()
---------------------------------------------

This is the most important HIP function for ML inference. It tells the GPU to execute a piece of code (a "kernel").

What is a GPU Kernel?
A kernel is a function that runs on thousands of GPU threads simultaneously.
CPU code (sequential):              GPU kernel (massively parallel):
for (int i = 0; i < 1000000; i++)   __global__ void relu(half* data, int n) {
    if (data[i] < 0)                    int i = threadIdx.x + blockIdx.x * 256;
        data[i] = 0;                    if (i < n && data[i] < 0)
                                            data[i] = 0;
                                    }

1 thread, 1 million iterations      262,144 threads, ~4 iterations each
Time: ~1 ms                         Time: ~0.002 ms (500x faster)

How hipLaunchKernel Works
hipLaunchKernelGGL(
    relu_kernel,           // which kernel to run
    dim3(1024, 1, 1),      // grid: 1024 blocks
```

```
    dim3(256, 1, 1),       // block: 256 threads per block
    0,                     // shared memory per block (bytes)
    stream,                // which HIP stream
    data_ptr,              // kernel argument 1: GPU data pointer
    n                      // kernel argument 2: data size
);
Total threads = 1024 blocks x 256 threads = 262,144 threads launched

The GPU Execution Hierarchy
Understanding how threads are organized is key to understanding kernel launches:
GRID (entire kernel launch)
|---- Block 0   (CDNA example -- Wave64; RDNA uses Wave32, so 8 wavefronts here)
|   |---- Wavefront 0 (64 threads)    <- AMD's basic execution unit
|   |---- Wavefront 1 (64 threads)       (NVIDIA calls this "warp" = 32 threads)
|   |---- Wavefront 2 (64 threads)       (RDNA uses Wave32 = 32 threads/wavefront)
|   `---- Wavefront 3 (64 threads)
|       Total: 256 threads in Block 0
|---- Block 1
|   |---- Wavefront 0 (64 threads)
|   |---- Wavefront 1 (64 threads)
|   |---- Wavefront 2 (64 threads)
|   `---- Wavefront 3 (64 threads)
|---- Block 2
|   `---- ...
`---- Block 1023
    `---- ...
Total: 1024 blocks x 256 threads = 262,144 threads

What Happens Inside the GPU When a Kernel is Launched
hipLaunchKernel(relu_kernel, grid(1024), block(256), 0, stream, args...)
    |
    v
+-----------------------------------------------------------------+
| HIP Runtime                                                     |
|   1. Package kernel arguments into a buffer                     |
|   2. Create an AQL (Architected Queuing Language) packet        |
|   3. Write AQL packet to the GPU's hardware queue               |
`-----------------+-------------------------------------------------+
                |
                v
+-----------------------------------------------------------------+
| GPU Command Processor (hardware)                                |
|   4. Reads AQL packet from queue                                |
|   5. Decodes: which kernel, how many blocks, how many threads |
|   6. Starts dispatching blocks to Compute Units (CUs)           |
`-----------------+-------------------------------------------------+
                |
                v
+-----------------------------------------------------------------+
| Compute Units (CUs) - the actual processors                     |
|                                                                 |
|   MI210: 104 CUs    |    MI300X: 304 CUs                       |
|                                                                 |
|   Each CU can run multiple blocks simultaneously                |
|   Each CU has:                                                  |
|     * 4 SIMD units (each executes one wavefront)                |
|     * 64 KB LDS (Local Data Share - fast shared memory)         |
|     * 256 vector registers per SIMD (for thread data)           |
|     * 4 Matrix/MFMA units (for FP16 matrix math)               |
|                                                                 |
|   Block 0 -> CU 0    Block 4 -> CU 4                            |
|   Block 1 -> CU 1    Block 5 -> CU 5                            |
|   Block 2 -> CU 2    ...                                        |
|   Block 3 -> CU 3    Block 1023 -> CU (1023 % 104) = CU 95     |
|                                                                 |
|   When a CU finishes a block, it picks up the next             |
|   unstarted block automatically (hardware scheduling)           |
`-----------------------------------------------------------------+

The AQL Packet - What HIP Actually Sends to the GPU
AQL (Architected Queuing Language) Packet - 64 bytes:
(Educational layout -- field sizes are approximate. See the HSA Platform
System Architecture Specification, Section 2.8 for exact ABI offsets.)
+---------------------------------------------------+
|  header:          0x0401  (kernel dispatch)       |  2 bytes
|  dimensions:      1       (1D grid)               |  2 bytes
|  workgroup_size:  [256, 1, 1]                    |  6 bytes
|  grid_size:       [262144, 1, 1]                 |  12 bytes
|  private_segment: 0                               |  4 bytes
|  group_segment:   0       (shared mem)            |  4 bytes
|  kernel_object:   0x7f... (GPU code address)      |  8 bytes
|  kernarg_address: 0x7f... (arguments in GPU mem)  |  8 bytes
|  completion_signal: handle (for sync)             |  8 bytes
```

```
|  reserved:        ...                            |  10 bytes
`-----------------------------------------------------+
Total: 64 bytes - this tiny packet triggers millions of GPU operations!
This packet is written to a special ring buffer in system memory that
the GPU's Command Processor continuously polls.

Kernel Launch is Asynchronous
+-------------------------------------------------------------------------+
|  BEGINNER'S NOTE: What Does "Asynchronous" Mean Here?                  |
|                                                                         |
|  GPU kernel launches are asynchronous -- the CPU sends a command to     |
|  the GPU's hardware queue and returns immediately, without waiting     |
|  for the GPU to finish. This is NOT the same as Python's async/await  |
|  (which uses cooperative coroutines on a single thread). GPU async     |
|  is hardware-level parallelism: the CPU and GPU run simultaneously    |
|  on separate processors.                                                |
|                                                                         |
|  Think of it like dropping letters in a mailbox -- you don't stand      |
|  there waiting for each letter to be delivered before dropping the     |
|  next one. You drop them all, then check for replies later.                                          |
|                                                                         |
|  Synchronous (blocking -- CPU waits):                                   |
|    result = gpu.compute(data)  # CPU stuck here until GPU is done      |
|    print(result)               # can only print after GPU finishes     |
|                                                                         |
|  Asynchronous (non-blocking -- CPU continues immediately):              |
|    gpu.compute_async(data)     # CPU returns INSTANTLY                 |
|    gpu.compute_async(data2)    # CPU queues more work immediately      |
|    gpu.wait()                  # NOW CPU waits for everything          |
|    print(result)                                                        |
|                                                                         |
|  The GPU works like a restaurant kitchen: the waiter (CPU) drops off   |
|  5 order tickets (kernel launches) in rapid succession, and the        |
|  kitchen (GPU) works through them one by one. The waiter does NOT      |
|  stand at the window waiting for each dish before placing the next.    |
`-------------------------------------------------------------------------+
This is a critical concept:
CPU thread timeline:
  +-------------------------------------------------------+
  | hipLaunchKernel(conv)  | hipLaunchKernel(relu)  | ...   |
  | <- returns instantly!   | <- returns instantly!   |       |
  | (~5-10 mus)             | (~5-10 mus)             |       |
  `-------------------------------------------------------+

  CPU doesn't wait! It just puts AQL packets in the queue and moves on.
GPU timeline:
  +-------------------------------------------------------+
  | ......waiting...... | [conv: 0.5ms]  |  [relu: 0.01ms]  |
  |  (GPU picks up work |                |                  |
  |   from queue)       |                |                  |
  `-------------------------------------------------------+

  GPU starts work when it gets to that packet in the queue.
This is why MIGraphX can issue dozens of kernel launches in microseconds - the CPU just fills the queue, the GPU works through it at it

MIGraphX Kernel Launch Examples
During a single inference, MIGraphX issues many different types of kernel launches:
TYPE 1: Library call (MIOpen convolution)
  miopenConvolutionForward(handle, ...)
    `---- Internally calls: hipLaunchKernel(ImplicitGemmV4R4, ...)

TYPE 2: Library call (rocBLAS GEMM)
  rocblas_gemm_ex(handle, ...)
    `---- Internally calls: hipLaunchKernel(Tensile_HGEMM_128x128, ...)

TYPE 3: JIT-compiled fused kernel
  hipModuleLaunchKernel(fused_pointwise_func, ...)
    `---- Launches MIGraphX's custom-generated kernel
TYPE 4: hipBLASLt fused GEMM
  hipblasLtMatmul(handle, ...)
    `---- Internally calls: hipLaunchKernel(fused_gemm_bias_gelu, ...)
All of these ultimately end up as AQL packets in the GPU queue.


4.4 Part 3: Synchronization - hipDeviceSynchronize()
-----------------------------------------------------

Why Synchronization Exists
+-------------------------------------------------------------------------+
|  BEGINNER'S NOTE: What is "Synchronization"?                           |
|                                                                         |
|  In Python threading, you might use a Lock or Event:                   |
|    event = threading.Event()                                            |
```

```
|    # Worker thread:                                                    |
|    do_work()                                                           |
|    event.set()   # "I'm done!"                                         |
|    # Main thread:                                                      |
|    event.wait()  # blocks until worker says "I'm done"                 |
|                                                                        |
|  GPU synchronization is the same concept. The GPU is like a worker    |
|  thread doing computation. The CPU needs to know when the GPU is       |
|  finished so it can safely read the results.                           |
|                                                                        |
|  Without synchronization: CPU reads output buffer while GPU is still   |
|  writing to it -> garbage data (a "race condition").                    |
|  With synchronization: CPU waits -> GPU finishes -> CPU reads correct   |
|  results.                                                              |
`------------------------------------------------------------------------+
Since kernel launches are asynchronous (CPU doesn't wait), you need a way to know when the GPU is actually done. Otherwise, you'd read
WITHOUT synchronization (BUG!):
  hipLaunchKernel(compute_result, ...);
  hipMemcpy(cpu_output, gpu_output, ...);  // GPU might not be done yet!
  printf("%f\n", cpu_output[0]);           // GARBAGE DATA!
WITH synchronization (correct):
  hipLaunchKernel(compute_result, ...);
  hipDeviceSynchronize();                  // CPU blocks here until GPU finishes
  hipMemcpy(cpu_output, gpu_output, ...);  // GPU is definitely done
  printf("%f\n", cpu_output[0]);           // correct result

The Synchronization Tools
HIP provides multiple synchronization mechanisms, ordered from coarsest to finest:

hipDeviceSynchronize() - Wait for EVERYTHING
hipDeviceSynchronize()
  CPU blocks until ALL work on ALL streams on this GPU is complete.

  This is the heaviest synchronization -- simple but can hurt performance
  because it forces ALL streams to drain completely.

  +-- Stream 0: [kernel A]--[kernel B]--[kernel C]--|
  |  Stream 1: [kernel D]--|                        |  <- CPU waits here
  |  Stream 2: [kernel E]--[kernel F]--|            |     for ALL to finish
  `----------------------------------------------------+
  Time cost: whatever the slowest stream takes

hipStreamSynchronize(stream) - Wait for ONE stream
hipStreamSynchronize(stream0)
  CPU blocks until all work on stream0 is complete.
  Other streams can still be running.

  +-- Stream 0: [kernel A]--[kernel B]--|  <- CPU waits only for this
  |  Stream 1: [kernel D]--[kernel E]--[kernel F]--  (still running, fine)
  `---------------------------------------------------
  Better than DeviceSync - only waits for what you need.

hipEventSynchronize(event) - Wait for a SPECIFIC point
hipEventRecord(event, stream0);        // mark a point in stream0
hipEventSynchronize(event);            // CPU waits until GPU reaches that point
  Stream 0: [kernel A]--[kernel B]--|event|--[kernel C]--
                                    ^
                                    CPU waits until GPU finishes kernel B
                                    kernel C might not be done yet - that's OK
  Most precise - wait for exactly what you need.

hipStreamWaitEvent(stream, event) - GPU-to-GPU sync (no CPU involved)
hipEventRecord(event1, stream1);
hipStreamWaitEvent(stream0, event1, 0);
  Stream 0: [kernel A]--------------[wait for event1]--[kernel C]--
  Stream 1: [kernel B]--|event1|
  Stream 0 pauses at the wait point until stream 1 records event1.
  THE CPU IS NOT INVOLVED! This is purely GPU-side synchronization.

  This is what MIGraphX uses for inter-stream dependencies.

How MIGraphX Uses Synchronization
DURING INFERENCE (typical pattern):
  // Phase 1: Queue up all work (CPU is blazing fast here)
  hipMemcpyAsync(input_gpu, input_cpu, size, H2D, stream0);   // async copy

  miopenConvolutionForward(handle0, ...);    // stream 0
  miopenConvolutionForward(handle1, ...);    // stream 1 (parallel branch)

  hipEventRecord(event_branch, stream1);     // mark when stream1 finishes
  hipStreamWaitEvent(stream0, event_branch); // stream0 waits (GPU-side only)

  hipLaunchKernel(add_residual, ..., stream0);  // needs both conv results
```

```
  hipLaunchKernel(matmul, ..., stream0);
  hipLaunchKernel(softmax, ..., stream0);

  hipMemcpyAsync(output_cpu, output_gpu, size, D2H, stream0); // async copy

  // Phase 2: Wait for result (only ONE sync at the very end)
  hipStreamSynchronize(stream0);  // CPU blocks here

  // Phase 3: Return result to Python
  return output_cpu;
Note: MIGraphX calls hipDeviceSynchronize or hipStreamSynchronize
only ONCE - at the very end. Everything else uses async operations
and GPU-side events. This keeps the CPU free and the GPU pipeline full.

The Cost of Synchronization
Operation                    Time Cost        When to Use
---------                    ---------        ----------
hipStreamWaitEvent           ~0 mus (CPU)      GPU-side sync between streams
(GPU-side)                   ~1-5 mus (GPU)    (MIGraphX uses this heavily)
hipEventSynchronize          ~2-5 mus          Wait for a specific point
(CPU blocks)
hipStreamSynchronize         ~2-10 mus         Wait for one stream to drain
(CPU blocks)                                  (MIGraphX uses at end of inference)
hipDeviceSynchronize         ~5-50 mus         Wait for ALL work
(CPU blocks)                                  (avoid during inference - too broad)


4.5 The Full Picture: One Inference Through HIP
-----------------------------------------------

session.run(input_image)
 +--- CPU THREAD -------------------------------------------------------+
 |                                                                      |
 |  hipMemcpyAsync(gpu_in, cpu_in, 600KB, H2D, s0)     <- ~1 mus CPU  |
 |  hipLaunchKernel(conv1, grid, block, 0, s0, args)    <- ~5 mus CPU  |
 |  hipLaunchKernel(conv2, grid, block, 0, s1, args)    <- ~5 mus CPU  |
 |  hipEventRecord(ev1, s1)                              <- ~1 mus CPU  |
 |  hipStreamWaitEvent(s0, ev1)                          <- ~1 mus CPU  |
 |  hipLaunchKernel(add_relu, grid, block, 0, s0, args) <- ~5 mus CPU  |
 |  hipLaunchKernel(gemm, grid, block, 0, s0, args)     <- ~5 mus CPU  |
 |  hipLaunchKernel(softmax, grid, block, 0, s0, args)  <- ~5 mus CPU  |
 |  hipMemcpyAsync(cpu_out, gpu_out, 4KB, D2H, s0)      <- ~1 mus CPU  |
 |  hipStreamSynchronize(s0)                             <- BLOCKS     |
 |  .................waiting............................              |
 |  return output  <- CPU resumes when GPU done                        |
 |                                                                      |
 |  Total CPU time queueing: ~30 mus (microseconds)                    |
 `----------------------------------------------------------------------+
 +--- GPU STREAM 0 -----------------------------------------------------+
 | [H2D copy]->[conv1: 0.3ms]->[wait ev1]->[add_relu: 0.02ms]->        |
 | [gemm: 0.1ms]->[softmax: 0.01ms]->[D2H copy]                       |
 `----------------------------------------------------------------------+
 +--- GPU STREAM 1 -----------------------------------------------------+
 | [conv2: 0.25ms]->[ev1 recorded]                                     |
 `----------------------------------------------------------------------+
 +--- GPU DMA ENGINE ---------------------------------------------------+
 | [H2D: 0.01ms]                              [D2H: 0.001ms]         |
 `----------------------------------------------------------------------+
  Total GPU time: ~0.43 ms (max of stream0, stream1)
  Total wall time: ~0.43 ms (GPU is the bottleneck, not CPU)

The Key Insight
The CPU spends ~30 microseconds queueing all work. The GPU spends ~430 microseconds (0.43 ms) actually doing the work. That means the G
Batch 1:  CPU[queue]------[wait]---------------CPU[queue]-----[wait]--
          GPU----------[compute batch 1]--------GPU-----[compute batch 2]--
Batch 2:                         CPU[prepare batch 2 input]
                                 (overlaps with GPU computing batch 1)
This is why asynchronous kernel launches via HIP are so critical - they enable pipelining between CPU and GPU work.


4.6 Part 4: Runtime Compilation - hipRTC and COMGR/LLVM
--------------------------------------------------------

This is where GPU machine code (ISA) is generated. COMGR/LLVM lives INSIDE
the HIP Runtime (Layer 5), and is invoked by Layer 4 (MIGraphX) when it
needs to compile JIT-generated HIP C++ kernel source code.

Why Runtime Compilation Exists
MIGraphX generates custom HIP C++ kernels for fused operations (e.g.,
a pointwise kernel that computes relu((x + bias) * scale) + residual).
These kernels don't exist as pre-compiled binaries -- they are generated
on-the-fly during prog.compile(). To turn them into executable
GPU code, MIGraphX calls hiprtcCompileProgram().
```

```
The hipRTC -> COMGR -> LLVM Pipeline
MIGraphX (Layer 4): generates HIP C++ source string
         |
         |  hiprtcCompileProgram(prog, ...)
         v
+---------------------------+
|    hipRTC Frontend        |  <- part of HIP Runtime (Layer 5)
|    (validates source)     |
`--------------+---------------+
            v
+---------------------------+
|     COMGR                 |  <- Code Object Manager
|  (wraps LLVM toolchain)   |
|                           |
|  Clang Frontend           |  <- parses HIP C++ into LLVM IR
|         v                 |
|  LLVM IR                  |  <- hardware-independent intermediate form
|    e.g., %r = fadd half %a, %b
|         v                 |
|  AMDGPU LLVM Backend      |  <- AMD-specific optimizations:
|    - Register allocation  |     register allocation,
|    - Instruction select   |     instruction selection,
|    - Vectorization        |     vectorization,
|    - Occupancy tuning     |     occupancy tuning
|         v                 |
|  AMDGPU ISA               |  <- actual machine instructions
|    gfx90a (MI210)         |     e.g., v_fma_f16 v0, v1, v2, v3
|    gfx942 (MI300X)        |
|         v                 |
|  Code Object (.co)        |  <- ELF binary
`--------------+----------------+
            v
+---------------------------+
|  hipModuleLoadData()      |  <- Layer 5 loads Code Object into GPU memory
|  hipModuleGetFunction()   |  <- Layer 5 returns kernel function handle
`---------------------------+

Target Architecture - What gfx90a / gfx942 Mean
Each AMD GPU has a target ID that tells the compiler which instructions are available:
GPU              Target     Matrix Cores    Key Feature
----------       ------     ------------    -----------
MI100 (CDNA1)    gfx908     MFMA v1         First AMD ML GPU
MI210 (CDNA2)    gfx90a     MFMA v2         Unified memory, 2x perf
MI300X (CDNA3)   gfx942     MFMA v3         Chiplet design, 192GB HBM3
COMGR compiles specifically for YOUR target. Code compiled for gfx90a
won't run on gfx942 and vice versa.

What MFMA Instructions Look Like
MFMA (Matrix Fused Multiply-Add) are the AMD Matrix Core instructions:
Single MFMA instruction:
  v_mfma_f32_32x32x8_f16
  Translation:
    v_mfma       = Vector Matrix FMA
    f32          = accumulate in FP32 (for precision)
    32x32x8      = computes a 32x32 output tile from 32x8 x 8x32 inputs
    f16          = input operands are FP16
  Each MFMA instruction produces:
    32 x 32 x 8 x 2 = 16,384 FP16 operations
  The instruction is pipelined over ~64 cycles, so each MFMA unit
  sustains 16,384 / 64 = 256 FP16 ops per clock cycle.
  An MI210 has 104 Compute Units, each with 4 MFMA units:
    104 x 4 x 256 = ~106,496 FP16 ops per clock cycle (sustained)
    At ~1.7 GHz: ~106K x 1.7 GHz ~= ~181 TFLOPS (peak FP16)
  This matches the published MI210 FP16 peak of ~181 TFLOPS.
This is why FP16 + Matrix Cores = enormous throughput.

The Final Code Object
+---------------------------------------------------------------------------+
|  BEGINNER'S NOTE: What is an "ELF binary" and a "Code Object"?        |
|                                                                         |
|  ELF = the standard file format for compiled programs on Linux.        |
|  A GPU "Code Object" is essentially an ELF but for the GPU:           |
|  - Contains compiled GPU machine code (ISA instructions)               |
|  - Tells the GPU: register counts, shared memory, target arch          |
|  - The HIP runtime loads this into GPU memory via hipModuleLoadData()  |
|                                                                         |
|  Think: .pyc files in Python (compiled bytecode), but for the GPU.    |
`---------------------------------------------------------------------------+
Code Object structure:
+-----------------------------------+
|  ELF Header                       | <- "this is for gfx90a"
|-----------------------------------|
```

```
|  .text section                  | <- GPU machine code (ISA)
|    kernel_conv_fused_bn_relu:   |
|      s_load_dword s0, ...       |    <- load kernel args
|      v_mfma_f32_32x32x8_f16    |    <- matrix multiply
|      v_pk_add_f16 v0, v1, v2   |    <- batch norm (fused)
|      v_pk_max_f16 v0, v0, 0    |    <- relu (fused)
|      ...                        |
|-------------------------------------|
|  .rodata section                | <- constant data
|-------------------------------------|
|  Metadata                       | <- workgroup size, register count,
|                                 |    shared memory usage
`-------------------------------------+
This binary gets loaded into GPU memory and is ready to execute.


4.7 Source Code Walkthrough -- HIP Runtime Internals
-----------------------------------------------------

  +-- DIAGRAM: HIP Runtime -- Call Flow Overview ----------------------------+
  |                                                                          |
  |  ALLOCATION PATH:                                                       |
  |  hipMalloc(&ptr, size)                                                  |
  |       |  public wrapper (tracing hooks)                                 |
  |       v                                                                  |
  |  ihipMalloc(ptr, size, flags)                                           |
  |       |  validate, check max size                                       |
  |       v                                                                  |
  |  amd::SvmBuffer::malloc(ctx, flags, size, align)                        |
  |       |  ROCclr abstraction                                             |
  |       v                                                                  |
  |  hsa_amd_memory_pool_allocate(pool, size, ...)                          |
  |       |  HSA runtime                                                    |
  |       v                                                                  |
  |  ioctl(fd, AMDGPU_IOC_ALLOC_MEMORY, ...)                               |
  |       |  kernel driver -> TTM allocator                                 |
  |       v                                                                  |
  |  Physical pages assigned (VRAM or GTT)                                  |
  |                                                                          |
  |  LAUNCH PATH:                                                           |
  |  hipModuleLaunchKernel(func, grid, block, sharedMem, stream, args)      |
  |       |  validate dims, stream                                          |
  |       v                                                                  |
  |  ihipLaunchKernelCommand -> new NDRangeKernelCommand(...)               |
  |       |  allocate command object, copy args                             |
  |       v                                                                  |
  |  command->enqueue()                                                     |
  |       |  ROCclr -> HSA: build 64-byte AQL packet                        |
  |       v                                                                  |
  |  write to doorbell MMIO register  -->  GPU CP wakes up, dispatches      |
  |  (returns immediately, GPU runs async)                                  |
  `--------------------------------------------------------------------------+

hipMalloc -- Public API Wrapper
  File: clr/hipamd/src/hip_memory.cpp:778-782

  hipError_t hipMalloc(void** ptr, size_t sizeBytes) {
      HIP_INIT_API(hipMalloc, ptr, sizeBytes);   // trace/profile hook
      CHECK_STREAM_CAPTURE_SUPPORTED();
      HIP_RETURN_DURATION(ihipMalloc(ptr, sizeBytes, 0),
                          ReturnPtrValue(ptr));
  }

ihipMalloc -- The Real Allocator
  File: clr/hipamd/src/hip_memory.cpp:338-388

  hipError_t ihipMalloc(void** ptr, size_t sizeBytes, unsigned int flags) {
      if (ptr == nullptr) return hipErrorInvalidValue;
      if (sizeBytes == 0) { *ptr = nullptr; return hipSuccess; }

      bool useHostDevice =
          (flags & CL_MEM_SVM_FINE_GRAIN_BUFFER) != 0;
      amd::Context* curDevContext =
          hip::getCurrentDevice()->asContext();
      amd::Context* amdContext =
          useHostDevice ? hip::host_context : curDevContext;

      const auto& dev_info = amdContext->devices()[0]->info();
      size_t max_device_size = IS_LINUX
          ? dev_info.maxMemAllocSize_
          : (dev_info.maxMemAllocSize_ + dev_info.maxPhysicalMemAllocSize_);

      if (max_device_size < sizeBytes) return hipErrorOutOfMemory;
```

```
      // * THE ACTUAL GPU ALLOCATION
      *ptr = amd::SvmBuffer::malloc(
          *amdContext, flags, sizeBytes, dev_info.memBaseAddrAlign_,
          useHostDevice ? curDevContext->svmDevices()[0] : nullptr);

      if (*ptr == nullptr) {
          size_t free = 0, total = 0;
          hipMemGetInfo(&free, &total);
          LogPrintfError("Allocation failed: required:%zu free:%zu total:%zu",
                         sizeBytes, free, total);
          return hipErrorOutOfMemory;
      }
      amd::Memory* memObj = getMemoryObject(*ptr, offset);
      memObj->getUserData().deviceId = hip::getCurrentDevice()->deviceId();
      return hipSuccess;
  }

hipModuleLaunchKernel -- Public Launch API
  File: clr/hipamd/src/hip_module.cpp:534-580

  hipError_t hipModuleLaunchKernel(
      hipFunction_t f,
      uint32_t gridDimX, uint32_t gridDimY, uint32_t gridDimZ,
      uint32_t blockDimX, uint32_t blockDimY, uint32_t blockDimZ,
      uint32_t sharedMemBytes, hipStream_t hStream,
      void** kernelParams, void** extra) {

      // Validate stream not destroyed
      if (!hip::isValid(hStream)) HIP_RETURN(hipErrorContextIsDestroyed);

      // Grid dimension limits
      constexpr auto int32_max = static_cast<uint64_t>(INT32_MAX);
      constexpr auto uint16_max = static_cast<uint64_t>(UINT16_MAX + 1);
      if (gridDimX > int32_max || gridDimY > uint16_max ...)
          HIP_RETURN(hipErrorInvalidValue);

      amd::HIPLaunchParams launch_params(gridDimX, gridDimY, gridDimZ,
                                         blockDimX, blockDimY, blockDimZ,
                                         sharedMemBytes);
      // Validate workgroup size and shared memory limits
      HIP_RETURN(ihipModuleLaunchKernel(f, launch_params, hStream,
                                        kernelParams, extra, ...));
  }

ihipLaunchKernelCommand -- Build the Command Object
  File: clr/hipamd/src/hip_module.cpp:352-441

  hipError_t ihipLaunchKernelCommand(
      amd::Command*& command, hipFunction_t f, ...) {
      hip::DeviceFunc* function = hip::DeviceFunc::asFunction(f);
      amd::Kernel* kernel = function->kernel();

      amd::NDRangeContainer ndrange(3, globalWorkOffset,
          launch_params.global_.Data(), launch_params.local_.Data());

      // * ALLOCATE THE DISPATCH COMMAND ON HEAP
      amd::NDRangeKernelCommand* kernelCommand =
          new amd::NDRangeKernelCommand(
              *stream, waitList, *kernel, ndrange,
              launch_params.sharedMemBytes_, ...);

      // Copy kernel arguments (kernelParams or extra path)
      // captureAndValidate() -- snapshot args into command buffer
      command = kernelCommand;
      return hipSuccess;
  }

command->enqueue() -- The Critical Moment
  File: clr/hipamd/src/hip_module.cpp:493-532

  // After building the command:
  command->enqueue();    // * KERNEL DISPATCHED TO GPU

  // Inside ROCclr + HSA Runtime:
  // 1. ROCclr translates NDRangeKernelCommand -> HSA dispatch
  // 2. HSA Runtime creates 64-byte AQL packet:
  //    struct hsa_kernel_dispatch_packet_s {
  //      uint16_t header;              // packet type + barriers
  //      uint16_t setup;               // dimensions
  //      uint16_t workgroup_size_x/y/z;
  //      uint32_t grid_size_x/y/z;
  //      uint32_t private_segment_size;
```

```
  //      uint32_t group_segment_size;  // shared memory (LDS)
  //      uint64_t kernel_object;       // ISA code address
  //      void*    kernarg_address;     // argument buffer
  //      hsa_signal_t completion_signal;
  //    };
  // 3. Packet written to hardware queue ring buffer
  // 4. Doorbell register write -> GPU interrupt
  // 5. GPU Command Processor -> wavefront dispatch -> CU execution

  command->release();    // free command envelope
  // Returns IMMEDIATELY -- GPU runs asynchronously

hiprtcCreateProgram -- Create the Source Program (called first)
  File: clr/hipamd/src/hiprtc/hiprtc.cpp:68-113

  hiprtcResult hiprtcCreateProgram(
      hiprtcProgram* prog, const char* src, const char* name,
      int numHeaders, const char* const* headers,
      const char* const* headerNames) {
      // Validates: prog not null, numHeaders >= 0
      auto* rtcProgram = new hiprtc::RTCCompileProgram(progName);
      // If name is null -> generate "CompileSourceXXXXXX"
      rtcProgram->addSource(std::string(src), progName);
      for (int i = 0; i < numHeaders; i++)
          rtcProgram->addHeader(headers[i], headerNames[i]);
      *prog = hiprtc::RTCCompileProgram::as_hiprtcProgram(rtcProgram);
      HIPRTC_RETURN(HIPRTC_SUCCESS);
  }

hiprtcCompileProgram -- Runtime Kernel Compilation (called second)
  File: clr/hipamd/src/hiprtc/hiprtc.cpp:115-154

  hiprtcResult hiprtcCompileProgram(
      hiprtcProgram prog, int numOptions, const char* const* options) {
      auto* rtcProgram =
          hiprtc::RTCCompileProgram::as_RTCCompileProgram(prog);
      bool fgpu_rdc = false;
      std::vector<std::string> compile_options;
      // Scan user flags for -fgpu-rdc, --hiprtc-no-builtin-header
      // Add defaults: -D__HIPCC_RTC__, -nogpuinc, -include hiprtc_runtime.h
      compile_options.insert(end, user_options.begin(), user_options.end());

      if (!rtcProgram->compile(compile_options, fgpu_rdc))
          HIPRTC_RETURN(HIPRTC_ERROR_COMPILATION);
      // Internally: COMGR -> LLVM AMDGPU -> gfx1151 ISA -> ELF code object
      HIPRTC_RETURN(HIPRTC_SUCCESS);
  }


4.8 Where Every Byte Lives -- Layer 5 (HIP Runtime)
---------------------------------------------------

  +-----------------------------------------------------------------+
  |  hipMalloc/hipMallocManaged buffers                             |
  |    Where: GPU-accessible memory via amd::SvmBuffer::malloc      |
  |    Physical location depends on HSA_XNACK:                      |
  |      XNACK=0 (discrete): VRAM pool only (HBM on MI210/MI300X) |
  |      XNACK=1 (APU):      VRAM (512 MB) + GTT (28 GB LPDDR5x) |
  |    hipMallocManaged: unified virtual address, migrates on fault |
  |    Alignment: dev_info.memBaseAddrAlign_ (typically 256 bytes)  |
  |    Tracked: amd::Memory objects in HIP runtime's internal map   |
  |                                                                  |
  |  Kernel arguments                                               |
  |    Where: packed into NDRangeKernelCommand (C++ heap)           |
  |    Then: copied to kernarg segment (GPU-visible system memory)  |
  |    Size: sum of all kernel parameter sizes (typically < 4 KB)   |
  |    Lifetime: valid until command->release()                     |
  |                                                                  |
  |  AQL dispatch packets (64 bytes each)                           |
  |    Where: ring buffer in GPU-visible system memory (mmap'd)     |
  |    Written by: HSA Runtime (called from ROCclr)                 |
  |    Read by: GPU Command Processor (hardware)                    |
  |    Contains: kernel code address, kernarg address, grid dims    |
  |                                                                  |
  |  hipRTC compiled code objects                                   |
  |    Where: initially returned as std::vector<char> (C++ heap)    |
  |    Then: hipModuleLoadData copies ELF to GPU code object heap   |
  |    GPU location: instruction cache (SRAM) + VRAM backing        |
  |                                                                  |
  |  HIP stream state                                               |
  |    Where: hip::Stream object in C++ heap                        |
  |    Contains: ROCclr command queue, device association            |
  |    Maps to: HSA software queue -> hardware queue                 |
```

```
  |                                                                  |
  |  Synchronization signals                                        |
  |    Where: 64-bit atomics in GPU-visible memory (SVM)            |
  |    Used by: hipDeviceSynchronize, hipStreamSynchronize           |
  |    Mechanism: CPU spins/waits on signal value change             |
  `------------------------------------------------------------------+


================================================================================
  CHAPTER 5: HSA Runtime, libhsa-runtime64.so (Layer 6)
  The Hardware Interface -- Agents, Queues, Packets, and Page Tables
================================================================================

+-- EXECUTIVE SUMMARY -- Layer 6: HSA Runtime ---------------------------+
|                                                                         |
|  What:  Hardware abstraction layer -- last userspace code before HW. |
|  Input: HIP API calls (hipLaunchKernel -> HSA dispatch packet).     |
|  Output: AQL packets on hardware queues + doorbell MMIO writes.     |
|                                                                         |
|  Key abstractions:                                                      |
|    Agent   -- a compute device (CPU agent, GPU agent)               |
|    Pool    -- a memory region (VRAM, GTT, kernarg, LDS)             |
|    Queue   -- ring buffer of AQL packets (user-mode, no syscall)    |
|    Signal  -- 64-bit atomic for GPU->CPU completion notification     |
|                                                                         |
|  Dispatch path (hot path, no kernel syscall):                          |
|    1. Write AQL dispatch packet to queue ring buffer               |
|    2. Update write index (atomic)                                      |
|    3. Write doorbell register (MMIO) -> GPU command processor wakes |
|    4. GPU reads packet, dispatches wavefronts to CUs               |
|    5. Completion signal decremented -> CPU sees result              |
|                                                                         |
|  XNACK / page fault handling:                                          |
|    HSA_XNACK=1 -> GPU page faults handled via retry (not crash).   |
|    On APU: fault = page table update only (~5-10 us, no PCIe).    |
|    ROCt (libhsakmt.so) bridges to kernel driver via KFD ioctls.   |
|                                                                         |
|  Key files: ROCR-Runtime (libhsa-runtime64.so), ROCT-Thunk         |
`-------------------------------------------------------------------------+

  +-- SOURCE REFERENCES -- Layer 6 (HSA Runtime) ----------------------------+
  |                                                                           |
  |  Repository: https://github.com/ROCm/ROCR-Runtime                     |
  |  Branch:     rocm-7.2.0                                                 |
  |                                                                           |
  |  File                                     Key Function                  |
  |  -----------------------------------     ---------------------         |
  |  runtime/hsa-runtime/core/runtime/       hsa_init(), hsa_iterate_agents|
  |    hsa_api_trace.cpp                                                    |
  |  runtime/hsa-runtime/core/runtime/       AqlQueue::SubmitPacket()      |
  |    amd_aql_queue.cpp                     DoorbellWrite()               |
  |  runtime/hsa-runtime/core/runtime/       hsa_signal_store_relaxed()    |
  |    amd_gpu_agent.cpp                     GpuAgent::CreateQueue()       |
  |  runtime/hsa-runtime/core/runtime/       MemoryPool::Allocate()        |
  |    amd_memory_region.cpp                 hsa_amd_memory_pool_allocate()|
  |                                                                           |
  |  KFD Thunk (ROCt):                                                      |
  |  Repository: https://github.com/ROCm/ROCT-Thunk-Interface             |
  |  File: src/openclose.c, src/memory.c, src/queues.c                    |
  |  (Bridges HSA to /dev/kfd ioctls into the kernel driver)              |
  |                                                                           |
  |  HSA Specification:                                                     |
  |  https://hsafoundation.com/standards/ (AQL packet format reference)   |
  |                                                                           |
  |  GitHub links:                                                          |
  |  https://github.com/ROCm/ROCR-Runtime/tree/rocm-7.2.0/               |
  |    runtime/hsa-runtime/core/runtime/                                    |
  `-----------------------------------------------------------------------------+

This is the deepest software layer - the last piece of code before we reach the actual GPU hardware and kernel driver. HSA Runtime is t

+-----------------------------------------------------------------------------+
|  BEGINNER'S CONTEXT: Why Does This Layer Exist?                        |
|                                                                           |
|  You might wonder: "If HIP already talks to the GPU, why do we need   |
|  another layer underneath?" Good question!                             |
|                                                                           |
|  Think of it like driving a car:                                       |
|    YOU         -> turn the steering wheel (Python / MIGraphX)          |
|    HIP         -> the steering column and power steering (translates    |
|                   your motion into mechanical action)                   |
|    HSA Runtime -> the axles, differential, and wheels (the actual       |
|                   mechanical connection to the road)                    |
```

```
|    Hardware    -> the road and tires (physics / silicon)                |
|                                                                          |
|  HIP provides a nice, clean API. HSA handles the messy details of     |
|  actually communicating with GPU hardware: writing command packets      |
|  into hardware queues, ringing "doorbell" registers to wake up the    |
|  GPU, managing hardware signals for completion notification.           |
|                                                                          |
|  As a Python user, you will NEVER call HSA directly. But understanding |
|  it explains WHY GPU operations are fast (userspace MMIO writes) vs    |
|  slow (ioctl system calls for memory allocation).                       |
`--------------------------------------------------------------------------+
```

## 5.1 What is HSA?

```
HSA = Heterogeneous System Architecture
HSA is an open standard (originally by the HSA Foundation, now driven by AMD) that defines how CPUs and GPUs (and other accelerators) s
Analogy: If HIP (Layer 5) is the steering wheel of a car, HSA Runtime is the transmission, axle, and drive shaft. You turn the wheel (
You (programmer) never directly call HSA functions for ML.
The call chain:
  MIGraphX -> hipLaunchKernel() -> [HIP Runtime] -> hsa_signal_store() -> [HSA Runtime] -> GPU

  You see HIP.
  HIP sees HSA.
  HSA sees hardware registers and ring buffers.

Where HSA Sits
+--------------------------------------------------+
|  MIGraphX / MIOpen / rocBLAS                  |  <- Application layer
|--------------------------------------------------|
|  HIP Runtime (libamdhip64.so)                 |  <- GPU API layer
|--------------------------------------------------|
|                                               |
|  HSA Runtime (libhsa-runtime64.so)            |  <-- YOU ARE HERE
|                                               |
|  The "firmware-like" software that directly   |
|  manipulates GPU hardware resources:          |
|    * Hardware command queues                   |
|    * GPU memory controllers                   |
|    * Inter-device signaling                   |
|    * Page table programming                   |
|    * GPU agent discovery                      |
|                                               |
|--------------------------------------------------|
|  Linux Kernel Driver (amdgpu.ko)              |  <- Kernel space
|--------------------------------------------------|
|  AMD GPU Hardware                             |  <- Silicon
`--------------------------------------------------+

The Three Core Responsibilities
(1) Hardware-level abstraction       -> "Discover and manage GPU devices"
(2) GPU command queues (AQL packets) -> "Send work to the GPU"
(3) Page table management (XNACK)    -> "Let CPU and GPU share memory"
```

## 5.2 Part 1: Hardware-Level Abstraction - "Who's Out There?"

```
What "Abstraction" Means Here
Your system might have 1 CPU, 8 GPUs, and various memory pools. HSA provides a unified way to discover and interact with all of them wi

Agents - HSA's Name for Processors
In HSA terminology, every processor is an agent:
hsa_iterate_agents() discovers:
+---------------------------------------------------------------+
|  Agent 0: CPU                                                 |
|    Type:         HSA_DEVICE_TYPE_CPU                          |
|    Name:         "AMD EPYC 9654"                              |
|    Cores:        96                                           |
|    Memory pools: DDR5 (512 GB)                                |
|                                                               |
|  Agent 1: GPU                                                 |
|    Type:         HSA_DEVICE_TYPE_GPU                          |
|    Name:         "gfx90a" (MI210)                             |
|    Compute Units: 104                                         |
|    Max Wavefronts per CU: 32                                  |
|    Wavefront size: 64 threads                                 |
|    Memory pools: HBM (64 GB), LDS (64 KB/CU)                |
|    Max queue size: 131072 packets                             |
|    ISA:          "amdgcn-amd-amdhsa--gfx90a:sramecc+:xnack-" |
|                                                               |
|  Agent 2: GPU                                                 |
```

```
|    Type:         HSA_DEVICE_TYPE_GPU                          |
|    Name:         "gfx90a" (MI210 #2)                          |
|    ... (same properties)                                      |
|                                                               |
|  ... up to Agent 8 for an 8-GPU system                       |
`---------------------------------------------------------------+

Memory Pools - What Memory is Available
Each agent has memory pools - regions of memory with different properties:
hsa_amd_agent_iterate_memory_pools(gpu_agent) discovers:
GPU Agent (gfx90a / MI210):
+---------------------------------------------------------------+
|  Pool 0: "Global" (HBM)                                       |
|    Size:           64 GB                                       |
|    Segment:        HSA_AMD_SEGMENT_GLOBAL                      |
|    Global flags:   KERNARG | FINE_GRAINED | COARSE_GRAINED     |
|    Accessible by:  this GPU (fast), other GPUs (via XGMI),     |
|                    CPU (via PCIe, slow)                         |
|    Bandwidth:      1,638 GB/s (local)                          |
|                                                                |
|  Pool 1: "Group" (LDS - Local Data Share)                      |
|    Size:           64 KB per Compute Unit                      |
|    Segment:        HSA_AMD_SEGMENT_GROUP                       |
|    Accessible by:  only wavefronts in the same workgroup       |
|    Bandwidth:      ~50,000 GB/s (per CU)                      |
|                                                                |
|  Pool 2: "Kernarg" (Kernel Argument memory)                    |
|    Size:           small pool                                  |
|    Purpose:        stores kernel launch arguments              |
|    Accessible by:  CPU (write) + GPU (read)                    |
`----------------------------------------------------------------+
CPU Agent:
+---------------------------------------------------------------+
|  Pool 0: "Global" (DDR5 System RAM)                            |
|    Size:           512 GB                                      |
|    Accessible by:  CPU (fast), GPU (via PCIe on discrete)     |
|    Bandwidth:      ~90 GB/s (12-ch DDR5-4800, server)         |
`----------------------------------------------------------------+

Fine-Grained vs Coarse-Grained Memory
This is a critical HSA concept:
COARSE-GRAINED memory:
  +-------------------------------------------------------+
  |  * Cached on both CPU and GPU                         |
  |  * NOT coherent - if GPU writes, CPU sees stale       |
  |    data until explicit flush/invalidate               |
  |  * FAST for single-device access                      |
  |  * Used for: weights, intermediate buffers            |
  |  * This is what hipMalloc() returns by default        |
  `-------------------------------------------------------+

  GPU writes 42 to address X
  CPU reads address X -> might see OLD value (cache not synced)
  Only after hipDeviceSynchronize -> CPU sees 42
FINE-GRAINED memory:
  +-------------------------------------------------------+
  |  * Cache-coherent between CPU and GPU                 |
  |  * If GPU writes, CPU sees updated value instantly    |
  |  * Supports atomic operations across CPU/GPU          |
  |  * SLOWER due to coherence overhead                   |
  |  * Used for: signals, flags, small shared state       |
  `-------------------------------------------------------+

  GPU writes 42 to address X
  CPU reads address X -> IMMEDIATELY sees 42 (coherent)
  No synchronization needed - hardware keeps caches in sync
Why this matters for ML inference:
Model weights (14 GB):      -> COARSE-GRAINED (GPU-only access, max bandwidth)
Kernel arguments (64 bytes): -> FINE-GRAINED (CPU writes, GPU reads immediately)
Completion signals:          -> FINE-GRAINED (GPU signals "done", CPU sees it)
Input/output buffers:        -> COARSE-GRAINED + explicit hipMemcpy

How HIP Maps to HSA - Agent Discovery
When you call hipGetDeviceCount() or hipSetDevice(0), here's what happens underneath:
hipGetDeviceCount(&count)
    |
    `----> hsa_iterate_agents(callback)
              |
              |---- Found agent type=CPU -> skip
              |---- Found agent type=GPU -> count++
              |---- Found agent type=GPU -> count++
              `---- ...
         return count = 8  (for 8-GPU system)
```

```
hipSetDevice(3)
    |
    `----> HSA runtime stores "current agent = Agent 4 (4th GPU)"
         All subsequent hipMalloc/hipLaunchKernel target this agent

MIGraphX's Chiplet Query
MIGraphX specifically uses HSA to detect chiplet topology on MI300X:
MI300X is not a monolithic chip - it's 8 XCDs (Accelerator Complex Dies)
glued together with an interconnect:
+-----------+-----------+-----------+-----------+
|  XCD 0  |  XCD 1  |  XCD 2  |  XCD 3  |
|  38 CUs |  38 CUs |  38 CUs |  38 CUs |
|-----------+---------+---------+-----------|
|  XCD 4  |  XCD 5  |  XCD 6  |  XCD 7  |
|  38 CUs |  38 CUs |  38 CUs |  38 CUs |
`------------+-----------+-----------+-----------+
         Infinity Fabric interconnect
Total: 304 CUs (8 x 38)
MIGraphX uses hsa_amd_agent_info() to query:
  - Number of XCDs
  - CUs per XCD
  - Memory proximity (which HBM stack is closest to which XCD)
This info is used to optimize kernel launch parameters - e.g., align
workgroups to XCD boundaries to minimize cross-chiplet traffic.


5.3 Part 2: GPU Command Queues - AQL Packets
---------------------------------------------

This is the heart of how work gets from software to hardware.

What is a Queue?
+-----------------------------------------------------------------------------+
|  BEGINNER'S NOTE: What is a "Ring Buffer" and a "Command Queue"?      |
|                                                                        |
|  A queue is like a line at a restaurant: first in, first served.       |
|                                                                        |
|  Python analogy:                                                       |
|    from collections import deque                                       |
|    queue = deque(maxlen=100)                                           |
|    queue.append("do conv")     # CPU adds work to the queue            |
|    queue.append("do relu")     # CPU adds more work                    |
|    task = queue.popleft()      # GPU picks up next task                |
|                                                                        |
|  A "ring buffer" is a fixed-size queue that wraps around. When you    |
|  reach the end, you go back to the beginning (like a circular track). |
|  This avoids ever needing to resize or move memory.                    |
|                                                                        |
|  The GPU hardware has a dedicated processor called the "Command        |
|  Processor" (CP) that continuously reads from this queue and           |
|  executes whatever it finds. The CPU writes work into the queue,      |
|  and the GPU consumes it -- they operate independently.                |
|                                                                        |
|  "AQL Packet" = a 64-byte data structure that describes one piece     |
|  of work. Think of it as a detailed "work order" with all the info    |
|  the GPU needs: which kernel to run, how many threads, where the      |
|  data is, where to store results, etc.                                |
`-----------------------------------------------------------------------------+
A queue is a ring buffer in memory that the GPU's Command Processor continuously reads from. Software writes work items (packets) to th
HSA Queue (ring buffer in memory):
+-----------------------------------------------------+
|  write_index -->  +-----------+                     |
|                   | Packet 5 | <- newest (to be     |
|                   |-------------|    executed)          |
|                   | Packet 4 |                      |
|                   |-------------|                       |
|  read_index --->  | Packet 3 | <- GPU executing this |
|                   |-------------|                       |
|                   | (empty)  | <- already executed   |
|                   |-------------|                       |
|                   | (empty)  |                      |
|                   `------------+                        |
|                                                       |
|  CPU writes at write_index (advances it forward)   |
|  GPU reads at read_index (advances it forward)     |
|  When read catches up to write -> queue empty       |
|  When write catches up to read -> queue full (wait) |
`-----------------------------------------------------+

Creating a Queue
hsa_queue_t* queue;
hsa_queue_create(
    gpu_agent,           // which GPU
```

```
    131072,              // queue size (max packets) - must be power of 2
    HSA_QUEUE_TYPE_SINGLE, // single-producer (one CPU thread)
    NULL, NULL,          // error callback
    UINT32_MAX,          // no private segment size limit
    UINT32_MAX,          // no group segment size limit
    &queue               // output: queue handle
);
```

The queue is allocated in system memory (fine-grained) so both
CPU and GPU can access it. The GPU's Command Processor has a
hardware pointer to this memory region.

AQL Packets - The Work Units

AQL = Architected Queuing Language - a standard packet format that HSA defines. Every piece of GPU work is described by a 64-byte AQL p

There are several packet types:

```
+-------------------------------------------------------------------+
|  AQL Packet Types                                                 |
|                                                                   |
|  1. Kernel Dispatch Packet (64 bytes)                            |
|     "Run this kernel with these parameters"                       |
|     -> This is what hipLaunchKernel creates                       |
|                                                                   |
|  2. Barrier AND Packet (64 bytes)                                |
|     "Wait until ALL of these signals are complete"                |
|     -> This is what hipStreamWaitEvent creates                    |
|                                                                   |
|  3. Barrier OR Packet (64 bytes)                                 |
|     "Wait until ANY of these signals is complete"                 |
|     -> Used for multi-stream coordination                         |
|                                                                   |
|  4. Agent Dispatch Packet (64 bytes)                             |
|     "Run a function on the host agent (CPU)"                      |
|     -> Rarely used in ML                                          |
`-------------------------------------------------------------------+
```

Kernel Dispatch Packet - In Full Detail

This is the most important packet type. Let's see every single field:

Kernel Dispatch Packet (64 bytes):

```
+-----------------------------------------------------------------+
| Offset | Size | Field                | Example Value          |
|----------+------+----------------------+------------------------|
|  0     | 2B   | header               | 0x0401                 |
|        |      |  |--- type: KERNEL_    |  (bits 0-7)            |
|        |      |  |  DISPATCH           |                        |
|        |      |  |--- barrier: 1       |  (bit 8: wait for      |
|        |      |  |                     |   previous packets)    |
|        |      |  |--- acquire_fence:   |  (bits 9-11: memory    |
|        |      |  |  SYSTEM             |   ordering)            |
|        |      |  `--- release_fence:   |  (bits 12-14: memory   |
|        |      |     SYSTEM             |   ordering)            |
|----------+------+----------------------+------------------------|
|  2     | 2B   | setup                | 0x0001                 |
|        |      |  `--- dimensions: 1    |  (1D/2D/3D grid)       |
|----------+------+----------------------+------------------------|
|  4     | 2B   | workgroup_size_x     | 256                    |
|  6     | 2B   | workgroup_size_y     | 1                      |
|  8     | 2B   | workgroup_size_z     | 1                      |
|----------+------+----------------------+------------------------|
|  10    | 2B   | (reserved)           | 0                      |
|----------+------+----------------------+------------------------|
|  12    | 4B   | grid_size_x          | 262144                 |
|  16    | 4B   | grid_size_y          | 1                      |
|  20    | 4B   | grid_size_z          | 1                      |
|----------+------+----------------------+------------------------|
|  24    | 4B   | private_segment_size | 0                      |
|        |      |  (scratch per thread)|                        |
|----------+------+----------------------+------------------------|
|  28    | 4B   | group_segment_size   | 16384                  |
|        |      |  (LDS per workgroup) | (16 KB shared mem)     |
|----------+------+----------------------+------------------------|
|  32    | 8B   | kernel_object        | 0x00007f12_34000000    |
|        |      |  (GPU code address)  | (points to ISA in HBM)|
|----------+------+----------------------+------------------------|
|  40    | 8B   | kernarg_address      | 0x00007f12_56000000    |
|        |      |  (kernel arguments)  | (GPU ptrs, constants) |
|----------+------+----------------------+------------------------|
|  48    | 8B   | (reserved)           | 0                      |
|----------+------+----------------------+------------------------|
|  56    | 8B   | completion_signal    | signal handle          |
|        |      |  (signal when done)  | (HSA signal object)    |
`----------+------+----------------------+-------------------------+
```

Total: 64 bytes - perfectly sized for one cache line

How a Kernel Launch Flows Through the Queue

```
Step-by-step: hipLaunchKernel -> GPU execution
  +-- CPU (your program, via HIP -> HSA) --------------------------+
  |                                                                |
  |  (1) Reserve slot in queue                                     |
  |     write_idx = hsa_queue_load_write_index_relaxed(queue)      |
  |     write_idx++                                                |
  |     hsa_queue_store_write_index_relaxed(queue, write_idx)      |
  |                                                                |
  |  (2) Write packet at that slot                                 |
  |     packet = &queue->base_address[write_idx % queue_size]      |
  |     packet->header = KERNEL_DISPATCH | BARRIER                 |
  |     packet->workgroup_size_x = 256                             |
  |     packet->grid_size_x = 262144                               |
  |     packet->kernel_object = kernel_code_gpu_address            |
  |     packet->kernarg_address = args_gpu_address                 |
  |     packet->completion_signal = signal                         |
  |                                                                |
  |  (3) Ring the doorbell (notify GPU)                            |
  |     hsa_signal_store_*(queue->doorbell_signal, write_idx)     |
  |     (ROCR uses _relaxed or _screlease depending on path)      |
  |     This writes to a MEMORY-MAPPED GPU REGISTER.              |
  |     The GPU's Command Processor is hardware-wired to           |
  |     watch this register. When it changes, it wakes up.         |
  |                                                                |
  `----------------------------------------------------------------+
                              |
                              v (doorbell triggers GPU)
  +-- GPU Command Processor (hardware) ----------------------------+
  |                                                                |
  |  (4) CP reads the new packet from the queue                    |
  |     - Decodes packet type: KERNEL_DISPATCH                     |
  |     - Extracts: kernel_object, grid/block dims, args          |
  |                                                                |
  |  (5) CP allocates resources                                    |
  |     - Assigns workgroups to Compute Units                      |
  |     - Allocates LDS (group_segment_size) on each CU           |
  |     - Allocates scratch memory (private_segment_size)          |
  |     - Sets up wavefronts with thread IDs                      |
  |                                                                |
  |  (6) CP dispatches wavefronts                                  |
  |     - Wavefronts start executing the kernel ISA               |
  |     - Each wavefront = 64 threads (CDNA) or 32 (RDNA)          |
  |                                                                |
  |  (7) When ALL wavefronts complete:                             |
  |     - CP decrements the completion_signal                      |
  |     - hsa_signal_value goes from 1 -> 0                        |
  |     - Anyone waiting on this signal wakes up                   |
  |                                                                |
  `----------------------------------------------------------------+

The Doorbell - How CPU Wakes Up the GPU
The "doorbell" is a MEMORY-MAPPED I/O (MMIO) register:
  Physical address: mapped by the kernel driver (amdgpu.ko)
  CPU sees it as:   a normal memory address (e.g., 0xFFFF_8800_0001_0000)
  GPU sees it as:   a hardware interrupt trigger
When CPU writes to this address:
  CPU                          GPU
  -----                        -----
  store(doorbell, new_idx) -> hardware signal fires
                             -> Command Processor wakes
                             -> reads queue at new_idx
                             -> starts processing packet
This is why kernel launches are so fast (~5 mus) - it's a single
memory write, not a system call or interrupt. The CPU doesn't even
need to enter the kernel (OS) - it's a pure userspace operation!

Signals - HSA's Synchronization Primitive
HSA signals are the foundation that HIP events are built on:
hsa_signal_t signal;
hsa_signal_create(1, 0, NULL, &signal);  // initial value = 1
// Packet's completion_signal = signal
// GPU decrements signal when kernel finishes: 1 -> 0
// CPU waiting:
hsa_signal_wait_scacquire(signal, HSA_SIGNAL_CONDITION_EQ, 0,
                          UINT64_MAX, HSA_WAIT_STATE_BLOCKED);
// Blocks until signal value == 0 (kernel done)
Internally, this maps to:
  hipEventRecord(event, stream)      -> associates signal with last packet in stream
  hipEventSynchronize(event)          -> hsa_signal_wait on that signal
  hipStreamWaitEvent(stream, event)   -> inserts Barrier packet that waits on signal

Signal wait strategies:
HSA_WAIT_STATE_BLOCKED:
```

```
  CPU thread goes to sleep (kernel schedules it out)
  Woken by interrupt when signal fires
  Latency: ~5-15 mus (context switch overhead)
  Power: LOW (CPU core sleeps)
  Used for: long waits (inference completion)
HSA_WAIT_STATE_ACTIVE:
  CPU thread spins in a tight loop checking signal value
  Latency: ~1-2 mus (no context switch)
  Power: HIGH (CPU core spinning at 100%)
  Used for: ultra-low-latency requirements


5.4 Part 3: Page Table Management & XNACK
-----------------------------------------

What Are Page Tables?
+-------------------------------------------------------------------------+
|  BEGINNER'S NOTE: Why Should I Care About Page Tables?                 |
|                                                                         |
|  See Chapter 0 section 0.15 for the basics of virtual memory.          |
|                                                                         |
|  Short version: page tables are lookup tables that translate "fake"    |
|  addresses (virtual) to "real" addresses (physical hardware locations).|
|  Both CPUs and GPUs use them. Without page tables, different programs  |
|  would step on each other's memory -- total chaos.                     |
|                                                                         |
|  For AI inference, the key implication is:                              |
|    - Every memory access on the GPU goes through a page table lookup  |
|    - If the translation is cached (TLB hit) -> fast (~1 cycle)         |
|    - If not cached (TLB miss) -> slow (~100-400 cycles of walking      |
|      through the page table data structure in memory)                  |
|    - For large sequential tensor accesses, TLB hits are >99%          |
|      so this is usually not a problem                                  |
|                                                                         |
|  TLB = Translation Lookaside Buffer. A small cache that stores        |
|  recent virtual->physical address translations. Like a speed-dial      |
|  list for frequently-called phone numbers.                             |
`-------------------------------------------------------------------------+
GPUs, like CPUs, use virtual memory. Every pointer you see (like 0x7f1234000000) is a virtual address. The hardware translates it to a
Virtual Address                    Physical Address
0x7f12_3400_0000  ---- page ---->  0x0000_0003_8000_0000  (HBM chip 2, bank 5)
                       table
                       lookup
Why virtual memory on GPUs?
  * Multiple programs can use the same GPU without conflicts
  * Memory can be non-contiguous physically but contiguous virtually
  * Enables unified memory (CPU + GPU share address space)
  * Enables memory overcommit (allocate more than physical HBM)

GPU Page Table Structure (GPUVM)
AMD GPUs use a multi-level page table, similar to CPU page tables:
GPU Virtual Address (48 bits): 0x7f12_3400_0000
Split into levels:
  +------------+------------+------------+------------+--------------+
  | PDB1 (9) | PDB0 (9) | PDE  (9) | PTE  (9) | Offset(12) |
  `-------+-------+------+-------+------+-------+------+-------+-------+--------+
       |          |          |          |           |
       v          v          v          v           v
  Page Dir    Page Dir   Page Dir   Page Table   4KB Page
  Base 1      Base 0     Entry      Entry        (physical)
  Each level is a 512-entry table (9 bits = 512 entries)
  Final page size: 4 KB (12-bit offset)
  Also supports 2 MB and 1 GB huge pages for efficiency

TLB - Translation Lookaside Buffer
Looking up page tables for every memory access would be incredibly slow. The GPU has a TLB cache:
GPU Compute Unit memory access:
  v_global_load_dword v0, [v1]    <- load from address in v1
  Step 1: Check L1 TLB (per-CU, ~32 entries)
          Hit? -> use cached translation (1 cycle)
          Miss v

  Step 2: Check L2 TLB (shared, ~512 entries)
          Hit? -> use cached translation (~10 cycles)
          Miss v

  Step 3: Hardware Page Table Walker
          Walks the 4-level page table in memory (~100-400 cycles)
          Caches result in TLB for next time

  For ML workloads: large sequential tensor accesses ->
  TLB hits are very high (>99%) because tensors are
  allocated in contiguous virtual memory
```

```
XNACK - The Unified Memory Magic
XNACK stands for "retry on page fault" - it's the hardware feature that enables unified memory (what hipMallocManaged relies on).
WITHOUT XNACK (xnack-):
  GPU accesses address 0x7f... that isn't in GPU page table
  -> GPU generates a fatal fault
  -> Kernel killed with SIGBUS or SIGSEGV
  -> Application crashes

  You MUST manually hipMemcpy data to GPU before accessing it.
WITH XNACK (xnack+):
  GPU accesses address 0x7f... that isn't in GPU page table
  -> GPU generates a RECOVERABLE page fault (not fatal!)
  -> GPU wavefront is PAUSED (other wavefronts keep running)
  -> Fault sent to HSA runtime -> kernel driver
  -> Driver migrates the page to GPU-accessible memory
  ->   (HBM on discrete GPU; page table update on APU)
  -> GPU page table updated
  -> Paused wavefront RETRIED (resumes execution)
  -> Access succeeds!

  Data migrates automatically on first touch.

The page fault flow in detail:
                          GPU Compute Unit
                                |
                    v_global_load v0, [address]
                                |
                                v
                       +---------------+
                       |  TLB Lookup  |
                       `---------+--------+
                              | MISS -> page table walk
                              v
                       +---------------+
                       |  Page Table  |
                       |    Walk      |
                       `---------+--------+
                              | PAGE NOT MAPPED!
                              v
               +--------------------------------+
               |  GPU generates page fault     |
               |  (XNACK = "retry" mechanism)  |
               |  Wavefront PAUSED             |
               `-----------------+-----------------+
                               |
               +---------------v----------------+
               |  Interrupt -> Kernel Driver    |
               |  (amdgpu.ko)                  |
               |                               |
               |  1. Find page in CPU RAM      |
               |  2. Allocate HBM page         |
               |  3. DMA copy: CPU -> GPU       |
               |     (via SDMA engine)         |
               |  4. Update GPU page table     |
               |  5. Invalidate old TLB entry  |
               |  6. Signal "retry ready"      |
               `-----------------+-----------------+
                               |
               +---------------v----------------+
               |  GPU retries the access        |
               |  TLB now has valid mapping     |
               |  Load succeeds!                |
               |  Wavefront RESUMES             |
               `-----------------------------------+
Time for a page fault: ~10-50 mus (MUCH slower than normal access: ~100 ns)
That's 100-500x slower - this is why managed memory has performance issues!

XNACK On vs Off - The Trade-Off
                     XNACK OFF (xnack-)         XNACK ON (xnack+)
                     -----------------          --------------------
ISA Target:          gfx90a:xnack-              gfx90a:xnack+

Kernel compilation:  Slightly faster code        Slightly slower code
                     (no retry logic in ISA)     (retry bookkeeping per load)

Page faults:         FATAL (crash)               Recoverable (auto-migrate)

hipMallocManaged:    Requires prefetch hints      Works transparently

Performance:         ~2-5% faster for pure GPU   ~2-5% slower overhead
                     workloads                    but unified memory works
```

```
Used in practice:    ML inference (explicit       Research / prototyping /
                     memory management)           MI300X unified memory
MIGraphX default:    xnack OFF                   (maximum performance)

MI300X and XNACK - A Special Case
MI300X architecture:
  +---------------------------------------------+
  |  CPU chiplets (EPYC cores)                |
  |  `---- Connected to ---+                     |
  |                        v                    |
  |              +----------------+              |
  |              |  Shared HBM3  |              |
  |              |   192 GB      |              |
  |              |  5,300 GB/s   |              |
  |              `-----------------+              |
  |                        ^                    |
  |  GPU chiplets (XCDs) -+                     |
  `---------------------------------------------+
  CPU and GPU access the SAME physical HBM.
  XNACK page faults are nearly free because
  there's no PCIe transfer - just a page table update.

  This makes hipMallocManaged nearly as fast as hipMalloc on MI300X!
  (Unlike MI210 where page faults trigger slow PCIe transfers)

XNACK on APU (Radeon 8060S / Strix Halo) -- Deep Dive

  On the 8060S, XNACK is not optional -- it's MANDATORY for useful inference.

  Why XNACK=1 is required on APU:
    * hipMalloc without XNACK -> allocates from 512 MB VRAM carve-out only
    * hipMallocManaged without XNACK -> fine-grained coherent only (bypasses L2)
    * With XNACK=1 -> unified virtual addressing across all 29 GB available
    * Page faults on APU are CHEAPER than on discrete GPUs:
      No PCIe transfer needed -- CPU and GPU share the same physical LPDDR5x
      Fault handling = page table update + TLB invalidation only (~5-10 mus)
      Compare: MI210 page fault = PCIe DMA copy (~20-50 mus)

  ISA compilation with xnack+:
    Compiler flag: --offload-arch=gfx1151 (RDNA 3.5 always has xnack capability)
    When HSA_XNACK=1 at runtime, ISA is built as: gfx1151:xnack+
    The +xnack ISA variant adds retry bookkeeping to every global memory load:
      v_global_load_b32 v0, v[1:2], off    // load instruction
      // With xnack+: hardware saves wavefront state before the load
      // If the load faults, wavefront is paused (not killed)
      // After fault is resolved, instruction is retried from saved state
    Without +xnack: the same fault kills the wavefront -> application crash

  ISA compilation with xnack-:
    When HSA_XNACK=0: ISA is built as: gfx1151:xnack-
    No retry logic -> slightly smaller code, ~2-5% faster memory access
    But hipMallocManaged pages that aren't pre-faulted -> SIGBUS/SIGSEGV

  Environment Variable Interaction Matrix:
    +----------------------+-----------------+-------------------------------+
    | HSA_XNACK          | Allocator      | Behavior                    |
    |----------------------+----------------+-------------------------------|
    | 0 (default)        | hipMalloc      | 512 MB VRAM only -> OOM     |
    | 0                  | hipMallocManaged| Fine-grain only, no L2     |
    | 1 (required APU)   | hipMalloc      | 512 MB VRAM only           |
    | 1                  | hipMallocManaged| Full unified memory, L2    |
    |                    |                | with coarse-grain (Fix D)   |
    | 1 + Fix B + Fix D  | hipMallocManaged| * OPTIMAL: unified memory  |
    |                    | + CoarseGrain  | + L2 caching + 29 GB pool  |
    `----------------------+-----------------+-------------------------------+

  Page migration flow on APU (with XNACK=1):
    First inference (cold start):
      1. compute_func runs -> prog.run_async()
      2. Kernel reads weight at virtual address 0x7f...
      3. GPU TLB miss -> page table walk -> page not mapped to GPU
      4. GPU page fault generated (XNACK retry)
      5. Wavefront paused, fault delivered to driver
      6. Driver: page already in LPDDR5x (shared memory)
         -> Just updates GPU page table to point to same physical page
         -> No DMA copy needed (APU shares physical memory)
      7. TLB invalidated, wavefront retried
      8. Subsequent accesses hit TLB -> normal speed

    Subsequent inferences (warm):
      All pages already mapped in GPU page table
      Zero page faults, zero migration overhead
      Infinity Cache (32 MB MALL) holds hot weights
```

```
  Key difference from discrete GPU page faults:
    Discrete (MI210): fault -> DMA copy over PCIe -> 20-50 mus per 4KB page
    APU (8060S): fault -> page table update only -> 5-10 mus per 4KB page
    APU is ~3-5x faster at handling page faults because no data movement


5.5 How HSA, HIP, and the Kernel Driver Interact
-------------------------------------------------

+-----------------------------------------------------------------+
|  USERSPACE (your process)                                       |
|                                                                  |
|  hipMalloc(&ptr, size)                                          |
|    `----> hsa_amd_memory_pool_allocate(pool, size, 0, &ptr)      |
|           `----> ioctl(fd, AMDGPU_IOC_ALLOC_MEMORY, ...)         |
|                  (system call to kernel driver)                  |
|                                                                  |
|  hipLaunchKernel(func, grid, block, args, sharedMem, stream)   |
|    `----> write AQL packet to queue                               |
|    `----> hsa_signal_store(doorbell, write_index)                 |
|           `----> MMIO write to GPU register (NO system call!)     |
|                                                                  |
|  hipDeviceSynchronize()                                         |
|    `----> hsa_signal_wait(signal, EQ, 0, ...)                    |
|           `----> spin or block until GPU decrements signal        |
|                                                                  |
|------------------------- system call boundary ---------------------|
|                                                                  |
|  KERNEL SPACE (amdgpu.ko driver)                                |
|                                                                  |
|  Handles:                                                       |
|    * Memory allocation (physical HBM pages)                     |
|    * Page table updates (GPUVM)                                 |
|    * GPU reset on hang                                          |
|    * Power management                                           |
|    * XNACK page fault handling                                  |
|    * PCIe / XGMI configuration                                  |
|                                                                  |
`-----------------------------------------------------------------+
KEY INSIGHT:
  Memory allocation -> requires system call (slow, ~microseconds)
  Kernel launch     -> NO system call! Pure userspace MMIO write (fast, ~nanoseconds)

  This is why kernel launches are so fast and why AMD can
  dispatch thousands of kernels per millisecond.

  +-- DIAGRAM: HSA / HIP / Kernel Driver Relationship ----------------------+
  |                                                                         |
  |              +-------------------------+                                |
  |              |   HIP Runtime           |                                |
  |              |  (libamdhip64.so)       |                                |
  |              |  "Nice clean API"       |                                |
  |              `------------+---------------+                                |
  |                          | calls                                        |
  |              +-----------v-------------+                                |
  |              |   HSA Runtime           |                                |
  |              | (libhsa-runtime64.so)   |                                |
  |              | "Hardware mechanics"    |                                |
  |              `------+---------------+-------+                                |
  |                   |             |                                         |
  |        +-----------v---+   +------v-----------+                          |
  |        |  FAST PATH  |   |  SLOW PATH      |                          |
  |        |  (no ioctl) |   |  (ioctl syscall) |                          |
  |        |             |   |                  |                          |
  |        |  AQL packet |   |  Memory alloc    |                          |
  |        |  + doorbell |   |  Page table setup |                          |
  |        |  MMIO write |   |  Queue creation   |                          |
  |        |             |   |                  |                          |
  |        |  ~40 ns     |   |  ~1-5 mus        |                          |
  |        `---------+--------+   `----------+-------------+                          |
  |               |                  |                                       |
  |               `------------+-----------+                                |
  |                            v                                            |
  |              +-------------------------+                                |
  |              |   amdgpu.ko driver      |                                |
  |              |   (kernel space)        |                                |
  |              `------------+---------------+                                |
  |                          v                                              |
  |              +-------------------------+                                |
  |              |   GPU Hardware          |                                |
  |              `-------------------------+                                |
  `-------------------------------------------------------------------------+
```

Key Takeaway
HSA Runtime is the invisible infrastructure that makes everything above it work. You never call it directly for ML, but every hipMalloc

5.6 Where Every Byte Lives -- Layer 6 (HSA Runtime)
---

```
  +-----------------------------------------------------------------+
  |  HSA agent descriptors                                          |
  |    Where: userspace structs (hsa_agent_t) in process memory     |
  |    Populated: at hsa_init() by reading topology from /sys/      |
  |    Contains: agent type (CPU/GPU), name, CU count, ISA info     |
  |                                                                  |
  |  Memory pools (hsa_amd_memory_pool_t)                           |
  |    Where: descriptors in userspace, physical pages in VRAM/GTT  |
  |    Types:                                                        |
  |      VRAM pool:     512 MB on-die (8060S) / 64 GB HBM (MI210)  |
  |      GTT pool:      28 GB pinned LPDDR5x (APU) / DDR5 (server) |
  |      Kernarg pool:  GPU-visible system memory for arguments     |
  |      Fine-grain:    coherent across CPU+GPU (bypasses L2)       |
  |      Coarse-grain:  GPU-cached (Fix D enables this for weights) |
  |                                                                  |
  |  Hardware queue (hsa_queue_t)                                   |
  |    Where: userspace struct + GPU-visible ring buffer (mmap'd)   |
  |    base_address:  points to ring buffer (power-of-2 size)       |
  |    doorbell_signal: MMIO register address (memory-mapped I/O)   |
  |    write_index:  64-bit atomic, incremented by producer (CPU)   |
  |    read_index:   64-bit atomic, incremented by consumer (GPU)   |
  |    Size:         4K-128K slots x 64 bytes per queue (power-of-2)|
  |                                                                  |
  |  AQL packets (in the ring buffer)                               |
  |    Where: ring buffer slots (64 bytes each, GPU-visible memory) |
  |    Written by: CPU (HSA Runtime writes header last, atomically) |
  |    Read by:    GPU Command Processor (hardware)                 |
  |    Header byte written LAST to make packet visible atomically   |
  |                                                                  |
  |  Doorbell register                                              |
  |    Where: MMIO BAR region (memory-mapped PCI register)          |
  |    Written by: hsa_signal_store -> userspace MMIO write          |
  |    Read by:    GPU hardware (interrupt -> CP fetches packets)    |
  |    Latency:    ~100 ns (no syscall, direct MMIO from userspace) |
  |                                                                  |
  |  Signals (hsa_signal_t)                                         |
  |    Where: 64-bit atomic in GPU-visible SVM memory               |
  |    Used for: completion notification, inter-queue dependencies  |
  |    Mechanism: GPU atomically decrements -> CPU waits/polls       |
  |                                                                  |
  |  Kernarg segment                                                |
  |    Where: GPU-visible system memory (allocated from kernarg pool)|
  |    Contains: copy of kernel arguments (tensor pointers, dims)   |
  |    Size: per-kernel, typically 64-256 bytes per launch           |
  |    Lifetime: until kernel completes                              |
  `-----------------------------------------------------------------+
```

```
==============================================================================
                        PART III -- THE SYSTEM LAYERS
==============================================================================
```

```
==============================================================================
  CHAPTER 6: amdgpu Kernel Driver (Layer 7)
  The Gatekeeper -- Registers, TTM, and Memory Pools
==============================================================================

+-- EXECUTIVE SUMMARY -- Layer 7: amdgpu Kernel Driver ------------------+
|                                                                         |
|  What:  Linux kernel module (amdgpu.ko) -- only code with HW access.|
|  Input: ioctl calls from HSA/ROCt (KFD_IOC_*) + doorbell writes.   |
|  Output: Configured GPU hardware, managed memory pools, interrupts. |
|                                                                         |
|  Three subsystems:                                                      |
|    KFD (/dev/kfd)  -- Kernel Fusion Driver: queue creation, process  |
|                       management, event signaling, GPU page faults   |
|    TTM              -- Translation Table Manager: manages VRAM pool  |
|                       (512 MB) + GTT pool (28 GB) + eviction/swap   |
|    GPUVM            -- GPU page tables: virtual -> physical mapping,  |
|                       per-process isolation (VMIDs), TLB management  |
|                                                                         |
|  APU specifics (Radeon 8060S):                                          |
|    VRAM = 512 MB carved from LPDDR5x (same physical memory as GTT) |
|    GTT  = 28 GB of pinned LPDDR5x (GPU-accessible system RAM)     |
```

```
|    No PCIe -- CPU and GPU on same die, same memory bus              |
|    Page faults (XNACK=1): driver updates GPUVM page table entries, |
|    no data copy needed (just remap) -> ~5-10 us per fault           |
|                                                                      |
|  Firmware: PSP (security), SMU (power/clocks), SDMA (DMA engine)   |
|  Key path: drivers/gpu/drm/amd/amdgpu/ + amdkfd/ in kernel source |
`----------------------------------------------------------------------+

  +-- SOURCE REFERENCES -- Layer 7 (Kernel Driver) --------------------------+
  |                                                                           |
  |  Repository: Linux kernel (upstream)                                     |
  |  Path:       drivers/gpu/drm/amd/                                        |
  |                                                                           |
  |  File                                          Key Function              |
  |  ---------------------------------------       ----------------------    |
  |  amdgpu/amdgpu_gem.c                          amdgpu_gem_create()       |
  |                                                 (BO allocation: ioctl) |
  |  amdgpu/amdgpu_cs.c                           amdgpu_cs_ioctl()         |
  |                                                 (command submission)    |
  |  amdgpu/amdgpu_ttm.c                          TTM buffer management    |
  |                                                 (VRAM/GTT pools)        |
  |  amdgpu/amdgpu_vm.c                           amdgpu_vm_bo_map()        |
  |                                                 (page table updates)    |
  |  amdgpu/amdgpu_gmc.c                          Memory controller init   |
  |  amdkfd/kfd_chardev.c                         KFD ioctl dispatch        |
  |                                                 (/dev/kfd interface)    |
  |  amdkfd/kfd_device_queue_manager.c             Queue management        |
  |  amdkfd/kfd_svm.c                             SVM range management     |
  |                                                 (XNACK page faults)     |
  |                                                                           |
  |  Online source (kernel.org):                                             |
  |  https://elixir.bootlin.com/linux/latest/source/                         |
  |    drivers/gpu/drm/amd/amdgpu/                                           |
  |    drivers/gpu/drm/amd/amdkfd/                                           |
  |                                                                           |
  |  ROCm KFD kernel driver:                                                 |
  |  https://github.com/ROCm/ROCK-Kernel-Driver/tree/rocm-7.2.0/           |
  |    drivers/gpu/drm/amd/                                                  |
  `---------------------------------------------------------------------------+

We've finally reached the bottom of the software stack - the kernel driver. This is the code that runs inside the Linux kernel with ful

+-------------------------------------------------------------------------+
|  BEGINNER'S CONTEXT: "Kernel" Confusion Reminder                        |
|                                                                          |
|  In this chapter, "kernel" means the LINUX KERNEL (the operating        |
|  system core), NOT a GPU kernel (computation function).                  |
|  See Chapter 0 section 0.4 for the full disambiguation.                 |
|                                                                          |
|  "Kernel driver" = a program that runs INSIDE the Linux OS with         |
|  special privileges. It can directly access hardware registers,         |
|  allocate physical memory, and handle interrupts.                       |
|                                                                          |
|  "Kernel space" vs "Userspace":                                         |
|    Userspace = where normal programs run (Python, MIGraphX, HIP, HSA) |
|      - CANNOT touch hardware directly                                    |
|      - Must ask the kernel driver via system calls (ioctl)              |
|    Kernel space = where the OS and drivers run (amdgpu.ko)              |
|      - CAN directly read/write GPU hardware registers                   |
|      - A bug here crashes the ENTIRE computer (not just your program)  |
|                                                                          |
|  As a Python developer, you never interact with this layer directly.   |
|  But when hipMalloc() allocates GPU memory, it internally makes an    |
|  ioctl system call that reaches this driver. The driver then            |
|  physically reserves memory pages on the GPU hardware.                  |
`-------------------------------------------------------------------------+


6.1 What is a Kernel Driver?
----------------------------

"Kernel" here means Linux kernel, NOT GPU kernel.
A kernel driver is a piece of code that runs as part of the Linux operating system itself, with unrestricted access to hardware. It's t
Analogy: Think of a bank vault.
* Userspace programs (HIP, MIGraphX) are the bank customers -- they fill out withdrawal slips
* The kernel driver is the vault teller -- the only person with keys to open the vault
* The GPU hardware is the vault itself -- contains valuable resources, only accessible through the teller
Just as customers cannot walk into the vault directly, userspace programs cannot touch hardware registers directly -- they must go thro
USERSPACE (your process):
  Can do:     call HIP functions, read/write shared memory
  Cannot do:  access hardware registers, allocate physical pages,
              program DMA engines, handle interrupts
```

```
KERNEL SPACE (amdgpu.ko):
  Can do:     EVERYTHING - hardware registers, physical memory,
              interrupts, DMA, power management
  Risk:       one bug = system crash (kernel panic), data corruption,
              or security vulnerability

Where amdgpu.ko Sits
+-----------------------------------------------------------------+
|  Userspace                                                      |
|  +------------+  +------------+  +------------+                  |
|  | MIGraphX |  | HIP      |  | HSA      |                  |
|  |          |-> | Runtime  |-> | Runtime  |                  |
|  `-------------+  `-------------+  `-------+-------+                  |
|                                     | ioctl() system calls     |
|                                     | + MMIO doorbell writes   |
|-------------------------------------+--------------------------|
|  Kernel Space                       v                          |
|  +-------------------------------------------------------+   |
|  |                                                       |   |
|  |  amdgpu.ko  <-- YOU ARE HERE                         |   |
|  |  (kernel module, loaded at boot)                      |   |
|  |                                                       |   |
|  |  +---------------+ +--------------+ +-------------------+ |   |
|  |  | GPU Memory  | |  Command   | |  Power/Thermal  | |   |
|  |  | Manager     | |  Submission| |  Management     | |   |
|  |  | (TTM)       | |  (scheduler)| |  (SMU interface)| |   |
|  |  `----------------+ `---------------+ `--------------------+ |   |
|  |  +---------------+ +--------------+ +-------------------+ |   |
|  |  | Interrupt   | |  Display   | |  PCIe / XGMI    | |   |
|  |  | Handler     | |  (KMS/DRM) | |  Link Manager   | |   |
|  |  `----------------+ `---------------+ `--------------------+ |   |
|  |                                                       |   |
|  `-------------------------------------------------------+   |
|                             |                                 |
|  Linux Kernel Infrastructure |                                |
|  +----------------------------v---------------------------+   |
|  |  DRM subsystem | TTM | PCIe core | IOMMU | scheduler  |   |
|  `-------------------------------------------------------+   |
|-----------------------------------------------------------------|
|  Hardware                                                       |
|  +-------------------------------------------------------+   |
|  |  AMD GPU Silicon (8060S / MI210 / MI300X)              |   |
|  |  Registers | Memory* | CUs | DMA | Command Processor  |   |
|  |  *HBM on discrete (MI210/MI300X), LPDDR5x on APU (8060S)|  |
|  `-------------------------------------------------------+   |
`-----------------------------------------------------------------+

The Three Core Responsibilities
(1) Direct hardware register access    -> "Talk to the GPU silicon"
(2) GPU memory manager (TTM subsystem) -> "Manage physical GPU memory"
(3) VRAM + GTT pool management         -> "Decide where data lives"


6.2 Part 1: Direct Hardware Register Access
--------------------------------------------

What Are Hardware Registers?
GPU registers are tiny memory locations built into the GPU silicon that control every aspect of the hardware. Think of them as the GPU
Types of GPU registers:
+-------------------------------------------------------------------+
|  MMIO Registers (Memory-Mapped I/O)                               |
|    * Mapped into CPU's address space by the PCIe BAR              |
|    * CPU reads/writes these like normal memory                    |
|    * But each read/write triggers a hardware action on the GPU    |
|                                                                   |
|  Examples:                                                        |
|    GRBM_STATUS         (0x8010): GPU status (busy/idle/fault)    |
|    SRBM_STATUS         (0x0E50): System block status             |
|    CP_RB_BASE          (0xC100): Command ring buffer base addr   |
|    GCEA_EDC_CNT        (0x????): ECC error counters              |
|    MMHUB_VM_FB_LOCATION(0x????): VRAM address window             |
`-------------------------------------------------------------------+
+-------------------------------------------------------------------+
|  Doorbell Registers (special MMIO)                                |
|    * Each queue has its own doorbell register                     |
|    * Writing to it wakes up the GPU's Command Processor           |
|    * Mapped to userspace (no system call needed!)                 |
|    * This is why kernel launches are fast (~5 mus)                |
`-------------------------------------------------------------------+
+-------------------------------------------------------------------+
|  Indirect Registers                                               |
|    * Not directly accessible - must write an index register,     |
|      then read/write a data register                              |
```

```
|    * Used for: GPU configuration, clock management, etc.         |
`-------------------------------------------------------------------+
```

PCIe BARs - How CPU Sees GPU Registers

The GPU's registers and memory are exposed to the CPU through PCIe Base Address Registers (BARs):

PCIe BARs for an AMD MI210:

```
BAR 0: VRAM aperture (GPU memory visible to CPU)
  Physical address: 0xE000_0000_0000 (example)
  Size:             256 MB (or resizable up to full VRAM)
  Purpose:          CPU can directly read/write GPU VRAM
                    through this window
BAR 2: Doorbell aperture
  Physical address: 0xE100_0000_0000 (example)
  Size:             8 MB
  Purpose:          Queue doorbell registers
                    HSA runtime maps this to userspace
BAR 4: Register aperture (MMIO)
  Physical address: 0xE200_0000_0000 (example)
  Size:             512 KB
  Purpose:          GPU control/status registers
                    ONLY kernel driver accesses this
```

You can see your GPU's BARs with:

```
  $ lspci -v -s 63:00.0
  Region 0: Memory at ... (64-bit, prefetchable) [size=16G]
  Region 2: Memory at ... (64-bit, prefetchable) [size=8M]
  Region 4: Memory at ... (32-bit, non-prefetchable) [size=512K]
```

What the Driver Does With Registers

GPU INITIALIZATION (at boot / module load):

```
  +-------------------------------------------------------------+
  |  amdgpu_device_init():                                       |
  |                                                              |
  |  1. Read GPU identity registers                              |
  |     RREG32(mmGRBM_CHIP_ID) -> "This is gfx90a (MI210)"   |
  |                                                              |
  |  2. Configure memory controller                              |
  |     WREG32(mmMC_VM_FB_LOCATION_BASE, vram_base_addr)      |
  |     WREG32(mmMC_VM_FB_LOCATION_TOP, vram_end_addr)        |
  |     "VRAM starts at physical address X, ends at Y"          |
  |                                                              |
  |  3. Initialize Command Processor                             |
  |     WREG32(mmCP_RB_BASE, ring_buffer_gpu_addr)            |
  |     WREG32(mmCP_RB_CNTL, ring_size | enable_bits)         |
  |     "Here's your command queue, start processing"            |
  |                                                              |
  |  4. Set up interrupt handling                                |
  |     WREG32(mmIH_RB_BASE, interrupt_ring_addr)             |
  |     "Send interrupts to this ring buffer"                    |
  |                                                              |
  |  5. Configure XGMI links (multi-GPU)                         |
  |     WREG32(mmXGMI_..., link_config)                          |
  |     "Connect GPU 0 <-> GPU 1 at 92 GB/s"                     |
  |                                                              |
  |  6. Load firmware (microcode) into GPU engines               |
  |     Upload CP firmware (Command Processor microcode)         |
  |     Upload SDMA firmware (DMA engine microcode)              |
  |     Upload SMU firmware (power management microcode)         |
  `-------------------------------------------------------------+
```

GPU Status Monitoring

The driver continuously reads status registers:

GRBM_STATUS register (0x8010) bit fields:

```
+-------------------------------------------------------------+
| Bit 31   | GUI_ACTIVE      | 1 = GPU is doing work         |
| Bit 30   | CP_BUSY         | 1 = Command Processor busy    |
| Bit 29   | CP_COHERENCY_BZ | 1 = cache flush in progress   |
| Bit 28   | CP_GDS_BUSY     | 1 = Global Data Share busy    |
| Bit 27   | DB_BUSY         | 1 = depth buffer busy (gfx)   |
| Bit 26   | CB_BUSY         | 1 = color buffer busy (gfx)   |
| Bit 25   | PA_BUSY         | 1 = primitive assembly busy   |
| Bit 24   | SC_BUSY         | 1 = scan converter busy       |
| Bit 22   | SX_BUSY         | 1 = shader export busy        |
| Bit 21   | SPI_BUSY        | 1 = shader processor busy     |
| Bit 19   | TA_BUSY         | 1 = texture addressing busy   |
| Bit 14   | GDS_BUSY        | 1 = global data share busy    |
| ...      | ...             |                               |
`-------------------------------------------------------------+
```

When you run `watch -n1 cat /sys/kernel/debug/dri/0/amdgpu_gpu_status` the driver reads this register and reports which parts are active.

GPU Reset - When Things Go Wrong

If a kernel hangs (infinite loop, deadlock), the driver must recover:

GPU Hang Detection:

```
  1. Driver sets a timer when work is submitted
  2. If completion signal doesn't fire within timeout (~10s):
     -> GPU is considered hung
GPU Reset Sequence:
  +-----------------------------------------------------------+
  |  amdgpu_device_gpu_recover():                              |
  |                                                            |
  |  1. Stop all userspace queues                              |
  |     Block new submissions                                  |
  |                                                            |
  |  2. Try "soft reset" first                                 |
  |     WREG32(mmGRBM_SOFT_RESET, reset_mask)                 |
  |     Wait for GPU to respond                                |
  |                                                            |
  |  3. If soft reset fails -> "hard reset" (mode 1)           |
  |     Write to PSP (Platform Security Processor)             |
  |     PSP resets the entire GPU                              |
  |                                                            |
  |  4. If hard reset fails -> "PCI link reset" (mode 2)       |
  |     Trigger PCIe FLR (Function Level Reset)                |
  |     Entire PCIe device resets                              |
  |                                                            |
  |  5. Reload firmware                                        |
  |  6. Reinitialize all hardware blocks                       |
  |  7. Restore userspace state (queues, page tables)          |
  |  8. Resume operations                                      |
  |                                                            |
  |  All running kernels are LOST. User programs get error     |
  |  codes. They must re-submit work.                          |
  `-----------------------------------------------------------+
```

## 6.3 Part 2: GPU Memory Manager - TTM Subsystem

```
What is TTM?
TTM = Translation Table Manager

+-------------------------------------------------------------------------+
|  BEGINNER'S NOTE: What is a "Subsystem"?                                |
|                                                                          |
|  A subsystem is a component within a larger system. The Linux kernel   |
|  is made up of many subsystems, each handling a different job:          |
|    - Networking subsystem: handles WiFi, Ethernet                       |
|    - Filesystem subsystem: handles reading/writing files                |
|    - TTM subsystem: handles GPU memory management                       |
|                                                                          |
|  Think: Python has "packages" (like os, sys, json). Linux kernel        |
|  has "subsystems" (like TTM, DRM, PCIe).                                |
|                                                                          |
|  "Buffer Object (BO)": a chunk of GPU memory that TTM tracks.           |
|  Every time you call hipMalloc(), TTM creates a BO internally.          |
|  It knows the size, location, who is using it, and whether it can       |
|  be moved or evicted if memory runs low.                                |
|                                                                          |
|  Python analogy: if hipMalloc is like "x = bytearray(size)",            |
|  then the BO is the internal bookkeeping object that the memory         |
|  manager creates to track that allocation.                              |
`-------------------------------------------------------------------------+
TTM is a Linux kernel subsystem (not AMD-specific) for managing GPU memory. It's used by AMD (amdgpu), older NVIDIA (nouveau), and othe
Analogy: If GPU HBM is a parking garage, TTM is the parking management system - it knows which spaces are empty, which are occupied, ca
Linux Memory Management Stack:
+-------------------------------------------+
|  Application (hipMalloc)                  |
|-------------------------------------------|
|  HSA Runtime (hsa_memory_pool_alloc)      |
|-------------------------------------------|
|  amdgpu driver (ioctl)                    |
|-------------------------------------------|
|  TTM (Translation Table Manager)          |  <- GPU memory OS
|    |---- Buffer Object (BO) management    |
|    |---- Placement policies               |
|    `---- Eviction / migration             |
|-------------------------------------------|
|  Physical GPU memory (HBM)                |
`-------------------------------------------+

Buffer Objects (BOs) - The Fundamental Unit
Every GPU memory allocation creates a Buffer Object (BO) - TTM's basic unit of memory management:
struct amdgpu_bo {
    struct ttm_buffer_object  tbo;          // TTM base object

    uint64_t                  size;          // allocation size (bytes)
```

```
    uint32_t                  preferred_domain; // where it should live
    uint32_t                  allowed_domains;  // where it CAN live

    struct amdgpu_bo_va       *bo_va;       // virtual address mappings
    struct reservation_object *resv;        // sync: who's using this BO

    uint32_t                  flags;        // VRAM-only, CPU-visible, etc.
    struct list_head          list;         // linked into per-process BO list
};
When MIGraphX calls hipMalloc(14 GB):
  -> HSA runtime calls ioctl(AMDGPU_IOC_ALLOC_MEMORY)
  -> amdgpu driver creates a BO:
      size = 14 GB
      preferred_domain = VRAM (GPU HBM)
      -> TTM finds 14 GB of free HBM pages
      -> Maps into GPU page table (GPUVM)
      -> Returns GPU virtual address to userspace

TTM Placement - Where Memory Lives
TTM manages placement domains - physical locations where a BO can reside:
+-----------------------------------------------------------------+
|  TTM Placement Domains for amdgpu                                |
|                                                                  |
|  +-------------------------------------------------------------+ |
|  |  DOMAIN: VRAM (GPU Video RAM / HBM)                         | |
|  |                                                             | |
|  |  Physical: HBM chips soldered to GPU                        | |
|  |  Size:     64 GB (MI210) / 192 GB (MI300X)                 | |
|  |  Bandwidth: 1,638 GB/s (MI210) / 5,300 GB/s (MI300X)      | |
|  |  Access:   GPU (fast), CPU (slow, via PCIe BAR)            | |
|  |                                                             | |
|  |  Used for: model weights, intermediate tensors,             | |
|  |            kernel code objects, workspace buffers            | |
|  |                                                             | |
|  |  This is where 99% of ML data lives during inference        | |
|  `-------------------------------------------------------------+ |
|                                                                  |
|  +-------------------------------------------------------------+ |
|  |  DOMAIN: GTT (Graphics Translation Table = System RAM)      | |
|  |                                                             | |
|  |  Physical: regular LPDDR5x RAM (pinned / non-swappable) | |
|  |  Size:     configurable (typically 50% of RAM)             | |
|  |  Bandwidth: ~256 GB/s (LPDDR5x, direct on APU)             | |
|  |             so effective: ~64 GB/s (PCIe Gen4 x16)          | |
|  |  Access:   CPU (fast), GPU (PCIe on discrete, same bus     | |
|  |            on APU)                                          | |
|  |                                                             | |
|  |  Used for: staging area for CPU<->GPU transfers,             | |
|  |            overflow when VRAM is full,                       | |
|  |            kernel arguments, command buffers                 | |
|  `-------------------------------------------------------------+ |
|                                                                  |
|  +-------------------------------------------------------------+ |
|  |  DOMAIN: SYSTEM (unpinned system RAM)                       | |
|  |                                                             | |
|  |  Physical: regular LPDDR5x RAM (can be swapped to disk!)| |
|  |  GPU access: NOT directly accessible                        | |
|  |                                                             | |
|  |  Used for: BOs that are not currently needed by GPU         | |
|  |            (evicted from VRAM and GTT)                       | |
|  `-------------------------------------------------------------+ |
`-----------------------------------------------------------------+

Memory Allocation Flow - The Full Path
hipMalloc(&ptr, 14_GB)
|
v
HSA Runtime: hsa_amd_memory_pool_allocate(gpu_pool, 14 GB, ...)
|
v ioctl(fd, DRM_AMDGPU_GEM_CREATE, ...)
|
v
+-- amdgpu driver (kernel space) ----------------------------------+
|                                                                   |
|  1. Create Buffer Object (BO)                                     |
|     bo = amdgpu_bo_create(adev, 14 GB, DOMAIN_VRAM)           |
|                                                                   |
|  2. TTM: find free VRAM pages                                     |
|     +-----------------------------------------------------+     |
|     |  TTM VRAM manager:                                 |     |
|     |                                                    |     |
|     |  VRAM: 64 GB total                                 |     |
|     |  +-------------------------------------------------+ |     |
```

```
|     |  |########.................................| |     |
|     |  | used    |        free: 52 GB                | |     |
|     |  | 12 GB   |                                   | |     |
|     |  `-----------------------------------------------+ |     |
|     |                                                  |     |
|     |  Request: 14 GB -> fits! Allocate at offset 12 GB  |     |
|     |                                                  |     |
|     |  After allocation:                               |     |
|     |  +-----------------------------------------------+ |     |
|     |  |########################....................| |     |
|     |  | 12 GB  |  14 GB (new)  |  free: 38 GB       | |     |
|     |  `-----------------------------------------------+ |     |
|     `-------------------------------------------------------+     |
|                                                                    |
|  3. Map into GPU page table (GPUVM)                                |
|     amdgpu_vm_bo_map(vm, bo, gpu_virtual_addr)                    |
|     Update GPU page tables so GPU can access this memory           |
|     GPU virtual addr: 0x00007f12_0000_0000                        |
|                                                                    |
|  4. Return handle to userspace                                     |
|                                                                    |
`--------------------------------------------------------------------+
|
v
HSA Runtime: maps BO handle -> userspace pointer
|
v
hipMalloc returns: ptr = 0x00007f12_0000_0000 (GPU virtual address)

Eviction - When VRAM is Full
What happens when you try to allocate more than the available VRAM?
Scenario: 64 GB VRAM, already 60 GB used, request for 8 GB
TTM: "Not enough VRAM! Need to evict something."
Eviction process:
+-------------------------------------------------------------------+
|                                                                    |
|  1. Find eviction candidates                                       |
|     TTM uses LRU (Least Recently Used) list                       |
|     Pick BOs that haven't been accessed recently                   |
|                                                                    |
|  2. Check if BO is currently in use by GPU                         |
|     reservation_object_test_signaled(bo->resv)                    |
|     If GPU is still using it -> can't evict yet, skip              |
|                                                                    |
|  3. Move BO from VRAM -> GTT (system RAM)                          |
|     +----------------+         +----------------+                  |
|     |  VRAM (HBM)   |  DMA   |  GTT (LPDDR5x)|                  |
|     |  [old BO data]| =====> |  [BO data]    |                   |
|     |  8 GB         | SDMA   |  8 GB         |                   |
|     |               | engine |               |                   |
|     `-----------------+         `-----------------+                  |
|     Time: 8 GB / 64 GB/s (PCIe, discrete) ~= 125 ms            |
|     APU: 8 GB / 256 GB/s (same bus) ~= 31 ms (faster)          |
|                                                                    |
|  4. Update GPU page table                                          |
|     Old BO's GPU virtual address now points to GTT location      |
|     If old BO's owner runs a kernel, it accesses via PCIe        |
|     (20-50x slower than VRAM - owner's perf degrades)              |
|                                                                    |
|  5. Allocate new BO in freed VRAM space                            |
|     New 8 GB BO gets the VRAM space                                |
|                                                                    |
|  If GTT also full:                                                 |
|     Move from GTT -> SYSTEM (swap to disk)                         |
|     This is catastrophically slow for ML!                          |
|                                                                    |
`-------------------------------------------------------------------+
This is why model size matters:
  Model fits in VRAM -> fast inference
  Model doesn't fit  -> eviction hell -> 10-100x slower


6.4 Part 3: VRAM + GTT Pool Management
---------------------------------------

Your System's Pools - What You Saw
$ cat /sys/class/drm/card0/device/mem_info_vram_total
   -> VRAM: the GPU's HBM
$ cat /sys/class/drm/card0/device/mem_info_gtt_total
   -> GTT: pinned system RAM reserved for GPU use
On your system with kernel 6.18.0+, the driver reports:
+-------------------------------------------------------------------+
|  VRAM Pool: 512 MB                                                 |
```

```
|  GTT Pool:  28 GB                                                |
|                                                                  |
|  This looks like an integrated GPU (APU) or a small discrete   |
|  GPU - NOT an MI210/MI300X data center GPU.                      |
|                                                                  |
|  For comparison:                                                 |
|    MI210:  VRAM = 64 GB HBM2e,  GTT = ~256 GB (half of RAM)   |
|    MI300X: VRAM = 192 GB HBM3,  GTT = ~512 GB (half of RAM)   |
|    Your system: VRAM = 512 MB,  GTT = 28 GB                   |
|                                                                  |
|  512 MB VRAM suggests an AMD APU (like Ryzen with integrated   |
|  Radeon graphics) where the GPU shares system RAM.               |
`------------------------------------------------------------------+

How VRAM and GTT Interact
The memory hierarchy from GPU's perspective:
Speed        +----------------------------+
fastest      |  LDS (per CU)            |  64 KB/CU, ~50 TB/s
  |          |  (managed by kernel code) |
  |          |----------------------------|
  |          |  L2 Cache                 |  16 MB (MI210), shared
  |          |----------------------------|
  |          |  VRAM / HBM              |  64-192 GB, 1.6-5.3 TB/s
  |          |  (managed by TTM)        |  <- PRIMARY storage for ML
  |          |----------------------------|
  |          |  GTT (System RAM)        |  ~256 GB, ~64 GB/s PCIe (discrete)
  |          |                          |  or ~256 GB/s same bus (APU)
  |          |  (managed by TTM)        |  <- OVERFLOW / staging
  |          |----------------------------|
  v          |  System swap (disk)      |  ~2 GB/s (SSD) or ~0.1 GB/s (HDD)
slowest      |  (managed by Linux VM)   |  <- DISASTER for ML
             `----------------------------+

APU Memory Management (Your 512 MB VRAM System)
Since your system has 512 MB VRAM, you likely have an APU (Accelerated Processing Unit) where CPU and GPU share system RAM:
APU Memory Layout:
+------------------------------------------------------------------+
|                    System RAM (e.g., 32 GB LPDDR5x)            |
|                                                                  |
|  +--------------------+  +-----------------------------------+  |
|  |  VRAM carve-out  |  |  Regular system RAM               |  |
|  |    512 MB        |  |    ~28 GB                         |  |
|  |                  |  |                                   |  |
|  |  Exclusive to GPU|  |  Shared: CPU normal use           |  |
|  |  Fastest for GPU |  |  GTT pool comes from here         |  |
|  |  (no contention) |  |  (pinned pages for GPU access)    |  |
|  `---------------------+  `-----------------------------------+  |
|                                                                  |
|  On APU: GPU accesses ALL RAM at the same speed               |
|  (no PCIe bottleneck - it's all on the same memory bus!)      |
|                                                                  |
|  The 512 MB carve-out is just a GUARANTEED reservation        |
|  The GPU can use GTT (28 GB) at nearly the same speed         |
|                                                                  |
|  APU bandwidth: ~256 GB/s (shared LPDDR5x bus)               |
|  Compare MI210: ~1,638 GB/s (dedicated HBM2e)               |
|  That's ~6x less bandwidth - APUs trade BW for integration    |
`------------------------------------------------------------------+

  +-- DIAGRAM: TTM Buffer Object Migration ----------------------------------+
  |                                                                          |
  |  When GPU memory is full, TTM moves BOs between pools:                 |
  |                                                                          |
  |  +----------------+    evict     +----------------+    swap    +----------+|
  |  |   VRAM        |----------->|    GTT         |--------->|  Swap  ||
  |  |  512 MB (APU) |<-----------|   28 GB        |<---------|  disk  ||
  |  |  64 GB (MI210)|   migrate   |   (sys RAM)    |  swap in  |        ||
  |  `-----------------+    back      `-----------------+           `-----------+|
  |                                                                          |
  |  Allocation flow for a 14 GB model on APU:                             |
  |                                                                          |
  |  hipMalloc(14 GB)                                                      |
  |    |---> Try VRAM (512 MB) -> FULL! Only 512 MB available               |
  |    |---> Overflow to GTT (28 GB) -> OK, 14 GB fits                      |
  |    |     TTM pins 14 GB of system RAM pages for GPU                    |
  |    |     GPUVM page table maps GPU VA -> physical LPDDR5x pages        |
  |    `---> If GTT also full -> swap old BOs to disk (very slow!)          |
  |                                                                          |
  |  With hipMallocManaged + XNACK=1 (your setup):                        |
  |    Pages start unmapped -> GPU accesses -> page fault ->                |
  |    driver maps page from wherever available -> GPU retries              |
  |    No explicit migration needed, demand-paged on access               |
  `--------------------------------------------------------------------------+
```

```
TTM's Internal Data Structures
TTM manages memory using these structures:
Per-GPU (amdgpu_device):
+-----------------------------------------------------------------+
|  VRAM manager (ttm_range_manager)                                |
|  +-------------------------------------------------------------+ |
|  |  Free list:    [0x000-0x1FF] (512 MB)                     | |
|  |  Allocated:    (empty initially)                          | |
|  |  Fragmentation map: tracks which pages are free           | |
|  |  Uses drm_mm (interval tree) for O(log n) alloc/free      | |
|  `-------------------------------------------------------------+ |
|                                                                  |
|  GTT manager (ttm_range_manager)                                 |
|  +-------------------------------------------------------------+ |
|  |  Free list:    28 GB of system RAM pages                  | |
|  |  Pinned pages: kernel locks these so they can't swap      | |
|  |  IOMMU mapped: so GPU can DMA to/from these pages         | |
|  `-------------------------------------------------------------+ |
|                                                                  |
|  Per-process BO tracking:                                        |
|  +-------------------------------------------------------------+ |
|  |  Process 1234 (MIGraphX):                                 | |
|  |    BO_0: 14 GB weights     -> VRAM offset 0x000            | |
|  |    BO_1: 512 MB workspace  -> VRAM offset 0x380000000      | |
|  |    BO_2: 64 MB outputs     -> VRAM offset 0x3A0000000      | |
|  |    BO_3: 4 KB kernel args  -> GTT                          | |
|  |                                                            | |
|  |  Process 5678 (another user):                              | |
|  |    BO_0: 8 GB model        -> VRAM offset 0x3C0000000      | |
|  |    ...                                                     | |
|  `-------------------------------------------------------------+ |
`-------------------------------------------------------------------+

SDMA Engine - The Data Mover
The GPU has dedicated SDMA (System DMA) engines that handle memory copies without using compute units:
SDMA Engines (MI210 has 4, MI300X has more):
+-----------------------------------------------------------------+
|  SDMA Engine 0                                                   |
|    * Copies data: VRAM <-> GTT (system RAM)                      |
|    * Used by: hipMemcpy(H2D), hipMemcpy(D2H)                     |
|    * Also used by: TTM eviction (VRAM -> GTT migration)          |
|    * Has its own command queue (independent of compute)          |
|    * Can run IN PARALLEL with GPU compute kernels!               |
|                                                                  |
|  GPU Compute: --[MatMul]--[Softmax]--[MatMul]--                  |
|  SDMA Engine: --[Copy next batch input H->D]------               |
|                                                                  |
|  This overlap is how MIGraphX hides data transfer latency        |
`-------------------------------------------------------------------+

Kernel Driver sysfs Interface - What You Can See
The driver exposes a wealth of information through /sys:
GPU Info:
  /sys/class/drm/card0/device/
  |---- mem_info_vram_total      -> 536870912 (512 MB)
  |---- mem_info_vram_used       -> current VRAM usage
  |---- mem_info_gtt_total       -> 30064771072 (28 GB)
  |---- mem_info_gtt_used        -> current GTT usage
  |---- gpu_busy_percent         -> 0-100% utilization
  |---- mem_busy_percent         -> memory controller utilization
  |---- current_link_speed       -> PCIe link speed
  |---- current_link_width       -> PCIe lane count (x16)
  |---- pp_cur_state             -> power state
  |---- hwmon/
  |   |---- temp1_input          -> GPU temperature (millidegrees C)
  |   |---- power1_average       -> current power draw (microwatts)
  |   `---- freq1_input          -> current GPU clock (Hz)
  `---- ras/                     -> ECC error counts (data center GPUs)
These files are the kernel driver reading hardware registers
and presenting them in human-readable form.


6.5 The Complete Flow - Register to Inference
---------------------------------------------

A single hipMalloc + hipLaunchKernel through the kernel driver:
+-- USERSPACE -------------------------------------------------------+
|                                                                     |
|  hipMalloc(&ptr, size)                                             |
|    -> ioctl(fd, DRM_AMDGPU_GEM_CREATE, {size, DOMAIN_VRAM})      |
|                                                                     |
|  hipLaunchKernel(func, grid, block, args, stream)                  |
```

```
|    -> HSA writes AQL packet to queue memory                       |
|    -> HSA writes to doorbell register (MMIO, no syscall)         |
|                                                                   |
|  hipDeviceSynchronize()                                          |
|    -> HSA spins/blocks on completion signal                       |
|                                                                   |
|--- KERNEL DRIVER (amdgpu.ko) -------------------------------------|
|                                                                   |
|  GEM_CREATE handler:                                             |
|    -> TTM allocates VRAM pages (drm_mm interval tree)            |
|    -> Programs GPU page table (GPUVM) via register writes         |
|    -> Returns BO handle                                           |
|                                                                   |
|  Doorbell write (no ioctl needed):                               |
|    -> CPU writes MMIO register mapped to userspace                |
|    -> GPU Command Processor hardware detects doorbell change      |
|    -> CP reads AQL packet from queue                              |
|    -> CP dispatches wavefronts to Compute Units                   |
|                                                                   |
|  Interrupt handler (when GPU signals completion):                |
|    -> GPU fires MSI interrupt                                     |
|    -> Driver's ISR reads interrupt status register                |
|    -> Updates completion signal value                              |
|    -> Wakes any waiting userspace threads                         |
|                                                                   |
|--- HARDWARE -------------------------------------------------------|
|                                                                   |
|  Command Processor: reads AQL, dispatches to CUs                 |
|  Compute Units: execute kernel ISA, access HBM                   |
|  Memory Controller: serves VRAM read/write requests              |
|  SDMA Engines: handle DMA copies (CPU<->GPU)                      |
|  Interrupt Controller: signals completion to driver              |
|                                                                   |
`-------------------------------------------------------------------+
```

Key Takeaway

The amdgpu kernel driver is the gatekeeper between software and silicon. It has two modes of operation:

1. Slow path (system calls): Memory allocation, GPU initialization, page table management, error recovery - these require ioctl calls t
2. Fast path (doorbell MMIO): Kernel launches and synchronization use memory-mapped registers that userspace can write directly - no sy

This split design is what makes AMD GPUs capable of launching thousands of kernels per second while still maintaining OS-level resource

## 6.6 Where Every Byte Lives -- Layer 7 (Kernel Driver)

```
  +-------------------------------------------------------------------+
  |  TTM Memory Pools (managed by the driver)                        |
  |                                                                   |
  |  VRAM pool (on-die SRAM or dedicated HBM)                        |
  |    8060S: 512 MB carved from LPDDR5x by BIOS                     |
  |    MI210: 64 GB HBM2e on-package                                 |
  |    Managed by: ttm_range_manager (free list + buddy allocator)   |
  |    Granularity: 4 KB pages                                       |
  |                                                                   |
  |  GTT pool (pinned system RAM, GPU-accessible)                    |
  |    8060S: ~28 GB (total RAM minus VRAM carve-out)                |
  |    MI210: ~28 GB (PCIe BAR mapped system RAM)                    |
  |    Pages pinned (non-swappable) for DMA access                   |
  |                                                                   |
  |  Page tables (GART -- GPU Address Remapping Table)               |
  |    Where: system RAM, managed by amdgpu_vm.c                     |
  |    Maps: GPU virtual addresses -> physical VRAM/GTT pages         |
  |    Updated by: driver on allocation, migration, XNACK faults    |
  |    Each process has its own GPU page table (VM isolation)         |
  |                                                                   |
  |  Buffer Objects (amdgpu_bo / ttm_buffer_object)                  |
  |    Where: kernel heap (struct per allocation)                    |
  |    Contains: size, placement (VRAM/GTT/SYSTEM), pin count,       |
  |              VM mapping list, backing pages                       |
  |    Tracks every hipMalloc/hipMallocManaged allocation             |
  |                                                                   |
  |  MMIO registers                                                  |
  |    Where: PCI BAR0/BAR2 memory regions                           |
  |    Doorbell BAR: written by userspace -> triggers GPU interrupt   |
  |    Register BAR: GPU control registers (clocks, power, etc.)     |
  |    Mapped to: userspace via mmap for fast-path access             |
  |                                                                   |
  |  sysfs entries                                                   |
  |    Where: /sys/class/drm/card0/device/                           |
  |    Virtual files exposing: VRAM usage, temperature, clocks,      |
  |    link speed, GPU utilization (read from hardware registers)    |
  |                                                                   |
  |  Firmware blobs                                                  |
```

```
  |    Where: loaded from /lib/firmware/amdgpu/ at module init      |
  |    Contains: GPU microcontroller firmware (PSP, SMU, SDMA)      |
  |    Loaded to: dedicated SRAM on the GPU die                     |
  `-----------------------------------------------------------------+


==================================================================================
  CHAPTER 7: AMD GPU Hardware (Layer 8)
  The Silicon -- Compute Units, Caches, and Memory
  Radeon 8060S, gfx1151, RDNA 3.5
==================================================================================

+-- EXECUTIVE SUMMARY -- Layer 8: Radeon 8060S Hardware ------------------+
|                                                                          |
|  Architecture: RDNA 3.5, gfx1151, TSMC 4nm, APU (integrated)       |
|  Compute:      40 CUs (20 WGPs), Wave32, 2x SIMD32/CU, WMMA 16x16 |
|                ~18 TFLOPS FP16 peak (estimated)                     |
|                                                                          |
|  Memory hierarchy (confirmed via rocminfo):                          |
|    L0 Vector Cache:  32 KB / CU     (~hundreds GB/s, ~20 cycles)   |
|    LDS:              64 KB / CU     (~50 TB/s aggregate)            |
|    L2 Cache:         2 MB shared    (~50 cycles)                    |
|    Infinity Cache:   32 MB MALL     (~1 TB/s, ~100 cycles)         |
|    LPDDR5x:          ~256 GB/s      (~300+ cycles), 28 GB GTT      |
|                                                                          |
|  Key differences vs datacenter (MI210/MI300X):                       |
|    No HBM -- shared LPDDR5x is the ONLY memory (256 GB/s vs 5.3 TB/s) |
|    RDNA (Wave32, WMMA) vs CDNA (Wave64, MFMA) -- ~10x less compute |
|    Same die as CPU -- no PCIe latency, but shared bandwidth          |
|                                                                          |
|  Performance reality:                                                |
|    < 32 MB working set -> Infinity Cache -> ~1 TB/s (near HBM-like) |
|    > 32 MB working set -> LPDDR5x bound -> ~256 GB/s (bottleneck)   |
|    Small quantized models (INT4/INT8 < 4 GB) run surprisingly well.|
`--------------------------------------------------------------------------+

  +-- SOURCE REFERENCES -- Layer 8 (Hardware) --------------------------------+
  |                                                                            |
  |  Hardware documentation:                                                 |
  |    RDNA 3.5 ISA Reference:                                               |
  |      https://www.amd.com/en/search/documentation/hub.html               |
  |      (search: "RDNA3 Instruction Set Architecture Reference Guide")   |
  |                                                                            |
  |    Strix Halo product page:                                              |
  |      https://www.amd.com/en/products/processors/ryzen/ai/300-series   |
  |                                                                            |
  |  On-device introspection tools:                                          |
  |    rocminfo               -> CU count, L2 size, memory pools, ISA      |
  |    rocm-smi               -> temperature, clocks, VRAM/GTT usage       |
  |    cat /sys/class/drm/card*/device/mem_info_vram_used                  |
  |    cat /sys/class/drm/card*/device/mem_info_gtt_used                  |
  |                                                                            |
  |  Verified specs (rocminfo + external):                                   |
  |    CU count:     40           (rocminfo: Compute Unit)                 |
  |    L2 cache:     2 MB         (rocminfo: L2 Cache Size)               |
  |    Inf. Cache:   32 MB        (rocminfo: L3 Cache Size / MALL)        |
  |    LPDDR5x BW:   ~256 GB/s   (8000 MT/s x 32 B = 256 GB/s peak)     |
  |    Wave size:    32           (rocminfo: Wavefront Size)               |
  |    gfx target:   gfx1151     (rocminfo: Name = gfx1151)              |
  `-----------------------------------------------------------------------------+

We've finally reached the bottom - actual silicon. No more software. This is transistors, wires, and physics. And your specific GPU te

+-----------------------------------------------------------------------------+
|  BEGINNER'S CONTEXT: What Happens at This Layer?                        |
|                                                                            |
|  Every layer above this was SOFTWARE -- code that humans wrote,          |
|  running on your CPU, telling the GPU what to do.                       |
|                                                                            |
|  This layer is HARDWARE -- physical transistors etched into silicon.    |
|  There is no code here; there are electrical circuits that perform     |
|  math operations when electricity flows through them.                   |
|                                                                            |
|  When MIGraphX launches a GPU kernel:                                   |
|    Layer 4: MIGraphX says "run this computation"                        |
|    Layer 5: HIP packages it into an AQL packet                          |
|    Layer 6: HSA writes it to the hardware queue                         |
|    Layer 7: The driver set up the hardware at boot                      |
|    Layer 8: Physical transistors fire, electrons flow through           |
|             logic gates, and millions of math operations happen          |
|             simultaneously -> THIS is where the actual AI math occurs   |
|                                                                            |
|  Understanding hardware helps you understand WHY some models run        |
```

```
|  fast and others run slow on your specific GPU. The hardware's         |
|  capabilities (compute units, cache size, memory bandwidth) set the   |
|  fundamental speed limits that no amount of software can overcome.     |
`-------------------------------------------------------------------------+
```

## 7.1 Your GPU - What You Actually Have

```
+-----------------------------------------------------------------+
|  Radeon 8060S - RDNA 3.5 Architecture                            |
|                                                                   |
|  ISA Target:      gfx1151                                         |
|  Architecture:    RDNA 3.5 (consumer/mobile)                      |
|  Process Node:    TSMC 4nm                                        |
|  GPU Type:        APU (integrated - shares RAM with CPU)          |
|  Dedicated VRAM:  512 MB (carved out of LPDDR5x)                  |
|  Accessible RAM:  ~28 GB (GTT - rest of system LPDDR5x)          |
|  Memory Bus:      256-bit LPDDR5x-8000 (~256 GB/s, shared)       |
|  L2 Cache:        2 MB (shared across all CUs)                    |
|  Infinity Cache:  32 MB MALL / L3 (~1 TB/s, memory-side LLC)     |
|                                                                   |
|  KEY FACT: This is NOT a data center GPU.                         |
|  It's an integrated GPU inside your AMD Ryzen APU.                |
`-------------------------------------------------------------------+
```

RDNA vs CDNA - Two Different GPU Families

AMD makes two completely separate GPU architectures. Understanding this is critical:

```
                RDNA 3.5 (yours)              CDNA 3 (MI300X)
                ================              ===============
Purpose:        Gaming + general              Pure ML/HPC compute
                compute + display
Products:       Radeon RX 7000/8000,          Instinct MI200/MI300
                APU iGPUs (yours)
Wavefront:      Wave32 (native)               Wave64 (native)
                (can emulate Wave64)
Matrix Unit:    WMMA (limited)                MFMA (powerful)
                16x16 tiles                   32x32 tiles
Display output: YES (HDMI, DP)                NO (headless)
Video decode:   YES (AV1, H.265)              NO
Ray tracing:    YES (RT accelerators)         NO
Memory:         Shared LPDDR5x (APU)          Dedicated HBM3
                or GDDR6 (discrete)
Target user:    Gamers, laptops,              Data centers,
                general purpose               ML training/inference
```

MATRIX INSTRUCTIONS -- WMMA vs MFMA (Critical for ML Performance)

Your Radeon 8060S (RDNA 3.5) uses WMMA (Wave Matrix Multiply-Accumulate):

```
  v_wmma_f32_16x16x16_f16
    - 16x16x16 tile: computes 16x16 output from 16x16 x 16x16 inputs
    - Ops per instruction: 16 x 16 x 16 x 2 = 8,192 FP16 ops
    - With 40 CUs at ~2.9 GHz: ~18 TFLOPS peak FP16 matrix throughput
```

MI210 / MI300X (CDNA) uses MFMA (Matrix Fused Multiply-Add):

```
  v_mfma_f32_32x32x8_f16
    - 32x32x8 tile: computes 32x32 output from 32x8 x 8x32 inputs
    - Ops per instruction: 32 x 32 x 8 x 2 = 16,384 FP16 ops
    - MI210 has 104 CUs x 4 MFMA units each: ~181 TFLOPS peak FP16
```

What this means for your 8060S:

```
  - Matrix throughput is ~10x LOWER than datacenter GPUs
  - BUT: for small models (< 4 GB), memory bandwidth is the bottleneck,
    not compute. Your Infinity Cache (32 MB MALL, ~1 TB/s) compensates.
  - Use INT4/INT8 quantized models to maximize throughput per byte.
  - MIGraphX automatically selects WMMA instructions for gfx1151.
```

## 7.2 Part 1: Compute Units - The Processors

RDNA 3.5 CU Architecture

The Compute Unit (CU) is the fundamental processing block. Your Radeon 8060S has 40 CUs. Here's what's inside each one:

RDNA 3.5 Work Group Processor (WGP) - contains 2 CUs:

```
+-------------------------------------------------------------------+
|                          WGP                                      |
|  +----------------------------+ +----------------------------+   |
|  |        CU 0                | |        CU 1                |   |
|  |  +-----------+-----------+   | |  +-----------+-----------+   |   |
|  |  | SIMD32  | SIMD32  |   | |  | SIMD32  | SIMD32  |   |   |
|  |  | Unit 0  | Unit 1  |   | |  | Unit 0  | Unit 1  |   |   |
|  |  |         |         |   | |  |         |         |   |   |
|  |  | 32 ALUs | 32 ALUs |   | |  | 32 ALUs | 32 ALUs |   |   |
|  |  `------------+-----------+   | |  `------------+-----------+   |   |
```

```
|  |  +-----------------------+ | |  +-----------------------+ |   |
|  |  |  Scalar Unit        | | |  |  Scalar Unit        | |   |
|  |  |  (control flow)     | | |  |  (control flow)     | |   |
|  |  `-----------------------+ | |  `-----------------------+ |   |
|  |  +-----------------------+ | |  +-----------------------+ |   |
|  |  |  WMMA Unit          | | |  |  WMMA Unit          | |   |
|  |  |  (AI matrix math)   | | |  |  (AI matrix math)   | |   |
|  |  `-----------------------+ | |  `-----------------------+ |   |
|  `-----------------------------+ `-----------------------------+   |
|  +-------------------------------------------------------------+   |
|  |  Shared LDS (Local Data Share): 128 KB per WGP          |   |
|  |  (64 KB per CU)                                          |   |
|  `-------------------------------------------------------------+   |
|  +-------------------------------------------------------------+   |
|  |  L0 Vector Cache: 32 KB per CU                           |   |
|  |  L0 Scalar Cache: 16 KB per CU                           |   |
|  |  Texture Units: for sampling (used in graphics)          |   |
|  |  RT Units: ray tracing accelerators (not used for ML)    |   |
|  `-------------------------------------------------------------+   |
`-----------------------------------------------------------------------+

Wave32 - RDNA's Native Execution Width
+-------------------------------------------------------------------------+
|  BEGINNER'S NOTE: What is a "Wavefront" and Why Does Size Matter?     |
|                                                                         |
|  Remember from Chapter 0: a wavefront is a group of GPU threads that  |
|  all execute the SAME instruction at the SAME time (SIMD).            |
|                                                                         |
|  Think of it like a marching band:                                     |
|    Wave32 = 32 musicians playing the same note simultaneously         |
|    Wave64 = 64 musicians playing the same note simultaneously         |
|                                                                         |
|  Bigger wavefronts (64) are more efficient for LARGE, REGULAR tasks   |
|  like AI matrix math -- you process more data per instruction.          |
|                                                                         |
|  Smaller wavefronts (32) are better for IRREGULAR tasks like game     |
|  shaders where different threads might need to do different things.   |
|                                                                         |
|  Your GPU uses Wave32 (optimized for gaming).                          |
|  Data center GPUs use Wave64 (optimized for AI math).                 |
|  This is one reason data center GPUs are faster for AI.               |
`-------------------------------------------------------------------------+
This is a key difference from CDNA:
CDNA (MI210/MI300X):              RDNA 3.5 (your GPU):
  Wavefront = 64 threads            Wavefront = 32 threads (native)
  (Wave64)                          (Wave32)
  64 threads execute the SAME       32 threads execute the SAME
  instruction in lockstep            instruction in lockstep
  4 SIMD16 units per CU              2 SIMD32 units per CU
  (execute as 64-wide wavefronts)    (each 32 ALUs wide)
Why Wave32?
Wave64 (CDNA):
  [Y] Better for large, regular workloads (ML matrix math)
  [Y] Amortizes instruction fetch/decode over 64 threads
  [N] Wastes threads if workload is irregular/small
Wave32 (RDNA):
  [Y] Better for irregular workloads (game shaders)
  [Y] Less thread divergence waste
  [Y] Lower latency for small tasks
  [N] Less efficient for large, regular ML math
For your GPU running ML inference:
  Wave32 means each wavefront processes 32 elements at a time
  vs 64 on CDNA - half the throughput per wavefront
  But RDNA has 2 SIMD32 units per CU, partially compensating

RDNA Wave32 Execution Example
ML operation: ReLU on 1024 elements
GPU launches: 1024 / 32 = 32 wavefronts
Wavefront 0 (threads 0-31):
  All 32 threads execute simultaneously:

  Clock 1: v_global_load_f16 v0, [address + tid*2]  // load 32 FP16 values
  Clock 2: v_max_f16 v0, v0, 0                      // relu: max(x, 0)
  Clock 3: v_global_store_f16 [address + tid*2], v0  // store 32 results

  32 SIMD lanes x 1 FP16 element each = 32 elements per wavefront

  Wavefront 1 handles elements 32-63, and so on...

  On a CU with 2 SIMD32 units:
    SIMD0 runs wavefront 0 -> elements 0-31
    SIMD1 runs wavefront 1 -> elements 32-63
    Both execute IN PARALLEL -> 64 elements per clock
```

```
WMMA - Your GPU's AI Accelerator
RDNA 3/3.5 introduced WMMA (Wave Matrix Multiply-Accumulate) instructions. These are the consumer version of CDNA's MFMA matrix cores:
WMMA instruction on RDNA 3.5:
  v_wmma_f32_16x16x16_f16
  Translation:
    v_wmma        = Wave Matrix Multiply-Accumulate
    f32           = accumulate in FP32
    16x16x16      = multiply 16x16 tile by 16x16 tile
    f16           = inputs are FP16
  Each WMMA instruction computes:
    16 x 16 x 16 x 2 = 8,192 FP16 operations
  Comparison with CDNA:
  +-------------------------------------------------------+
  |  CDNA (MI210) MFMA:                                   |
  |    v_mfma_f32_32x32x8_f16                             |
  |    32 x 32 x 8 x 2 = 16,384 ops per instruction       |
  |    Per CU: 4 MFMA units                               |
  |    Total: 65,536 ops/instruction/CU                   |
  |    x 104 CUs = ~6.8M ops/instruction (whole GPU)      |
  |    Pipelined over ~64 cycles -> ~106K sustained ops/clk |
  |                                                       |
  |  RDNA 3.5 (yours) WMMA:                               |
  |    v_wmma_f32_16x16x16_f16                            |
  |    16 x 16 x 16 x 2 = 8,192 ops per instruction       |
  |    Per CU: 1 WMMA unit (per SIMD)                     |
  |    Total: ~16,384 ops/instruction/CU (2 SIMDs)        |
  |    x 40 CUs = ~655K ops/instruction (whole GPU)       |
  |                                                       |
  |  Ratio: MI210 is ~10x more matrix throughput           |
  `-------------------------------------------------------+

Compute Unit Count Comparison
GPU                    CUs     Arch      Matrix Ops/Instruction (approx)
---------------        ----    ----      ------------------------------
Your Radeon 8060S      40      RDNA 3.5  ~655K
Radeon RX 7900 XTX     96     RDNA 3     ~1.5M
MI210                  104     CDNA 2     ~6.8M
MI300X                 304     CDNA 3     ~19.8M
Your GPU has roughly:
  ~10x fewer matrix ops than MI210
  ~30x fewer matrix ops than MI300X
For ML inference, matrix throughput is the #1 factor.


7.3 Part 2: L2 Cache - The Speed Multiplier
---------------------------------------------

Why Cache Matters So Much On Your GPU
Your GPU has no dedicated VRAM (just 512 MB carved from LPDDR5x). All "VRAM" accesses actually go to LPDDR5x system RAM at ~256 GB/s. I
Memory Hierarchy - Your Radeon 8060S (confirmed via rocminfo):
                   Bandwidth          Latency      Size
                   ---------          -------      ----
  Registers        inf (instant)        0 cycles     256 KB/CU
       |
       v
  L0 Cache         ~hundreds GB/s     ~20 cycles   32 KB/CU
  (per CU)
       |
       v
  LDS              ~50 TB/s total     ~20 cycles   64 KB/CU
  (per CU)         (across all CUs)                (128 KB/WGP)
       |
       v
  L2 Cache         (shared across     ~50 cycles   2 MB
  (shared)          all 40 CUs)
       |
       v
  Infinity    -->  ~1 TB/s           ~100 cycles   32 MB (MALL / L3)
  Cache            effective
       |
       v
  LPDDR5x     -->  ~256 GB/s         ~300+ cycles  28 GB (GTT)
  (system)

Infinity Cache (MALL) - The Critical Layer
The Radeon 8060S has a two-tier GPU cache: a 2 MB L2 shared across all CUs,
backed by the 32 MB Infinity Cache (Memory Access Last Level / MALL).
The MALL is the memory-side last-level cache that sits between L2 and LPDDR5x.
(Confirmed via rocminfo: L2 = 2048 KB, L3 = 32768 KB.)

+-------------------------------------------------------------------+
|  L2 Cache:        2 MB (shared across all 40 CUs)                 |
```

```
|  Infinity Cache:  32 MB (MALL -- memory-side last-level cache)    |
|  Inf. Cache BW:   ~1 TB/s (measured by Chips and Cheese)         |
|  Line size:       128 bytes                                      |
|                                                                  |
|  For ML inference, this means:                                   |
|                                                                  |
|  If your working data fits in 32 MB (Infinity Cache):            |
|    -> ~1,000 GB/s effective bandwidth (~4x faster than LPDDR5x!) |
|    -> Feels like having HBM, sort of                             |
|                                                                  |
|  If your working data does NOT fit in 32 MB:                     |
|    -> Falls through to LPDDR5x at 256 GB/s                       |
|    -> ~4x slower than Infinity Cache hit                          |
|    -> This is the bottleneck for most ML workloads               |
`------------------------------------------------------------------+

Infinity Cache Hit Rate - Why It Determines Your Performance
Example: Matrix Multiply 512x512 FP16
  Matrix A: 512 x 512 x 2 bytes = 512 KB
  Matrix B: 512 x 512 x 2 bytes = 512 KB
  Matrix C: 512 x 512 x 2 bytes = 512 KB
  Total: 1.5 MB -> FITS in L2 and Infinity Cache!

  Performance: runs at ~1 TB/s effective -> GOOD

Example: Matrix Multiply 4096x4096 FP16
  Matrix A: 4096 x 4096 x 2 bytes = 32 MB
  Matrix B: 4096 x 4096 x 2 bytes = 32 MB
  Matrix C: 4096 x 4096 x 2 bytes = 32 MB
  Total: 96 MB -> DOES NOT fit in 32 MB Infinity Cache!

  Performance: runs at ~256 GB/s (LPDDR5x bound) -> ~4x SLOWER
  Must tile the computation to reuse Infinity Cache effectively

Compare to MI210:
  L2 Cache: 16 MB
  VRAM (HBM): 64 GB at 1,638 GB/s
  Even cache misses go to HBM at 1,638 GB/s (~6x your LPDDR5x)

Cache Tiling - How Kernels Stay Fast
Smart GPU kernels break large matrices into tiles that fit in cache:
Naive MatMul (cache-unfriendly):
  for each row of A:
    for each column of B:
      C[row][col] = dot(A[row], B[col])

  Each dot product reads an entire row of A (4096 x 2B = 8 KB)
  and an entire column of B (4096 x 2B = 8 KB)
  These constantly evict each other from L2 cache
  -> TERRIBLE cache hit rate -> stuck at 256 GB/s (LPDDR5x bound)
Tiled MatMul (cache-friendly):
  for each 16x16 tile of C:
    load 16x16 tile of A into LDS  (512 bytes - fits easily)
    load 16x16 tile of B into LDS  (512 bytes - fits easily)
    compute 16x16 output tile using data in LDS

  Working set per tile: 512 + 512 = 1 KB -> fits in LDS AND L2!
  -> Excellent cache hit rate -> runs near 1 TB/s
This tiling is what MIGraphX's WMMA kernels and rocBLAS do automatically.


7.4 Part 3: Shared LPDDR5x - The Bandwidth Reality
--------------------------------------------------

APU Memory Architecture
This is the fundamental reality of your system:
Traditional Discrete GPU:              Your APU:
+-----------+      +------------+         +--------------------------------+
|   CPU   | PCIe |   GPU    |         |   CPU + GPU on same chip      |
|         |<====>|          |         |                               |
|  DDR5   |64GB/s|  GDDR6/  |         |   +-----------+ +------------+  |
|  RAM    |      |  HBM     |         |   |Zen 5    | |RDNA 3.5  |  |
|         |      |1000+GB/s |         |   |CPU cores| |GPU CUs   |  |
`------------+      `-------------+         |   `-------+------+ `--------+------+  |
                                       |        |            |        |
  Separate chips, separate memory      |        v            v        |
  GPU has dedicated high-bandwidth     |   +-----------------------+   |
  memory                               |   |   Memory Controller  |   |
                                       |   `-------------+------------+   |
                                       |              |               |
                                       `----------------+----------------+
                                                      |
                                              +-------v-------+
```

```
                                        |  LPDDR5x   |
                                        |   Shared    |
                                        |   ~32 GB    |
                                        |  ~256 GB/s  |
                                        `----------------+

                                   ONE pool of memory, shared
                                   between CPU and GPU

LPDDR5x Bandwidth Breakdown
Your Strix Halo memory configuration:
  LPDDR5x-8000, 256-bit bus:
    Theoretical: 8000 MT/s x 32 bytes (256-bit) = 256 GB/s
    Real-world:  ~200-220 GB/s (~80-85% efficiency - typical)
  This bandwidth is SHARED between CPU and GPU:
  +-------------------------------------------------------------+
  |  LPDDR5x Bus: ======================== ~256 GB/s total     |
  |                                                              |
  |  When GPU is working hard on inference:                      |
  |    GPU uses:  ~180 GB/s  (most of the bandwidth)            |
  |    CPU gets:  ~40 GB/s   (remaining share)                  |
  |    CPU programs may slow down during GPU inference           |
  |                                                              |
  |  When both are idle:                                         |
  |    Either can burst to ~200+ GB/s                            |
  |                                                              |
  |  Compare to MI210:                                           |
  |    GPU HBM: 1,638 GB/s (dedicated, no sharing)              |
  |    CPU DDR: separate bus (no contention)                     |
  |    Ratio: MI210 has ~6x more GPU bandwidth AND               |
  |           doesn't steal from the CPU                         |
  `-------------------------------------------------------------+

What ~256 GB/s Means for ML Inference
Operation: MatMul for one transformer layer (e.g., in a small LLM)
  Weight matrix: 4096 x 4096 x 2 bytes (FP16) = 32 MB
  Input vector:  1 x 4096 x 2 bytes = 8 KB

  This is memory-bandwidth bound (read 32 MB of weights for tiny compute):
    Time on your GPU:    32 MB / 256 GB/s = 0.125 ms per layer
    Time on MI210:       32 MB / 1,638 GB/s = 0.020 ms per layer
    Time on MI300X:      32 MB / 5,300 GB/s = 0.006 ms per layer

  For a 24-layer model:
    Your GPU:    0.125 x 24 = 3.0 ms per token
    MI210:       0.020 x 24 = 0.47 ms per token
    MI300X:      0.006 x 24 = 0.14 ms per token

  Your GPU is ~6x slower than MI210 for memory-bound LLM inference.
  BUT: it's still 3.0 ms/token = ~333 tokens/second for a small model.
  That's completely usable for small models!

The 512 MB VRAM Carve-Out Explained
Your 32 GB system RAM:
+-------------------------------------------------------------------+
|                                                                    |
|  +--------------+                                                  |
|  |  VRAM      |  512 MB - BIOS-configured carve-out            |
|  |  Carve-out |                                                  |
|  |            |  * Invisible to Linux (not in free RAM)         |
|  |            |  * Exclusively for GPU - no CPU contention       |
|  |            |  * Used for: framebuffer (display), critical     |
|  |            |    GPU data structures, page tables              |
|  |            |  * TTM domain: VRAM                              |
|  |            |  * NOT enough for ML models (only 512 MB)       |
|  |----------------|                                                |
|  |            |                                                  |
|  |  System    |  ~28 GB - GTT (GPU-accessible pinned RAM)       |
|  |  RAM       |                                                  |
|  |  (shared)  |  * Linux manages this as normal RAM             |
|  |            |  * GPU accesses via same memory controller       |
|  |            |    (NO PCIe - direct access on APU!)            |
|  |            |  * TTM domain: GTT                               |
|  |            |  * ML model weights go here                      |
|  |            |  * Access speed: same ~256 GB/s as VRAM         |
|  |            |    (on APU, VRAM and GTT are same speed!)       |
|  |            |                                                  |
|  |  + CPU     |  ~3.5 GB - OS + applications                    |
|  |  Reserved  |                                                  |
|  |            |                                                  |
|  `---------------+                                                |
|                                                                    |
|  IMPORTANT APU INSIGHT:                                            |
```

```
|  On a discrete GPU: VRAM (fast HBM) >> GTT (slow PCIe)         |
|  On YOUR APU:       VRAM ~= GTT (both same LPDDR5x speed!)     |
|  The 512 MB limit barely matters because GTT is just as fast.   |
`-----------------------------------------------------------------+
```

## 7.5 Your GPU's Full Die Layout

```
Radeon 8060S APU Die (simplified):
+-----------------------------------------------------------------+
|                                                                  |
|  +-----------------------------------+  +-----------------------+  |
|  |      Zen 5 CPU Cores            |  |   XDNA NPU          |  |
|  |  +-------++-------++-------++-------+ |  |   (AI accelerator)  |  |
|  |  |Core0||Core1||Core2||Core3| |  |   16-50 TOPS INT8    |  |
|  |  `--------+`--------+`--------+`--------+ |  |   (separate from GPU)|  |
|  |  +-------++-------++-------++-------+ |  `-----------------------+  |
|  |  |Core4||Core5||Core6||Core7| |                              |
|  |  `--------+`--------+`--------+`--------+ |  +-----------------------+  |
|  |  L3 Cache: 16-24 MB            |  |   Media Engine       |  |
|  `-----------------------------------+  |   AV1/H.265 decode  |  |
|                                          |   AV1 encode        |  |
|  +-----------------------------------+  `-----------------------+  |
|  |      RDNA 3.5 GPU               |                              |
|  |  +-------++-------++-------++-------+ |  +----------------------+  |
|  |  |WGP 0||WGP 1||WGP 2||WGP 3| |  |  Display Engine      |  |
|  |  |2 CUs||2 CUs||2 CUs||2 CUs| |  |  HDMI 2.1            |  |
|  |  `--------+`--------+`--------+`--------+ |  |  DisplayPort 2.1     |  |
|  |  +-------++-------++-------++-------+ |  `-----------------------+  |
|  |  |WGP 4||WGP 5||WGP 6||WGP 7| |                              |
|  |  |2 CUs||2 CUs||2 CUs||2 CUs| |                              |
|  |  `--------+`--------+`--------+`--------+ |                              |
|  |  L2: 2 MB + Infinity Cache: 32 MB|                              |
|  |  Render Backends (display)      |                              |
|  `-----------------------------------+                              |
|                                                                     |
|  +-------------------------------------------------------------+  |
|  |              Infinity Fabric / Memory Controller           |  |
|  |              LPDDR5x 256-bit interface                     |  |
|  `-------------------------------------------------------------+  |
|                          |  |                                      |
`--------------------------+--+--------------------------------------+
                           |  |
                     +-------v--v-------+
                     |   LPDDR5x      |
                     |   2 channels   |
                     |  ~256 GB/s     |
                     `------------------+
Notice: your APU has three compute engines - CPU, GPU, and XDNA NPU. For ML inference, the NPU (XDNA) is actually designed specifically
```

## 7.6 How a Kernel Actually Executes on Your Hardware

```
Let's trace a fused ReLU+Add kernel through the actual silicon:
MIGraphX launches: fused_relu_add(input, residual, output, N=65536)
Grid: 256 blocks x 256 threads per block = 65536 threads
Wavefront size: 32 (Wave32 on RDNA 3.5)

Step 1: Command Processor receives AQL packet
  +-------------------------------------------------------------+
  |  CP reads from hardware queue (doorbell was rung)       |
  |  Decodes: 256 blocks, 256 threads/block, kernel address |
  |  Each block needs: 256/32 = 8 wavefronts                |
  |  Total wavefronts: 256 x 8 = 2048 wavefronts            |
  `-------------------------------------------------------------+

Step 2: CP dispatches workgroups to WGPs
  +-------------------------------------------------------------+
  |  WGP 0: receives blocks 0-3   (32 wavefronts)          |
  |  WGP 1: receives blocks 4-7   (32 wavefronts)          |
  |  WGP 2: receives blocks 8-11  (32 wavefronts)          |
  |  ...                                                    |
  |  WGP 7: receives blocks 28-31 (32 wavefronts)          |
  |                                                         |
  |  Remaining blocks 32-255 wait in a hardware queue       |
  |  As WGPs finish blocks, they pick up new ones           |
  `-------------------------------------------------------------+

Step 3: Inside one CU - wavefront execution
  +-------------------------------------------------------------+
  |  SIMD32 Unit 0 executes Wavefront 0 (threads 0-31):    |
```

```
  |                                                          |
  |  Cycle 1: s_load_dword s0, [kernarg]                    |
  |           // load N from kernel arguments (scalar)      |
  |                                                          |
  |  Cycle 2-5: v_global_load_b32 v0, [input_ptr + tid*2]  |
  |           // each of 32 threads loads its FP16 pair     |
  |           // 32 threads x 4 bytes = 128 bytes           |
  |           // = one cache line from L2/LPDDR5x            |
  |                                                          |
  |  Cycle 6-9: v_global_load_b32 v1, [residual + tid*2]   |
  |           // load residual values                        |
  |                                                          |
  |  Cycle 10: v_pk_max_f16 v0, v0, 0                       |
  |           // packed ReLU: max(x, 0) on 2 FP16 values   |
  |           // "pk" = packed - processes 2 FP16s at once! |
  |           // 32 threads x 2 FP16 = 64 ReLU ops/cycle   |
  |                                                          |
  |  Cycle 11: v_pk_add_f16 v0, v0, v1                      |
  |           // packed add: relu(x) + residual             |
  |           // 32 threads x 2 FP16 = 64 add ops/cycle    |
  |                                                          |
  |  Cycle 12-15: v_global_store_b32 [output + tid*2], v0  |
  |           // store results back to memory               |
  |                                                          |
  |  Meanwhile, SIMD32 Unit 1 runs Wavefront 1 in parallel |
  `-----------------------------------------------------------+

Step 4: Memory access path
  +-----------------------------------------------------------+
  |  v_global_load issues 128-byte request                  |
  |       |                                                  |
  |       v                                                  |
  |  L0 Cache (32 KB per CU) - check                        |
  |  Hit? -> return in ~20 cycles                             |
  |  Miss v                                                  |
  |                                                          |
  |  L2 Cache (2 MB shared) - check                            |
  |  Hit? -> return in ~50 cycles                             |
  |  Miss v                                                  |
  |                                                          |
  |  Infinity Cache (32 MB MALL) - check                     |
  |  Hit? -> return in ~100 cycles -> ~1 TB/s bandwidth      |
  |  Miss v                                                  |
  |                                                          |
  |  LPDDR5x System RAM - fetch                              |
  |  Return in ~300+ cycles -> ~256 GB/s bandwidth           |
  |  (wavefront is stalled during this wait, but GPU         |
  |   switches to another ready wavefront - latency hiding) |
  `-----------------------------------------------------------+

Latency Hiding - How GPUs Stay Busy
+-------------------------------------------------------------------------+
|  BEGINNER'S NOTE: What is "Latency Hiding"?                            |
|                                                                         |
|  Imagine you're at a cafe. You order coffee (takes 3 minutes to brew).|
|  Instead of standing there doing nothing for 3 minutes, you:           |
|    - Check your email while waiting                                     |
|    - Reply to a message                                                 |
|    - When coffee is ready, you pick it up and continue                  |
|                                                                         |
|  GPUs do exactly this: when one wavefront is waiting for slow          |
|  memory (LPDDR5x: ~300 cycles), the GPU instantly switches to another |
|  wavefront that is ready to compute. Zero cost to switch -- the GPU    |
|  hardware manages this automatically.                                   |
|                                                                         |
|  This is why GPUs need THOUSANDS of threads -- not because the math    |
|  needs that many, but because many of them will be waiting for memory  |
|  at any given time, and the GPU needs other threads to stay busy.      |
|                                                                         |
|  If the GPU does not have enough wavefronts in flight, it will stall  |
|  waiting for memory -- wasting its compute capability. This is called  |
|  low "occupancy."                                                       |
`-------------------------------------------------------------------------+
Even with LPDDR5x latency, the GPU hides it by keeping many wavefronts in flight:
One wavefront stalls on memory load (300 cycles to LPDDR5x):
  Wavefront 0: [compute]--[LOAD...waiting 300 cycles for LPDDR5x...]--[compute]

  Without latency hiding: GPU idle for 300 cycles = HUGE waste
  With latency hiding (multiple wavefronts):

  Wavefront 0: [compute]--[LOAD...stalled.........................]--[comp]
  Wavefront 1:             [compute]--[LOAD...stalled.............]--[comp]
  Wavefront 2:                          [compute]--[LOAD...stalled]--[comp]
```

```
  Wavefront 3:                                    [compute]--[LOAD...    ]
  ...

  GPU switches between wavefronts instantly (zero-cost context switch)
  While one wavefront waits for LPDDR5x, others are computing

  To fully hide 300 cycles of LPDDR5x latency:
    Need ~300/10 = ~30 wavefronts in flight per CU
    Your CU supports up to ~16-32 wavefronts
    -> Can partially but not fully hide LPDDR5x latency
    -> This is why bandwidth is the bottleneck, not compute

  +-- DIAGRAM: Complete Kernel Execution Pipeline ---------------------------+
  |                                                                           |
  |  hipLaunchKernel(fused_relu_add, grid=256, block=256, stream)         |
  |       |                                                                  |
  |       v                                                                  |
  |  +-----------------------------------------------+                      |
  |  |  1. HSA: Write AQL packet -> ring doorbell    |  ~40 ns              |
  |  `-----------------------+--------------------------+                      |
  |                          v                                               |
  |  +-----------------------------------------------+                      |
  |  |  2. Command Processor: Decode packet         |  ~100 ns             |
  |  |     256 blocks x 8 waves/block = 2048 waves  |                      |
  |  `-----------------------+--------------------------+                      |
  |                          v                                               |
  |  +-----------------------------------------------+                      |
  |  |  3. Dispatch: Assign blocks -> 20 WGPs        |                      |
  |  |     WGP 0 gets blocks 0-3 (32 wavefronts)    |                      |
  |  |     ...WGP 19 gets blocks 76-79              |                      |
  |  `-----------------------+--------------------------+                      |
  |                          v                                               |
  |  +-----------------------------------------------+                      |
  |  |  4. SIMD32 executes wavefronts:              |                      |
  |  |     Fetch -> Decode -> Execute -> Writeback     |                      |
  |  |                                               |                      |
  |  |     v_global_load (data from LPDDR5x/cache)  |                      |
  |  |     v_pk_max_f16  (ReLU on 2 FP16 values)   |  ~10-15 cycles       |
  |  |     v_pk_add_f16  (Add residual)             |  per wavefront       |
  |  |     v_global_store (write result)            |  (compute only)      |
  |  `-----------------------+--------------------------+                      |
  |                          v                                               |
  |  +-----------------------------------------------+                      |
  |  |  5. All 2048 waves complete -> signal done    |                      |
  |  |     hsa_signal decrement -> CPU sees result   |  ~0.02-0.5 ms total |
  |  `-----------------------------------------------+                      |
  `-------------------------------------------------------------------------+


7.7 Your GPU's Realistic ML Performance
----------------------------------------

+-----------------------------------------------------------------+
|  Radeon 8060S - ML Inference Reality Check                      |
|                                                                 |
|  Peak FP16 compute:    ~18 TFLOPS (estimated, 40 CUs + WMMA)   |
|  Memory bandwidth:     ~256 GB/s (shared LPDDR5x)               |
|  Max model size:       ~24 GB (28 GB GTT minus OS overhead)     |
|                                                                 |
|  Achievable performance (bandwidth-limited):                    |
|  +-------------------------------------------------------+  |
|  |  Tiny model (< 32 MB weights): Infinity Cache hides LPDDR5x| |
|  |    -> Runs at effective ~1 TB/s -> nearly GPU-fast          |  |
|  |    -> Example: MobileNet classifier = very fast            |  |
|  |                                                           |  |
|  |  Small model (32 MB - 500 MB): LPDDR5x bandwidth bound   |  |
|  |    -> Runs at ~256 GB/s                                    |  |
|  |    -> Example: ResNet-50 = usable (~30-50 ms/image)       |  |
|  |                                                           |  |
|  |  Medium model (500 MB - 4 GB): LPDDR5x bound, still fits |  |
|  |    -> Example: Small LLM (1-2B params FP16) = ~100 tok/s  |  |
|  |                                                           |  |
|  |  Large model (4+ GB): LPDDR5x bound, may pressure system |  |
|  |    -> Example: 7B model FP16 (14 GB) = ~10-20 tok/s       |  |
|  |    -> System may feel sluggish (GPU hogs bandwidth)        |  |
|  |                                                           |  |
|  |  Huge model (14+ GB): technically fits in GTT but         |  |
|  |    -> Very slow + system becomes nearly unusable           |  |
|  |    -> Strongly recommend quantized models (INT4/INT8)      |  |
|  `-------------------------------------------------------+  |
|                                                                 |
|  YOUR SWEET SPOT: quantized models (INT4/INT8) under 4 GB      |
|  A 7B model quantized to INT4: ~3.5 GB -> runs well on your GPU |
```

```
`-----------------------------------------------------------------+


7.8 The Complete 8-Layer Stack - On YOUR Hardware
-------------------------------------------------

Layer 1: Python onnxruntime          -> session.run(input)
Layer 2: ORT (C++)                   -> graph optimization
Layer 3: MIGraphX EP (C++ plugin)    -> calls Layer 4 to parse, quantize, compile
Layer 4: libmigraphx.so             -> fusion, scheduling, emit launches
Layer 5: libamdhip64.so (HIP)       -> hipMalloc, hipLaunchKernel
Layer 6: libhsa-runtime64.so (HSA)  -> AQL packets, doorbell, signals
Layer 7: amdgpu.ko (kernel driver)  -> TTM, 512MB VRAM + 28GB GTT, registers
Layer 8: Radeon 8060S silicon  <-- YOU ARE HERE
         |
         |---- 40 RDNA 3.5 CUs (Wave32, WMMA matrix units)
         |---- 2 MB L2 + 32 MB Infinity Cache / MALL (~1 TB/s)
         |---- Shared LPDDR5x (~256 GB/s - still the bottleneck)
         |---- No dedicated HBM (APU architecture)
         `---- Same die as Zen 5 CPU + XDNA NPU

For ML on this hardware:
  * Compute is NOT your bottleneck (WMMA units are underutilized)
  * Memory bandwidth IS your bottleneck (~256 GB/s shared LPDDR5x)
  * Infinity Cache hits are your best friend (~1 TB/s vs 256 GB/s)
  * Quantization (INT4/INT8) is your biggest lever
    (smaller model = more fits in cache = faster)
  * Small, quantized models run surprisingly well
  * Large FP16 models will be painfully slow

  +-- DIAGRAM: Complete 8-Layer Stack on YOUR Radeon 8060S ------------------+
  |                                                                          |
  |  +-- L1 -------------------------------------------------------------+  |
  |  | Python: session = ort.InferenceSession("model.onnx")             |  |
  |  |         result  = session.run(None, {"input": img})              |  |
  |  `---------------------------------+-------------------------------------+  |
  |                                    | pybind11 / ctypes                    |
  |  +-- L2 -------------------------v-----------------------------------+  |
  |  | ORT C++: Load ONNX -> optimize graph -> partition GPU/CPU           |  |
  |  `---------------------------------+-------------------------------------+  |
  |                                    | GetCapability() + Compile()          |
  |  +-- L3 -------------------------v-----------------------------------+  |
  |  | MIGraphX EP: adapter layer, 3 calls into libmigraphx             |  |
  |  `---------------------------------+-------------------------------------+  |
  |                                    | C API: parse / quantize / compile    |
  |  +-- L4 -------------------------v-----------------------------------+  |
  |  | libmigraphx.so: 50+ passes -> MLIR/JIT -> schedule -> GPU program  |  |
  |  `---------------------------------+-------------------------------------+  |
  |                                    | hipMalloc / hipLaunchKernel          |
  |  +-- L5 -------------------------v-----------------------------------+  |
  |  | HIP (libamdhip64.so): malloc -> SvmBuffer, launch -> NDRange cmd  |  |
  |  `---------------------------------+-------------------------------------+  |
  |                                    | AQL packet + doorbell write          |
  |  +-- L6 -------------------------v-----------------------------------+  |
  |  | HSA (libhsa-runtime64.so): queues, signals, memory pools          |  |
  |  `---------------------------------+-------------------------------------+  |
  |                                    | ioctl (setup) / MMIO (dispatch)      |
  |  +-- L7 -------------------------v-----------------------------------+  |
  |  | amdgpu.ko: TTM alloc, page tables, 512MB VRAM + 28GB GTT         |  |
  |  `---------------------------------+-------------------------------------+  |
  |                                    | registers + memory bus               |
  |  +-- L8 -------------------------v-----------------------------------+  |
  |  | SILICON: 40 CUs | 2MB L2 | 32MB MALL | LPDDR5x 256 GB/s        |  |
  |  `-----------------------------------------------------------------------+  |
  |                                                                          |
  |  Setup (once): L1->L2->L3->L4 compile (seconds to minutes)               |
  |  Inference (per-call): L4->L5->L6->L7->L8 dispatch (milliseconds)        |
  `--------------------------------------------------------------------------+

Your Radeon 8060S is genuinely capable of running ML inference - just not at data center speeds. For local, private, small-model infere


7.9 Where Every Byte Lives -- Layer 8 (Silicon)
-----------------------------------------------

  +-------------------------------------------------------------------+
  |  REGISTER FILES (per Compute Unit)                                |
  |    VGPRs (Vector General Purpose Registers):                      |
  |      1536 x 32-bit per CU (RDNA 3.5), shared across wavefronts|
  |      Each wavefront of 32 threads gets a private slice          |
  |      Holds: intermediate computation values, accumulators       |
  |    SGPRs (Scalar General Purpose Registers):                      |
  |      104 x 32-bit per CU, shared within a wavefront            |
```

```
|      Holds: loop counters, addresses, uniform values            |
|                                                                  |
|  LDS (Local Data Share) -- per CU                                |
|    Size: 64 KB per CU (128 KB per WGP, RDNA 3.5)              |
|    Bandwidth: ~several TB/s (on-die SRAM, single-cycle access) |
|    Holds: shared tiles for matrix multiply, reduction buffers   |
|    Explicitly managed by the kernel (not a cache)               |
|                                                                  |
|  L0 Instruction Cache -- per CU                                  |
|    Size: 32 KB per CU                                           |
|    Holds: decoded ISA instructions (ELF .text section)          |
|    Fed from: L1 instruction cache                               |
|                                                                  |
|  L0 Scalar Cache -- per CU                                       |
|    Size: 16 KB per CU                                           |
|    Holds: scalar data (kernel arguments, constants)             |
|    Read by: SGPR loads (s_load_dword instructions)              |
|                                                                  |
|  L1 Vector Cache -- per CU (RDNA) / per Shader Array (CDNA)     |
|    Size: 32 KB per CU (RDNA 3.5)                               |
|    Holds: data from global memory accesses (v_load/v_store)     |
|    Write policy: write-through to L2                            |
|                                                                  |
|  L2 Cache -- shared across all CUs                               |
|    8060S: 2 MB L2 + 32 MB Infinity Cache/MALL (RDNA 3.5)      |
|    MI210: 16 MB (CDNA 2)                                        |
|    MI300X: 256 MB (CDNA 3)                                      |
|    Bandwidth: ~1 TB/s (8060S MALL), ~5.3 TB/s (MI300X)         |
|    Holds: recently accessed weights, activations, code objects  |
|    THIS is the critical cache for ML inference on APU -- a       |
|    model that fits in 32 MB MALL runs at ~1 TB/s instead of    |
|    256 GB/s from LPDDR5x (~4x speedup)                          |
|                                                                  |
|  LPDDR5x (off-die, shared with CPU) -- 8060S                    |
|    Total: ~29 GB available to GPU (after VRAM carve-out)        |
|    Bandwidth: ~256 GB/s (shared bus with CPU)                   |
|    Holds: model weights, activation tensors, GPU heap           |
|    Accessed via: VRAM carve-out (fast) or GTT pool (same speed) |
|                                                                  |
|  HBM (on-package, dedicated) -- MI210/MI300X                    |
|    MI210: 64 GB HBM2e at 1,638 GB/s                            |
|    MI300X: 192 GB HBM3 at 5,300 GB/s                           |
|    Holds: all GPU allocations (weights, activations, scratch)   |
|                                                                  |
|  GPU Instruction Cache (hardware managed)                        |
|    Holds: ELF code objects loaded by hipModuleLoadData          |
|    Fed from: VRAM/LPDDR5x backing store                        |
|    Each CU caches hot instructions in its L0 I-cache            |
|                                                                  |
|  Command Processor SRAM                                          |
|    Holds: fetched AQL packets being decoded                     |
|    Reads from: hardware queue ring buffer (GPU-visible memory)  |
|    Outputs: wavefront dispatch commands to CU schedulers        |
`------------------------------------------------------------------+
```

```
================================================================================
                    PART IV -- REFERENCES AND APPENDICES
================================================================================
```

```
================================================================================
  APPENDIX A: Libraries Used by ORT + MIGraphX -- The Complete Map
================================================================================

+-------------------------------------------------------------------------+
|  BEGINNER'S CONTEXT: What is a "Dependency"?                            |
|                                                                          |
|  In Python:                                                              |
|    import numpy      # numpy is a "dependency" of your script          |
|    import pandas     # pandas depends on numpy (transitive dependency) |
|                                                                          |
|  You can see all dependencies with: pip install -r requirements.txt    |
|                                                                          |
|  The same concept applies in C++. MIGraphX depends on rocBLAS          |
|  (for matrix math), MIOpen (for convolutions), HIP (for GPU access),  |
|  and many more libraries. Each library is a compiled .so file that     |
|  provides specific functionality.                                       |
|                                                                          |
|  This appendix maps out the ENTIRE dependency tree -- which library     |
|  calls which, and what each one does. Think of it as the              |
|  "pip freeze" or "requirements.txt" of the AMD inference stack.       |
`-------------------------------------------------------------------------+
```

The Full Dependency Stack

NOTE: This appendix groups libraries into 4 DEPENDENCY LAYERS (1-4) for simplicity. These do NOT map 1:1 to the 8-LAYER STACK in the main book (Layers 1-8). The main book's layering is the authoritative model.

```
+-----------------------------------------------------------------------+
|                     YOUR PYTHON CODE                                  |
|                session.run(input)                                     |
|-----------------------------------------------------------------------|
|                                                                       |
|  DEP GROUP 1: ONNX Runtime (ORT)  [~= Book Layers 1-2]                |
|  |---- onnx (protobuf parser)                                         |
|  |---- protobuf (Google Protocol Buffers)                             |
|  |---- abseil-cpp (Google's C++ base library)                         |
|  |---- flatbuffers (ORT's optimized model format)                     |
|  `---- onnxruntime core (graph optimizer, session, allocator)         |
|                                                                       |
|-----------------------------------------------------------------------|
|                                                                       |
|  DEP GROUP 2: MIGraphX EP + Library  [~= Book Layers 3-4]             |
|  |---- MIGraphX core (graph IR, optimizer, compiler, scheduler)       |
|  |---- nlohmann/json (JSON config parsing)                            |
|  |---- msgpack (serialization for model caching)                      |
|  |---- sqlite3 (performance database / kernel cache)                  |
|  |---- pybind11 (Python bindings)                                     |
|  `---- Eigen (CPU fallback GEMM)                                      |
|                                                                       |
|-----------------------------------------------------------------------|
|                                                                       |
|  DEP GROUP 3: AMD GPU Math Libraries (the heavy lifters)              |
|  |---- MIOpen          (convolutions, pooling, batchnorm, LRN)       |
|  |---- rocBLAS         (GEMM / matrix multiply)                      |
|  |---- hipBLASLt       (lightweight GEMM, mixed precision)           |
|  |---- hipBLAS         (BLAS interface, required by hipBLASLt)       |
|  |---- Composable Kernel (CK) (JIT fused GEMM, attention kernels)    |
|  `---- rocMLIR         (MLIR compiler for custom fused kernels)      |
|                                                                       |
|-----------------------------------------------------------------------|
|                                                                       |
|  DEP GROUP 4: AMD Runtime / Compiler  [~= Book Layers 5-6]            |
|  |---- HIP Runtime     (GPU API: memory, streams, kernel launch)     |
|  |---- hipRTC          (runtime compilation of HIP kernels)          |
|  |---- HSA Runtime     (low-level hardware interface, chiplet query) |
|  `---- COMGR / LLVM    (compiler: HIP C++ -> AMDGPU ISA)             |
|                                                                       |
`-----------------------------------------------------------------------+
```

```
  +-- DIAGRAM: Full Dependency Tree ----------------------------------------+
  |                                                                         |
  |  session.run(input)    <- your Python code                              |
  |       |                                                                 |
  |       |---> onnxruntime (ORT core)                                      |
  |       |     |---> protobuf (ONNX file parsing)                          |
  |       |     |---> abseil-cpp (C++ utilities)                            |
  |       |     `---> flatbuffers (.ort format)                             |
  |       |                                                                 |
  |       `---> MIGraphX EP -> libmigraphx.so                               |
  |              |---> MIOpen (convolution kernels)                         |
  |              |     `---> composable_kernel (fused ops)                  |
  |              |---> rocBLAS (GEMM / matrix multiply)                     |
  |              |     `---> Tensile (auto-tuned GEMM generator)            |
  |              |---> hipBLASLt (lightweight GEMM)                         |
  |              |---> rocMLIR (MLIR fused kernel compiler)                 |
  |              |---> msgpack, sqlite3, nlohmann/json (utils)              |
  |              |                                                          |
  |              `---> HIP Runtime (libamdhip64.so)                         |
  |                     |---> hipRTC -> COMGR (JIT compilation)             |
  |                     |              `---> LLVM (gfx1151 backend)         |
  |                     |                                                   |
  |                     `---> HSA Runtime (libhsa-runtime64.so)             |
  |                           `---> ROCt/libhsakmt (KFD ioctls)            |
  |                                 `---> amdgpu.ko (kernel driver)         |
  |                                       `---> GPU Silicon                 |
  `-------------------------------------------------------------------------+
```

Detailed Breakdown: What Each Library Does

ORT's Own Dependencies

Library

What It Does

Why ORT Needs It

ONNX

Defines the .onnx model format

Parse model files into graph
Protobuf (Google)
Binary serialization format
ONNX files are protobuf-encoded
Abseil (Google)
C++ utility library (strings, containers, sync)
Used by protobuf v4+ internally
FlatBuffers
Zero-copy serialization
ORT's .ort optimized format (faster than protobuf)

MIGraphX's Own Dependencies (from requirements.txt)
Library
Version
What It Does
abseil-cpp
20250512.0
C++ base utilities (used by protobuf)
protobuf
v30.0
Parse ONNX protobuf when MIGraphX calls parse_onnx_buffer()
nlohmann/json
v3.8.0
Read/write JSON config files and metadata
pybind11
latest
Creates migraphx Python module so you can call MIGraphX from Python
msgpack
cpp-3.3.0
Fast binary serialization for caching compiled models
sqlite3
3.50.4
Performance database - stores auto-tuning results for kernels
Composable Kernel
latest
AMD's JIT kernel library for fused GEMM and attention
Eigen
5.0.1
CPU fallback matrix math (when GPU can't handle an op)
rocMLIR
latest
MLIR-based compiler for generating custom fused GPU kernels

AMD GPU Math Libraries (from CMakeLists.txt)
These are the performance-critical libraries - where the actual GPU compute happens.
Library
CMake Option
What It Computes
When MIGraphX Uses It
MIOpen
MIGRAPHX_USE_MIOPEN=ON
Convolutions, pooling, batch norm, LRN, activation
Every CNN model (ResNet, YOLO, etc.)
rocBLAS
MIGRAPHX_USE_ROCBLAS=ON
General Matrix Multiply (GEMM)
Every MatMul / Linear layer
hipBLASLt
MIGRAPHX_USE_HIPBLASLT=ON
Lightweight GEMM with mixed-precision and fused epilogues
Modern transformer MatMul + bias + activation
hipBLAS
Required by hipBLASLt
BLAS interface layer
Dependency, not called directly
Composable Kernel (CK)
MIGRAPHX_USE_COMPOSABLEKERNEL=ON
JIT-compiled fused GEMM, fused attention (GEMM+Softmax+GEMM)
Transformer attention blocks
rocMLIR
MIGRAPHX_ENABLE_MLIR=ON
MLIR compiler that generates fused GPU kernels from subgraphs
Complex fused ops that no library has a pre-built kernel for

AMD Runtime Layer (from GPU CMakeLists.txt)
Library
What It Does
HIP Runtime (hip-runtime-amd)
The GPU API - allocate memory, launch kernels, manage streams
hipRTC
Runtime compilation - compiles HIP C++ kernel source code into GPU binaries at runtime
HSA Runtime (hsa-runtime64)
Low-level hardware abstraction - MIGraphX uses it for chiplet query (detecting MI300X chiplet topology)

COMGR / LLVM
The actual compiler chain: HIP -> LLVM IR -> AMDGPU ISA machine code

Which Library Handles Which Op - The Dispatch Table
When MIGraphX sees an instruction, here's where it goes:

```
INSTRUCTION              LIBRARY           WHY THIS ONE
-------------            -------           ------------
Convolution 2D/3D   ->   MIOpen        ->   Auto-tuned conv kernels (ImplicitGemm, Winograd)
MatMul / GEMM       ->   rocBLAS       ->   Hand-tuned assembly GEMM for each GPU arch
MatMul + bias + act ->   hipBLASLt     ->   Fused GEMM epilogue (fewer memory round-trips)
GEMM + Softmax + GEMM -> CK (JIT)     ->   Fused multi-head attention in one kernel
Pooling             ->   MIOpen        ->   Max/Avg pooling
BatchNorm           ->   MIOpen        ->   Fused with conv when possible
LRN                 ->   MIOpen        ->   Local Response Normalization
Pointwise ops       ->   MIGraphX JIT  ->   ReLU, GELU, Add, Mul - compiled via hipRTC/COMGR
(relu, gelu, add...)
Complex fusions     ->   rocMLIR       ->   When MIGraphX detects a fusible subgraph that
                                            no single library covers, it generates MLIR
                                            and compiles a custom kernel
CPU fallback ops    ->   Eigen         ->   Rare ops not supported on GPU
```

Visual: How Libraries Map to the rocm-libraries Super-Repo
The ROCm/rocm-libraries repo consolidates 24 libraries. Here are the ones MIGraphX directly uses, highlighted:

```
rocm-libraries/
|---- projects/
|   |---- composablekernel/   * MIGraphX uses (fused GEMM, attention)
|   |---- hipblas/            * MIGraphX uses (required by hipBLASLt)
|   |---- hipblas-common/
|   |---- hipblaslt/          * MIGraphX uses (lightweight fused GEMM)
|   |---- hipcub/
|   |---- hipfft/
|   |---- hiprand/
|   |---- hipsolver/
|   |---- hipsparse/
|   |---- hipsparselt/
|   |---- hiptensor/
|   |---- miopen/             * MIGraphX uses (conv, pool, batchnorm)
|   |---- rocblas/            * MIGraphX uses (GEMM)
|   |---- rocfft/
|   |---- rocprim/
|   |---- rocrand/
|   |---- rocsolver/
|   |---- rocsparse/
|   |---- rocthrust/
|   |---- rocwmma/
|   `---- ...
`---- shared/
    `---- tensile/            * Used internally by rocBLAS (GEMM code generator)
```

6 out of 24 libraries in the super-repo are directly used by MIGraphX. Plus rocMLIR (which lives in its own repo at ROCm/rocMLIR).

Summary Table - All Dependencies in One Place

Layer
Library
Role
Critical for Performance?
ORT
onnx
Model format
No (one-time parse)
ORT
protobuf
Serialization
No (one-time parse)
ORT
abseil
C++ utilities
No
MIGraphX
protobuf
Re-parse subgraph
No (one-time)
MIGraphX
nlohmann/json
Config files
No
MIGraphX
msgpack
Model cache serialization
No
MIGraphX
sqlite3
Kernel perf database
Moderate (caches tuning)

```
MIGraphX
pybind11
Python API
No
MIGraphX
Eigen
CPU fallback GEMM
No (only for fallback)
AMD Math
MIOpen
Conv, pool, BN
YES - critical
AMD Math
rocBLAS
GEMM
YES - critical
AMD Math
hipBLASLt
Fused GEMM
YES - critical
AMD Math
Composable Kernel
JIT fused attention
YES - critical
AMD Math
rocMLIR
MLIR kernel compiler
YES - critical
AMD Runtime
HIP Runtime
GPU API
YES - foundation
AMD Runtime
hipRTC
Runtime compilation
YES - for JIT
AMD Runtime
HSA Runtime
Hardware interface
YES (chiplet detection)
AMD Runtime
COMGR/LLVM
GPU compiler
YES - foundation
The bold YES entries are what determine your inference speed. Everything else is infrastructure/plumbing.


================================================================================
  APPENDIX B: The Complete API Call Chain -- All 8 Layers, Unified
================================================================================

PHASE 1: SESSION CREATION (One-Time Setup)
------------------------------------------

+-----------------------------------------------------------------------------+
|  BEGINNER'S CONTEXT: What is an "API Call Chain"?                          |
|                                                                             |
|  When you call ONE Python function, it triggers a CASCADE of internal  |
|  function calls across multiple libraries. Like dominoes falling:      |
|                                                                             |
|  Python analogy:                                                            |
|    pandas.read_csv("data.csv")                                             |
|      -> pandas calls Python's open()                                        |
|      -> open() calls the C library's fopen()                                |
|      -> fopen() calls the Linux kernel's open() system call                 |
|      -> The kernel accesses the filesystem driver                           |
|      -> The driver talks to the SSD hardware                                |
|                                                                             |
|  One simple Python call -> 6 layers of function calls!                      |
|                                                                             |
|  This appendix shows the same chain for:                                   |
|    ort.InferenceSession("model.onnx")   <- Phase 1 (setup)                 |
|    session.run(None, {"input": data})    <- Phase 2 (inference)             |
|                                                                             |
|  Each triggers calls cascading through all 8 layers from Python        |
|  down to the GPU hardware. Understanding this chain is key to         |
|  understanding the ENTIRE system.                                          |
`-----------------------------------------------------------------------------+

  +-- DIAGRAM: End-to-End API Call Chain (L1 -> L8) --------------------------+
  |                                                                            |
  |  ort.InferenceSession("model.onnx")          session.run(input)        |
  |  =========== SETUP ===========               === INFERENCE ===         |
```

```
  |                                                                        |
  |  L1: Python API call                        L1: session.run()         |
  |    |                                          |                       |
  |  L2: Model::Load -> Partition                L2: execute compute_func  |
  |    |                                          |                       |
  |  L3: EP::Compile -> 3 MIGraphX calls          | (skipped, compiled)    |
  |    |                                          |                       |
  |  L4: parse -> quantize -> 50+ passes         L4: program::eval()       |
  |    |   (fusion, scheduling, JIT)              |  walk instruction list |
  |    |                                          |                       |
  |  L5: hiprtcCompileProgram (JIT)             L5: hipLaunchKernel xN    |
  |    |   hipMalloc (weight buffers)             |  hipMemcpy (I/O)      |
  |    |                                          |                       |
  |  L6: AQL packets -> queues                   L6: AQL dispatch + signal |
  |    |   ioctl for memory setup                 |  doorbell MMIO write   |
  |    |                                          |                       |
  |  L7: TTM allocates BOs, page tables         L7: (already set up)      |
  |    |                                          |                       |
  |  L8: GPU programs page table regs           L8: CUs execute kernels   |
  |                                                                        |
  |  Time: seconds to minutes                   Time: milliseconds        |
  `------------------------------------------------------------------------+

The Complete API Call Chain - InferenceSession()
session = ort.InferenceSession("model.onnx",
    providers=["MIGraphXExecutionProvider", "CPUExecutionProvider"])
This single Python line triggers hundreds of API calls cascading through all 8 layers. Here's every significant one:

Layer 1 -> Layer 2: Python to ORT Core
PYTHON                              ORT C++ (onnxruntime.so)
------                              ------------------------
ort.InferenceSession(path,          -> InferenceSession::InferenceSession()
    providers=[...])                    |
                                        |---- Model::Load(path)
                                        |     |
                                        |     |---- ReadProtoFromBinary(path, &model_proto)
                                        |     |     Uses Google protobuf to deserialize
                                        |     |     the .onnx file from disk into a
                                        |     |     ModelProto C++ object
                                        |     |
                                        |     |---- model_proto.graph()
                                        |     |     -> GraphProto containing:
                                        |     |       node[0..N]: operations (Conv, Relu...)
                                        |     |       initializer[0..M]: weight tensors
                                        |     |       input[]: model input shapes
                                        |     |       output[]: model output shapes
                                        |     |
                                        |     `---- Graph::Resolve()
                                        |           Shape inference on every node
                                        |           Type checking
                                        |           Build in-memory graph structure
                                        |
                                        |---- GraphTransformerManager::ApplyTransformers()
                                        |     ORT-level optimizations (before EP):
                                        |     |---- ConstantFolding
                                        |     |---- CommonSubexpressionElimination
                                        |     |---- MatMulAddFusion
                                        |     |---- ReshapeFusion
                                        |     |---- ConvBNFusion
                                        |     `---- ... (30+ ORT-level transforms, separate from MIGraphX's 50+)
                                        |
                                        |---- SessionState::PartitionGraphForEPs()
                                        |     |
                                        |     |---- For each registered EP (in priority order):
                                        |     |     MIGraphXExecutionProvider::GetCapability(graph)
                                        |     |       -> returns list of nodes this EP can handle
                                        |     |       -> "I can run: Conv, Relu, MatMul, Softmax..."
                                        |     |       -> "I cannot run: CustomOp_xyz"
                                        |     |
                                        |     |---- Assign unclaimed nodes to CPUExecutionProvider
                                        |     |
                                        |     `---- Group consecutive same-EP nodes into subgraphs
                                        |           Subgraph 1: [nodes 0-45] -> MIGraphX
                                        |           Subgraph 2: [node 46]    -> CPU
                                        |           Subgraph 3: [nodes 47-82]-> MIGraphX
                                        |
                                        `---- For each EP subgraph:
                                              EP::Compile(subgraph)
                                              -> triggers Layer 3

Layer 2 -> Layer 3: ORT to MIGraphX EP (Thin Adapter)
ORT CORE                                MIGRAPHX EP (C++ plugin in ORT)
--------                                -------------------------------
```

```
EP::GetCapability(graph)           ->    MIGraphXExecutionProvider::GetCapability()
                                            |
                                            |---- Examine every node in the graph
                                            |     "Can MIGraphX handle this op?"
                                            |     Conv [Y], Relu [Y], MatMul [Y], CustomOp [N]
                                            `---- Return: list of supported node groups

EP::Compile(subgraph)              ->    MIGraphXExecutionProvider::Compile()
                                            |
                                            |  The EP is a THIN ADAPTER that makes
                                            |  3 C API calls into Layer 4. It does NOT
                                            |  do optimization, fusion, or compilation.
                                            |
                                            |---- Serialize subgraph back to ONNX protobuf
                                            |     model_proto.SerializeToString(&buffer)
                                            |     (ORT gives a subgraph; MIGraphX wants
                                            |      raw ONNX bytes to parse its own way)
                                            |
                                            |---- CALL 1: migraphx::parse_onnx_buffer()  ---> Layer 4
                                            |     Layer 4 re-parses ONNX protobuf
                                            |     Builds MIGraphX internal IR
                                            |     Returns: migraphx::program
                                            |
                                            |---- CALL 2: migraphx::quantize_fp16(prog)  ---> Layer 4
                                            |     Layer 4 converts safe ops to FP16
                                            |     Inserts cast ops at FP32 boundaries
                                            |     Returns: modified program (mixed precision)
                                            |
                                            |---- CALL 3: prog.compile(target, options)   ---> Layer 4
                                            |
                                            |     Layer 4 does ALL the heavy work:
                                            |       50+ graph optimization passes
                                            |       operator fusion (pointwise, CK, MLIR)
                                            |       kernel selection (MIOpen, rocBLAS, JIT)
                                            |       JIT compilation via hipRTC         ---> Layer 5
                                            |         (COMGR/LLVM: HIP C++ -> GPU ISA)
                                            |       memory planning (memory_coloring)
                                            |       instruction scheduling
                                            |     Returns: compiled program (ready to run)
                                            |
                                            `---- Store compiled program for later run()

compute_func(inputs)               ->    MIGraphXExecutionProvider compute callback
                                            |
                                            |---- Map ORT tensors to MIGraphX arguments
                                            |---- CALL: prog.run_async(args, stream)     ---> Layer 4
                                            |     Layer 4 walks execution plan:
                                            |       hipMemcpy, hipLaunchKernel, etc.   ---> Layer 5
                                            `---- Return: output tensors

Layer 3 -> Layer 4: MIGraphX EP to libmigraphx.so
(See Chapter 3 for detailed context on every optimization pass,
 fusion type, kernel selection, and memory planning step.
 The EP simply calls these C APIs -- Layer 4 does all the work.)

MIGRAPHX EP                          LIBMIGRAPHX.SO (core library)
-----------                          -----------------------------
parse_onnx_buffer(buf, len)     ->    onnx_parser::parse()
                                          |
                                          |---- protobuf::Parse(buf, &model_proto)
                                          |     Google protobuf deserializes bytes
                                          |
                                          |---- For each NodeProto in graph:
                                          |     Create migraphx::instruction
                                          |     |---- op_type -> migraphx::op::convolution
                                          |     |              migraphx::op::dot (matmul)
                                          |     |              migraphx::op::relu
                                          |     |              migraphx::op::softmax
                                          |     |              ... (~150 supported ops)
                                          |     |---- inputs -> references to other instructions
                                          |     `---- attributes -> kernel_size, strides, etc.
                                          |
                                          `---- Return: migraphx::program
                                                Contains migraphx::module "main"
                                                with ordered list of instructions

quantize_fp16(program)           ->    quantize_fp16_pass::apply()
                                          |
                                          |---- For each instruction:
                                          |     Is this op safe for FP16?
                                          |     |---- YES (conv, matmul, relu, add, mul):
                                          |     |     Change output type to FP16
                                          |     |     Change weight type to FP16
```

```
                                        |     `---- NO (softmax, layernorm reductions):
                                        |           Keep in FP32
                                        |           Insert convert(fp16->fp32) before
                                        |           Insert convert(fp32->fp16) after
                                        |
                                        `---- Return: modified program (mixed precision)

compile(program, target("gpu")) ->    program::compile()
                                        |
                                        |---- PASS GROUP 1: Graph Optimizations
                                        |   |---- simplify_reshapes()
                                        |   |---- simplify_algebra()
                                        |   |     x * 1 -> x,  x + 0 -> x
                                        |   |---- eliminate_common_subexpression()
                                        |   |---- dead_code_elimination()
                                        |   |---- constant_folding()
                                        |   |     Pre-compute weight reshapes, transposes
                                        |   |---- rewrite_batchnorm()
                                        |   |     BN(x) -> x * scale + bias
                                        |   `---- (repeat) REPEAT until graph stops changing
                                        |
                                        |---- PASS GROUP 2: Fusion
                                        |   |---- fuse_pointwise()
                                        |   |     Chain of add/mul/relu -> single JIT kernel
                                        |   |---- fuse_ops()
                                        |   |     Conv+BN+Relu -> MIOpen fused call
                                        |   |---- fuse_ck()
                                        |   |     GEMM+Softmax+GEMM -> CK fused attention
                                        |   `---- fuse_mlir()
                                        |         Complex patterns -> rocMLIR compiled kernel
                                        |
                                        |---- PASS GROUP 3: Lowering (map to GPU)
                                        |   |---- gpu::lowering()
                                        |   |     For each instruction:
                                        |   |       conv -> gpu::miopen_convolution
                                        |   |       dot  -> gpu::rocblas_gemm
                                        |   |       matmul -> gpu::hip_gemm (hipBLASLt)
                                        |   |       fused_pointwise -> gpu::code_object_op
                                        |   |
                                        |   |---- Kernel selection:
                                        |   |     MIOpen: miopenFindSolutions() ---> Layer 5
                                        |   |       Auto-tunes conv kernels on YOUR GPU
                                        |   |       Benchmarks 4-10 kernel variants
                                        |   |       Picks fastest, caches in ~/.config/miopen/
                                        |   |
                                        |   |     rocBLAS: rocblas_gemm_ex_get_solutions()
                                        |   |       Finds best GEMM kernel for matrix sizes
                                        |   |
                                        |   |     JIT kernels: compile_hip() ---> Layer 5
                                        |   |       Generates HIP C++ source for fused ops
                                        |   |       Compiles via hipRTC or COMGR/LLVM
                                        |   |       Produces GPU code object (.co)
                                        |   |
                                        |   |     rocMLIR: rockCompiler::compile()
                                        |   |       MLIR IR -> LLVM IR -> AMDGPU ISA
                                        |   |
                                        |   `---- dead_code_elimination() (cleanup)
                                        |
                                        |---- PASS GROUP 4: Memory Planning
                                        |   |---- memory_coloring()
                                        |   |     Analyze instruction lifetimes
                                        |   |     Reuse memory when tensors don't overlap
                                        |   |     Plan: "instruction @5 output reuses @2's buffer"
                                        |   |
                                        |   `---- Allocate GPU memory:
                                        |         hipMalloc(&weights_buf, 14 GB) ---> Layer 5
                                        |         hipMalloc(&workspace, 512 MB)  ---> Layer 5
                                        |         hipMalloc(&intermediates, ...)  ---> Layer 5
                                        |         hipMemcpy(weights_buf, host_weights,
                                        |                   size, H2D)           ---> Layer 5
                                        |
                                        |---- PASS GROUP 5: Scheduling
                                        |   |---- schedule_model::schedule()
                                        |   |     Analyze instruction dependencies
                                        |   |     Assign to HIP streams (parallel lanes)
                                        |   |     Insert sync events between streams
                                        |   |
                                        |   `---- Final instruction order:
                                        |         @0: param("input") on stream0
                                        |         @1: fused_conv_bn_relu on stream0
                                        |         @2: conv_branch on stream1
                                        |         @3: event_record(ev1, stream1)
                                        |         @4: stream_wait(stream0, ev1)
```

```
                                        |         @5: add_residual on stream0
                                        |         @6: matmul on stream0
                                        |         ...
                                        |
                                        `---- Return: compiled program
                                              (execution plan + GPU buffers + kernel handles)

Layer 4 -> Layer 5: libmigraphx.so to HIP Runtime
(See Chapter 4 for detailed context on hipMalloc, hipLaunchKernel, hipMemcpy,
 stream synchronization, and the hipRTC/COMGR/LLVM compilation pipeline.)

LIBMIGRAPHX.SO                       HIP RUNTIME (libamdhip64.so)
--------------                       ----------------------------

=== DURING COMPILE (setup) ===

Memory allocation:
  hipMalloc(&ptr, 14 GB)         ->    hipMalloc()
                                        |---- Check internal memory pool
                                        |     Pool has free block? -> return (fast, ~1 mus)
                                        |     No free block? -> call HSA ---> Layer 6
                                        `---- Return: device pointer

Weight transfer:
  hipMemcpy(gpu, cpu, size, H2D) ->    hipMemcpy()
                                        |---- Determine transfer type:
                                        |     hipMemcpyHostToDevice
                                        |---- Pin source memory (if needed)
                                        |---- Program SDMA engine via HSA ---> Layer 6
                                        `---- Wait for DMA completion

Kernel compilation (JIT):
  hiprtcCompileProgram(prog)     ->    hiprtcCompileProgram()
                                        |---- HIP C++ source -> COMGR/LLVM
                                        |     Clang frontend -> LLVM IR
                                        |     AMDGPU backend -> gfx1151 ISA
                                        |     Linker -> code object (ELF binary)
                                        `---- Return: compiled module handle

  hipModuleLoadData(&mod, co)    ->    hipModuleLoadData()
                                        |---- Parse ELF code object
                                        |---- Upload ISA to GPU memory ---> Layer 6
                                        `---- Return: module handle

  hipModuleGetFunction(&func,    ->    hipModuleGetFunction()
      mod, "kernel_name")               |---- Look up symbol in module
                                        `---- Return: function handle

=== DURING INFERENCE (every run) ===

Input transfer:
  hipMemcpyAsync(gpu_in, cpu_in, ->    hipMemcpyAsync()
      size, H2D, stream)                |---- Queue async DMA on stream
                                        |---- hsa_amd_memory_async_copy() ---> Layer 6
                                        `---- Return immediately (async!)

Library kernel calls:
  miopenConvolutionForward(      ->    [MIOpen internally calls]
      handle, ...)                      hipLaunchKernel(ImplicitGemmV4, ...)
                                          |---- Package kernel args
                                          |---- Write AQL packet ---> Layer 6
                                          `---- Return immediately (async!)

  rocblas_gemm_ex(               ->    [rocBLAS internally calls]
      handle, ...)                      hipLaunchKernel(Tensile_HGEMM, ...)
                                          |---- Package kernel args
                                          |---- Write AQL packet ---> Layer 6
                                          `---- Return immediately (async!)

  hipblasLtMatmul(               ->    [hipBLASLt internally calls]
      handle, ...)                      hipLaunchKernel(fused_gemm, ...)
                                          |---- Package kernel args
                                          |---- Write AQL packet ---> Layer 6
                                          `---- Return immediately (async!)

JIT kernel calls:
  hipModuleLaunchKernel(         ->    hipModuleLaunchKernel()
      func, grid, block,                |---- Package kernel args into GPU memory
      sharedMem, stream, args)          |---- Write AQL packet ---> Layer 6
                                        `---- Return immediately (async!)

Stream sync:
  hipEventRecord(event, stream1) ->    hipEventRecord()
```

```
                                        |---- Associate HSA signal with stream position
                                        `---- hsa_signal_create() ---> Layer 6

  hipStreamWaitEvent(stream0,    ->    hipStreamWaitEvent()
      event1)                           |---- Insert barrier AQL packet in stream0
                                        |     that waits on event1's signal
                                        `---- hsa_queue_store() ---> Layer 6

Output transfer:
  hipMemcpyAsync(cpu_out,        ->    hipMemcpyAsync()
      gpu_out, size, D2H, stream)       |---- Queue async DMA
                                        `---- Return immediately (async!)

Final sync:
  hipStreamSynchronize(stream0)  ->    hipStreamSynchronize()
                                        |---- hsa_signal_wait() ---> Layer 6
                                        |     Blocks CPU until GPU finishes all
                                        |     queued work on stream0
                                        `---- Return when GPU done

Layer 5 -> Layer 6: HIP Runtime to HSA Runtime
(See Chapter 4 section 4.3 and Chapter 5 for additional context on AQL packet
 construction, doorbell writes, and signal management details.)

HIP RUNTIME                           HSA RUNTIME (libhsa-runtime64.so)
-----------                           ---------------------------------

=== INITIALIZATION ===

hipInit(0)                       ->    hsa_init()
                                        |---- Load HSA runtime library
                                        |---- Open /dev/kfd (kernel driver) ---> Layer 7
                                        |---- Enumerate hardware:
                                        |     hsa_iterate_agents(callback)
                                        |       Agent 0: CPU (AMD EPYC/Ryzen)
                                        |       Agent 1: GPU (gfx1151, Radeon 8060S)
                                        |
                                        |---- For each GPU agent:
                                        |     hsa_amd_agent_iterate_memory_pools()
                                        |       Pool 0: VRAM (512 MB) coarse-grained
                                        |       Pool 1: GTT (28 GB) fine-grained
                                        |       Pool 2: LDS (per-CU, 64 KB)
                                        |       Pool 3: Kernarg (small, fine-grained)
                                        |
                                        `---- Create default queues per GPU

=== MEMORY ALLOCATION ===

hipMalloc(&ptr, size)            ->    hsa_amd_memory_pool_allocate(
                                          vram_pool, size, 0, &ptr)
                                        |
                                        |---- ioctl(kfd_fd,                    ---> Layer 7
                                        |       KFD_IOC_ALLOC_MEMORY_OF_GPU,
                                        |       {size, GPU_ID, VRAM_TYPE})
                                        |     Kernel driver:
                                        |       TTM allocates physical pages
                                        |       Programs GPU page table (GPUVM)
                                        |       Returns kernel BO handle
                                        |
                                        |---- hsa_amd_agents_allow_access()
                                        |     If CPU also needs access:
                                        |       Map into CPU page table too
                                        |
                                        `---- Return: GPU virtual address

hipFree(ptr)                     ->    hsa_amd_memory_pool_free(ptr)
                                        |---- ioctl(kfd_fd,                    ---> Layer 7
                                        |       KFD_IOC_FREE_MEMORY_OF_GPU)
                                        |     TTM frees physical pages
                                        |     Removes page table entries
                                        `---- Return memory to pool

=== KERNEL LAUNCH ===

hipLaunchKernel(func, grid,      ->    [HIP builds AQL packet internally]
    block, args, shared, stream)
                                       1. Allocate kernarg memory:
                                          hsa_amd_memory_pool_allocate(
                                              kernarg_pool, args_size, ...)
                                          Copy kernel args (GPU pointers, constants)
                                          into kernarg buffer

                                       2. Reserve queue slot:
```

```
                                        write_idx = hsa_queue_load_write_index_relaxed(queue)
                                        hsa_queue_store_write_index_relaxed(queue, write_idx+1)

                                     3. Write AQL Kernel Dispatch packet (64 bytes):
                                        packet->header          = KERNEL_DISPATCH | BARRIER
                                        packet->setup           = dimensions (1D/2D/3D)
                                        packet->workgroup_size  = {block.x, block.y, block.z}
                                        packet->grid_size       = {grid.x * block.x, ...}
                                        packet->kernel_object   = func's GPU code address
                                        packet->kernarg_address = kernarg buffer address
                                        packet->group_segment   = shared memory size
                                        packet->completion_signal = signal handle

                                     4. Ring the doorbell:
                                        hsa_signal_store_screlease(
                                            queue->doorbell_signal,
                                            write_idx)
                                        ---> This is an MMIO write to GPU register ---> Layer 7
                                        ---> NO system call! Pure userspace!
                                        ---> Takes ~nanoseconds

                                     5. Return immediately (GPU works async)

=== SYNCHRONIZATION ===

hipStreamSynchronize(stream)     ->    Get completion signal for last packet on stream

                                     hsa_signal_wait_scacquire(
                                         signal,
                                         HSA_SIGNAL_CONDITION_EQ,
                                         0,        // wait until value == 0
                                         UINT64_MAX, // no timeout
                                         HSA_WAIT_STATE_BLOCKED)

                                     Internally:
                                       |---- Spin briefly (active wait, ~1 mus)
                                       |---- If signal still not 0:
                                       |     ioctl(kfd_fd,                  ---> Layer 7
                                       |         KFD_IOC_WAIT_EVENTS)
                                       |     Kernel puts thread to sleep
                                       |     GPU interrupt wakes it when done
                                       `---- Return when signal == 0

=== DATA TRANSFER ===

hipMemcpyAsync(dst, src,         ->    hsa_amd_memory_async_copy(
    size, direction, stream)               dst, dst_agent,
                                           src, src_agent,
                                           size, ...)
                                       |
                                       |---- Enqueue DMA operation on SDMA engine
                                       |     ioctl(kfd_fd,                  ---> Layer 7
                                       |         KFD_IOC_COPY_MEMORY)
                                       |     Programs SDMA engine registers
                                       |     SDMA copies data independently
                                       |     of GPU compute units
                                       |
                                       `---- Return immediately (async)

=== PAGE FAULT (XNACK, if enabled) ===

GPU accesses unmapped page       ->    [Hardware generates page fault interrupt]

                                     hsa_amd_svm_page_fault_handler()  ---> Layer 7
                                       |---- ioctl -> kernel driver handles fault
                                       |     Find page in CPU RAM
                                       |     Migrate to GPU VRAM (DMA copy)
                                       |     Update GPU page table
                                       |     Resume faulting wavefront
                                       `---- [transparent to application]

Layer 6 -> Layer 7: HSA Runtime to Kernel Driver
(Also see Chapter 5 section 5.5 and Chapter 6 for additional context on ioctl calls,
 TTM allocation, page table updates, and interrupt handling details.)

HSA RUNTIME                          AMDGPU KERNEL DRIVER (amdgpu.ko)
-----------                          --------------------------------

=== DEVICE DISCOVERY ===

hsa_init() opens /dev/kfd        ->    kfd_open()
                                       |---- Verify user permissions
                                       |---- Create per-process GPU context
```

```
                                        |     struct kfd_process
                                        |     Tracks all BOs, queues, events
                                        `---- Return file descriptor

hsa_iterate_agents()             ->    ioctl(KFD_IOC_GET_SYSTEM_PROPERTIES)
                                        |---- Read GPU identity registers:
                                        |     RREG32(mmGRBM_CHIP_ID)
                                        |       -> chip_family = GFX1151
                                        |     RREG32(mmGC_CONFIG)
                                        |       -> num_cu, num_se, num_sh
                                        |     RREG32(mmMC_ARB_RAMCFG)
                                        |       -> memory config
                                        |
                                        `---- Return: topology info
                                              GPU name, CU count, memory sizes,
                                              ISA version, cache sizes, etc.

=== MEMORY MANAGEMENT ===

hsa_amd_memory_pool_allocate()   ->    ioctl(KFD_IOC_ALLOC_MEMORY_OF_GPU)
                                        |
                                        |---- amdgpu_amdkfd_gpuvm_alloc_memory()
                                        |     |
                                        |     |---- amdgpu_bo_create()
                                        |     |     TTM subsystem:
                                        |     |     |---- ttm_bo_init_reserved()
                                        |     |     |     Create buffer object (BO)
                                        |     |     |
                                        |     |     |---- ttm_bo_validate()
                                        |     |     |     Domain = VRAM?
                                        |     |     |     |---- amdgpu_vram_mgr_new()
                                        |     |     |     |     drm_mm_insert_node()
                                        |     |     |     |     Find free range in VRAM
                                        |     |     |     |     (interval tree, O(log n))
                                        |     |     |     |     Mark pages as allocated
                                        |     |     |     |
                                        |     |     |     Domain = GTT?
                                        |     |     |     |---- amdgpu_gtt_mgr_new()
                                        |     |     |     |     Allocate from system RAM
                                        |     |     |     |     Pin pages (prevent swap)
                                        |     |     |     |     Setup IOMMU mapping
                                        |     |     |     |
                                        |     |     |     Not enough VRAM?
                                        |     |     |     `---- EVICTION:
                                        |     |     |           Find LRU victim BO
                                        |     |     |           DMA copy: VRAM -> GTT
                                        |     |     |           (SDMA engine, ~PCIe speed)
                                        |     |     |           Update victim's page table
                                        |     |     |           Free VRAM pages
                                        |     |     |           Retry allocation
                                        |     |     |
                                        |     |     `---- ttm_tt_populate()
                                        |     |           Allocate physical page frames
                                        |     |
                                        |     `---- Return: BO handle
                                        |
                                        |---- amdgpu_amdkfd_gpuvm_map_memory()
                                        |     Update GPU page table (GPUVM):
                                        |     |---- amdgpu_vm_bo_map()
                                        |     |     Walk 4-level page table
                                        |     |     PDB1 -> PDB0 -> PDE -> PTE
                                        |     |     Write PTE entries with physical addresses
                                        |     |     Set permissions (read/write/execute)
                                        |     |
                                        |     |---- amdgpu_vm_update_ptes()
                                        |     |     Program GPU page table via SDMA:
                                        |     |     WREG32(mmSDMA_...) ---> Layer 8
                                        |     |     SDMA writes PTEs to GPU-accessible memory
                                        |     |
                                        |     `---- Flush TLB:
                                        |           WREG32(mmGCVM_INVALIDATE_ENG_REQ, ...)
                                        |           GPU TLB caches cleared ---> Layer 8
                                        |
                                        `---- Return: GPU virtual address to userspace

hsa_amd_memory_pool_free()       ->    ioctl(KFD_IOC_FREE_MEMORY_OF_GPU)
                                        |---- Unmap from GPU page table
                                        |---- TTM releases physical pages
                                        |     drm_mm_remove_node()
                                        `---- BO destroyed

=== QUEUE MANAGEMENT ===
```

```
hsa_queue_create()                ->    ioctl(KFD_IOC_CREATE_QUEUE)
                                         |
                                         |---- Allocate queue ring buffer
                                         |     amdgpu_bo_create(queue_size * 64 bytes)
                                         |     (64 bytes per AQL packet)
                                         |
                                         |---- Allocate doorbell page
                                         |     Map doorbell MMIO register to userspace
                                         |     mmap() on doorbell BAR region
                                         |     -> HSA can now write doorbell from userspace!
                                         |
                                         |---- Program hardware queue:
                                         |     WREG32(mmCP_HQD_PQ_BASE, ring_gpu_addr) -> L8
                                         |     WREG32(mmCP_HQD_PQ_RPTR, 0)              -> L8
                                         |     WREG32(mmCP_HQD_PQ_WPTR, 0)              -> L8
                                         |     WREG32(mmCP_HQD_ACTIVE, 1)               -> L8
                                         |     "Activate this hardware queue"
                                         |
                                         `---- Return: queue handle + doorbell address

Doorbell write (userspace MMIO)  ->    [NO IOCTL -- direct hardware access!]
  hsa_signal_store(doorbell, idx)       CPU writes to memory-mapped GPU register
                                         |---- PCIe BAR2 mapped to userspace
                                         |     virtual addr -> GPU doorbell register
                                         |
                                         `---- GPU Command Processor detects ---> Layer 8
                                               doorbell change and reads queue

=== INTERRUPT HANDLING ===

GPU completes kernel              ->    [GPU fires MSI/MSI-X interrupt]
                                         |
                                         |---- amdgpu_irq_handler() (ISR)
                                         |     RREG32(mmIH_RB_RPTR)
                                         |     Read interrupt ring buffer
                                         |     Determine source: compute, SDMA, fault...
                                         |
                                         |---- If compute completion:
                                         |     Signal the HSA completion signal
                                         |     Wake any thread in hsa_signal_wait()
                                         |
                                         |---- If page fault (XNACK):
                                         |     amdgpu_vm_handle_fault()
                                         |     Migrate page, update page table
                                         |     Resume faulting wavefront
                                         |
                                         `---- If GPU hang detected:
                                               amdgpu_device_gpu_recover()
                                               Soft reset -> hard reset -> PCI FLR

=== POWER / THERMAL ===

[Driver monitors continuously]    ->    SMU (System Management Unit) interface
                                         WREG32(mmMP1_SMN_C2PMSG_x, command)
                                         |---- Read temperature sensors
                                         |---- Adjust clock speeds
                                         |---- Throttle if overheating
                                         `---- Report via sysfs:
                                               /sys/class/drm/card0/device/hwmon/
                                                 temp1_input, power1_average, freq1_input

Layer 7 -> Layer 8: Kernel Driver to Silicon
(Also see Chapter 6 section 6.5 and Chapter 7 for additional context on Command
 Processor dispatch, CU execution, memory hierarchy, and SDMA transfers.)

AMDGPU DRIVER (register writes)     GPU HARDWARE (Radeon 8060S, gfx1151)
--------------------------------     -------------------------------------

=== DOORBELL -> COMMAND PROCESSOR -> COMPUTE ===

MMIO write to doorbell register  ->   Command Processor (CP) hardware block
                                       |
                                       |---- CP detects doorbell value changed
                                       |     (hardware comparator, ~1 cycle)
                                       |
                                       |---- CP reads AQL packet from queue in memory
                                       |     DMA read from queue ring buffer
                                       |     64 bytes -> one cache line
                                       |
                                       |---- CP decodes packet:
                                       |     Type: KERNEL_DISPATCH
                                       |     Grid: 256 blocks x 256 threads
                                       |     Kernel: address 0x7f12...
```

```
                                        |     Args: address 0x7f34...
                                        |     Shared mem: 16 KB per block
                                        |
                                        |---- CP dispatches to Shader Engines:
                                        |
                                        |   +-- Shader Engine 0 -------------------+
                                        |   |                                      |
                                        |   |  +--- WGP 0 ----------------------+ |
                                        |   |  |  CU 0:                        | |
                                        |   |  |    SIMD32[0]: wavefront 0-7   | |
                                        |   |  |    SIMD32[1]: wavefront 8-15  | |
                                        |   |  |    WMMA unit: matrix ops      | |
                                        |   |  |    LDS: 64 KB shared memory   | |
                                        |   |  |    L0$: 32 KB vector cache    | |
                                        |   |  |                               | |
                                        |   |  |  CU 1:                        | |
                                        |   |  |    SIMD32[0]: wavefront 16-23 | |
                                        |   |  |    SIMD32[1]: wavefront 24-31 | |
                                        |   |  |    ...                        | |
                                        |   |  `---------------------------------+ |
                                        |   |  +--- WGP 1 ----------------------+ |
                                        |   |  |  ...                          | |
                                        |   |  `---------------------------------+ |
                                        |   `----------------------------------------+
                                        |
                                        `---- CP manages dispatch queue:
                                              As CUs finish blocks, CP sends new ones
                                              Until all 256 blocks are complete
                                              Then decrements completion signal

=== COMPUTE UNIT EXECUTION ===

Wavefront executes kernel ISA      ->   SIMD32 hardware pipeline
                                        |
                                        |---- Instruction fetch:
                                        |     Read ISA from L0 instruction cache
                                        |     If miss -> fetch from L2 -> LPDDR5x
                                        |
                                        |---- Scalar instructions (uniform across wavefront):
                                        |     s_load_dword s0, [kernarg]
                                        |       Scalar unit reads kernel arguments
                                        |       s0 = N (number of elements)
                                        |
                                        |---- Vector instructions (per-thread):
                                        |     v_global_load_b32 v0, [addr + tid*2]
                                        |       32 threads each load 4 bytes
                                        |       128 bytes total (one cache line)
                                        |       |
                                        |       |---- Check L0 vector cache (32 KB) -> L8 cache
                                        |       |     Hit? -> return in ~20 cycles
                                        |       |     Miss v
                                        |       |---- Check L2 (2 MB) -> Infinity Cache (32 MB) -> L8
                                        |       |     L2 hit: ~50 cycles; MALL hit: ~100 cycles
                                        |       |     (~1 TB/s effective MALL bandwidth)
                                        |       |     Miss v
                                        |       `---- Fetch from LPDDR5x system RAM -> L8 DRAM
                                        |             ~300+ cycles, ~256 GB/s
                                        |             Wavefront STALLS (switched out)
                                        |             Another wavefront runs instead
                                        |
                                        |---- ALU instructions:
                                        |     v_pk_max_f16 v0, v0, 0    (packed ReLU)
                                        |       32 SIMD lanes x 2 FP16 per lane = 64 ReLUs
                                        |       Executes in 1 cycle
                                        |
                                        |     v_pk_add_f16 v0, v0, v1   (packed Add)
                                        |       64 FP16 additions in 1 cycle
                                        |
                                        |---- WMMA instructions (matrix acceleration):
                                        |     v_wmma_f32_16x16x16_f16 acc, a, b
                                        |       16x16 matrix tile multiply
                                        |       8,192 FP16 ops in a few cycles
                                        |       Used by rocBLAS / hipBLASLt for GEMM
                                        |
                                        |---- LDS (Local Data Share) access:
                                        |     ds_write_b32 v0, [lds_offset]
                                        |     ds_read_b32 v1, [lds_offset]
                                        |       64 KB per CU, ~20 cycle latency
                                        |       Used for workgroup-level data sharing
                                        |       Tiled GEMM stores tiles here
                                        |
                                        `---- Store results:
                                              v_global_store_b32 [addr + tid*2], v0
```

```
                                        Write through L0 -> L2 -> LPDDR5x
                                        Write-back caching

=== MEMORY SYSTEM ===

GPU memory access                  ->   Memory hierarchy hardware
                                        |
                                        |---- L0 Vector Cache (per CU):
                                        |     Size: 32 KB, 4-way associative
                                        |     Line: 128 bytes
                                        |     Latency: ~20 cycles
                                        |     Bandwidth: ~hundreds GB/s per CU
                                        |
                                        |---- L2 Cache (shared):
                                        |     Size: 2 MB
                                        |     Latency: ~50 cycles
                                        |
                                        |---- Infinity Cache / MALL (memory-side LLC):
                                        |     Size: 32 MB
                                        |     Line: 128 bytes
                                        |     Latency: ~100 cycles
                                        |     Bandwidth: ~1 TB/s aggregate
                                        |     <- YOUR BEST FRIEND for ML!
                                        |     If working set < 32 MB, you're golden
                                        |
                                        |---- Memory Controller -> LPDDR5x:
                                        |     256-bit LPDDR5x-8000
                                        |     Bandwidth: ~256 GB/s (shared with CPU!)
                                        |     Latency: ~300+ GPU cycles
                                        |     <- YOUR BOTTLENECK for large models
                                        |
                                        `---- VRAM carve-out (512 MB):
                                              Same physical LPDDR5x
                                              Same bandwidth as GTT
                                              But guaranteed not stolen by CPU
                                              On APU: VRAM ~= GTT in speed

=== DMA ENGINE (SDMA) ===

hipMemcpy / TTM eviction           ->   SDMA (System DMA) engine
                                        |
                                        |---- Independent of compute CUs
                                        |     Can copy data WHILE CUs compute
                                        |
                                        |---- For APU (your system):
                                        |     Memory-to-memory copy within LPDDR5x
                                        |     Bandwidth: limited by LPDDR5x (~256 GB/s)
                                        |     BUT: competes with CPU + GPU for bandwidth
                                        |
                                        `---- For discrete GPU (MI210):
                                              Copies across PCIe: ~64 GB/s
                                              Or across XGMI: ~92 GB/s (GPU-to-GPU)

=== INTERRUPT GENERATION ===

Kernel completion                  ->   Interrupt controller hardware
                                        |
                                        |---- CP decrements completion signal
                                        |     Atomic decrement: signal value 1 -> 0
                                        |
                                        |---- If anyone waiting (BLOCKED state):
                                        |     Generate MSI-X interrupt
                                        |     Interrupt routed to CPU core via APIC
                                        |     CPU runs amdgpu_irq_handler()  ---> Layer 7
                                        |     Handler wakes sleeping thread
                                        |
                                        `---- If active polling (ACTIVE state):
                                              No interrupt needed
                                              CPU sees signal change on next poll
                                              (~1-2 mus latency)


PHASE 2: INFERENCE (Every session.run())
----------------------------------------

The Complete Timed API Call Chain - One Inference
output = session.run(None, {"input": input_data})
Here is every significant call, timed, from Python to silicon and back:
TIME       LAYER   API CALL                                          WHAT HAPPENS
-----      -----   --------                                          ------------
 0 mus      L1      session.run(None, {"input": data})                Python entry
 1 mus      L2      InferenceSession::Run()                           ORT dispatch
 2 mus      L2        RunImpl() -> find compiled EP programs
```

```
 3 mus      L2        compute_func(subgraph1)                        Start MIGraphX
 4 mus      L3        MIGraphXExecutionProvider compute callback
 5 mus      L3          Copy input numpy -> ORT tensor
 6 mus      L4          program.run({input_arg})                     MIGraphX starts
--- GPU WORK QUEUEING (CPU issues all commands rapidly) ---
 7 mus      L5          hipMemcpyAsync(gpu_in, cpu_in, H2D, s0)      Queue input copy
 8 mus      L6            hsa_amd_memory_async_copy(...)              Program SDMA
 9 mus      L7              ioctl(KFD_IOC_COPY_MEMORY)               SDMA register write
10 mus      L8              SDMA engine starts DMA: LPDDR5x -> VRAM   Hardware copies
12 mus      L5          hipLaunchKernel(fused_conv_bn_relu, s0)       Queue conv kernel
13 mus      L6            hsa_queue_store(packet, queue)              Write AQL packet
14 mus      L6            hsa_signal_store(doorbell, idx)             Ring doorbell
14 mus      L7              [MMIO write - no syscall]                 Doorbell register
14 mus      L8              CP reads AQL, dispatches to CUs           Hardware runs conv
16 mus      L5          hipLaunchKernel(conv_branch, s1)              Queue parallel conv
17 mus      L6            [write AQL + doorbell on stream 1]          Second stream
17 mus      L8              CP dispatches second kernel to free CUs
19 mus      L5          hipEventRecord(ev1, s1)                       Mark stream 1 point
20 mus      L5          hipStreamWaitEvent(s0, ev1)                   Stream 0 waits
21 mus      L6            [insert Barrier AQL packet in s0 queue]     GPU-side sync
23 mus      L5          hipLaunchKernel(fused_add_relu, s0)           Queue fused pointwise
24 mus      L6            [AQL + doorbell]
26 mus      L5          rocblas_gemm_ex(handle, ...)                  Queue GEMM
27 mus      L5            hipLaunchKernel(Tensile_HGEMM, s0)
28 mus      L6              [AQL + doorbell]
30 mus      L5          hipLaunchKernel(fused_layernorm, s0)          Queue LayerNorm
31 mus      L6            [AQL + doorbell]
33 mus      L5          hipLaunchKernel(fused_softmax, s0)            Queue Softmax
34 mus      L6            [AQL + doorbell]
36 mus      L5          hipblasLtMatmul(handle, ...)                  Queue fused GEMM
37 mus      L5            hipLaunchKernel(fused_gemm_gelu, s0)
38 mus      L6              [AQL + doorbell]
40 mus      L5          hipMemcpyAsync(cpu_out, gpu_out, D2H, s0)    Queue output copy
41 mus      L6            hsa_amd_memory_async_copy(...)
--- CPU DONE QUEUEING (~40 mus total) - NOW WAITS ---
43 mus      L5          hipStreamSynchronize(stream0)                 CPU blocks
44 mus      L6            hsa_signal_wait_scacquire(signal, EQ, 0)   Spin, then sleep
--- GPU IS STILL WORKING (executing queued kernels) ---
~50 mus     L8          SDMA completes input copy                     Data in GPU memory
~100 mus    L8          CU finishes fused_conv_bn_relu                Stream 0 kernel done
~350 mus    L8          CU finishes conv_branch                       Stream 1 kernel done
~350 mus    L8          CP sees ev1, unblocks stream 0                GPU-side sync
~370 mus    L8          CU finishes fused_add_relu                    Pointwise done
~500 mus    L8          CU finishes GEMM (rocBLAS)                    Matrix multiply done
~520 mus    L8          CU finishes fused_layernorm                   Norm done
~530 mus    L8          CU finishes fused_softmax                     Softmax done
~650 mus    L8          CU finishes fused_gemm_gelu (hipBLASLt)       Last compute done
~660 mus    L8          SDMA copies output to CPU RAM                 D2H transfer
~665 mus    L8          CP decrements completion signal -> 0           Signal fires!
--- GPU DONE - CPU WAKES UP ---
~665 mus    L8          MSI-X interrupt -> CPU                         Hardware interrupt
~667 mus    L7          amdgpu_irq_handler() -> wake signal waiter     ISR runs
~670 mus    L6          hsa_signal_wait returns (signal == 0)         HSA returns
~671 mus    L5          hipStreamSynchronize returns                   HIP returns
~672 mus    L4          program.run() returns result                   MIGraphX done
~673 mus    L3          MIGraphXExecutionProvider compute_func returns EP done
~674 mus    L2          InferenceSession::Run() returns                ORT done
~675 mus    L1          session.run() returns numpy array              Python gets result
TOTAL: ~675 mus (0.675 ms) for a small model


================================================================================
  APPENDIX C: Summary Tables and Key Insights
================================================================================

+-------------------------------------------------------------------------+
|  BEGINNER'S CONTEXT: How to Read These Tables                           |
|                                                                          |
|  This appendix condenses everything from Chapters 1-7 into quick      |
|  reference tables. If you have read the chapters, these are your       |
|  "cheat sheets." If you haven't, they will not make much sense --      |
|  read the chapters first, then come back here for quick lookups.      |
|                                                                          |
|  Table C.1: Shows every API call at every layer (the "dictionary")    |
|  Table C.2: How many calls happen in one inference (the "count")      |
|  Table C.3: Where data crosses between layers and how (the "map")     |
|  Section C.4: The single most important insight of the entire book    |
`-------------------------------------------------------------------------+


C.1 All Layers, All APIs, One Table
-------------------------------------
```

| LAYER | COMPONENT | KEY API CALLS | WHAT IT DOES |
|---|---|---|---|
| 1 | Python / onnxruntime | ort.InferenceSession() | User-facing API |
| | | session.run() | Load model, run inference |
| | | session.get_providers() | NumPy arrays in/out |
| 2 | ORT Core (C++) onnxruntime.so | Model::Load() | Parse ONNX protobuf |
| | | Graph::Resolve() | Build in-memory graph |
| | | ApplyTransformers() | Generic graph optimizations |
| | | EP::GetCapability() | Ask EPs what they can run |
| | | PartitionGraphForEPs() | Split graph into subgraphs |
| | | EP::Compile() | Hand subgraphs to EPs |
| | | EP::compute_func | Run inference |
| 3 | MIGraphX EP (C++ plugin) | parse_onnx_buffer() | Re-parse ONNX into MIGraphX |
| | | quantize_fp16() | Convert to mixed FP16/FP32 |
| | | compile(target="gpu") | Full GPU compilation |
| | | program.run() | Execute compiled program |
| 4 | libmigraphx.so (MIGraphX core) | simplify_reshapes() | Graph optimizations |
| | | simplify_algebra() | Algebraic simplifications |
| | | dead_code_elimination() | Remove unused ops |
| | | constant_folding() | Pre-compute constants |
| | | fuse_pointwise() | Fuse element-wise chains |
| | | fuse_ops() [MIOpen] | Fuse conv patterns |
| | | fuse_ck() [Composable Kernel] | Fuse GEMM/attention |
| | | fuse_mlir() [rocMLIR] | Compile custom fused kernels |
| | | memory_coloring() | Plan memory reuse |
| | | schedule_model() | Multi-stream scheduling |
| | | miopenConvolutionForward() | Convolution execution |
| | | rocblas_gemm_ex() | Matrix multiply execution |
| | | hipblasLtMatmul() | Fused GEMM execution |
| 5 | libamdhip64.so (HIP Runtime) | hipInit() | Initialize HIP runtime |
| | | hipGetDeviceCount() | How many GPUs? |
| | | hipSetDevice() | Select GPU |
| | | hipMalloc() | Allocate GPU memory |
| | | hipMallocManaged() | Unified CPU+GPU memory |
| | | hipFree() | Free GPU memory |
| | | hipMemcpy() | Sync copy CPU<->GPU |
| | | hipMemcpyAsync() | Async copy CPU<->GPU |
| | | hipLaunchKernel() | Launch GPU kernel |
| | | hipModuleLaunchKernel() | Launch JIT kernel |
| | | hipStreamCreate() | Create execution stream |
| | | hipEventCreate() | Create sync event |
| | | hipEventRecord() | Mark stream position |
| | | hipStreamWaitEvent() | GPU-side stream sync |
| | | hipStreamSynchronize() | CPU waits for stream |
| | | hipDeviceSynchronize() | CPU waits for ALL work |
| | | hiprtcCompileProgram() | JIT compile HIP kernel |
| | | hipModuleLoadData() | Load compiled code object |
| | | hipModuleGetFunction() | Get kernel function handle |
| 6 | libhsa-runtime64.so (HSA Runtime) | hsa_init() | Initialize HSA |
| | | hsa_iterate_agents() | Discover CPUs/GPUs |
| | | hsa_agent_get_info() | Query GPU properties |
| | | hsa_amd_agent_iterate_ memory_pools() | Discover memory pools (VRAM, GTT, LDS) |
| | | hsa_amd_memory_pool_allocate() | Allocate from pool |
| | | hsa_amd_memory_pool_free() | Return to pool |
| | | hsa_amd_agents_allow_access() | Share memory between agents |
| | | hsa_amd_memory_async_copy() | DMA transfer |
| | | hsa_queue_create() | Create GPU command queue |
| | | hsa_queue_load_write_index() | Reserve queue slot |
| | | hsa_queue_store_write_index() | Commit queue slot |
| | | [write AQL packet to queue] | 64-byte kernel dispatch |
| | | hsa_signal_create() | Create sync signal |
| | | hsa_signal_store_screlease() | Ring doorbell (MMIO write) |
| | | hsa_signal_wait_scacquire() | Wait for GPU completion |
| | | hsa_signal_destroy() | Free signal |
| 7 | amdgpu.ko (Kernel Driver) | ioctl(KFD_IOC_ALLOC_MEMORY_OF_GPU) | Allocate physical GPU pages |
| | | ioctl(KFD_IOC_FREE_MEMORY_OF_GPU) | Free physical GPU pages |
| | | ioctl(KFD_IOC_MAP_MEMORY_TO_GPU) | Update GPU page tables |
| | | ioctl(KFD_IOC_CREATE_QUEUE) | Setup HW queue + doorbell |
| | | ioctl(KFD_IOC_DESTROY_QUEUE) | Tear down HW queue |
| | | ioctl(KFD_IOC_WAIT_EVENTS) | Sleep until GPU signals |
| | | ioctl(KFD_IOC_COPY_MEMORY) | Program SDMA for DMA copy |
| | | RREG32(register_offset) | Read GPU hardware register |
| | | WREG32(register_offset, val) | Write GPU hardware register |
| | | TTM: ttm_bo_init_reserved() | Create buffer object |
| | | TTM: ttm_bo_validate() | Place BO in VRAM/GTT |

```
|       |                     | TTM: ttm_bo_move()            | Evict/migrate BO                |
|       |                     | drm_mm_insert_node()          | Allocate VRAM range             |
|       |                     | amdgpu_vm_bo_map()            | Map BO into GPUVM              |
|       |                     | amdgpu_vm_update_ptes()       | Write page table entries        |
|       |                     | amdgpu_irq_handler()          | Handle GPU interrupts          |
|---------+---------------------+-------------------------------+-------------------------------|
|   8   | Radeon 8060S        | Command Processor              | Reads AQL, dispatches blocks  |
|       | (gfx1151, RDNA 3.5) | SIMD32 units (per CU)         | Execute 32 threads in lockstep|
|       |                     | Scalar Unit (per CU)           | Control flow, constants       |
|       |                     | WMMA unit (per CU)             | 16x16 matrix multiply          |
|       |                     | LDS (64 KB per CU)             | Workgroup shared memory        |
|       |                     | L0 Vector Cache (32 KB/CU)     | Per-CU data cache              |
|       |                     | L2 (2 MB) + MALL (32 MB)       | ~1 TB/s, shared by all CUs    |
|       |                     | Memory Controller              | LPDDR5x interface, ~256 GB/s   |
|       |                     | SDMA Engine                    | Async DMA copies               |
|       |                     | Interrupt Controller           | Completion signals -> CPU       |
`----------+---------------------+-------------------------------+-------------------------------+
```

## C.2 API Call Count Summary - One Inference

```
One session.run() call triggers approximately:
  Layer 1 (Python):           1 call    (session.run)
  Layer 2 (ORT Core):        ~5 calls   (dispatch, tensor copy, EP compute)
  Layer 3 (MIGraphX EP):     ~3 calls   (compute, input/output handling)
  Layer 4 (libmigraphx):    ~15 calls   (kernel launches via libraries)
  Layer 5 (HIP):            ~30 calls   (malloc, memcpy, launch, sync, events)
  Layer 6 (HSA):            ~50 calls   (AQL packets, signals, doorbells)
  Layer 7 (Kernel Driver):   ~5 ioctls  (most work is doorbell MMIO, no syscall)
  Layer 8 (Hardware):         ~15 kernel dispatches + ~2 DMA transfers
  Total software calls: ~110
  Total time: ~0.5-5 ms (depending on model size)

  Of that time:
    CPU queueing overhead:  ~40-100 mus  (issuing all calls)
    GPU actual compute:     ~0.4-5 ms   (the real work)
    Sync overhead:          ~2-10 mus    (waiting for signal)

  +-- DIAGRAM: API Call Amplification -- One session.run() -------------------+
  |                                                                           |
  |  Layer        Calls    Time                                              |
  |  -----        -----    ----                                              |
  |  L1 Python  |  1    |#........................|  ~1 mus            |
  |  L2 ORT     |  5    |##.......................|  ~5 mus            |
  |  L3 EP      |  3    |#........................|  ~2 mus            |
  |  L4 MIGraphX| 15    |###......................|  ~10 mus           |
  |  L5 HIP     | 30    |#####....................|  ~20 mus           |
  |  L6 HSA     | 50    |########................|  ~50 mus           |
  |  L7 Driver  |  5    |#........................|  ~10 mus (ioctls) |
  |  L8 GPU     | 15    |##########################|  ~0.4-5 ms < 99%|
  |             ------                                                        |
  |  Total:     ~110 calls                                                   |
  |                                                                           |
  |  CPU overhead (~100 mus) vs GPU compute (~0.4-5 ms) = <2% overhead      |
  |  Key: most "calls" are in-process (zero-cost); only ioctl crosses       |
  |  the kernel boundary; the GPU does 99% of the real work.                |
  `---------------------------------------------------------------------------+
```

## C.3 The Boundary Map - What Crosses Where

```
BOUNDARY                    MECHANISM              COST
--------                    ---------              ----
Python -> C++ (L1->L2)       pybind11 / C API       ~1 mus
                            PyObject -> OrtValue
ORT -> MIGraphX EP (L2->L3)  C++ virtual function   ~0 (same process)
                            EP::Compile/compute_func
EP -> libmigraphx (L3->L4)   C++ shared library     ~0 (same process)
                            dlopen/dlsym linkage
libmigraphx -> HIP (L4->L5)  C++ function call      ~0 (same process)
                            hipLaunchKernel(...)
HIP -> HSA (L5->L6)          C++ function call      ~0 (same process)
                            hsa_queue_store(...)
HSA -> Driver (L6->L7)       ioctl() SYSTEM CALL     ~1-5 mus (context switch)
  (memory allocation)       user -> kernel boundary  EXPENSIVE, done rarely
HSA -> GPU (L6->L8)          MMIO doorbell write     ~100 ns (PCIe posted write)
  (kernel launch)           NO system call!          CHEAP, done frequently
Driver -> GPU (L7->L8)       WREG32 register write   ~100 ns (MMIO)
                            Via PCIe BAR4
GPU -> Driver (L8->L7)       MSI-X interrupt          ~1-5 mus (interrupt latency)
  (completion)              Hardware -> ISR
```

```
Driver -> HSA (L7->L6)       Signal update +          ~1-2 mus
  (wake waiter)             wake_up_process()
HSA -> HIP -> Python         Return chain             ~1-2 mus total
  (L6->L5->...->L1)           Unwinding the call stack

  +-- DIAGRAM: Layer Boundary Crossing Map ---------------------------------+
  |                                                                          |
  |  L1 Python  ---- pybind11/C API ---- L2 ORT Core                      |
  |               (PyObject -> OrtValue)    ~1 mus                            |
  |                                                                          |
  |  L2 ORT     ---- C++ virtual call ---- L3 MIGraphX EP                   |
  |               (same process)            ~0 ns                            |
  |                                                                          |
  |  L3 EP      ---- dlopen linkage ---- L4 libmigraphx.so                 |
  |               (same process)          ~0 ns                              |
  |                                                                          |
  |  L4 MIGraphX ---- C++ call ---- L5 HIP Runtime                         |
  |               (hipLaunchKernel)    ~0 ns                                 |
  |                                                                          |
  |  L5 HIP     ---- C++ call ---- L6 HSA Runtime                          |
  |               (AQL packet write)  ~0 ns                                  |
  |                                                                          |
  |  L6 HSA     =======-+-===============-+-======== L7 / L8                 |
  |              SLOW PATH:      FAST PATH:                                  |
  |              ioctl()          MMIO doorbell                              |
  |              ~1-5 mus         ~40 ns                                     |
  |              (alloc, setup)   (kernel launch)                            |
  |                                                                          |
  |  L7 Driver  ---- WREG32 / DMA ---- L8 GPU Silicon                      |
  |               (register writes)                                          |
  |                                                                          |
  |  L8 GPU     ---- MSI-X interrupt ---- L7 -> L6 -> L5 -> L1 (return)     |
  |               (completion signal)     ~2-5 mus unwind                    |
  `--------------------------------------------------------------------------+


C.4 The Key Design Insight
---------------------------

The entire 8-layer stack is engineered around one principle:
SETUP (rare, can be slow):
  Session creation: ~100 ms - 10 seconds
  Crosses user/kernel boundary many times (ioctl)
  Allocates memory, compiles kernels, auto-tunes
  DONE ONCE
INFERENCE (frequent, must be fast):
  Each run: ~0.5-5 ms
  CPU issues ~30 HIP calls in ~40 mus (just filling queues)
  ZERO system calls for kernel launches (doorbell MMIO)
  Only ONE sync point at the very end
  GPU does all real work asynchronously
This split is why:
  First run:  SLOW  (compile + tune + allocate)
  Every run after: FAST (just queue + execute + sync)
The entire stack, from your Python session.run() through 7 software layers to the RDNA 3.5 silicon in your Radeon 8060S, is optimized s

  +-- DIAGRAM: Setup vs Inference -- The Key Design Split -------------------+
  |                                                                          |
  |  TIME ------------------------------------------------------------> |
  |                                                                          |
  |  +---- SETUP (once) -------------------------+  +-- INFERENCE (repeat) --+|
  |  |                                        |  |                       ||
  |  |  Model::Load      (~100 ms)            |  |  session.run() #1   ||
  |  |  GraphTransformers (~50 ms)            |  |  session.run() #2   ||
  |  |  Partition          (~10 ms)           |  |  session.run() #3   ||
  |  |  parse_onnx        (~200 ms)           |  |       ...           ||
  |  |  quantize_fp16     (~100 ms)           |  |  session.run() #N   ||
  |  |  compile (50+ passes + JIT) (~1-10 sec)  |  |                      ||
  |  |  hipMalloc (ioctl)  (~5 ms)            |  |  Each: ~0.5-5 ms    ||
  |  |                                        |  |  No ioctl!          ||
  |  |  Total: seconds to minutes             |  |  No recompile!     ||
  |  |  Many ioctl system calls (slow)         |  |  Just doorbell MMIO ||
  |  `-----------------------------------------+  `-------------------------+|
  |                                                                          |
  |  Why first run is slow:   compile + allocate + tune (SETUP)            |
  |  Why subsequent runs fly: pre-compiled -> queue AQL -> doorbell -> done  |
  `--------------------------------------------------------------------------+


==============================================================================
  APPENDIX D: Annotated Source Code Walkthroughs
==============================================================================
```

How to read these: Every line of real code is shown with a deep plain-English explanation. Lines marked [SKIP] are logging/bookkeeping -- ignore them on first read. Lines marked [KEY] are the important ones. File paths refer to the ROCm 7.2.0 source tree.

## D.1 Function 1: hipMalloc -- "Allocate GPU Memory"

```
  +-- DIAGRAM: hipMalloc -- Wrapper to Hardware ------------------------------+
  |                                                                            |
  |  YOUR CODE:  hipMalloc(&dev_ptr, 14 * 1024*1024*1024)  // 14 GB      |
  |       |                                                                    |
  |       v   (1) public wrapper -- 4 lines                                   |
  |  hipMalloc(void** ptr, size_t sizeBytes)                                  |
  |       |   HIP_INIT_API -> tracing/profile hook                            |
  |       |   CHECK_STREAM_CAPTURE_SUPPORTED -> validation                     |
  |       v   (2) internal -- real allocator                                   |
  |  ihipMalloc(ptr, size, flags)                                             |
  |       |   check: ptr != null, size != 0, size < max                      |
  |       v   (3) AMD runtime allocation                                      |
  |  amd::SvmBuffer::malloc(ctx, flags, size, align)                          |
  |       |   ROCclr abstraction layer                                        |
  |       v   (4) HSA memory pool                                             |
  |  hsa_amd_memory_pool_allocate(pool, size, ...)                            |
  |       |   HSA runtime -> KFD interface                                     |
  |       v   (5) kernel driver ioctl                                         |
  |  ioctl(fd, AMDGPU_IOC_ALLOC_MEMORY, {size, type, ...})                   |
  |       |   TTM subsystem selects VRAM or GTT pool                           |
  |       v                                                                    |
  |  Physical pages reserved, GPU page tables updated                         |
  |  dev_ptr now valid for GPU access                                         |
  `----------------------------------------------------------------------------+
```

File: clr/hipamd/src/hip_memory.cpp
Line: 778
GitHub: https://github.com/ROCm/clr/blob/rocm-7.2.0/hipamd/src/hip_memory.cpp#L778

What it does in ONE sentence: Reserves a chunk of device memory on the GPU and tells you where it is.

```
LINE 778 | hipError_t hipMalloc(void** ptr, size_t sizeBytes) {
         |
         |   hipError_t  -> "this function returns a success or error code"
         |                  (0 = hipSuccess, other numbers = specific errors)
         |
         |   hipMalloc   -> "the function name -- HIP Memory Allocate"
         |
         |   void** ptr  -> "first input: a pointer-to-pointer"
         |                  WHY double pointer? Because hipMalloc needs to WRITE BACK
         |                  the address of the newly allocated memory. You pass the
         |                  ADDRESS of your pointer variable, and hipMalloc writes
         |                  the new GPU address into it.
         |
         |   size_t sizeBytes -> "second input: how many bytes to allocate"
         |                       Example: sizeBytes = 1048576 means "allocate 1 MB"
         |
LINE 779 |   HIP_INIT_API(hipMalloc, ptr, sizeBytes);
         |
         |   [SKIP] Macro that logs: "hipMalloc was called with these arguments."
         |
LINE 780 |   CHECK_STREAM_CAPTURE_SUPPORTED();
         |
         |   [SKIP] Macro checking if stream capture mode is active. Not relevant here.
         |
LINE 781 |   HIP_RETURN_DURATION(ihipMalloc(ptr, sizeBytes, 0), ReturnPtrValue(ptr));
         |
         |   [KEY] This is the ONLY important line in the entire function.
         |
         |   HIP_RETURN_DURATION(...)  -> macro that:
         |     1. Starts a timer
         |     2. Calls ihipMalloc (the REAL implementation)
         |     3. Stops the timer and records duration
         |     4. Returns the result
         |
         |   ihipMalloc(ptr, sizeBytes, 0)  -> calls the INTERNAL implementation
         |     - ptr = same pointer-to-pointer we received
         |     - sizeBytes = same size we received
         |     - 0 = flags (no special flags, just normal device memory)
         |     - The "i" prefix means "internal"
         |
LINE 782 | }
```

```
Summary: hipMalloc is a 4-line function. It logs, then calls ihipMalloc.
ALL the real work happens in ihipMalloc (see next function).


D.2 Function 2: ihipMalloc -- "The REAL Memory Allocator"
---------------------------------------------------------

  +-- DIAGRAM: ihipMalloc Decision Flow -------------------------------------+
  |                                                                          |
  |  ihipMalloc(ptr, sizeBytes, flags)                                      |
  |       |                                                                  |
  |       v  STEP 1: Validate inputs                                        |
  |  +--------------------------------+                                      |
  |  | ptr == null? -> return ERROR   |                                      |
  |  | size == 0?   -> *ptr=null, OK  |                                      |
  |  `-------------+---------------------+                                     |
  |              v  STEP 2: Pick context                                     |
  |  +-----------------------------------------------+                       |
  |  | flags have SVM_FINE_GRAIN?                    |                       |
  |  |   Yes -> use host_context (CPU-side, coherent) |                      |
  |  |   No  -> use current device context (GPU-side) |                     |
  |  `-------------+-------------------------------------+                     |
  |              v  STEP 3: Check size limit                                 |
  |  +-----------------------------------------------+                       |
  |  | sizeBytes > maxMemAllocSize? -> return OOM     |                      |
  |  | On your APU: max depends on GTT pool (~28 GB) |                       |
  |  `-------------+-------------------------------------+                     |
  |              v  STEP 4: ALLOCATE (the key line!)                         |
  |  +-----------------------------------------------+                       |
  |  | *ptr = amd::SvmBuffer::malloc(ctx, flags, sz)|                       |
  |  |   -> HSA -> KFD ioctl -> physical pages reserved|                      |
  |  |   -> GPU page tables updated                   |                      |
  |  `-------------+-------------------------------------+                     |
  |              v  STEP 5: Check and return                                 |
  |  +-----------------------------------------------+                       |
  |  | *ptr == null? -> log OOM, return error         |                      |
  |  | Otherwise -> tag with device ID, return success|                      |
  |  `-----------------------------------------------+                       |
  `-----------------------------------------------------------------------------+

File: clr/hipamd/src/hip_memory.cpp
Line: 338
GitHub: https://github.com/ROCm/clr/blob/rocm-7.2.0/hipamd/src/hip_memory.cpp#L338

What it does in ONE sentence: Validates inputs, checks available memory,
allocates from the GPU memory pool, and writes the address back to the caller.

LINE 338 | hipError_t ihipMalloc(void** ptr, size_t sizeBytes, unsigned int flags) {
         |
         |   "unsigned int flags" -> a number where each BIT is a yes/no switch
         |     - Bit 0 might mean "fine-grained memory" (CPU+GPU simultaneous access)
         |     - When flags = 0, all switches are OFF -> normal device memory
         |
         |   ============================================
         |   STEP 1: VALIDATE INPUTS
         |   ============================================
         |
LINE 339 |   if (ptr == nullptr) {
LINE 340 |     return hipErrorInvalidValue;
LINE 341 |   }
         |
         |   [KEY] "Did the caller give us a valid address to write back to?"
         |   - nullptr = "no address" -> can't write the result -> error
         |
LINE 342 |   if (sizeBytes == 0) {
LINE 343 |     *ptr = nullptr;
LINE 344 |     return hipSuccess;
LINE 345 |   }
         |
         |   [KEY] "Did they ask for 0 bytes?"
         |   - Allocating 0 bytes is legal -- write nullptr, return success
         |   - This is a common C++ pattern: handle trivial case early
         |
         |   ============================================
         |   STEP 2: DETERMINE MEMORY TYPE
         |   ============================================
         |
LINE 347 |   bool useHostDevice = (flags & CL_MEM_SVM_FINE_GRAIN_BUFFER) != 0;
         |
         |   [KEY] Bitwise AND checks if fine-grained flag is set.
         |   For normal hipMalloc (flags=0): useHostDevice = false
         |
LINE 348 |   amd::Context* curDevContext = hip::getCurrentDevice()->asContext();
```

```
          |
          |   [KEY] Get the current GPU device's context -- holds device handles,
          |   memory pools, queue objects for interacting with this specific GPU.
          |
LINE 349  |   amd::Context* amdContext = useHostDevice ? hip::host_context : curDevContext;
          |
          |   [KEY] Choose context: host (CPU) context for fine-grained, else GPU context
          |
          |   ============================================
          |   STEP 3: CHECK AVAILABLE MEMORY
          |   ============================================
          |
LINE 355  |   const auto& dev_info = amdContext->devices()[0]->info();
          |
          |   [KEY] Get device info struct (max memory, alignment, capabilities)
          |
LINE 358  |   size_t max_device_size = IS_LINUX
LINE 359  |       ? dev_info.maxMemAllocSize_
LINE 360  |       : (dev_info.maxMemAllocSize_ + dev_info.maxPhysicalMemAllocSize_);
          |
          |   [KEY] On Linux: max per-allocation size = maxMemAllocSize_
          |
LINE 362  |   if ((useHostDevice && dev_info.maxPhysicalMemAllocSize_ < sizeBytes) ||
LINE 363  |       (!useHostDevice && max_device_size < sizeBytes)) {
LINE 364  |     return hipErrorOutOfMemory;
LINE 365  |   }
          |
          |   [KEY] Pre-check: fail FAST if requested size exceeds device limits
          |
          |   ============================================
          |   STEP 4: THE ACTUAL ALLOCATION (most important)
          |   ============================================
          |
LINE 367  |   *ptr = amd::SvmBuffer::malloc(*amdContext, flags, sizeBytes,
LINE 368  |                                 dev_info.memBaseAddrAlign_,
          |                                 useHostDevice ? curDevContext->svmDevices()[0] : nullptr);
          |
          |   [KEY] THIS IS THE MOST IMPORTANT LINE IN THE ENTIRE FUNCTION.
          |
          |   amd::SvmBuffer::malloc() calls deeper into AMD ROCclr runtime:
          |     -> HSA Runtime (hsa_amd_memory_pool_allocate)
          |       -> KFD kernel driver (ioctl)
          |         -> Physical pages reserved, GPU page tables updated
          |
          |   FOR YOUR APU (Radeon 8060S):
          |   Since CPU and GPU share LPDDR5x RAM, this reserves a region of the
          |   SHARED LPDDR5x pool and sets up page table entries for GPU access.
          |
          |   ============================================
          |   STEP 5: CHECK AND RETURN
          |   ============================================
          |
LINE 370  |   if (*ptr == nullptr) {
LINE 371-381  [SKIP] Error logging: logs requested vs available memory for debugging
LINE 381  |     return hipErrorOutOfMemory;
LINE 382  |   }
          |
LINE 383  |   size_t offset = 0;
LINE 384  |   amd::Memory* memObj = getMemoryObject(*ptr, offset);
LINE 386  |   memObj->getUserData().deviceId = hip::getCurrentDevice()->deviceId();
          |
          |   [SKIP] Bookkeeping: record which GPU device owns this allocation
          |
LINE 387  |   return hipSuccess;
LINE 388  | }

FULL CALL CHAIN:
  hipMalloc (line 778)  -- public wrapper (4 lines)
    `---- ihipMalloc (line 338)  -- validation + real work
          `---- amd::SvmBuffer::malloc (line 367)  -- AMD runtime allocation
                `---- HSA Runtime (hsa_amd_memory_pool_allocate)
                      `---- KFD kernel driver (ioctl)
                            `---- Physical memory pages reserved, page tables updated


D.3 Function 3: migraphx_parse_onnx_buffer -- "Parse an ONNX Model"
-------------------------------------------------------------------

  +-- DIAGRAM: ONNX Parse -- Bytes to Graph IR -------------------------------+
  |                                                                           |
  |  INPUT: raw bytes (e.g. 14 GB model.onnx file, protobuf-encoded)         |
  |       |                                                                   |
  |       v                                                                   |
```

```
  |  +-------------------------------------------------------+           |
  |  | Google Protobuf Library                                |           |
  |  |  model_proto.ParseFromArray(data, size)                |           |
  |  |  Decodes binary -> structured onnx::ModelProto          |           |
  |  |  Contains: graph, nodes[], initializers[], metadata     |           |
  |  `----------------------------+-------------------------------+           |
  |                               v                                         |
  |  +-------------------------------------------------------+           |
  |  | MIGraphX onnx_parser                                   |           |
  |  |  For each ONNX node:                                   |           |
  |  |    |---- look up op type (Conv, MatMul, Add, ...)       |           |
  |  |    |---- create migraphx::instruction                   |           |
  |  |    `---- connect to inputs (edges in graph)             |           |
  |  |  Load initializers -> weight tensors in graph           |           |
  |  `----------------------------+-------------------------------+           |
  |                               v                                         |
  |  OUTPUT: migraphx::program                                             |
  |  +-------------------------------------------------------+           |
  |  | Complete in-memory graph (IR)                          |           |
  |  |  instructions: [@0=load, @1=conv, @2=relu, @3=matmul...]|           |
  |  |  weights: embedded in graph (float32 tensors)           |           |
  |  |  shapes: inferred for every edge                        |           |
  |  `-------------------------------------------------------+           |
  `-----------------------------------------------------------------------+
```

File: AMDMIGraphX/src/api/api.cpp
Line: 2109
GitHub: https://github.com/ROCm/AMDMIGraphX/blob/rocm-7.2.0/src/api/api.cpp#L2109

What it does in ONE sentence: Takes raw ONNX model bytes (protobuf binary) and builds an in-memory graph of operations (a MIGraphX "program").

```
LINE 2109 | extern "C" migraphx_status migraphx_parse_onnx_buffer(
          |
          |   extern "C"         -> PUBLIC API function callable from any language
          |   migraphx_status    -> Returns success/error code
          |   migraphx_parse_onnx_buffer -> Function name: parse ONNX from memory buffer
          |
LINE 2110 |     migraphx_program_t* out,
          |
          |   "Where to put the result"
          |   - migraphx_program_t is a HANDLE (opaque pointer to migraphx_program struct)
          |   - migraphx_program_t* means "pointer to a handle" -> write handle back
          |   - Same pattern as hipMalloc's void**
          |
LINE 2111 |     const void* data,
          |
          |   "The raw ONNX model bytes" (protobuf-encoded, not modified by this function)
          |
LINE 2112 |     size_t size,
          |
          |   "How many bytes of data"
          |
LINE 2113 |     migraphx_onnx_options_t options)
          |
          |   "Parsing options" (batch size, input shape overrides, etc.)
          |
LINE 2114 | {
LINE 2115 |     auto api_error_result = migraphx::try_([&] {
          |
          |   [KEY] migraphx::try_() is a SAFETY WRAPPER that catches any C++
          |   exception and converts it to a status code. Without this, a C++
          |   exception would crash the entire program.
          |
LINE 2116 |         if(options == nullptr)
LINE 2117 |             MIGRAPHX_THROW(migraphx_status_bad_param, "Bad parameter options: Null pointer");
          |
          |   [SKIP] Safety check for null options
          |
LINE 2118 |         *out = allocate<migraphx_program_t>(
LINE 2119 |             migraphx::parse_onnx_buffer((data), (size), (options->object)));
          |
          |   [KEY] TWO things happen here (read inside-out):
          |
          |   INNER: migraphx::parse_onnx_buffer(data, size, options->object)
          |     This calls the REAL parser in onnx/onnx.cpp line 107:
          |     1. Feeds raw bytes to Google protobuf library to decode
          |     2. Walks every ONNX node (operation) in the decoded model
          |     3. Creates a MIGraphX "instruction" for each node
          |     4. Connects instructions based on ONNX data flow edges
          |     5. Loads weight values (initializers) into the graph
          |     6. Returns a complete migraphx::program object
          |
```

```
          |   OUTER: allocate<migraphx_program_t>(...)
          |     Wraps the C++ program in a C API handle:
          |     1. Creates migraphx_program struct on the heap
          |     2. Stores program inside the struct's "object" field
          |     3. Returns pointer to the struct (the handle)
          |
LINE 2120 |     });
LINE 2121 |     return api_error_result;
          |
          |   [KEY] Return status: migraphx_status_success or an error code
          |
LINE 2122 | }

FULL CALL CHAIN:
  migraphx_parse_onnx_buffer (api.cpp:2109) -- C API front door
    `---- migraphx::parse_onnx_buffer (onnx.cpp:107) -- internal parser
          `---- parse_onnx_from (onnx.cpp:41) -- template that creates parser
                `---- onnx_parser::parse_from() -- walks protobuf, creates instructions


D.4 Function 4: quantize_fp16 -- "Convert FP32 to FP16"
-------------------------------------------------------

  +-- DIAGRAM: FP16 Quantization -- 4-Pass Pipeline -------------------------+
  |                                                                           |
  |  INPUT: program in FP32 (32-bit floats, ~14 GB for 7B model)          |
  |       |                                                                  |
  |       v  PASS 1: normalize_ops                                          |
  |  +-----------------------------------------------+                      |
  |  | Standardize op representations                |                      |
  |  | (consistent patterns for later passes)        |                      |
  |  `-------------------------+-------------------------+                     |
  |                          v  PASS 2: optimize_module (FP32)              |
  |  +-----------------------------------------------+                      |
  |  | Constant folding in FP32 (max precision)      |                      |
  |  | 3.14159 * 2.0 -> 6.28318 (computed now)       |                      |
  |  | Algebraic simplification: x + 0 -> x           |                      |
  |  `-------------------------+-------------------------+                     |
  |                          v  PASS 3: truncate_float_pass  <- THE KEY    |
  |  +-----------------------------------------------+                      |
  |  | For each instruction: insert FP32->FP16 casts  |                      |
  |  | [FP32 tensor]--> convert -->[FP16 tensor]--> op |                      |
  |  | Insert FP16->FP32 at boundaries if needed      |                      |
  |  `-------------------------+-------------------------+                     |
  |                          v  PASS 4: optimize_module (cleanup)          |
  |  +-----------------------------------------------+                      |
  |  | Remove redundant converts: FP32->FP16->FP32     |                      |
  |  | Fold any new constant expressions              |                      |
  |  `-------------------------+-------------------------+                     |
  |                          v                                              |
  |  OUTPUT: program in FP16 (~7 GB, 2x less memory, 2x faster compute)  |
  `-----------------------------------------------------------------------------+

File: AMDMIGraphX/src/quantization.cpp
Line: 68
GitHub: https://github.com/ROCm/AMDMIGraphX/blob/rocm-7.2.0/src/quantization.cpp#L68

What it does in ONE sentence: Walks the entire program and converts 32-bit
floating-point operations to 16-bit (half precision) for better GPU throughput.

WHY? FP16 uses half the memory of FP32:
  - 2x less memory bandwidth needed (the #1 bottleneck on GPUs)
  - 2x more data fits in cache
  - AMD GPUs have dedicated FP16 hardware that runs 2x faster than FP32
  - Trade-off: slightly less precision (usually acceptable for inference)

LINE 68  | void quantize_fp16(program& prog, const std::vector<std::string>& ins_names)
         |
         |   void             -> "returns nothing -- modifies program in-place"
         |   program& prog    -> "the program to modify" (& = reference, not a copy)
         |   const std::vector<std::string>& ins_names -> "which op types to convert"
         |     - Default: {"all"} = convert ALL operations
         |     - Could pass {"add", "mul"} to convert only those types
         |
LINE 69  | {
LINE 70  |     run_passes(prog,
         |
         |   [KEY] run_passes() takes a program and a LIST of passes, runs each
         |   pass on the program IN ORDER -- like an assembly line.
         |
LINE 71  |                {normalize_ops{},
         |
         |   PASS 1: normalize_ops
```

```
         |     "Standardize all operations to a common format"
         |     Different ONNX models may represent the same math differently
         |     (e.g., "subtract" as "add with negation"). Normalizing ensures
         |     later passes see consistent patterns.
         |
LINE 72  |                 optimize_module{{"quantizelinear", "dequantizelinear"}},
         |
         |   PASS 2: optimize_module (in FP32)
         |     "Simplify the graph while still in high precision"
         |     - Constant folding: 3.14159 * 2.0 -> 6.28318 computed now
         |     - Algebraic simplification: x + 0 -> x, x * 1 -> x
         |     WHY before FP16? FP32 has ~7 decimal digits vs ~3 for FP16.
         |     Folding constants in FP32 first preserves maximum accuracy.
         |     The {"quantizelinear", "dequantizelinear"} means: don't optimize
         |     away quantize/dequantize ops -- they are intentional.
         |
LINE 73  |                 truncate_float_pass{ins_names, shape::half_type},
         |
         |   PASS 3: truncate_float_pass  <<<< THIS IS THE ACTUAL FP16 CONVERSION
         |     For each instruction matching ins_names:
         |     1. Insert convert FP32 -> FP16 before the instruction (inputs)
         |     2. The instruction now operates on FP16 data
         |     3. Insert convert FP16 -> FP32 after if downstream needs FP32
         |     shape::half_type = the target precision (16-bit float)
         |
LINE 74  |                 optimize_module{{"quantizelinear", "dequantizelinear"}}},
         |
         |   PASS 4: optimize_module (cleanup)
         |     "Clean up the mess left by FP16 conversion"
         |     - Remove redundant converts: FP32->FP16->FP32 when not needed
         |     - Fold new constant expressions created by conversion
         |
LINE 75  |                quant_tracer());
         |
         |   [SKIP] Optional logging/tracing for debugging
         |
LINE 76  | }

SUMMARY: quantize_fp16 runs exactly 4 passes in order:
  normalize -> optimize(FP32) -> CONVERT to FP16 -> optimize(cleanup)

FULL CALL CHAIN:
  migraphx_quantize_fp16 (api.cpp) -- C API entry
    `---- quantize_fp16 (quantization.cpp:68) -- 4-pass pipeline
          `---- run_passes(prog, {
                |---- normalize_ops{}         -- standardize op representations
                |---- optimize_module{}       -- constant fold in FP32 (preserve precision)
                |---- truncate_float_pass{}   -- INSERT FP32->FP16 converts (the actual conversion)
                `---- optimize_module{}       -- clean up redundant converts
             }, quant_tracer())


D.5 Function 5: program::compile -- "Compile Everything for the GPU"
--------------------------------------------------------------------

  +-- DIAGRAM: program::compile -- 50+ Pass Pipeline ------------------------+
  |                                                                          |
  |  INPUT: parsed + quantized MIGraphX program (IR graph)                  |
  |       |                                                                  |
  |       v  (1) target::get_passes(ctx, options) -> pass list              |
  |                                                                          |
  |  +---- Phase 1: CLEANUP ----------------------------------------------+  |
  |  | simplify_qdq -> dead_code_elimination -> simplify_reshapes         |  |
  |  | eliminate_common_subexpression -> constant_folding                 |  |
  |  `----------------------------+-----------------------------------------+  |
  |                               v                                          |
  |  +---- Phase 2: FUSION -----------------------------------------------+  |
  |  | fuse_attention -> fuse_pointwise -> fuse_reduce -> fuse_concat    |  |
  |  | fuse_mlir (GEMM+pw) -> fuse_ck (Composable Kernels)             |  |
  |  `----------------------------+-----------------------------------------+  |
  |                               v                                          |
  |  +---- Phase 3: LIBRARY COMPILATION ----------------------------------+  |
  |  | compile_miopen (conv/pool) -> compile_hipblaslt (GEMM)           |  |
  |  | compile_ops -> hipRTC JIT -> COMGR/LLVM -> gfx1151 ELF .co        |  |
  |  | This phase generates ACTUAL GPU machine code binaries            |  |
  |  `----------------------------+-----------------------------------------+  |
  |                               v                                          |
  |  +---- Phase 4: SCHEDULING + MEMORY ----------------------------------+  |
  |  | memory_coloring (buffer reuse) -> schedule (multi-stream assign)  |  |
  |  | Determines: which kernel on which stream, in what order          |  |
  |  `----------------------------+-----------------------------------------+  |
  |                               v                                          |
  |  +---- Phase 5: VALIDATION -------------------------------------------+  |
```

```
  |  | validate() -> find_dangling_reference() -> finalize()              |  |
  |  `----------------------------+----------------------------------------+  |
  |                               v                                          |
  |  OUTPUT: fully compiled GPU program (ready to execute)                  |
  |    - Pre-compiled kernels (ELF code objects loaded in GPU memory)       |
  |    - Memory plan (buffer addresses and reuse map)                       |
  |    - Stream schedule (execution order per stream)                       |
  `-------------------------------------------------------------------------+

File: AMDMIGraphX/src/program.cpp
Line: 313
GitHub: https://github.com/ROCm/AMDMIGraphX/blob/rocm-7.2.0/src/program.cpp#L313

What it does in ONE sentence: Takes the optimized graph and transforms it into
actual GPU machine code through 50+ optimization and compilation passes.

LINE 313 | void program::compile(const target& t, compile_options options)
         |
         |   program::compile -> "the compile method BELONGS TO the program class"
         |
         |   const target& t  -> "which hardware to compile for"
         |     - In our case: target is "gpu" (AMD GPU)
         |     - The target knows: GPU architecture (gfx1151), available libraries
         |       (MIOpen, hipBLASLt), number of streams, etc.
         |
         |   compile_options options -> compilation settings:
         |     - offload_copy: should MIGraphX handle CPU<->GPU copies?
         |     - fast_math: allow approximations for speed?
         |     - exhaustive_tune: spend more time finding fastest algorithms?
         |
LINE 314 | {
LINE 316 |     assert(not this->is_compiled());
         |
         |   [SKIP] Safety: "make sure we haven't compiled already" (compile once only)
         |
LINE 317 |     this->impl->targets = {t};
         |
         |   [KEY] Save the target into the program
         |
LINE 318 |     this->impl->contexts = {t.get_context()};
         |
         |   [KEY] Create a GPU context -- initializes device, discovers properties,
         |   sets up memory pools, creates default streams. This is the first point
         |   where actual GPU hardware is touched.
         |
LINE 320 |     if(enabled(MIGRAPHX_TRACE_COMPILE{}))
LINE 321 |         options.trace = tracer{std::cout};
         |
         |   [SKIP] Enable compilation tracing if env var is set
         |
LINE 325 |     auto&& passes = t.get_passes(this->impl->contexts.front(), options);
         |
         |   [KEY] *** THIS IS THE BIG ONE ***
         |
         |   t.get_passes() goes to target.cpp line 94.
         |   It returns a LIST of 50+ optimization/compilation passes including:
         |     - Dead code elimination (remove useless operations)
         |     - Operation fusion (merge adjacent operations)
         |     - MIOpen compilation (convolution kernels)
         |     - hipBLASLt compilation (matrix multiply kernels)
         |     - HIP kernel generation (custom kernels via hipRTC)
         |     - Memory planning (buffer reuse)
         |     - Multi-stream scheduling (parallel execution)
         |
         |   See Appendix D.8 for the full target::get_passes() pass list.
         |
LINE 326 |     run_passes(*this, passes, options.trace);
         |
         |   [KEY] *** RUN ALL 50+ PASSES ***
         |
         |   AFTER this line finishes (seconds to minutes), the program contains:
         |     - Pre-compiled GPU kernels (from MIOpen, hipBLASLt, CK)
         |     - Custom-compiled HIP kernels (via hipRTC -> LLVM -> gfx1151 ISA)
         |     - Memory allocation plan (which buffers, sizes, reuse mapping)
         |     - Multi-stream execution schedule (which kernel on which stream)
         |
LINE 327 |     auto mods = this->get_modules();
         |
LINE 329 |     for(const auto& mod : reverse(mods))
LINE 330 |     {
LINE 331 |         auto invalid = mod->validate();
LINE 332 |         if(invalid != mod->end())
LINE 333 |             MIGRAPHX_THROW("Invalid module " + mod->name() + "...");
```

```
LINE 337 |         auto dangling = mod->find_dangling_reference();
LINE 338 |         if(dangling != mod->end())
LINE 339 |             MIGRAPHX_THROW("Dangling reference in module " + mod->name() + "...");
LINE 344 |         mod->finalize(this->impl->contexts);
         |
         |   [KEY] VALIDATION AND FINALIZATION
         |   - validate(): check every instruction is correct (no broken connections)
         |   - find_dangling_reference(): check for pointers to deleted instructions
         |   - finalize(): last-minute setup (write kernel metadata)
         |   - If anything is wrong -> crash with error (catches compiler bugs)

FULL CALL CHAIN:
  program::compile (program.cpp:313)
    |---- target::get_passes (target.cpp:94) -> returns 50+ passes
    |---- run_passes() -> executes them all:
    |     |---- dead_code_elimination (many times)
    |     |---- fuse_attention -> fuse multi-head attention
    |     |---- fuse_mlir -> fuse GEMM+pointwise via MLIR
    |     |---- fuse_ck -> use Composable Kernel library
    |     |---- compile_miopen -> compile conv/pool via MIOpen
    |     |---- compile_hipblaslt -> compile GEMM via hipBLASLt
    |     |---- compile_ops -> generate + compile custom HIP kernels
    |     |     `---- compile_hip_src() -> hiprtcCompileProgram() (crosses to Layer 5)
    |     |---- memory_coloring -> plan buffer reuse
    |     `---- schedule -> assign operations to streams
    `---- validate + finalize all modules


D.6 Function 6: hipModuleLaunchKernel -- "Send a Kernel to the GPU"
-------------------------------------------------------------------

  +-- DIAGRAM: Kernel Launch -- Software to Hardware -------------------------+
  |                                                                            |
  |  hipModuleLaunchKernel(func, gridX, gridY, gridZ,                         |
  |                        blockX, blockY, blockZ, shMem, stream, args)       |
  |       |                                                                   |
  |       v  validate dims, stream                                            |
  |  ihipLaunchKernelCommand                                                  |
  |       |  create NDRangeKernelCommand on heap                              |
  |       |  copy kernel args into command buffer                            |
  |       v                                                                   |
  |  command->enqueue()                                                       |
  |       |                                                                   |
  |       v  ROCclr translates to HSA dispatch                                |
  |  +-------------------------------------------+                            |
  |  |  AQL Packet (64 bytes, hardware format)  |                             |
  |  |  +---------------------------------------+|                            |
  |  |  | header:    packet type + barriers   ||                             |
  |  |  | workgroup: blockX x blockY x blockZ ||                              |
  |  |  | grid:      gridX x gridY x gridZ    ||                              |
  |  |  | kernel_object: ISA code address      ||                             |
  |  |  | kernarg_address: argument buffer     ||                             |
  |  |  | completion_signal: hsa_signal_t      ||                             |
  |  |  `---------------------------------------+|                             |
  |  `---------------------+------------------------+                          |
  |                        | write to queue ring buffer                       |
  |                        v                                                  |
  |  DOORBELL MMIO WRITE (~40 ns) -> GPU Command Processor wakes up           |
  |       |                                                                   |
  |       v  GPU dispatches wavefronts to CUs                                 |
  |  [=== KERNEL EXECUTES ON GPU HARDWARE ===]                                |
  |       |                                                                   |
  |       v  completion signal decremented                                    |
  |  CPU returns IMMEDIATELY (async -- GPU runs in parallel)                  |
  `---------------------------------------------------------------------------+

File: clr/hipamd/src/hip_module.cpp
Line: 534
GitHub: https://github.com/ROCm/clr/blob/rocm-7.2.0/hipamd/src/hip_module.cpp#L534

What it does in ONE sentence: Takes a compiled GPU kernel and dispatches it
to the GPU hardware for execution.

LINE 534 | hipError_t hipModuleLaunchKernel(
LINE 535 |     hipFunction_t f,
         |
         |   "The compiled kernel to launch"
         |   - hipFunction_t is a HANDLE to a loaded GPU kernel
         |   - Previously compiled (by hipRTC or offline) and loaded (hipModuleLoadData)
         |
         |     uint32_t gridDimX, uint32_t gridDimY, uint32_t gridDimZ,
         |
         |   "GRID dimensions -- how many thread BLOCKS"
```

```
         |   - gridDimX x gridDimY x gridDimZ = total blocks
         |   - Example: 1024 x 1 x 1 -> 1024 blocks
         |
LINE 536 |     uint32_t blockDimX, uint32_t blockDimY, uint32_t blockDimZ,
         |
         |   "BLOCK dimensions -- how many threads PER block"
         |   - Example: 256 x 1 x 1 -> 256 threads/block
         |   - TOTAL = 1024 x 256 = 262,144 threads
         |   - AMD GPUs execute in groups of 32 or 64 ("wavefronts")
         |
LINE 537 |     uint32_t sharedMemBytes,
         |
         |   "Shared memory per block (bytes)"
         |   - LDS (Local Data Share) -- fast on-chip memory shared within a block
         |
LINE 538 |     hipStream_t hStream,
         |
         |   "Which GPU stream to use" (streams are command queues; different
         |   streams can run in parallel)
         |
LINE 539 |     void** kernelParams, void** extra)
         |
         |   "Kernel arguments" -- kernelParams is an array of pointers to each arg
         |
LINE 540 |   HIP_INIT_API(hipModuleLaunchKernel, ...);
         |
         |   [SKIP] Init/logging macro (sets up tracing, TLS error state)

This function delegates to ihipModuleLaunchKernel (line 443) which does:

LINE 443-492: VALIDATION
  - Is the stream valid? Does its device match current device?
  - Is the kernel function handle non-null?
  - Are grid/block dimensions legal? (not too large, not zero)

LINE 493-497: BUILD THE LAUNCH COMMAND
  amd::Command* command = nullptr;
  hip::Stream* hip_stream = hip::getStream(hStream);
  status = ihipLaunchKernelCommand(command, f, launch_params, hip_stream, ...);

Inside ihipLaunchKernelCommand (line 352):
  LINE 359 | function = hip::DeviceFunc::asFunction(f);   // unwrap handle
  LINE 360 | kernel = function->kernel();                  // get compiled GPU code
  LINE 363 | ndrange = NDRangeContainer(3, offset, global, local);  // execution dims
  LINE 374 | kernelCommand = new NDRangeKernelCommand(...);  // create command envelope
  LINE 416-429: Copy kernel arguments into command
  LINE 432 | kernelCommand->captureAndValidate();          // validate everything

Back in ihipModuleLaunchKernel -- the critical dispatch:
  LINE 518 |   command->enqueue();
         |
         |   [KEY] *** THIS IS WHERE THE KERNEL GOES TO THE GPU ***
         |
         |   What happens inside command->enqueue():
         |   1. Command enters ROCclr (AMD's compute runtime layer)
         |   2. ROCclr creates an HSA dispatch:
         |      - Looks up kernel code object (ELF binary)
         |      - Gets arguments buffer address
         |      - Packages into an AQL dispatch packet (64 bytes)
         |   3. HSA writes AQL packet into the HARDWARE QUEUE
         |      (memory region the GPU hardware continuously polls)
         |   4. HSA writes to the DOORBELL REGISTER (memory-mapped address)
         |      -> signals the GPU's command processor to check the queue
         |   5. GPU COMMAND PROCESSOR reads AQL packet
         |      -> allocates wavefronts on Compute Units -> execution begins
         |   6. Kernel EXECUTES: thousands of threads running simultaneously
         |
  LINE 529 |   command->release();    // free command envelope (GPU has its copy)
  LINE 531 |   return hipSuccess;     // returns IMMEDIATELY -- kernel may still run!

FULL CALL CHAIN:
  hipModuleLaunchKernel (line 534) -- public API
    `---- ihipModuleLaunchKernel (line 443) -- validation + dispatch
          |---- ihipLaunchKernel_validate (line 274) -- check dimensions
          |---- ihipLaunchKernelCommand (line 352) -- build the command
          |     |---- new NDRangeKernelCommand() -- create command object
          |     `---- copy arguments + captureAndValidate()
          `---- command->enqueue() (line 518/521) -- submit to GPU
                `---- ROCclr -> HSA -> AQL packet -> hardware queue -> doorbell -> GPU executes


D.7 Function 7: hiprtcCompileProgram -- "Compile Code to GPU Machine Code at Runtime"
--------------------------------------------------------------------------------
```

```
  +-- DIAGRAM: Runtime Compilation -- Source to Binary -----------------------+
  |                                                                            |
  |  INPUT: HIP C++ source (text string generated by MIGraphX JIT)            |
  |  +-------------------------------------------+                             |
  |  | __global__ void fused_kernel(half* x, ...) |                            |
  |  | {                                          |                            |
  |  |   int i = threadIdx.x + blockIdx.x * 256; |                            |
  |  |   half t = (x[i] + bias[i]) * scale[i];  |                            |
  |  |   out[i] = (t > 0) ? t : 0;              |                            |
  |  | }                                          |                            |
  |  `---------------------+------------------------+                            |
  |                        v                                                   |
  |  hiprtcCompileProgram(prog, opts: ["--offload-arch=gfx1151", "-O3"])      |
  |       |                                                                    |
  |       v  COMGR (Code Object Manager)                                       |
  |  +-------------------------------------------+                             |
  |  |  LLVM AMDGPU Backend:                     |                             |
  |  |  (a) Parse: HIP C++ -> LLVM IR            |                             |
  |  |  (b) Optimize: const fold, vectorize      |                             |
  |  |  (c) ISA select: gfx1151 instructions     |                             |
  |  |  (d) Register alloc: VGPRs, SGPRs         |                             |
  |  |  (e) Emit: binary machine code            |                             |
  |  `---------------------+------------------------+                            |
  |                        v                                                   |
  |  OUTPUT: ELF Code Object (GPU binary)                                      |
  |  +-------------------------------------------+                             |
  |  | .text:     gfx1151 ISA instructions       |                             |
  |  | metadata:  VGPR=24, SGPR=16, LDS=0        |                             |
  |  | symbols:   "fused_kernel"                 |                             |
  |  | Ready for: hipModuleLoadData() -> launch  |                             |
  |  `-------------------------------------------+                             |
  `-----------------------------------------------------------------------------+
```

File: clr/hipamd/src/hiprtc/hiprtc.cpp
Line: 115
GitHub: https://github.com/ROCm/clr/blob/rocm-7.2.0/hipamd/src/hiprtc/hiprtc.cpp#L115

What it does in ONE sentence: Takes HIP C++ source code (as a text string)
and compiles it into GPU machine code (binary) for your specific GPU architecture.

```
LINE 115 | hiprtcResult hiprtcCompileProgram(hiprtcProgram prog, int numOptions,
         |                                   const char* const* options) {
         |
         |   hiprtcResult  -> success or a specific compilation error
         |   hiprtcProgram prog -> handle created by hiprtcCreateProgram(),
         |     contains HIP C++ source code as text
         |   int numOptions -> how many compiler flags
         |   const char* const* options -> array of flag strings
         |     Example: ["--offload-arch=gfx1151", "-O3"]
         |     --offload-arch=gfx1151 = YOUR GPU (RDNA 3.5 = Radeon 8060S)
         |
LINE 116 |   HIPRTC_INIT_API(prog, numOptions, options);
         |   [SKIP] Logging macro
         |
         |   ============================================
         |   STEP 1: UNWRAP THE PROGRAM HANDLE
         |   ============================================
         |
LINE 118 |   auto* rtcProgram = hiprtc::RTCCompileProgram::as_RTCCompileProgram(prog);
         |
         |   [KEY] Convert opaque handle to the real RTCCompileProgram object
         |
         |   ============================================
         |   STEP 2: PARSE COMPILER OPTIONS
         |   ============================================
         |
LINE 120 |   bool fgpu_rdc = false;
LINE 121 |   bool no_builtin_header = false;
LINE 122 |   std::vector<std::string> opt, compile_options;
         |
LINE 125 |   for (int i = 0; i < numOptions; i++) {
LINE 126 |     if (options[i] == "-fgpu-rdc") fgpu_rdc = true;
LINE 130 |     if (options[i] != "--hiprtc-no-builtin-header")
LINE 131 |       opt.push_back(options[i]);
LINE 133 |     else no_builtin_header = true;
LINE 135 |   }
         |
         |   [KEY] Scan options: extract special flags, pass rest through
         |
         |   ============================================
         |   STEP 3: ADD DEFAULT COMPILER FLAGS
         |   ============================================
```

```
         |
LINE 140 |   if (!no_builtin_header) {
LINE 141 |     compile_options.push_back("-D__HIPCC_RTC__");
LINE 142 |     compile_options.push_back("-nogpuinc");
LINE 143 |     compile_options.push_back("-include");
LINE 144 |     compile_options.push_back("hiprtc_runtime.h");
LINE 145 |   }
         |
         |   [KEY] Default flags: define __HIPCC_RTC__ macro, include runtime header
         |
LINE 147 |   compile_options.insert(end(compile_options), begin(opt), end(opt));
         |
         |   [KEY] Merge: [defaults...] + [user options...]
         |
         |   ============================================
         |   STEP 4: THE ACTUAL COMPILATION
         |   ============================================
         |
LINE 149 |   if (!rtcProgram->compile(compile_options, fgpu_rdc)) {
         |
         |   [KEY] *** THIS IS THE ACTUAL COMPILATION ***
         |
         |   rtcProgram->compile() triggers a deep chain:
         |
         |   1. Calls into COMGR (Code Object Manager)
         |      COMGR is AMD's compiler orchestration library
         |
         |   2. COMGR invokes the LLVM AMDGPU compiler backend
         |
         |   3. LLVM processing pipeline:
         |      a. PARSING: HIP C++ source -> LLVM IR (intermediate representation)
         |      b. OPTIMIZATION: constant folding, loop unrolling, vectorization
         |      c. ISA SELECTION: choose AMDGPU instructions for gfx1151
         |      d. REGISTER ALLOCATION: decide which GPU registers to use
         |      e. CODE EMISSION: produce binary machine code
         |
         |   4. Output: an ELF binary containing:
         |      - AMDGPU machine code (ISA) for gfx1151
         |      - Metadata (register count, shared memory usage)
         |      - Symbol names (kernel function names for loading)
         |
LINE 150 |     HIPRTC_RETURN(HIPRTC_ERROR_COMPILATION);  // compilation failed
LINE 151 |   }
         |
LINE 153 |   HIPRTC_RETURN(HIPRTC_SUCCESS);   // compiled binary stored in rtcProgram
LINE 154 | }                                   // retrieve with hiprtcGetCode()

FULL CALL CHAIN:
  hiprtcCompileProgram (hiprtc.cpp:115) -- public API
    `---- rtcProgram->compile() -- internal compilation
          `---- COMGR (Code Object Manager) -- compiler orchestration
                `---- LLVM AMDGPU Backend -- actual compiler
                      |---- Front-end: HIP C++ -> LLVM IR
                      |---- Optimizer: constant folding, vectorization, etc.
                      |---- Code generator: LLVM IR -> AMDGPU ISA for gfx1151
                      `---- Output: ELF binary with GPU machine code


D.8 target::get_passes() -- The Complete 50+ Pass List
-------------------------------------------------------

  +-- DIAGRAM: 50+ Pass Pipeline -- Grouped by Phase ------------------------+
  |                                                                          |
  |  INPUT: raw MIGraphX IR (after parse + quantize)                        |
  |       |                                                                  |
  |       v  GROUP 1: CLEANUP (lines 94-210)                                |
  |  +-------------------------------------------------------------------+  |
  |  | split_single_dyn_dim -> dead_code_elimination -> simplify_dyn_ops |  |
  |  | simplify_qdq -> simplify_reshapes -> eliminate_common_subexpr     |  |
  |  | propagate_constant -> dead_code_elimination (again)               |  |
  |  | convert_type (BF16/FP8 -> FP32 if unsupported)                   |  |
  |  `-----------------------------------+-------------------------------+  |
  |                                      v  GROUP 2: FUSION (lines 211-240)  |
  |  +-------------------------------------------------------------------+  |
  |  | fuse_attention -> fuse_pointwise -> fuse_reduce -> fuse_concat    |  |
  |  | fuse_mlir (GEMM + pointwise via MLIR JIT)                        |  |
  |  | fuse_ck (Composable Kernel library patterns)                     |  |
  |  | dead_code_elimination (after each fusion)                        |  |
  |  `-----------------------------------+-------------------------------+  |
  |                                      v  GROUP 3: LIBRARY COMPILE (241-260) |
  |  +-------------------------------------------------------------------+  |
  |  | compile_miopen (Conv/Pool -> MIOpen library kernels)              |  |
  |  | compile_hipblaslt (GEMM -> hipBLASLt tuned kernels)              |  |
```

```
  |  | compile_ops (remaining -> HIP C++ -> hipRTC -> COMGR -> ELF)       |  |
  |  `--------------------------------+-----------------------------------+  |
  |                                   v  GROUP 4: SCHEDULE + MEMORY (261+)  |
  |  +-----------------------------------------------------------------+  |
  |  | memory_coloring (buffer reuse: 7 temporaries -> 3 buffers)       |  |
  |  | schedule (multi-stream: parallel kernels on separate queues)     |  |
  |  | dead_code_elimination (final cleanup)                            |  |
  |  `--------------------------------+-----------------------------------+  |
  |                                   v                                       |
  |  OUTPUT: fully compiled, scheduled GPU program                           |
  `--------------------------------------------------------------------------+
```

File: AMDMIGraphX/src/targets/gpu/target.cpp
Lines: 94-277
GitHub: https://github.com/ROCm/AMDMIGraphX/blob/rocm-7.2.0/src/targets/gpu/target.cpp#L94

This function returns the ordered list of ALL compilation passes that
program::compile() (D.5) runs. Shown here in 4 groups as they appear in source.

GROUP 1: Cleanup and Normalization (target.cpp lines 94-210)

```
  std::vector<pass> target::get_passes(migraphx::context& gctx,
                                       const compile_options& options) const
  {
      auto& ctx = any_cast<context>(gctx);
      ctx.set_exhaustive_tune_flag(options.exhaustive_tune);
      ctx.load_problem_cache();
      // Build unsupported_types set -- types needing conversion to float
      // BF16 support conditional on gfx_has_bf16_intrinsics()
      // FP8 support conditional on gfx_has_fp8fnuz/ocp_intrinsics()
      return {
          split_single_dyn_dim{},    // handle dynamic dimensions
          dead_code_elimination{},
          simplify_dyn_ops{},        // simplify dynamic shape operations
          dead_code_elimination{},
          normalize_ops{},           // standardize op representations
          dead_code_elimination{},
          eliminate_identity{},      // remove pass-through operations
          dead_code_elimination{},
```

GROUP 2: Fusion Passes (target.cpp lines 228-240)

```
          fuse_attention{...},       // Transformer Multi-Head Attention fusion
          dead_code_elimination{},   //   Q*K^T + scale + mask + softmax + V -> 1 kernel
          optimize_module{},
          fuse_pointwise_reduce{},   // fuse element-wise + reduction ops
          dead_code_elimination{},
          fuse_ck{},                 // Composable Kernel library (AMD hand-tuned GEMM/Conv)
          dead_code_elimination{},
          fuse_mlir{&ctx},           // MLIR-based GEMM fusion for unsupported shapes
          dead_code_elimination{},
          fuse_concat{},             // concatenation optimization
          dead_code_elimination{},
```

GROUP 3: Lowering and Library Compilation (target.cpp lines 241-263)

```
          auto_contiguous{},         // insert contiguous operations where needed
          dead_code_elimination{},
          lowering{&ctx, options.offload_copy},  // abstract ops -> GPU ops (MatMul -> gpu::gemm)
          eliminate_contiguous{"gpu::contiguous"},
          dead_code_elimination{},
          eliminate_concat{concat_gpu_optimization{}},
          dead_code_elimination{},
          compile_miopen{&gctx},     // convolution via MIOpen (AMD's cuDNN equivalent)
          dead_code_elimination{},   //   *** CROSSES TO LAYER 5 (MIOpen calls HIP) ***
          fuse_ops{&ctx, options.fast_math},
          dead_code_elimination{},
          compile_hipblaslt{&gctx},  // GEMM with fused epilogues (bias + activation)
          dead_code_elimination{},   //   *** CROSSES TO LAYER 5 (hipBLASLt calls HIP) ***
          replace_allocate{gpu_allocation_model{}, options.offload_copy},
          dead_code_elimination{},
          adjust_allocation{gpu_allocation_model{}},
          dead_code_elimination{},
          compile_ops{&ctx, ...},    // custom HIP kernels via hipRTC
          dead_code_elimination{},   //   *** CROSSES TO LAYER 5 (hiprtcCompileProgram) ***
```

GROUP 4: Scheduling and Memory (target.cpp lines 264-277)

```
          promote_literals{},
          dead_code_elimination{},
          write_literals{&ctx},      // copy constant data to GPU
          schedule{gpu::schedule_model{ctx.get_current_device().nstreams()}, ...},
                                     // multi-stream parallel scheduling
```

```
        memory_coloring{"hip::allocate"},  // buffer reuse (50-80% memory savings)
        sync_device{},             // insert synchronization barriers
        preallocate_param{"scratch", gpu_allocation_model{}},
        dead_code_elimination{},
        eliminate_allocation{"hip::allocate"},
        check_context<context>{},  // final sanity check
        normalize_ops{},
        dead_code_elimination{},
        eliminate_identity{}        // final cleanup
    };
}
```

D.9 Complete Numbered Pass Table (50+ passes from target.cpp)
---

```
+-- DIAGRAM: Pass Pipeline -- Numbered Execution Order ---------------------+
|                                                                           |
|  IR Graph In --> [1-12 CLEANUP] --> [13-24 NORMALIZE] -->            |
|                                                                           |
|  --> [25-31 FUSION ***] --> [32-40 COMPILE ***] -->               |
|                                                                           |
|  --> [41-48 SCHEDULE+MEM ***] --> Compiled GPU Program Out           |
|                                                                           |
|  *** = highest impact passes:                                          |
|  #25 fuse_attention   (MHA -> 1 kernel, huge for Transformers)         |
|  #29 fuse_mlir        (GEMM+pointwise -> MLIR JIT kernel)              |
|  #35 compile_miopen   (Conv -> MIOpen Winograd/FFT/GEMM)               |
|  #37 compile_hipblaslt (GEMM -> hipBLASLt tuned kernel)                |
|  #40 compile_ops      (remaining -> hipRTC JIT -> ELF binary)          |
|  #43 schedule         (multi-stream parallel dispatch)                 |
|  #44 memory_coloring  (buffer reuse: 50-80% memory saved)             |
`---------------------------------------------------------------------------+
```

Source: AMDMIGraphX/src/targets/gpu/target.cpp lines 94-277
GitHub: https://github.com/ROCm/AMDMIGraphX/blob/rocm-7.2.0/src/targets/gpu/target.cpp#L181

| # | Pass Name | What It Does | Why It Matters |
|---|---|---|---|
| 1 | split_single_dyn_dim | Handle models with dynamic (variable) batch sizes | Flexibility for different input sizes |
| 2 | dead_code_elimination | Remove ops whose outputs are never used | Less work, less memory |
| 3 | simplify_dyn_ops | Simplify operations with dynamic shapes | Cleaner graph |
| 4 | normalize_ops | Standardize operation representations | Consistent patterns for later passes |
| 5 | eliminate_identity | Remove "do nothing" identity operations | Cleanup |
| 6 | simplify_qdq | Simplify quantize/dequantize patterns | Cleaner quantization |
| 7 | rewrite_quantization | Rewrite quantization patterns for GPU | GPU-friendly quantization |
| 8 | rewrite_rnn | Rewrite recurrent neural network ops | Better RNN performance |
| 9 | eliminate_data_type | Convert unsupported types to supported ones | gfx1151 supports float, half, int8, etc. |
| 10 | simplify_reshapes | Merge/eliminate unnecessary reshapes | Less metadata overhead |
| 11 | eliminate_pad | Remove unnecessary padding operations | Less memory waste |
| 12 | insert_pad | Add padding where convolution needs it | Required for correct conv |
| 13 | inline_module | Inline small submodules into main module | More optimization opportunities |
| 14 | rewrite_pooling | Rewrite pooling for GPU efficiency | Better pooling performance |
| 15 | rewrite_gelu | Rewrite GELU activation for fast math | Faster activation |
| 16 | optimize_module | Constant folding + algebraic simplification | Pre-compute constants |
| 17 | layout_convolution | Choose memory layout (NCHW vs NHWC) | Optimal layout = faster memory access |
| 18 | prefuse_ops | Prepare operations for later fusion passes | Sets up fusion patterns |
| 19 | rewrite_reduce | Rewrite reduction operations | Better GPU reduction |
| 20 | rewrite_topk | Rewrite top-k operations | Better sorting/selection |
| 21 | rewrite_low_precision | Rewrite ops for low-precision (FP8/BF16) paths | Correct handling of reduced-precision ops |
| 22 | rewrite_dot [conditional] | Rewrite dot/GEMM operations for GPU | Better GEMM patterns |
| 23 | propagate_precision | Propagate data types through the graph | Correct types everywhere |
| 24 | fp8_ocp_to_fnuz [cond.] | Convert FP8 OCP format to FNUZ if needed | Hardware-specific FP8 variant |
| 25 | fuse_attention *** | Fuse multi-head attention into one kernel<br>Q*K^T + scale + mask + softmax + V -> 1 kernel | HUGE gain for Transformers (LLaMA, BERT)<br>Keeps data in fast on-chip memory |
| 26 | optimize_module (again) | Clean up after attention fusion | Simplification round |
| 27 | fuse_pointwise_reduce | Fuse element-wise + reduction ops | Fewer kernel launches |
| 28 | fuse_ck *** [conditional] | Use Composable Kernel (CK) library | Pre-written optimized GEMM/Conv kernels |
| 29 | fuse_mlir *** [conditional] | Fuse GEMM+pointwise using MLIR compiler | Generates specialized fused kernels |
| 30 | fuse_concat | Fuse concatenation operations | Fewer memory copies |
| 31 | auto_contiguous | Ensure contiguous memory layouts | Required for many GPU ops |
| 32 | lowering *** | Convert abstract ops -> GPU ops (MatMul -> gpu::gemm) | Specific algorithm + tile sizes |
| 33 | eliminate_contiguous | Remove unnecessary contiguous ops from lowering | Cleanup |
| 34 | eliminate_concat | Optimize concat with GPU-specific implementation | Faster concat |
| 35 | compile_miopen *** [cond.] | Use MIOpen for convolution/pooling (if built with) | AMD's cuDNN (Winograd, FFT, implicit GEMM) |
| 36 | fuse_ops | General fusion for remaining patterns | More kernel fusion |
| 37 | compile_hipblaslt *** [c.] | Use hipBLASLt for GEMM with fused epilogues | MatMul + Bias + Activation -> 1 kernel |
| 38 | replace_allocate | Replace abstract allocs with GPU-specific ones | Prepare for actual GPU memory |
| 39 | adjust_allocation | Adjust sizes for alignment requirements | GPU memory must be aligned |
| 40 | compile_ops *** | Generate HIP C++ source, compile via hipRTC<br>*** CROSSES TO LAYER 5 *** | Custom kernels for remaining ops<br>(hiprtcCompileProgram call) |
| 41 | promote_literals | Move small constants into efficient storage | Better memory access |
| 42 | write_literals | Write constant values to GPU memory | Constants ready for use |

```
 43 schedule ***                 Assign operations to GPU streams                     Independent ops overlap in parallel
 44 memory_coloring ***          Plan buffer reuse (non-overlapping lifetimes share) 50-80% memory savings
 45 sync_device                  Insert synchronization points between streams       Correct ordering where needed
 46 preallocate_param            Pre-allocate scratch memory                          Temporary workspace
 47 eliminate_allocation         Remove allocation ops no longer needed               Final cleanup
 48 check_context                Verify GPU context is valid                          Safety check
 49 normalize_ops (final)        Final normalization                                  Clean graph
 50 eliminate_identity (final)   Final identity removal                               Clean graph

Note: *** marks performance-critical passes. [conditional] / [cond.] / [c.] means
the pass is gated by #if or env vars and may not run in all builds. dead_code_elimination
runs between nearly every pass (omitted from numbering for readability).


D.10 Key Source File Reference Table
------------------------------------

  +-- DIAGRAM: Source File Map by Layer -------------------------------------+
  |                                                                           |
  |  LAYER   REPO                     KEY FILES                              |
  |  -----   --------------------     ------------------------------         |
  |  L2      microsoft/onnxruntime    inference_session.cc                   |
  |          (ORT Core)               graph_partitioner.cc                   |
  |                                   sequential_executor.cc                 |
  |          +-------------------------------------------+                   |
  |  L3      | onnxruntime/core/providers/migraphx/    |                    |
  |          | migraphx_execution_provider.cc           |                    |
  |          | migraphx_allocator.cc                    |                    |
  |          `-------------------------------------------+                   |
  |  L4      ROCm/AMDMIGraphX        api.cpp (C API entry points)           |
  |          (libmigraphx.so)        onnx.cpp (ONNX parser)                 |
  |                                   quantization.cpp (FP16 pipeline)       |
  |                                   program.cpp (compile + eval)           |
  |                                   target.cpp (50+ pass list)             |
  |                                   compile_hip.cpp (JIT via hipRTC)       |
  |                                                                           |
  |  L5      ROCm/clr                hip_memory.cpp (hipMalloc)              |
  |          (libamdhip64.so)        hip_module.cpp (hipLaunchKernel)        |
  |                                   hiprtc.cpp (runtime compiler)          |
  |                                                                           |
  |  L6      ROCm/ROCR-Runtime       hsa_api.cpp (HSA entry points)          |
  |          (libhsa-runtime64.so)   amd_gpu_agent.cpp (queue mgmt)          |
  |                                                                           |
  |  L7      linux/drivers/gpu/drm/amdgpu/                                   |
  |          (amdgpu.ko)             amdgpu_gem.c (BO allocation)            |
  |                                   amdgpu_cs.c (command submit)           |
  `---------------------------------------------------------------------------+

LAYER 2 -- ONNX Runtime (C++ engine)
  Repository: https://github.com/microsoft/onnxruntime
  File                                            Line    Purpose
  ----------------------------------------------- ------- -------------------------------
  onnxruntime/core/session/inference_session.cc     varies  InferenceSession constructor + Run()
  onnxruntime/core/framework/graph_partitioner.cc   varies  Graph partitioning to EPs
  onnxruntime/core/framework/sequential_executor.cc varies  Sequential node execution

LAYER 3 -- MIGraphX Execution Provider
  Repository: https://github.com/microsoft/onnxruntime (contrib EP)
  File                                            Line    Purpose
  ----------------------------------------------- ------- -------------------------------
  onnxruntime/core/providers/migraphx/              varies  GetCapability, Compile, compute_func
    migraphx_execution_provider.cc
  onnxruntime/core/providers/migraphx/              varies  GPU memory allocator (Fixes B + D):
    migraphx_allocator.cc                                   hipMallocManaged + CoarseGrain

LAYER 4 -- MIGraphX Library (libmigraphx.so)
  Repository: https://github.com/ROCm/AMDMIGraphX (branch: rocm-7.2.0)
  File                                            Line    Purpose
  ----------------------------------------------- ------- -------------------------------
  src/api/api.cpp                                   2109    migraphx_parse_onnx_buffer (C API)
  src/api/api.cpp                                   2270    migraphx_quantize_fp16 (C API)
  src/api/api.cpp                                   1714    migraphx_program_compile (C API)
  src/api/api.cpp                                   1774    migraphx_program_run (C API)
  src/onnx/onnx.cpp                                 107     parse_onnx_buffer (internal parser)
  src/quantization.cpp                              68      quantize_fp16 (4-pass FP16 pipeline)
  src/program.cpp                                   313     program::compile (compilation driver)
  src/program.cpp                                   588     program::eval (inference hot path)
  src/targets/gpu/target.cpp                        94      target::get_passes (50+ pass list)
  src/targets/gpu/compile_hip.cpp                   202     compile_hip_src_with_hiprtc
  src/targets/gpu/compile_hip.cpp                   229     compile_hip_src (main entry)
  src/targets/gpu/compile_hip.cpp                   161     hiprtc_program::compile (calls hipRTC)
  src/targets/gpu/compile_miopen.cpp                varies  MIOpen integration
  src/targets/gpu/compile_hipblaslt.cpp             varies  hipBLASLt integration
```

```
LAYER 5 -- HIP Runtime (libamdhip64.so)
  Repository: https://github.com/ROCm/clr (branch: rocm-7.2.0)
  File                                          Line    Purpose
  --------------------------------------------- ------- ------------------------------
  hipamd/src/hip_memory.cpp                     778     hipMalloc (public API)
  hipamd/src/hip_memory.cpp                     338     ihipMalloc (internal allocator)
  hipamd/src/hip_memory.cpp                     817     hipMemcpy (data transfer)
  hipamd/src/hip_module.cpp                     534     hipModuleLaunchKernel (kernel dispatch)
  hipamd/src/hip_module.cpp                     443     ihipModuleLaunchKernel (internal)
  hipamd/src/hip_module.cpp                     352     ihipLaunchKernelCommand (build cmd)
  hipamd/src/hiprtc/hiprtc.cpp                  115     hiprtcCompileProgram (runtime compile)
```

## D.11 Profiling and Observability Tools

```
  +-- DIAGRAM: Profiling Tool Stack -- Which Tool Answers Which Question -----+
  |                                                                          |
  |  QUESTION                          TOOL & MODE                          |
  |  -------------------------         ------------------------------       |
  |  "Which kernel is slowest?"        rocprofv3 --kernel-trace             |
  |  "What HIP calls were made?"       rocprofv3 --hip-trace               |
  |  "How much data was copied?"       rocprofv3 --memory-copy-trace       |
  |  "Full timeline in Perfetto?"      rocprofv3 --sys-trace -f pftrace    |
  |  "Per-kernel min/max/avg?"         rocprofv3 --stats                   |
  |  "CPU+GPU overlap timeline?"       rocprof-sys (omnitrace)             |
  |  "GPU temp/clock/memory now?"      rocm-smi / watch rocm-smi          |
  |  "What ISA was compiled?"          roc-obj-ls / roc-obj-extract        |
  |  "MIGraphX pass timings?"          MIGRAPHX_TRACE_COMPILE=1           |
  |  "Function-level C++ tracing?"     uftrace (integrated in benchmark)  |
  |                                                                          |
  |  Output formats:                                                         |
  |  +--------------+  +--------------+  +--------------------+              |
  |  | .db (SQLite)|  | .csv       |  | .pftrace         |               |
  |  | queryable   |  | spreadsheet|  | Perfetto UI      |               |
  |  | sql queries |  | compatible |  | interactive      |               |
  |  `---------------+  `---------------+  `---------------------+               |
  `---------------------------------------------------------------------------+
```

When diagnosing "is it memory or compute?" or "where is the bottleneck?", use these tools from the ROCm ecosystem:

```
TOOL              WHAT IT DOES                                      WHEN TO USE
----              ------------                                      -----------
rocprofv3         Next-gen GPU profiler (replaces rocprof v1/v2)   "What happened during inference?"
(rocprofiler-     Provides 6 trace modes and stats collection.     "Where is time spent?"
 sdk)             Default output: SQLite3 .db files (queryable).   "Is XNACK causing overhead?"
                  Also supports Perfetto .pftrace for timelines.

                  Trace modes (used in this project):
                    --kernel-trace        GPU kernel dispatch times, names, queues
                    --hsa-trace           HSA Runtime API calls (agent, queue, signal ops)
                    --memory-copy-trace   hipMemcpy / DMA copy events with sizes + direction
                    --hip-trace           HIP API calls (hipMalloc, hipLaunchKernel, etc.)
                    --sys-trace           System-wide Perfetto trace (combines all above)
                    --stats               Per-kernel aggregate statistics (min/max/avg/total)

                  Example commands:
                    rocprofv3 --kernel-trace -o output_dir -- python3 run_model.py
                    rocprofv3 --hip-trace -o output_dir -- python3 run_model.py
                    rocprofv3 --sys-trace -o output_dir -- python3 run_model.py

                  Output format (ROCm 7.2+):
                    Each mode produces a .db (SQLite3) file in the output directory.
                    Query with: sqlite3 output_dir/*.db "SELECT * FROM rocprofiler_kernel_trace;"
                    System trace also produces a .pftrace file for Perfetto UI
                    (open at ui.perfetto.dev for interactive timeline visualization).

                  Notes for gfx1151:
                    Hardware PMC counter collection (--pmc) is limited -- many counters
                    (TCP, UTCL1, UTCL2, TCC, SQ) are not yet defined for RDNA 3.5 in
                    the rocprofiler-sdk counter definitions. Use trace modes instead.

rocprof (v1)      Legacy hardware counter profiling                  "Which kernel is slowest?"
                  Records GPU kernel execution times, occupancy,   "Is my kernel memory-bound
                  memory throughput, cache hit rates.                or compute-bound?"
                  Usage: rocprof --stats ./my_program
                         rocprof -i counters.txt ./my_program
                  NOTE: Being superseded by rocprofv3 in ROCm 7.x+

rocprof-sys       System-wide tracing (formerly Omnitrace)         "Where does time go across
(omnitrace)       Captures CPU+GPU timeline, API calls, kernel     the full stack?"
```

```
                    launches, memory transfers, and thread activity   "Is the CPU starving the GPU?"
                    in a single timeline view (Chrome tracing format)
                    Usage: rocprof-sys-instrument -- ./my_program
                           rocprof-sys-run -- ./my_program

rocm-smi            GPU status monitor                               "Is the GPU busy right now?"
                    Shows temperature, clock speeds, memory usage,    "How much VRAM is used?"
                    power draw, and utilization percentage.
                    Usage: rocm-smi (snapshot)
                           watch -n 1 rocm-smi (live monitoring)

roc-obj-ls /        Inspect compiled GPU binaries                     "What ISA did my kernel
roc-obj-extract     List kernels inside a code object, extract ISA    compile to?"
                    assembly, check target architecture matches

For MIGraphX specifically:
  MIGRAPHX_TRACE_COMPILE=1  -> prints each compilation pass and its duration
  MIGRAPHX_TRACE_EVAL=1     -> prints each kernel launch during inference
  MIGRAPHX_GPU_DEBUG=1      -> extra validation during GPU operations

Profiling Data Collected for This Project:
  Full rocprofv3 traces were captured for two models under both XNACK=1 and
  XNACK=0 settings on the Radeon 8060S (gfx1151) under Xen PVH dom0:

  1. DeepSeek-R1 1.5B (6.7 GB FP16) -- fits within GTT without oversubscription
     Traces: kernel, stats, hip-trace, timeline (Perfetto), xnack0/xnack1
     Output: CSV format (rocprofv3 older output mode)
     Scripts: profiling/profile_deepseek.py, profiling/analyze_xnack.py

  2. OpenVLA-7B Full (28 GB FP32) -- oversubscribes available GPU memory
     Traces: kernel, hsa, memory-copy, hip, sys-trace, stats (both XNACK settings)
     Output: SQLite3 .db format (rocprofv3 default in ROCm 7.2)
     Script: profiling/profile_openvla.py

  Key finding: ORT + MIGraphX uses explicit hipMalloc/hipMemcpy for the vast
  majority of memory operations. Only 3 hipMallocManaged calls were observed
  (from the custom migraphx_allocator.cc). HMM/SVM demand-paging is not
  heavily exercised by the current ORT MIGraphX EP code path.

  All traces are archived at: profiling/ (local) and
  \\park-corp\ATG-Public\SST-Team\profiling\ (network share).
  See profiling/README.txt for methodology, file layout, and SQL query examples.


D.12 Precision Formats Beyond FP16 -- BF16, FP8, INT4/INT8
-----------------------------------------------------------

FP16 (half) is not the only reduced-precision format. Modern ML inference
uses several formats, each with different trade-offs:

FORMAT    BITS  RANGE            PRECISION        GPU SUPPORT (gfx1151)
------    ----  -----            ---------        ---------------------
FP32      32    +-3.4e38         ~7 digits        Full (baseline)
FP16      16    +-65504          ~3.3 digits      Full (WMMA accelerated)
BF16      16    +-3.4e38         ~3.4 digits      Conditional (gfx_has_bf16_intrinsics)
FP8 E4M3  8     +-448            ~1.5 digits      Conditional (gfx_has_fp8ocp_intrinsics)
FP8 E5M2  8     +-57344          ~1.0 digits      Conditional (gfx_has_fp8fnuz_intrinsics)
INT8       8    -128 to 127      Exact integers   Supported (quantized inference)
INT4       4    -8 to 7          Exact integers   Software-emulated (packing)

BF16 (Brain Floating Point 16):
  Same range as FP32 (8-bit exponent) but only ~3 decimal digits of precision.
  Preferred over FP16 for training because it avoids overflow on large gradients.
  MIGraphX supports BF16 via migraphx_quantize_bf16() -- same 4-pass pattern
  as FP16 but uses shape::bf16_type instead of shape::half_type.
  target.cpp conditionally enables BF16 based on hardware capability:
    if(gpu::gfx_has_bf16_intrinsics()) { unsupported_types.erase(bf16_type); }

INT4/INT8 Quantization:
  Weights stored as 4-bit or 8-bit integers with a scale factor per group.
  Dramatically reduces memory: a 7B parameter model goes from:
    FP32: 28 GB -> FP16: 14 GB -> INT8: 7 GB -> INT4: 3.5 GB
  On your 8060S with 512 MB VRAM carve-out:
    - FP16 7B model: DOES NOT FIT (14 GB >> 512 MB)
    - INT4 7B model: still too large (3.5 GB >> 512 MB)
    - INT4 1.5B model: ~750 MB -- marginal, uses system RAM spill
    - INT4 500M model: ~250 MB -- FITS in VRAM carve-out, good performance
  MIGraphX supports INT8 natively. INT4 support is evolving.

FP8 (8-bit Floating Point):
  Two flavors: E4M3 (more precision, less range) and E5M2 (more range, less precision).
  Primarily used on MI300X (CDNA3) for training. Your gfx1151 may have partial support.
  MIGraphX's target.cpp checks for FP8 support at compile time:
```

```
    if(gpu::gfx_has_fp8ocp_intrinsics()) { /* enable FP8 OCP path */ }
    if(gpu::gfx_has_fp8fnuz_intrinsics()) { /* enable FP8 FNUZ path */ }


+-----------------------------------------------------------------------------+
|  FINAL NOTE FOR BEGINNERS: What You Just Learned                            |
|                                                                             |
|  You now understand what happens when you write these two statements:       |
|                                                                             |
|    session = ort.InferenceSession("model.onnx",                             |
|        providers=["MIGraphXExecutionProvider"])                             |
|    output = session.run(None, {"input": data})                              |
|                                                                             |
|  Behind those 2 statements of Python:                                       |
|    - A protobuf file is deserialized into a C++ graph (Chapter 1)           |
|    - The graph is optimized and split for GPU/CPU (Chapter 1)               |
|    - The EP delegates to MIGraphX (Ch 2 calls -> Ch 3 does the work)        |
|    - 50+ optimization passes fuse operations together (Chapter 3)           |
|    - HIP allocates GPU memory and launches kernels (Chapter 4)              |
|    - HSA writes AQL packets to hardware queues (Chapter 5)                  |
|    - The kernel driver manages physical memory (Chapter 6)                  |
|    - Transistors fire across 40 Compute Units (Chapter 7)                   |
|    - ~110 software API calls happen in ~40 microseconds                     |
|    - The GPU does actual math in ~0.5-5 milliseconds                        |
|    - A numpy array appears in your Python variable                          |
|                                                                             |
|  All 8 layers work together seamlessly. You never see any of this           |
|  complexity -- and that is the beauty of well-designed software.            |
|                                                                             |
|  Key takeaways for your Radeon 8060S:                                       |
|    - Memory bandwidth (~256 GB/s LPDDR5x) is your bottleneck, not compute |
|    - Use quantized models (INT4/INT8) for best performance                  |
|    - Small models (< 4 GB) run surprisingly well on your APU                |
|    - The Infinity Cache (32 MB MALL, ~1 TB/s) is your best friend           |
`-----------------------------------------------------------------------------+


================================================================================
  APPENDIX E: Deployment Engineering -- Lessons, Infrastructure, Workarounds
================================================================================

This appendix consolidates practical deployment knowledge gained from shipping
23 ONNX model components across 20 base models on the AMD Strix Halo APU
(Radeon 8060S, gfx1151) under a Xen hypervisor with 29 GB RAM.

NOTE: The custom ORT 1.25.0 build includes four source-level fixes documented
in detail in docs/R1_R2_OFFICIAL_REPORT.txt Section 5 as Fix A through Fix D:
  Fix A: Double allocation prevention (upstream PR #27551)
  Fix B: hipMallocManaged allocator (custom, APU-specific)
  Fix C: C++20 build compatibility ([=] -> [=, this])
  Fix D: Coarse-grained coherence (hipMemAdviseSetCoarseGrain)

E.1 Ten Engineering Lessons
----------------------------

  1. APU != Discrete GPU (Fixes B + D)
     512 MB VRAM + 28 GB GTT means unified memory (HSA_XNACK=1 +
     hipMallocManaged) is mandatory for any model > 500 MB. The standard
     hipMalloc path only sees the 512 MB carve-out.
     Additionally, hipMallocManaged() defaults to fine-grained coherence
     which bypasses the GPU Infinity Cache. Fix D adds
     hipMemAdviseSetCoarseGrain after every allocation so that model
     weights (read-many, write-once) benefit from the 32 MB MALL at
     ~1 TB/s instead of falling through to LPDDR5x at ~256 GB/s.

  2. Split Large VLMs
     Monolithic 10B+ parameter ONNX graphs hit C++ template depth limits
     during MIGraphX compilation (stack overflow in hipRTC). The
     encoder/decoder split pattern (used for Whisper, LLaMA-3.2-11B)
     compiles each half independently and runs them sequentially.

  3. onnxsim Is Essential
     MIGraphX's ONNX parser has known issues with certain reshape patterns
     (e.g., MobileSAM). Running python -m onnxsim model.onnx model_sim.onnx
     before deployment resolves most parse failures by simplifying the graph.

  4. BFloat16 Is Not Portable
     Many PyTorch models export with bfloat16 weights. ORT and MIGraphX on
     RDNA 3.5 (gfx1151) do not support bfloat16 inference. Convert to
     float16 before deployment:
       from onnxconverter_common import float16
       model_fp16 = float16.convert_float_to_float16(onnx.load("model.onnx"))
     CogACT-7B required this conversion to run on MIGraphX.
```

```
5. Verify EP Activation
   ORT silently falls back to CPUExecutionProvider if the GPU EP fails
   to load. ALWAYS check after session creation:
     providers = session.get_providers()
     assert providers[0] == 'MIGraphXExecutionProvider', \
         f"Expected MIGraphX EP, got {providers[0]}"

6. Subprocess Isolation for Large Models
   Running each model in its own Python subprocess prevents:
     * GPU memory fragmentation across models
     * GPU state corruption from one model affecting another
     * Swap pressure from accumulated allocations
   Pattern: multiprocessing.Process per model, join() after each.

7. MIGraphX Compiler Tuning
   Large graphs (LLaMA encoder, OpenVLA, CogACT) require all of:
     export MIGRAPHX_GPU_HIP_FLAGS="-ftemplate-depth=2048
                                    -Wno-error=stack-exhausted"
     export MIGRAPHX_GPU_COMPILE_PARALLEL=1
     ulimit -s unlimited
   Without these, compilation either crashes or exhausts memory.

8. No Shortcuts (LD_PRELOAD Rejection)
   An LD_PRELOAD shim (hip_alloc_redirect.cpp) was built to intercept
   hipMalloc->hipMallocManaged at the dynamic linker level. It worked
   in testing but was explicitly rejected as a production approach:
     * Fragile: intercepts ALL hipMalloc calls, not just ORT's
     * Non-portable: breaks if library load order changes
     * Unmaintainable: invisible to developers, no stack trace
   The proper fix (source-level allocator change, Fix B) is transparent
   and auditable.

9. Xen Is Strictly Harder Than Baremetal
   If all 23 models pass under Xen (29 GB RAM, hypervisor overhead,
   PCI virtualization), they will run on baremetal with identical env
   variables. Expected baremetal improvement: 10-20% faster compilation
   (more RAM -> less swap thrashing), equal or better inference latency.

10. Double Allocation Is Silent and Deadly (Fix A)
    When ORT partitions a graph for MIGraphX EP, it includes constant
    initializers (model weights) as subgraph inputs. ORT allocates GPU
    memory for those weights -- but MIGraphX ALSO loads them internally
    via parse_onnx_buffer(). Every weight allocated twice. Fix A (upstream
    PR #27551) skips constant initializers when building subgraph inputs,
    halving GPU memory usage for weight-heavy models like OpenVLA-7B.


+-- DIAGRAM: Deployment Decision Tree --------------------------------------+
|                                                                           |
|  Model ready for deployment?                                             |
|       |                                                                   |
|       v                                                                   |
|  +-- Check model size (FP16) --+                                          |
|  |                             |                                          |
|  |--- < 512 MB --------> hipMalloc OK, VRAM fits                         |
|  |                                                                        |
|  |--- 512 MB - 28 GB --> Need hipMallocManaged + HSA_XNACK=1             |
|  |                       Need no_system_mem_limit=Y                      |
|  |                       Check: swap space >= model size                 |
|  |                                                                        |
|  `--- > 28 GB ----------> Must split model (encoder/decoder)              |
|                           Or quantize to INT4/INT8 first                 |
|       |                                                                   |
|       v                                                                   |
|  +-- Check for bfloat16 weights --+                                       |
|  |  Yes -> convert to float16     |                                       |
|  |  No  -> proceed                |                                       |
|  `-----------------+------------------+                                       |
|                  v                                                        |
|  +-- Run onnxsim on model --+                                             |
|  |  Simplifies graph for   |                                             |
|  |  MIGraphX parser        |                                             |
|  `-----------------+------------+                                           |
|                  v                                                        |
|  +-- Set environment ----------------------------------+                  |
|  |  HSA_XNACK=1                                       |                  |
|  |  no_system_mem_limit=Y                              |                  |
|  |  MIGRAPHX_GPU_HIP_FLAGS="-ftemplate-depth=2048 ..."|                  |
|  |  MIGRAPHX_GPU_COMPILE_PARALLEL=1                    |                  |
|  |  ulimit -s unlimited                                |                  |
|  `-----------------+---------------------------------------+                  |
|                  v                                                        |
```

```
  |  +-- Run inference, verify EP active --+                                |
  |  |  assert providers[0] ==            |                                |
  |  |    'MIGraphXExecutionProvider'      |                                |
  |  `-------------------------------------+                                |
  `-------------------------------------------------------------------------+
```

E.2 Infrastructure Engineering
---

```
  Swap Engineering
  +-----------------------------------------------------------------------+
  |  Problem: Xen VM has 29.34 GB RAM. MIGraphX kernel compiler for   |
  |  large graphs (LLaMA encoder, OpenVLA) requires 40-60+ GB during  |
  |  template-heavy hipRTC compilation.                                |
  |                                                                    |
  |  Solution: ~400 GB total swap                                      |
  |    /swapfile       ~8 GB (original)                                |
  |    /swapfile200g   ~200 GB (activated from existing)               |
  |    /swapfile2      ~162 GB (created fresh)                         |
  |                                                                    |
  |  Peak usage: LLaMA encoder compilation -> ~44 GB swap usage        |
  |              during 55-minute compilation                          |
  |                                                                    |
  |  WARNING: Do NOT swapoff a swapfile while a process holds swap     |
  |  pages -- the process will hang indefinitely. Leave all active.    |
  `-----------------------------------------------------------------------+

  MIGraphX Compilation Cache
  +-----------------------------------------------------------------------+
  |  Location: /tmp/migraphx/                                          |
  |  Contents: compiled GPU kernels (ELF code objects) per model       |
  |  Size:     1-11 GB per model (larger for complex graphs)           |
  |  Behavior: first compilation writes cache, subsequent runs reuse   |
  |  Clear:    rm -rf /tmp/migraphx/  (forces full recompilation)     |
  |                                                                    |
  |  On Xen VM: this cache consumed 11 GB, contributing to the disk   |
  |  space crisis. Monitor with: du -sh /tmp/migraphx/                |
  `-----------------------------------------------------------------------+

  Disk Space Management
  +-----------------------------------------------------------------------+
  |  Trigger: "OSError: [Errno 28] No space left on device"            |
  |  during onnxsim of MobileSAM Full.                                 |
  |                                                                    |
  |  Root cause:                                                       |
  |    /tmp/migraphx/:     11 GB of compilation cache                  |
  |    llama32_onnx/:      130 GB total, 66 GB unused intermediates   |
  |                                                                    |
  |  Files purged (78 GB freed):                                       |
  |    * Abandoned monolithic ONNX exports (llama32_vision_full,       |
  |      llama32_full_ext): ~42 GB                                     |
  |    * Intermediate encoder exports: ~4 GB                           |
  |    * Pre-optimization decoder data: ~20 GB                         |
  |    * Loose .weight / onnx__MatMul_* files: ~12 GB                  |
  |    * MIGraphX cache: 11 GB (recompiles on next run)                |
  |                                                                    |
  |  Prevention: always clean ONNX export intermediates after split.  |
  |  Monitor: df -h /tmp && du -sh /tmp/migraphx/                     |
  `-----------------------------------------------------------------------+
```

E.3 Per-Model Workaround Patterns
---

```
  +-- DIAGRAM: Workaround Pattern Quick Reference --------------------------+
  |                                                                         |
  |  SYMPTOM                      ROOT CAUSE              FIX              |
  |  ------------------           --------------          -----------      |
  |  "bf16 not supported"         gfx1151 has no BF16     Convert bf16    |
  |                                intrinsics              -> fp16 offline  |
  |                                                                         |
  |  "parse error on reshape"     MIGraphX parser bug     Run onnxsim     |
  |                                with complex reshapes   on model first  |
  |                                                                         |
  |  "stack overflow in hipRTC"   Graph too deep for       Split encoder/  |
  |                                C++ template depth      decoder apart   |
  |                                                                         |
  |  "template depth exceeded"    Deep recursion during    Set -ftemplate  |
  |                                JIT compilation          -depth=2048    |
  |                                                                         |
  |  "wrong inference results"    ORT fusions conflict     Set ORT_DISABLE |
  |                                with MIGraphX fusions   _ALL level      |
  |                                                                         |
```

```
  |  "out of memory (512 MB)"    hipMalloc only sees       Use hipMalloc  |
  |                                VRAM carve-out           Managed + XNACK|
  `-------------------------------------------------------------------------+

  These patterns apply to specific model families, not to MIGraphX in general.

  Pattern 1: BFloat16 -> Float16 Conversion (CogACT, some HuggingFace models)
    Problem:  Model exported with bf16 weights; MIGraphX on gfx1151 has no
              bf16 intrinsic support (only gfx942+ has BF16 hardware)
    Fix:      Convert offline before deployment
              onnxconverter-common float16 conversion (see Lesson 4)

  Pattern 2: onnxsim for Reshape Bugs (MobileSAM)
    Problem:  MIGraphX parser fails on certain reshape sequences
    Fix:      python -m onnxsim model.onnx model_sim.onnx
              Simplifies graph, eliminates problematic reshape patterns

  Pattern 3: Encoder/Decoder Split (Whisper, LLaMA-3.2-11B)
    Problem:  Monolithic 10B+ graph exceeds C++ template depth during
              MIGraphX compilation (stack overflow in hipRTC)
    Fix:      Export encoder and decoder as separate ONNX files
              Run sequentially: encoder output -> decoder input
              Each half compiles independently (manageable graph size)

  Pattern 4: Template Depth Flags (LLaMA, OpenVLA, CogACT)
    Problem:  Deep graph -> deep C++ template recursion -> compile crash
    Fix:      MIGRAPHX_GPU_HIP_FLAGS="-ftemplate-depth=2048
                                      -Wno-error=stack-exhausted"
              Plus: ulimit -s unlimited (prevent stack segfault)

  Pattern 5: Disable ORT Graph Optimizations (YOLOv12, CenterPoint)
    Problem:  ORT's fusion passes produce subgraphs that confuse MIGraphX
    Fix:      sess_options.graph_optimization_level = ORT_DISABLE_ALL
              Let MIGraphX handle all optimizations


E.4 Complete Deployment Checklist
----------------------------------

  +-- DIAGRAM: Deployment Flow -- Step by Step ------------------------------+
  |                                                                          |
  |  +-- 1. ENVIRONMENT -----------------------------------------------------+|
  |  | export HSA_XNACK=1                                                  ||
  |  | export no_system_mem_limit=Y                                        ||
  |  | export MIGRAPHX_GPU_HIP_FLAGS="-ftemplate-depth=2048 ..."          ||
  |  | export MIGRAPHX_GPU_COMPILE_PARALLEL=1                              ||
  |  | ulimit -s unlimited                                                 ||
  |  `---------------------------------+--------------------------------------+|
  |                                    v                                     |
  |  +-- 2. MODEL PREP ------------------------------------------------- + |
  |  | bf16 weights? -> convert to fp16                                   | |
  |  | Parse errors? -> run onnxsim                                       | |
  |  | > 10B params? -> split encoder/decoder                             | |
  |  `---------------------------------+--------------------------------------+|
  |                                    v                                     |
  |  +-- 3. SESSION CREATE --------------------------------------------- + |
  |  | session = ort.InferenceSession("model.onnx", providers=[...])    | |
  |  | assert session.get_providers()[0] == 'MIGraphXExecutionProvider' | |
  |  | (first load: slow = compile + cache -- this is normal)            | |
  |  `---------------------------------+--------------------------------------+|
  |                                    v                                     |
  |  +-- 4. INFERENCE -------------------------------------------------- + |
  |  | result = session.run(None, {"input": data})                       | |
  |  | (subsequent runs: fast -- cached kernels, warm page tables)       | |
  |  `---------------------------------+--------------------------------------+|
  |                                    v                                     |
  |  +-- 5. MONITOR ---------------------------------------------------- + |
  |  | rocm-smi (GPU temp/util) | du -sh /tmp/migraphx/ (cache)       | |
  |  | free -h (RAM/swap)       | dmesg | grep amdgpu (errors)        | |
  |  `------------------------------------------------------------------- + |
  `-------------------------------------------------------------------------+

  Before first run:
    [ ] export HSA_XNACK=1
    [ ] export no_system_mem_limit=Y
    [ ] export MIGRAPHX_GPU_HIP_FLAGS="-ftemplate-depth=2048
            -Wno-error=stack-exhausted"
    [ ] export MIGRAPHX_GPU_COMPILE_PARALLEL=1
    [ ] ulimit -s unlimited
    [ ] Verify swap: swapon --show (need >> model size for compilation)
    [ ] Verify disk: df -h /tmp (need ~15 GB free for MIGraphX cache)
    [ ] Custom ORT 1.25.0 installed (with Fixes A-D)
```

```
  Per model:
    [ ] Run onnxsim on the model if parse fails
    [ ] Convert bf16 -> fp16 if model has bfloat16 weights
    [ ] Split encoder/decoder for 10B+ parameter models
    [ ] Set ORT_DISABLE_ALL if model produces wrong results

  After session creation:
    [ ] Verify: session.get_providers()[0] == 'MIGraphXExecutionProvider'
    [ ] First inference is slow (compilation + page faults) -- this is normal
    [ ] Subsequent inferences are fast (cached kernels + warm page tables)

  Monitoring:
    [ ] rocm-smi -> GPU utilization, VRAM usage, temperature
    [ ] du -sh /tmp/migraphx/ -> compilation cache size
    [ ] free -h -> RAM/swap usage during compilation
    [ ] dmesg | grep amdgpu -> driver-level errors or page faults


================================================================================
                           END OF TEXTBOOK
================================================================================
```